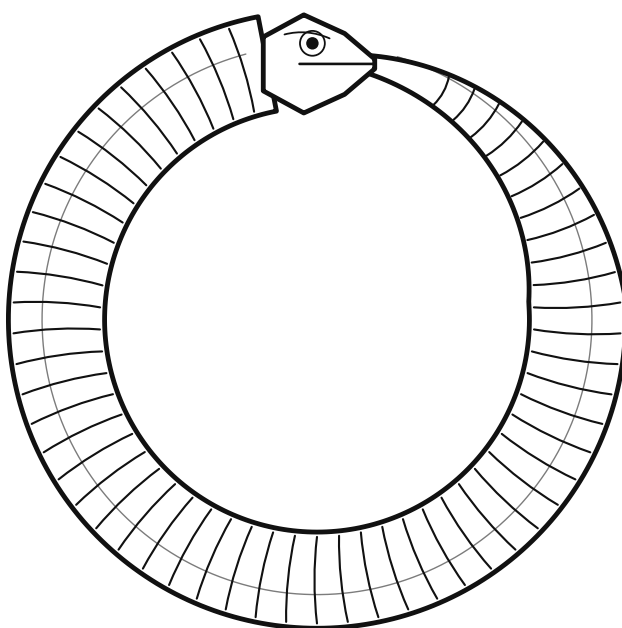


THE SELF-TYPESETTING LIBRARY

Learning Typst

A hands-on guide to the Typst typesetting system



The ouroboros — a book that typesets itself.

Lars De Richter

with Claude (Opus 4.8)

In the spirit of the classic animal books — an homage, not one of them.

Learning Typst

A hands-on guide to the Typst typesetting system

Lars De Richter, with Claude (Opus 4.8)

Contents

Preface	2
Who this book is for	2
How to read this book	2
Conventions used in this book	2
About the examples	3
A note on how this book was made	3
1 Why Typst?	4
1.1 What Typst actually is	4
1.2 A little history	4
1.3 Markup meets code	5
1.4 How Typst compares	5
1.5 What you'll be able to do	6
1.6 What you've got	6
1.7 Exercises	6
2 Getting Typst running	8
2.1 Three ways to run Typst	8
2.2 Installing the command-line binary	8
2.3 Your first document	9
2.4 The watch loop	10
2.5 Editor setup: TinyMist	10
2.6 Output formats and handy flags	10
2.7 Project structure and <code>--root</code>	11
2.8 What you've got	11
2.9 Exercises	12
3 Markup: the content layer	13
3.1 Paragraphs and whitespace	13
3.2 Headings	13
3.3 Emphasis: italic and bold	14
3.4 Raw text: code and verbatim	14
3.5 Lists	14
3.6 Links	15
3.7 Smart quotes and punctuation	15
3.8 Escapes and comments	16
3.9 A taste of labels and references	16
3.10 Two symbols you'll meet soon	16
3.11 What you've got	17
3.12 Exercises	17
4 Text and fonts	18
4.1 Styling text: the <code>text</code> function	18
4.2 Choosing a typeface	18
4.3 Size, weight, and style	19
4.4 Color	19
4.5 Decorations	20
4.6 Figures, ligatures, and other OpenType niceties	20
4.7 Language and hyphenation	21
4.8 What you've got	21
4.9 Exercises	22
5 Pages and layout	23
5.1 The page is one function	23
5.2 Headers and footers	24
5.3 Page numbers	24
5.4 Columns	25
5.5 The shape of a paragraph	25

5.6	Spacing and breaks by hand	26
5.7	Fills and watermarks	26
5.8	What you've got	27
5.9	Exercises	27
6	Figures and images	28
6.1	Placing an image	28
6.2	Wrapping it in a figure	29
6.3	Referencing figures	29
6.4	Figures aren't only for images	30
6.5	Sizing and fit	30
6.6	Letting figures float	31
6.7	What you've got	31
6.8	Exercises	31
7	Tables and grids	33
7.1	The shape of a table	33
7.2	Sizing the columns	34
7.3	Alignment	34
7.4	Fills and zebra striping	35
7.5	Header and footer rows	35
7.6	Spanning cells	36
7.7	Strokes, lines, and the booktabs look	36
7.8	Spacing: inset and gutters	37
7.9	grid : the same engine, minus the meaning	37
7.10	What you've got	38
7.11	Exercises	38
8	Math and equations	39
8.1	Entering math mode	39
8.2	Symbols have names, not backslashes	39
8.3	Superscripts and subscripts	40
8.4	Fractions and roots	40
8.5	Functions and operators	40
8.6	Sums, integrals, and other big operators	41
8.7	Matrices, vectors, and cases	41
8.8	Lining equations up	42
8.9	Text, styles, and dropping into code	42
8.10	Numbering and cross-references	42
8.11	What you've got	43
8.12	Exercises	43
9	Set rules	45
9.1	What a set rule actually is	45
9.2	Where a set rule reaches	46
9.3	A tour of the set rules you'll actually use	46
9.4	Numbering patterns, in one place	47
9.5	Setting a default only when a condition holds	48
9.6	The mental model to keep	48
9.7	What you've got	49
9.8	Exercises	49
10	Show rules	50
10.1	Set tunes, show transforms	50
10.2	The function form: rebuilding an element	50
10.3	The show-set form: set rules, but selective	51
10.4	Selectors: what a rule can match	51
10.5	Matching text, not elements	52
10.6	The document-wide show rule	53
10.7	Two things that bite	53
10.8	What you've got	54

10.9 Exercises	54
11 References and cross-references	55
11.1 Labels: naming a thing	55
11.2 Referencing: @name and #ref	55
11.3 Referencing headings (numbering required)	56
11.4 Supplements: the word before the number	56
11.5 Customizing every reference	57
11.6 Outlines: the table of contents	57
11.7 Lists of figures and tables	58
11.8 Coming from LaTeX	58
11.9 The whole point: they never lie	58
11.10 What you've got	58
11.11 Exercises	59
12 Citations and bibliographies	60
12.1 The shortest possible bibliography	60
12.2 Where the sources live: YAML or BibTeX	61
12.3 Getting APA right	62
12.4 The many shapes of a citation	63
12.5 Only what you cite	63
12.6 Choosing a style	64
12.7 What you've got	64
12.8 Exercises	64
13 From markup to code	66
13.1 The two worlds	66
13.2 Naming a value with #let	66
13.3 Values have types	67
13.4 Operators	68
13.5 Two kinds of block	69
13.6 Putting values back: interpolation	70
13.7 A first taste of methods	70
13.8 What you've got	71
13.9 Exercises	71
14 Functions and closures	72
14.1 Defining a function	72
14.2 Returning content, returning a value	73
14.3 Named arguments and defaults	73
14.4 Passing a whole block: the content-argument trick	73
14.5 However many arguments you like	74
14.6 Functions are values	75
14.7 Closures: functions that remember	75
14.8 Pre-filling arguments with .with()	76
14.9 What you've got	76
14.10 Exercises	76
15 Control flow	78
15.1 Choosing with if	78
15.2 Repeating with for	79
15.3 Counting with range	80
15.4 Repeating with while	80
15.5 Steering a loop: break and continue	81
15.6 A note on scope	81
15.7 The payoff: data in, document out	81
15.8 What you've got	82
15.9 Exercises	82
16 Arrays, dictionaries, and strings	84
16.1 Arrays: values in a row	84
16.2 The transformer trio	85

16.3 Dictionaries: values with names	86
16.4 Strings: sequences of characters	87
16.5 Putting it together: an array of dictionaries	88
16.6 What you've got	89
16.7 Exercises	89
17 Context, state, and counters	91
17.1 The value you can't have yet	91
17.2 Counters	92
17.3 State	93
17.4 measure and layout	94
17.5 Tying it together	95
17.6 What you've got	95
17.7 Exercises	95
18 Your own functions	97
18.1 Machinery in one file, content in another	97
18.2 The three ways to import	98
18.3 Building a small style library	99
18.4 Bundling a whole look into one function	100
18.5 import brings definitions; include brings content	100
18.6 Keeping the library tidy	101
18.7 What you've got	102
18.8 Exercises	102
19 Templates	103
19.1 A template is a function	103
19.2 Giving the template something to configure	103
19.3 Configuration versus content	104
19.4 Moving the template into its own file	105
19.5 A real article template	105
19.6 Templates you can install	107
19.7 What you've got	107
19.8 Exercises	108
20 Packages	109
20.1 Typst Universe	109
20.2 Why the version is pinned	110
20.3 A short tour of Universe	110
20.4 Starting from a template package	111
20.5 Publishing your own package	111
20.6 What you've got	113
20.7 Exercises	113
21 Advanced layout	114
21.1 Placing things exactly	114
21.2 Boxes and blocks	115
21.3 Stacking	115
21.4 Nudging within the flow	116
21.5 Transforming content	116
21.6 Filling space with <code>#repeat</code>	116
21.7 Reserving space without drawing	117
21.8 Asking the document about itself	117
21.9 Drawing your own shapes	118
21.10 A door to native code	118
21.11 What you've got	118
21.12 Exercises	119
22 Designing a book template	120
22.1 The shape of the project	120
22.2 The theme: every knob in one place	120
22.3 The template function	121

22.4	Chapters and parts	122
22.5	The running head, from <code>query</code>	123
22.6	The contents page	124
22.7	The admonition boxes	124
22.8	Theming the code blocks	125
22.9	The index	125
22.10	Tying it together in <code>main.typ</code>	126
22.11	What you've got	127
22.12	Exercises	127
23	Building the book	128
23.1	The book as a manifest	128
23.2	One file per chapter	129
23.3	Front matter, body, and back matter	129
23.4	Building it	130
23.5	Problems that only appear at length	131
23.6	Where this is going	131
23.7	What you've got	132
23.8	Exercises	132
24	The Pandoc bridge	133
24.1	Pandoc, the universal converter	133
24.2	The three-step pipeline	133
24.3	Teaching Pandoc new tricks with a Lua filter	134
24.4	The head: the Chapter 22 template, reused	136
24.5	Running it	137
24.6	What's simplified, honestly	137
24.7	What you've got	137
24.8	Exercises	137
Appendix A	• Solutions to the exercises	139
	Chapter 1 – Why Typst?	139
	Chapter 2 – Getting Typst running	139
	Chapter 3 – Markup: the content layer	139
	Chapter 4 – Text and fonts	140
	Chapter 5 – Pages and layout	141
	Chapter 6 – Figures and images	142
	Chapter 7 – Tables and grids	143
	Chapter 8 – Math and equations	144
	Chapter 9 – Set rules	145
	Chapter 10 – Show rules	146
	Chapter 11 – References and cross-references	147
	Chapter 12 – Citations and bibliographies	148
	Chapter 13 – From markup to code	148
	Chapter 14 – Functions and closures	149
	Chapter 15 – Control flow	150
	Chapter 16 – Arrays, dictionaries, and strings	151
	Chapter 17 – Context, state, and counters	153
	Chapter 18 – Your own functions	154
	Chapter 19 – Templates	155
	Chapter 20 – Packages	156
	Chapter 21 – Advanced layout	157
	Chapter 22 – Designing a book template	158
	Chapter 23 – Building the book	159
	Chapter 24 – The Pandoc bridge	160
Appendix B	• From LaTeX to Typst	162
	Document setup	162
	Text formatting	163
	Sectioning and cross-references	163

Lists	164
Math	164
Tables	165
Figures and images	166
Bibliography and citations	166
Custom commands and packages	167
Things that are just gone	167
Appendix C • From Word to Typst	168
The one shift that explains all the others	168
How do I do X? — the quick reference	168
Page setup	169
Headers, footers, and page numbers	169
Tables and pictures	170
Tables of contents and cross-references	170
Equations	170
Find & Replace for formatting	170
Templates	171
Mail merge, i.e. a hundred certificates	171
Track Changes and version history	171
What Word still does better	171
Appendix D • Quick reference	173
Markup (Ch 3)	173
Text and fonts (Ch 4)	173
Pages and paragraphs (Ch 5)	174
Positioning and layout (Ch 21)	175
Figures and images (Ch 6)	175
Tables and grids (Ch 7)	175
Math (Ch 8)	176
Scripting: values and functions (Ch 13–14)	176
Control flow (Ch 15)	177
Arrays, dictionaries, strings (Ch 16)	177
Set and show rules (Ch 9–10)	177
References and bibliography (Ch 11–12)	177
Context, counters, state (Ch 17, 21)	178
Files and packages (Ch 18, 20)	178
Command line (Ch 2)	178
Appendix E • Where to go next	180
The official documentation	180
The web app	180
Typst Universe	180
The community forum	180
GitHub: the source and the registry	180
Community chat	180
Curated lists	181
Staying current	181
How to ask a good question	181
Where to practice	181
A last word	181
Appendix F • Pandoc, in practice	182
What Pandoc is	182
The commands worth knowing	182
Filters, concisely	184
Pandoc + Typst: an idea bank	185

Preface

There is a particular kind of misery known to anyone who has tried to make a document look right. You have something to say — a report, a thesis, a set of lecture notes, a recipe for your grandmother's stew — and somewhere between the saying and the printing, the tools turn on you. A heading drifts onto its own lonely page. An image lands three paragraphs from where you put it. You spend a Sunday afternoon fighting a word processor for control of a single table, and the table wins.

Typst is a quiet rebellion against that afternoon.

It is a *typesetting system*: software whose entire job is to take your content and lay it out beautifully on a page. In that sense it shares a mission with LaTeX, the system that has typeset most of the world's mathematics for forty years. But where LaTeX arrived in the 1980s and shows its age, Typst arrived in 2023 and feels like it. You write in a clean, readable markup — closer to Markdown than to anything with a backslash — and Typst compiles it into a polished PDF in milliseconds, telling you in plain language when something is wrong instead of vomiting three hundred lines of cryptic warnings.

This book teaches you Typst from the first `= Hello` to writing your own reusable packages. By the end you'll be able to typeset an essay, a scientific paper with citations, a book, or a template you hand to two hundred students — and you'll understand not just *which* buttons to press but *why* the system works the way it does.

Who this book is for

You, probably. Specifically:

- **People who write things that need to look good** — students, researchers, teachers, technical writers, self-publishers, hobbyists with a manuscript in a drawer.
- **Refugees from word processors** who are tired of documents that reflow into chaos the moment they add a sentence.
- **Refugees from LaTeX** who love the output but not the ceremony. (You'll feel at home fast. There are little "Coming from LaTeX" asides throughout to translate.)

You do **not** need to know LaTeX. You do **not** need to be a programmer. If you can edit a plain-text file and you're not frightened of the occasional curly brace, you have everything required. Typst does contain a genuine little programming language — one of its best features — and Part IV teaches it from absolute zero, assuming nothing.

What you do need is a habit of *trying things*. Typst compiles so fast that the right way to learn it is to keep a document open, change something, and watch what happens. This book is built for that habit. Read it near a keyboard.

How to read this book

Front to back, if you can. The chapters build on each other, and later ones assume the vocabulary of earlier ones. But the book is also organized so you can raid it:

- **Part I** gets Typst installed and teaches you to write basic marked-up content — the stuff you type before you think about styling.
- **Part II** covers everyday documents: text, fonts, pages, figures, tables, and Typst's spectacular math support.
- **Part III** introduces *set* and *show* rules — the two ideas that turn Typst from a markup language into a styling engine — plus references and bibliographies.
- **Part IV** teaches the scripting language properly: values, functions, loops, and the genuinely tricky concept of *context*.
- **Part V** is about reuse: your own functions, templates, and packages.
- **Part VI** is the capstone. We build an O'Reilly-style book template in Typst, then wire up a pipeline that typesets *this very book* from its Markdown source. The book, in other words, learns to print itself.

If you're impatient and already technical, you could skim Part I, skip to the part you need, and refer back. Nothing will break. If you're new to all of this, resist the urge to skip — the early chapters plant seeds that later ones grow.

Conventions used in this book

Throughout the book you'll see a few recurring signposts.

`Constant width` is used for anything you'd type or that Typst would show you — function names like `image`, snippets of code, filenames like `main.typ`, and terminal commands.

Longer code appears in blocks, labeled by language:

= A heading

Some ***emphasized*** text and a bit of `#text(fill: blue)[colour]`.

```
typst compile main.typ out.pdf
```

Set off from the main text, you'll find boxes. They're the descendants of those little animal icons in old O'Reilly books, and they mean:

NOTE

A clarification or a useful fact worth pausing on.

TIP

A shortcut or a nicer way to do what you just learned.

IMPORTANT

Something you genuinely need to get right.

WARNING

A common trap. People fall in here. Mind the gap.

And plain sidebars translate from other worlds:

Coming from LaTeX. Where you would have written `\documentclass{article}` and a wall of preamble, Typst asks for nothing. A blank file is already a valid document.

About the examples

Every example in this book is a real, runnable project, collected in the `examples/` folder of the book's repository. Each lives in its own numbered directory with the source (`main.typ`), a short note on what it shows, and a committed rendering so you can see the result without lifting a finger.

They are not decorative. Every one of them compiles — cleanly, with no errors — against **Typst 0.15.0**, and there's a script that rebuilds and checks all of them at once. When the book claims a snippet produces a certain result, it does, because a compiler was made to prove it before the page went to press. If you find one that doesn't, that's a bug, and you should feel entitled to complain.

To run any example yourself:

```
cd examples/001-hello-typesetting
typst compile main.typ out.pdf
```

Then open `out.pdf`. That's the whole loop. You'll be doing it in your sleep by Chapter 3.

A note on how this book was made

Full disclosure, because you deserve it: this book was written with heavy assistance from an AI system, working under human direction and — crucially — under human quality control. Every chapter was reviewed. Every example was compiled by a real Typst binary, not imagined. The goal was a book good enough that its origins wouldn't matter; you'll have to judge whether that worked.

It's released under a Creative Commons license (BY-NC-SA 4.0) because the person who commissioned it learns best from books like this one, noticed that no such book existed for Typst yet, and figured that if it helped him, it might help you too. Share it. Teach from it. Improve it. Just don't sell it, and pass along the same freedoms you were given.

Now. Let's go find out why Typst is worth your afternoon — the good kind, this time.

Why Typst?

Here is a small file. Six lines, no ceremony:

= Hello, Typst

Typesetting used to mean wrestling with your software. Not here. You write plain text with a little markup, and Typst turns it into a properly typeset page – real ligatures, automatic hyphenation, and micro-typography you never had to ask for.

Feed it to Typst and, in the time it takes to blink, you get a PDF: a crisp heading, a neat paragraph, proper typography, the works. No document class to choose. No preamble to copy from a forum post you half understand. No "package float not found." You wrote content; you got a document.

That gap – between how much you wrote and how much you got – is the reason this book exists. Let's talk about why it's so wide.

1.1 What Typst actually is

Typst is a **typesetting system**. That word is worth unpacking, because it's the whole game.

A word processor like Microsoft Word or Google Docs is a *what-you-see-is-what-you-get* editor. You manipulate the document's appearance directly, by clicking and dragging, and the file stores a tangle of content and formatting fused together. It's wonderful for a two-page letter and increasingly hostile as your document grows.

A typesetting system takes a different deal. You write your content as plain text, marked up to describe its *structure* – "this is a heading," "this word is emphasized," "here is a list" – and you describe the *appearance* separately, as rules. Then a program reads both and does the fussy work of arranging ink on the page: line breaking, hyphenation, spacing, page breaks, keeping figures near their captions. You say *what*; the system figures out *where*.

This is the model LaTeX has used to typeset scientific literature for four decades. Typst uses it too. The difference is roughly forty years of hindsight.

NOTE

"Typesetting" is an old printing word. It once meant physically arranging metal type in a frame. The craft survived the metal; the software that does it now inherited both the name and the exacting standards.

1.2 A little history

Typst is young. It grew out of the work of two graduate students, Laurenz Mäde and Martin Haug, at the Technical University of Berlin, who had done enough LaTeX to know exactly what hurt. Their theses turned into a real system, written from scratch in the Rust programming language, and in 2023 it went public and open source.

Two design decisions from that origin shape everything you'll experience:

- **It's fast.** Typst was built around *incremental compilation* – when you change one line, it recomputes only what that line affects. In practice the preview updates as fast as you can type. Compare this to LaTeX, where a big document can take a coffee break to build, and the difference in how you *work* is profound. Fast feedback changes learning from "guess, wait, despair" to "try, see, adjust."
- **It's designed, not accreted.** LaTeX is a set of macros piled on top of TeX, Donald Knuth's original 1978 engine, with layer upon layer added by thousands of hands over decades. It's magnificent and it's a sediment. Typst was designed as one coherent thing, with a single consistent language for content, styling, and scripting. There's a lot less to hold in your head.

TIP

You don't have to install anything to try Typst. There's a web app at typst.app with a live preview, a bit like Google Docs crossed with a code editor. We'll set up both the web app and the command-line version in Chapter 2. For now, you can follow along just by reading.

1.3 Markup meets code

Most of what you write in Typst is *markup*: lightweight symbols that mean "heading" or "emphasis" or "list item." If you've written Markdown, it will feel familiar. A `=` starts a heading. Asterisks around a word make it **bold**. Hyphens make a bullet list. That's Example 001 in the book's repository ([examples/001-hello-typesetting/](https://github.com/typst/typst-examples/blob/main/001-hello-typesetting/)), and it renders exactly what you'd hope:

Hello, Typst — a bold heading over a cleanly set paragraph and a tidy bullet list, all from plain text.

But here's where Typst leaves ordinary markup behind. Underneath the friendly surface sits a real programming language, and you can drop into it any time you like by typing `#`. Watch what that buys you ([examples/002-markup-meets-code/](https://github.com/typst/typst-examples/blob/main/002-markup-meets-code/)):

```
= Powers of two
This list wasn't typed out by hand — a loop generated it:

#for n in range(1, 11) [
  / $2^#n$: #calc.pow(2, n)
]
```

That `#for` runs a loop right inside your document. Each pass produces one line of a list, computing `2` to the power of `n` as it goes. The result is a clean table of the powers of two from 2 up to 1024 — and if you changed the `11` to `101`, you'd get a hundred rows without touching the list itself.

Stop and appreciate how strange and wonderful that is. Your document is also a *program that writes the document*. Need a multiplication table, a calendar, a chart labeled from a data file, a hundred certificates with a hundred different names? You don't do it by hand. You describe it once and let the machine elaborate. Part IV of this book teaches that language properly, from the ground up. For now, just file away the headline: **in Typst, content and computation live in the same file.**

NOTE

Don't worry about the `$2^#n$` yet — the dollar signs mean "this is math," and Typst's math typesetting is so good it gets its own chapter (Chapter 8). The point here is only that markup, math, and code all mix freely.

1.4 How Typst compares

You almost certainly arrived here from another tool. Here's an honest map of the neighborhood.

1.4.1 Versus word processors (Word, Google Docs, Pages)

Word processors are direct-manipulation editors, and for short, simple documents that's exactly right — nobody should write a birthday-party invitation in Typst. The trouble starts as documents grow. Formatting and content are welded together, so a consistent change ("make every figure caption smaller") means hunting through the whole file by hand. Long documents reflow unpredictably. The files are opaque binaries, so you can't meaningfully track changes in tools like Git, and "track changes" inside the app is a poor substitute.

Typst inverts all of that. Your source is plain text, so it's diffable, searchable, and scriptable. Styling is centralized in rules, so one change propagates everywhere. And because layout is computed from your structure, a hundred-page document behaves as predictably as a one-page one.

Coming from Word. The mental shift is this: you stop *formatting* and start *labeling*. Instead of selecting a line and clicking a bigger font, you mark it as a heading and decide, once, in one place, what headings look like. It feels like more work for the first ten minutes and less work forever after.

1.4.2 Versus LaTeX

This is the close comparison, because Typst and LaTeX aim at the same target: beautiful, structured, automated documents, especially ones full of mathematics. If you know LaTeX, you already believe in the plain-text approach; the question is only whether to switch.

What Typst gives you over LaTeX: near-instant compilation and preview; a syntax you can read without wincing; error messages written for humans; and one unified scripting language instead of TeX's notorious macro system, where defining a function can feel like performing surgery with oven mitts.

What LaTeX still gives you over Typst: forty years of accumulated packages for every niche imaginable, and its status as the *lingua franca* of academic publishing — many journals still want LaTeX, full stop. Typst's ecosystem is younger and smaller, though it's growing quickly and already covers the common cases well (Chapter 20).

Coming from LaTeX. The single biggest relief is the feedback loop. Save, and the preview is already updated — no build, no wait, no scrolling through a log to find the one real error among forty warnings. It changes how you work more than any syntax difference.

1.4.3 Versus Markdown

Markdown and Typst look like cousins, and their basic markup really is similar. But Markdown was designed to be converted *into* something else — usually HTML — and it deliberately stops at simple structure. It has no opinion about page size, no real story for fine typography, no standard for typesetting an equation, no way to compute part of your document. It's a lovely notation for content and nothing more, by design.

Typst is a Markdown-like *input* wired to a full typesetting engine and a programming language *behind* it. You get the readability of Markdown and the power of LaTeX in one place. And the two worlds aren't even enemies: in Chapter 24 we build a pipeline that converts Markdown *to* Typst, which is exactly how this book turns its own Markdown chapters into a printed PDF.

1.4.4 Versus InDesign and friends

Page-layout software like Adobe InDesign is built for *visual*, manual design — magazines, posters, packaging — where a human places every element for effect. That's a different craft from what Typst does. Typst shines on *structured, content-driven* documents where consistency and automation matter more than hand-tuned art direction: books, papers, reports, theses, documentation. Use the right tool. This book is about the second kind.

1.5 What you'll be able to do

By the time you reach the last page, you'll be able to:

- Write essays, reports, and letters that look professionally typeset, from plain text, in minutes.
- Typeset serious mathematics without tears (Chapter 8).
- Manage citations and produce a correct APA bibliography (Chapter 12).
- Build tables that would make a spreadsheet jealous (Chapter 7).
- Write your own functions to automate repetitive layout (Chapters 14 and 18).
- Package your styles into a reusable template and share it (Chapters 19, 20).
- Assemble a full book — chapters, table of contents, index, the lot — and typeset it from Markdown (Part VI).

That last one isn't a hypothetical. It's this book. The thing you're reading was built with the very techniques it teaches, and the final chapters walk you through exactly how.

1.6 What you've got

You now know:

- **What Typst is** — a typesetting system that turns marked-up plain text into beautifully laid-out PDFs, in the LaTeX tradition but built fresh.
 - **Where it came from** — a young, Rust-based, open-source project designed for speed and coherence.
 - **Its central idea** — markup and a real programming language in the same file, so documents can compute themselves.
 - **How it stacks up** against word processors, LaTeX, Markdown, and layout software, and when each is the right call.
- That's the pitch. The rest of the book is the delivery. Next up: getting Typst running on your own machine (or in your browser) and compiling your first document with your own hands.

1.7 Exercises

Solutions are in Appendix A.

- 1.1. In your own words, what's the difference between how a word processor and a typesetting system treat the *content* and the *appearance* of a document? Why does that difference matter more as a document gets longer?
- 1.2. Name a document type for which a word processor is genuinely the better tool than Typst, and say why. (Being able to see the other side is a sign you understand the trade-off.)
- 1.3. Look again at the powers-of-two example. Without running anything, predict what you'd get if you changed `range(1, 11)` to `range(1, 4)`. Then, once you've installed Typst in the next chapter, check yourself.
- 1.4. (*Stretch.*) Skim the Typst reference documentation for two minutes — just the landing page and one link that catches your eye. You won't understand most of it yet, and that's fine. The goal is only to see the shape of the territory before we start walking it.

Getting Typst running

Installing a typesetting system has a reputation. If you have ever set up LaTeX, you remember it: a download measured in gigabytes, an afternoon spent feeding a package manager, and a folder somewhere on your disk you will never fully understand.

Typst is one file. You run a single command, wait a second or two, and you have a program that already knows how to typeset a book — fonts included. No folder of mysteries. When a newer version lands, you replace the one file.

So this is a short chapter with a big payoff. By the end you'll have Typst on your machine, you'll have compiled a document with your own hands, and — this is the part that actually matters — you'll have set up the fast edit-and-preview loop that turns learning Typst into something closer to play than study.

2.1 Three ways to run Typst

There are three ways to put Typst to work, and they are not rivals. Most people end up using two of them.

- **The web app** at `typst.app`. A free account gets you an in-browser editor with a live preview beside your text and real-time collaboration, a bit like Google Docs crossed with Overleaf. Nothing to install. It's the fastest way to try Typst, and a genuinely good place to write.
- **The command-line binary**. This is the `typst` program you run in a terminal, and it's the focus of this book. It compiles a source file into a PDF (or PNG, or SVG), it scripts and automates cleanly, and it's the same engine everything else is built on.
- **An editor with the Typst extension**. Your normal code editor — VS Code, Neovim, and others — can gain autocomplete, a preview pane, and live error underlines. This isn't really a fourth engine; it's the command-line world with a comfortable front end bolted on. We'll set it up near the end of the chapter.

If you just want to feel Typst move, open the web app right now and paste in the six-line file from Chapter 1. The preview appears as you type. Come back when you're ready to work locally — the rest of this book assumes the command line, because that's what you'll build real projects and templates on.

2.2 Installing the command-line binary

The commands below are the current, official ones. Pick your platform. Each one downloads a single self-contained binary; there is no second step.

2.2.1 macOS

If you have Homebrew:

```
brew install typst
```

2.2.2 Windows

With the built-in Windows package manager, `winget` :

```
winget install --id Typst.Typst
```

If you prefer Scoop or Chocolatey, Typst is packaged for both (`scoop install typst` , `choco install typst`). Any of them is fine; they all fetch the same binary.

2.2.3 Linux

Typst is in a growing number of distribution repositories — search your package manager for `typst` , or check Repology to see what your distro ships. There's also a Snap (`snap install typst`). Distro packages sometimes lag a version or two behind, so if you need the newest release, grab the prebuilt binary straight from the GitHub releases page: download the archive for your architecture, unpack it, and put the `typst` file somewhere on your `PATH` (such as `~/.local/bin`).

2.2.4 The universal route: Rust's Cargo

If you have the Rust toolchain installed, one command works on every platform and always builds the latest release:

```
cargo install --locked typst-cli
```

This compiles Typst from source, so it takes a few minutes and asks for nothing but Rust. It's the reliable fallback when a package manager doesn't have what you want.

2.2.5 Confirm it worked

Whatever route you took, open a fresh terminal and ask Typst who it is:

```
typst --version
```

You should see something like:

```
typst 0.15.0
```

This book is written and tested against **0.15.0**. A newer version will run every example here just fine; if you're on something much older and a snippet misbehaves, updating is the first thing to try.

WARNING

If instead you get `command not found: typst` (or on Windows, `'typst' is not recognized...`), the install worked but your shell can't find the binary — it isn't on your `PATH`. Two fixes usually do it: open a brand-new terminal window (so it re-reads your configuration), and if you installed by hand, make sure the folder holding `typst` is actually listed in `PATH`. This is the single most common first-day snag, and it's never as broken as it looks.

Coming from LaTeX. There is no TeX distribution to install here — no multi-gigabyte MacTeX or TeX Live, no `tlmgr`, no waiting for `latexmk` to settle. Typst is one binary that ships its own fonts and its own math engine. "Install" means "download a file," and "the fonts" means "they're already inside."

2.3 Your first document

Time to make something. Create a folder, and in it a plain-text file called `main.typ`. The name doesn't matter to Typst, but `main.typ` is the convention this book (and most projects) uses for the entry point. Put a couple of lines in it:

```
= My first Typst document
```

```
Hello. This paragraph is here so the page has something to typeset. Notice
that you did not choose a font, a size, or a margin — Typst picked sensible
defaults so you could get straight to writing.
```

That `=` at the start of the first line marks a heading, exactly as in Chapter

1. Everything else is just text. Now, from inside that folder, run:

```
typst compile main.typ out.pdf
```

Open `out.pdf`. There's your document: a bold heading over a neatly set, properly typeset paragraph, on a clean page. You wrote structure; Typst supplied the typography.

That single command is the whole idea, so let's name its parts. `main.typ` is your **source** — the human-readable file you edit. `out.pdf` is the **output** — the finished document Typst produced. **Compiling** is the act of turning one into the other, and it's the verb you'll use a thousand times. The source is the thing you keep and change; the output is disposable, regenerated on demand, and usually not even worth saving in version control.

Notice that you never told Typst you wanted a PDF. It figured that out from the `.pdf` on the output name. Typst infers the format from the extension, so `out.png` would have given you an image and `out.svg` a vector graphic — more on that shortly. The runnable version of this document is in `examples/003-first-document/`.

NOTE

The example file adds one line you didn't type: `#set page(width: 12cm, height: auto, margin: 1.5cm) .` That's only there to keep the rendered preview in this book small and tidy. Delete it and you get an ordinary A4 page. Anything starting with `#` is Typst code rather than content; we start pulling that thread properly in Chapter 3.

2.4 The watch loop

Compiling by hand, once, is a fine way to see that Typst works. It is a miserable way to actually write, because you'd be retyping that command after every edit. Typst has a better mode, and building the habit of using it is probably the most valuable thing in this chapter:

```
typst watch main.typ out.pdf
```

`watch` compiles once, then sits there watching the file. Every time you save `main.typ`, it recompiles — automatically, in milliseconds. Your terminal turns into a running log:

```
watching main.typ
writing to out.pdf
[12:41:26] compiling ...
[12:41:26] compiled successfully in 14.28 ms
```

Save again and a new line appears. Make a mistake — a stray bracket, a misspelled function — and instead of silence you get a clear error pointing at the line, and the moment you fix it the next save goes green again. You leave `watch` running in a corner while you work and forget it's there.

The trick that makes this sing is to pair `watch` with a PDF viewer that notices when its file changes on disk and reloads on its own. Then your loop becomes: edit, save, glance at the preview, edit again — no clicking, no re-running anything. Good auto-reloading viewers include **Skim** on macOS, **zathura** on Linux, and **SumatraPDF** on Windows. (The editor extension in the next section gives you a preview pane that reloads too, which for many people replaces the separate viewer entirely.)

TIP

Some PDF viewers — Adobe Acrobat is the usual culprit — hold a lock on a file while it's open, and won't refresh it. If `watch` starts complaining that it can't write the output, or the PDF just never updates, that's almost always the viewer. Switch to one of the reloading viewers above and the problem evaporates.

2.5 Editor setup: Tinymist

You can write Typst in any text editor. But the moment you install the Typst extension for your editor, the experience changes character: you get autocomplete for function names and their arguments, red underlines on errors before you even save, a preview pane docked next to your text, and one-keystroke formatting to tidy your source.

The extension to install is **Tinymist**. In VS Code, open the Extensions panel, search for "Tinymist Typst," and click install — that's the whole setup. It's the standard choice and the one the community has consolidated around.

Tinymist's real cleverness is that it's built on a *language server* — a background program that understands Typst and speaks a protocol many editors know how to talk to. So Tinymist isn't VS Code-only. Neovim, Emacs, Helix, Zed and others can all connect to the same language server and get the same autocomplete and error-checking. Whatever editor you already love, there's a good chance it can speak Typst through Tinymist. If you take one recommendation from this chapter, take this one: install it early, and let it teach you the function names as you go.

2.6 Output formats and handy flags

A PDF is the usual destination, but it isn't the only one. Because Typst infers the format from the output extension, you can aim the same source at different targets just by renaming the output — or by saying so explicitly with `--format`. Using the small document in `examples/004-output-formats/`:

```
typst compile main.typ out.pdf
typst compile main.typ out.png
typst compile main.typ out.svg
```

PDF is your portable, printable default. PNG is a raster image — pixels — handy for pasting a page into a slide or a web post. SVG is a vector image that stays razor-sharp at any size, good for logos, diagrams, or anything you'll scale up.

PNG and SVG come with one wrinkle worth knowing before it surprises you: they hold a single image, so Typst writes **one file per page**. For a one-page document `out.png` is fine. For anything longer, you have to tell Typst where the page number goes, using a `{p}` placeholder in the output name:

```
typst compile report.typ page-{p}.png
```

That produces `page-1.png`, `page-2.png`, and so on. (Leave the `{p}` out of a multi-page export and Typst stops and tells you it's missing — a helpful error, not a mysterious one.) A few more flags earn their keep:

- `--ppi <number>` sets the resolution of PNG export in pixels per inch. The default is 144, which is fine on screen; bump it to `--ppi 300` when you need print-quality pixels.
- `--open` opens the finished file in your default viewer the moment it's done, so `typst compile main.typ out.pdf --open` compiles and previews in one go.
- `--pages <range>` exports only the pages you ask for. `--pages 1` gives you just the first page; `--pages 2-4,6` gives pages 2 through 4 and page 6. Handy when a document is long and you only want to eyeball one part.

2.7 Project structure and `--root`

A "project" in Typst sounds grander than it is. It's a folder with a `.typ` file in it. As your document grows it gathers company — an image or two, maybe a data file, perhaps a second `.typ` file you pull in — but it's still just a folder of plain files, and you can zip it up and email it to a colleague and it will compile on their machine exactly as it did on yours.

That portability depends on how paths work, which is where `--root` comes in. When you refer to a file by an *absolute* path — one that starts with a `/`, like `/images/logo.png` — Typst resolves it against the **project root**, not against your whole hard drive. By default the root is the folder your main file lives in. For safety, Typst refuses to read anything *outside* the root, so a document can't quietly reach into the rest of your filesystem.

Sometimes you want the root to sit higher up — say your document lives in a `chapters/` subfolder but shares an `images/` folder with its siblings. That's what the `--root` flag is for: it points the root at a directory you choose. This book's own examples are compiled from the repository root exactly this way:

```
typst compile --root . examples/005-document-metadata/main.typ out.pdf
```

The `--root .` says "treat the current directory as the project root," so absolute paths inside the document resolve from there.

2.7.1 Setting document metadata

While we're looking at real projects: a PDF carries hidden fields — a title, an author — that don't print on the page but show up in a reader's "Document Properties" panel and in the browser tab when someone opens the file online. You set them with `#set document(...)`, as in `examples/005-document-metadata/`:

```
#set document(
  title: "A Short Field Guide to Puddles",
  author: "R. Waterhouse",
)

= A short field guide to puddles
Some body text goes here.
```

The `title` and `author` never appear in the layout — they're metadata, written into the PDF itself. It's a small, professional touch that costs one line, and `#set` is a rule you'll meet properly in Chapter 9.

TIP

You don't always have to start from a blank file. `typst init` scaffolds a whole project from a template — for instance, `typst init @preview/charged-ieee` pulls a template from Typst Universe, the package registry, and drops a ready-to-edit folder in front of you. We'll dig into packages and templates in Chapters 19 and 20; file the command away for now.

2.8 What you've got

You went from an empty machine to a working Typst setup. Concretely, you can now:

- **Install Typst** on macOS, Windows, or Linux, and confirm it with `typst --version` — and you know that "command not found" just means it isn't on your `PATH`.
- **Choose where to run it** — the browser for a quick try, the command line for real work, an editor with Tinymist for comfort.
- **Compile a document** with `typst compile main.typ out.pdf`, and explain what source, output, and compiling each mean.
- **Live-preview your work** with `typst watch` paired with an auto-reloading viewer — the loop you'll use for everything from here on.
- **Export to PDF, PNG, or SVG**, including multi-page images with a `{p}` template, and reach for `--ppi`, `--open`, and `--pages` when you need them.
- **Organize a project** as a folder of files, steer absolute paths with `--root`, and stamp a PDF with a title and author via `#set document(...)`.

The next chapter finally slows down on the thing you've been typing past — the markup itself — and shows you how much of a document you can build with nothing but plain text and a handful of symbols.

2.9 Exercises

Solutions are in Appendix A.

2.1. Install Typst on your machine using the method for your platform, then run `typst --version`. Write down what it prints. If you get "command not found," diagnose it before moving on — a fresh terminal window fixes most cases.

2.2. Compile `examples/003-first-document/` to a PDF and open it. Then change the heading text, save, and compile again. Confirm the PDF updates. (You've now done by hand the loop that `watch` automates.)

2.3. Run `typst watch main.typ out.pdf` on that same file with an auto-reloading viewer open beside it. Edit a word, save, and watch the preview change without you touching the terminal. Then type a deliberate error — delete the `]` from a piece of code, say — and describe what the terminal shows you.

2.4. Take `examples/004-output-formats/` and export it to PNG twice: once at the default resolution and once with `--ppi 300`. Compare the two files' sizes and how sharp they look when you zoom in. Which would you send to a printer, and which to a chat message?

2.5. (*Stretch.*) Start a project of your own: a folder with a `main.typ` that uses `#set document(title: ..., author: ...)` and a heading of your choosing. Compile it, open the PDF, and find your title and author in the viewer's document-properties panel. Bonus: run the compile with `--open` so it previews itself.

Markup: the content layer

Open a blank file and type a sentence. Compile it. You get that sentence set in a real typeface, with proper kerning and ligatures and its lines broken to fit — a paragraph, and a handsome one, from a line you typed without a thought. You never asked for any of that. Typst assumed it, because prose is the default. A file with nothing special in it is already a document.

Everything else in this chapter is a small, deliberate deviation from that default: a stroke of punctuation that tells Typst "this part is a heading," "this word is emphasized," "these three lines are a list." That punctuation is *markup*, and it is the layer you'll spend most of your writing life in. It is deliberately quiet — a hyphen here, an asterisk there — so that your source still reads like the thing it describes, not like a wall of tags.

If you've written Markdown, a lot of this will feel like coming home, with the furniture moved slightly. Read the "Coming from Markdown" box near the end before you assume anything transfers unchanged; a few of the pieces are in different places, and one or two will trip you if you don't look.

By the end of the chapter you'll be able to hand-write a structured, multi-section document — headings, emphasis, lists, links, the lot — without touching a single styling rule. Styling comes later. First you learn to say *what things are*.

3.1 Paragraphs and whitespace

A paragraph is just text with a blank line above and below it. That's the whole rule. To start a new paragraph, leave a blank line; to keep writing the same one, don't.

Inside a paragraph, Typst treats whitespace generously — which is to say, it mostly ignores how much of it you typed. Runs of spaces collapse to one. A line break in your source is just another space. So this:

```
This sentence is broken across
several lines with odd
spacing in the source.
```

comes out as one tidy line of prose, respaced and rewrapped to fit the page. Your source can be as ragged as you like; the output won't care. This is a feature, not a quirk: you get to wrap your source at a comfortable width (this book wraps at 80 columns) without those wraps leaking into the page.

Now and then you *do* want a line to break exactly where you say — an address, a line of a poem, the closing of a letter. For that, end the line with a backslash:

```
Rue de la Paix 12\
1000 Brussels\
Belgium
```

The trailing `\` forces a hard line break *within* the same paragraph. It's the one case where your source's line ending survives into the output. See `examples/006-headings-and-paragraphs/` for headings and both kinds of break in one small file.

NOTE

A blank line and a backslash are not the same thing. A blank line starts a new paragraph, which carries paragraph spacing above it. A backslash breaks the line but stays inside the paragraph, tight against the line before. Use the one that matches what you mean.

3.2 Headings

A heading is an equals sign, a space, and the heading text:

```
= Chapter title
== A section
=== A subsection
```

One `=` is a top-level heading; each extra `=` drops you a level, down to six (`=====`) if you ever need that far, which you almost never will. The space after the equals signs matters — `=heading` with no space is just literal text.

That's the whole syntax. Notice what's missing: no numbers. Typst does not print "1.2.3" in front of your headings unless you ask it to, and asking is a one-line *set rule* — `#set heading(numbering: "1.")` — which we hold until Chapter 9, where set rules get the introduction they deserve. For now, write your headings as plain structure and let the numbering wait.

Coming from Markdown. Yes, it's `=`, not `#`. In Typst the `#` is reserved for something bigger (dropping into code — hold that thought), so headings took the next most obvious symbol. More equals signs means a *deeper* heading, the same way more `#` means deeper in Markdown.

3.3 Emphasis: italic and bold

Wrap a word or phrase in underscores to *emphasize* it, and in asterisks to make it *strong*:

```
Wrap a word in underscores for _emphasis_, which renders italic,
and in asterisks for *strong*, which renders bold.
```

They nest, so you can have bold with a slice of italic inside it:

```
*Read this now, it is _actually_ important.*
```

Under the hood, `_...` is shorthand for a function called `emph` and `*...*` for one called `strong`. You can call them the long way — `#emph[like this]` and `#strong[like this]` — and you'll occasionally want to when you're generating text from code rather than typing it by hand. That `#` is the doorway into Typst's programming language; Chapter 13 walks through it properly. For plain writing, the underscores and asterisks are all you need. Example [examples/007-emphasis-and-raw/](#) shows emphasis, nesting, and raw text together.

Coming from Markdown. Both delimiters are *single* here. Markdown makes you double the asterisks for bold (`**bold**`) and reserves single ones for italic; Typst uses `*` for strong and `_` for emphasis, one character each. Fewer keystrokes, one fewer thing to miscount.

3.4 Raw text: code and verbatim

Sometimes you need Typst to keep its hands off — to print exactly what you typed, markup characters and all. That's *raw text*, and you get it with backticks.

For a snippet mid-sentence, wrap it in single backticks: run ``typst watch main.typ`` to rebuild on every save. Whatever sits between the backticks is verbatim, so ``*this stays literal*`` prints its asterisks instead of turning bold. Handy, given how much of this chapter is about characters that normally *do* something.

For a whole block — a code listing, a config file, a terminal session — fence it with three backticks, and optionally name the language right after the opening fence:

```
```rust
fn main() {
 println!("Hello, Typst!");
}
```
```

Everything inside is preserved character for character: indentation, blank lines, the works. The language tag (`rust` here) switches on syntax highlighting, so keywords, strings, and the rest come out colored. Leave the tag off and you get the same verbatim block without the colors. Typst knows a long list of languages; if it doesn't recognize the tag, it simply skips the highlighting rather than complaining.

3.5 Lists

Three kinds of list cover almost everything you'll write, and all three are one character at the start of a line.

A hyphen makes a bullet:

- Bread
- Cheese
- A quiet afternoon

A plus sign makes a numbered list, and Typst does the counting — you never type the numbers, so reordering items never means renumbering by hand:

- + Boil the water.
- + Add the pasta.
- + Wait, impatiently.

Nest either kind by indenting, and mix them freely — bullets under numbers, numbers under bullets, however the content wants to be shaped:

- Fruit
 - + Apples
 - + Pears
- Vegetables
 - Leafy
 - Root

Indentation is what creates the nesting, so keep it consistent within a list. `examples/008-lists/` collects all of these, plus the third kind.

That third kind is the *term list*, for pairing a name with its definition — a glossary, a cast of characters, a set of options:

- / **Markup:** the content layer you type by hand.
- / **Set rule:** a way to change a default, met in Chapter 9.

A term list item is a forward slash, the term, a colon, and the description. Typst sets the term apart (bold, by default) and lays the description beside it.

WARNING

A block sitting flush against a list — a fenced code block, say — ends the list, and a following `+` item starts counting again from one. If you want a code block to belong *inside* an item, indent it to line up under that item's text. The meeting note in `examples/011-a-structured-note/` does exactly this to keep a shell block tucked inside step two.

3.6 Links

Type a bare URL and Typst turns it into a working link automatically:

The docs live at `https://typst.app/docs` — go read them.

A naked URL is honest but ugly. To hang a link on some friendlier words, use the `link` function, which takes the address in quotes and the visible text in square brackets:

Read `#link("https://typst.app/docs")`[the official docs] when you get stuck.

That `#link(...)[...]` is your first real look at Typst's function-call shape: a `#`, a name, the arguments in parentheses, and content in square brackets. You'll see that shape everywhere once you start styling, so it's worth clocking now even though functions proper wait for Chapter 13. Links and quotes share `examples/009-links-and-quotes/`.

Links don't only point outward. You can also link *within* your document — jump to a heading, a figure, an equation — using labels, which is the next stop.

3.7 Smart quotes and punctuation

Here's where Typst quietly earns its keep. You type plain, unfussy characters and get proper typography without thinking about it.

Type straight quotes and Typst curls them for you, and it does so *language-awarely* — the right quotation marks for the document's language, not a one-size guess:

"Double quotes" and 'single quotes' and don't-panic apostrophes all come out as the correct curly marks.

Dashes work by counting hyphens. One hyphen is a hyphen. Two (--) make an *en dash*, the one for ranges: pages 12--18. Three (---) make an *em dash*, the long one for a break in thought---like this. And three dots (...) become a single, properly spaced ellipsis, not three cramped full stops.

One more, easy to miss and genuinely useful: a tilde (~) is a *non-breaking space*. It looks like an ordinary space but Typst won't break the line there, so you can keep a number glued to its unit — 10~kg, Chapter~9, Fig.~3 — and never suffer a "10" alone at the end of a line with its "kg" orphaned on the next. `examples/010-symbols-and-escapes/` demonstrates every one of these.

3.8 Escapes and comments

If markup characters do things, you'll eventually need to print one *without* it doing its thing — a literal asterisk, a real hash, an underscore in a filename. Put a backslash in front of it:

```
\*not strong\*, a real \# hash, and a file called my\_notes.txt
```

The backslash escapes exactly the next character. It's the same backslash that forces a line break at the end of a line; in the middle of one, it defuses whatever follows.

The backslash also reaches any character at all by its Unicode codepoint, with `\u{...}` and the hexadecimal number:

```
A check mark \u{2713}, an arrow \u{2192}, a heart \u{2764}.
```

Finally, notes to yourself that never reach the page. Two slashes start a comment that runs to the end of the line; `/*` and `*/` bracket a comment that can span several:

```
This prints. // this does not
This also prints. /* and none
of this multi-line aside does */
```

Comments are for scaffolding — a reminder, a disabled paragraph, a note to a co-author. They vanish at compile time and leave no trace in the PDF.

3.9 A taste of labels and references

You can pin a *label* on almost anything by writing it in angle brackets right after the thing, and then point at it from elsewhere with an `@`:

```
#set heading(numbering: "1.")
```

```
= Introduction <intro>
```

```
As we saw in @intro, markup comes first.
```

Typst fills in `@intro` with a live cross-reference — a number and a link that stay correct even as you shuffle the document around. This is one of the payoffs of describing structure instead of formatting: the machine knows what "the introduction" is, so it can always find it.

That's the whole idea in one bite. One catch worth naming now: pointing at a *heading* only works once heading numbering is switched on, which is why that `#set heading(numbering: "1.")` line sits at the top of the example — without it, Typst stops and tells you it *cannot reference heading without numbering*. Labels on other things don't need it. References can also carry their own wording, and Chapter 11 gives labels and cross-references the full treatment; file the syntax away for now.

3.10 Two symbols you'll meet soon

While we're naming special characters, two more are worth flagging so they don't surprise you later.

A dollar sign opens *math*: `$a^2 + b^2$` typesets an equation inline, and Typst's math is good enough to get its own chapter (Chapter 8). And the `#` you keep seeing drops out of markup and into *code* — the little programming language under the surface, where `#link` and `#emph` and `#set` all come from. Chapter 13 opens that door deliberately. For now, just recognize the two symbols: `$` means math, `#` means code.

Coming from Markdown. The quick translation table, because the differences bite:

- **Emphasis is single, both ways.** `*strong*` and `_emph_` — no doubling.
- **Headings are =, not #.** More equals, deeper heading.
- **# means code, not a heading.** It's the single biggest gotcha coming from Markdown: a stray `#` isn't a title, it's an attempt to run something.
- **Links are backwards.** Markdown's `[text](url)` becomes `#link("url")[text]` — address first, visible text in the brackets.
- **Raw text is the same.** Backticks and triple-backtick fences work just as you'd expect, language tag and all.

3.11 What you've got

You can now write the entire content layer of a document by hand:

- **Paragraphs** from blank lines, with a trailing `\` for a hard line break, and the freedom to wrap your source however you like.
- **Headings** with `=` through `=====`, structure without numbering (that's a set rule, Chapter 9).
- **Emphasis** (`_..._`) and **strong** (`*...*`), nesting freely, with `#emph` / `#strong` waiting in code mode.
- **Raw text** — inline backticks and fenced blocks with syntax highlighting — for anything that must stay verbatim.
- **Lists:** bulleted (`-`), numbered (`+`), nested, mixed, and term lists (`/ Term: ...`).
- **Links**, bare or labeled with `#link`, and a first taste of `<label> / @reference` cross-references (Chapter 11).
- **Typographic niceties** for free: smart quotes, en/em dashes, the ellipsis, and the `~` non-breaking space.
- **Escapes** (`\*`, `\#`, `\u{...}`) and **comments** (`///`, `/* ... */`).

That is enough to write a clean, well-structured multi-section document — something you'd actually be happy to hand in or send out. It won't be styled to your taste yet; that starts in Chapter 4 with text and fonts, and turns into real power in Chapter 9 with set and show rules. But the words, and the shape of the words, are entirely in your hands now.

3.12 Exercises

Solutions are in Appendix A.

3.1. Write a short document with a title, two sections (each a `==` heading), and a paragraph or two of your own text under each. Compile it and confirm the headings stand out from the body.

3.2. Take a paragraph and, without adding or removing any words, make one phrase *emphasized*, one *strong*, and one a bit of `inline raw code`. Then add a second paragraph containing a bulleted list of three items and turn one of those bullets into a two-item nested list.

3.3. Typeset this line so it comes out exactly right: a range written as an en dash ("pages 30–34"), a break in thought as an em dash, a quantity whose number and unit never split across a line ("5 km"), and a real, literal asterisk in the sentence. Check the compiled PDF to be sure each one landed.

3.4. Build a small "reading list" as a term list: three books, each term the title and each description a one-line note. Then add, below it, a labeled link to where someone could buy or borrow one of them.

3.5. (*Stretch.*) Rebuild `examples/011-a-structured-note/` from scratch without looking at it — a meeting note or a recipe with a title, a couple of sections, at least one numbered list and one bulleted list, an emphasized word, a link, and a fenced code (or ingredient) block nested inside a list item. Compile it and fix anything that doesn't render the way you meant, paying special attention to whether your nested block breaks the numbering.

Text and fonts

Everything you have typeset so far arrived in the same clothes: Libertinus Serif, eleven points, black on white, set ragged along the right edge. You never chose any of it. Typst did, the moment you started typing, because a document has to look like *something*, and those are sensible defaults.

This chapter is about changing them — every visible property of type, from the whole typeface down to whether the digit 7 sits on the baseline or dips below it. It's the first chapter where you stop describing *what things are* and start deciding *how they look*.

4.1 Styling text: the `text` function

One function controls the appearance of type, and it is called `text`. You have been using it all along without naming it; it's the thing Typst wraps around every character on the page. Call it explicitly to restyle a stretch of words:

```
The word #text(fill: red)[danger] stands out.
```

`#text(...)` takes named arguments — the properties you want to change — and a chunk of content in square brackets. Everything in the brackets gets those properties; everything outside keeps the defaults.

That's fine for one word. For a whole document you don't want to wrap every paragraph by hand. Instead you set the property once, as a *set rule*:

```
#set text(font: "New Computer Modern", size: 12pt)
```

A set rule changes a default for everything after it. Put that line at the top of your file and the entire document switches to twelve-point New Computer Modern; put it halfway down and only the second half changes. Set rules are one of the two big ideas that turn Typst from a markup language into a styling engine, and Chapter 9 gives them the full treatment. For now the shape is all you need: `#set text(...)` means "from here on," `#text(...)[...]` means "just this."

Almost everything in the rest of this chapter is an argument to that one function. Learn the arguments and you have learned text styling.

4.2 Choosing a typeface

The `font` argument takes the name of a font family, as a string:

```
#set text(font: "Libertinus Serif")
```

To find out which names are legal on your machine, ask Typst:

```
typst fonts
```

It prints every font family it can see — the ones bundled with Typst plus every one installed on your system. That list is where the trouble starts, so let's talk about reproducibility before you fall for a font you'll later regret.

4.2.1 Bundled fonts, and why to prefer them

Typst ships with a handful of fonts baked into the program itself. On any machine, in any browser, with nothing installed, these are always present and always render identically:

- **Libertinus Serif** — the default, a warm and readable book serif.
- **New Computer Modern** — the classic look of LaTeX, for when you want a document to *read* as academic.
- **New Computer Modern Math** — its partner for equations (Chapter 8).
- **DejaVu Sans Mono** — a monospace face for code and anything that needs fixed-width columns.

Notice what's missing: a general-purpose sans-serif. Typst bundles no Helvetica-alike. If you want clean sans type, you're reaching for a system font — and that's exactly where "it looked fine on my laptop" goes to die.

When you name a font Typst can't find, it doesn't stop. It prints a warning —

```
warning: unknown font family: helvetica
```

— falls back to a font it *does* have, and carries on. Your document still compiles, but it no longer looks the way you wrote it, and on a colleague's machine it may look different again. Every example in this book sticks to the bundled fonts for that reason: they render the same everywhere and they never warn. If you're writing something that has to travel — a template for two hundred students, a paper for a co-author — do the same. If you're the only person who will ever compile the file and you have Garamond installed, name Garamond and enjoy it.

Coming from LaTeX. Remember the ritual: `\usepackage{fontspec}`, switching to XeLaTeX or LuaLaTeX, `\setmainfont{...}`, and a separate spell for the math font. In Typst, choosing a font is one line — `#set text(font: "...")` — with no package, no engine flag, and no preamble. The font is just another property.

4.2.2 Fallback lists and missing glyphs

You can hand `font` a *list* of families instead of one, in parentheses:

```
#set text(font: ("Libertinus Serif", "DejaVu Sans Mono"))
```

Typst reads that as an order of preference. For each glyph it tries the first family; if that family has no glyph for the character — an obscure symbol, say, or a script the font never covered — it moves to the next, and the next, until something can draw it. This is how a mostly-Latin document can hold a stray mathematical arrow or a single Chinese character without you lifting a finger: the first font handles the ordinary letters, and a later font quietly supplies the rest. Behind your list Typst keeps its own last-resort fonts, so a glyph is almost never simply lost.

`examples/012-fonts-and-families/` sets one sentence in each of three bundled faces so you can compare them side by side, and shows the list syntax at work.

4.3 Size, weight, and style

Three arguments cover most of what you'll adjust from one day to the next.

Size is `size`, in points or in ems:

```
#text(size: 8pt)[fine print] and #text(size: 18pt)[a big word]
```

A point (`pt`) is an absolute unit — 72 to the inch, the currency of type since the days of metal. An **em** is relative: `1em` is the current font size, so `1.5em` is half again as large as whatever surrounds it. Ems are the better choice inside anything reusable, because they scale with the context instead of nailing themselves to one fixed size.

Weight is `weight`, given as a number from 100 to 900 in steps of 100, or as a name:

```
#text(weight: "bold")[heavy] and #text(weight: 300)[light]
```

400 is "regular" and 700 is "bold"; the names ("regular", "medium", "bold", and a few more) are just aliases for the round numbers. One catch: a font can only give you the weights it actually ships. Libertinus Serif has a regular and a bold and little in between, so asking for 300 gets you the nearest weight it owns, not a true light cut. A font with nine real weights will honour all nine.

Style is `style`, one of "normal", "italic", or "oblique". Italic is the separate cursive design a type designer drew; oblique is a mechanical slant of the upright. Most serif fonts give you a real italic, so asking for oblique usually lands you on the italic anyway.

`examples/013-size-weight-style/` lays all three axes out in a row. And recall from Chapter 3 that `*strong*` and `_emph_` are markup shortcuts for bold and italic — you don't need the long form for ordinary writing.

4.4 Color

Text color is the `fill` argument — "fill" because you're filling the letter shapes with a color, the same word Typst uses for shapes and backgrounds. Four ways of naming a color come up constantly:

```
#text(fill: rgb("#1a7f37"))[a hex green]
#text(fill: red)[a named color]
#text(fill: luma(40%))[a gray]
#text(fill: cmyk(75%, 0%, 70%, 5%))[a print green]
```

- `rgb("#rrggbb")` takes a hex string — the same one you'd paste out of a color picker or a website's style guide.
- Named colors (`red`, `blue`, `green`, `orange`, `purple`, `navy`, and a couple dozen more) are there for quick work and rough drafts.
- `luma(...)` is grayscale in a single number: `luma(0%)` is black, `luma(100%)` is white, and everything between is a gray. Handy for setting text a notch softer than dead black.
- `cmyk(...)` names a color by its cyan, magenta, yellow, and black percentages, the way a printer thinks. If your document is going to a professional press, a CMYK fill lands more predictably than RGB; for anything headed to a screen or an ordinary PDF, `rgb` is the friendlier choice.

4.5 Decorations

A small cluster of functions draw lines on, through, or behind text, or reshape it outright. Each takes its content in brackets:

```
#underline[underlined] #overline[overlined] #strike[struck out]
#highlight[highlighted] #smallcaps[Small Capitals]
```

- `#underline`, `#overline`, and `#strike` draw a line under, over, and through their content.
- `#highlight` lays a colored band behind the text, like a marker pen; it defaults to a soft yellow.
- `#smallcaps` sets lowercase letters as small capitals — capital letterforms at roughly the height of a lowercase x — the classic way to typeset an acronym or a name without shouting.

Each of these is a real function with its own options. `#highlight(fill: rgb("#fff3b0"))[...]` recolors the band, and `#underline(stroke: 1.5pt + red, offset: 3pt)[...]` thickens the rule and drops it lower. You rarely need the options, but they're there when a house style demands them. [examples/014-color-and-decorations/](#) collects the lot.

Coming from Word. The highlighter button and the font-color dropdown you know from the toolbar are `#highlight[...]` and `#text(fill: ...)[...]` here. The difference is that in Typst you define the color once, in a set rule, and every highlight across a hundred pages matches it — no hunting for the exact same yellow in a swatch grid each time.

4.6 Figures, ligatures, and other OpenType niceties

Good fonts carry more than one shape for some characters, and Typst lets you choose between them. The most useful choices are about numbers.

4.6.1 Lining vs old-style figures

By default, digits are **lining figures**: all one height, sitting on the baseline like capitals — 0123456789. That's what you want in a table or a heading.

Set `number-type: "old-style"` and you get **old-style figures**, which rise and fall like lowercase letters, some of them dipping a little descender below the baseline. They look at home in running prose, where a lining figure sticks out like a tourist:

```
Founded in #text(number-type: "old-style")[1789].
```

4.6.2 Tabular vs proportional figures

The other choice is width, set with `number-width`. **Proportional** figures are individually spaced — a 1 is narrower than an 8 — which reads naturally in a sentence. **Tabular** figures are all forced to one width, so numbers stacked in a column line up digit for digit, ones under ones, without a ragged edge. You want tabular in tables and anywhere numbers must align; proportional everywhere else.

```
#text(number-width: "tabular")[1,111]
```

[examples/015-figures-and-numbers/](#) shows both figure styles and a small table where the tabular column lines up cleanly and, for contrast, a proportional one that doesn't.

4.6.3 Ligatures and features

When two letters would collide — the f and the i, whose hook and dot fight for the same sliver of space — a good font supplies a single combined glyph called a **ligature**, and Typst uses them automatically. Usually that's exactly what you

want. Now and then, in a technical context, you'd rather see the raw letters; `#text(ligatures: false)[...]` turns them off.

Beyond these named options, a font may carry any number of **OpenType features** — alternate letterforms, swashes, whole stylistic sets — reached through the `features` argument by their four-letter tags, as in `features: ("ss01",)`. It's a specialist's door; most people never open it, and the fonts bundled with Typst keep it modest. Just know that `features` exists, and read the font's own documentation on the rare day you need it.

4.6.4 Letter tracking

To add or remove space *between* letters — for a title, an acronym, or a run of small caps that reads too tight — set `tracking`, in ems:

```
#text(tracking: 0.15em)[S P A C E D]
```

Positive values open the text up; negative values pull it in. A touch of tracking on small caps or all-caps text is a classic typographic move; a lot of it on body text is a classic mistake.

NOTE

`tracking` is space between *letters*. Space between *words*, between *lines*, and between *paragraphs* is a matter of layout, not text, and lives on `par` and `page` — `#set par(...)` and friends — which is Chapter 5's business.

4.7 Language and hyphenation

Two arguments tell Typst what language your text is written in:

```
#set text(lang: "en", region: "US")
```

`lang` is a two-letter language code; `region` is an optional two-letter country code. Together they set more than metadata. They decide which **hyphenation patterns** Typst uses to break long words at the margin — English words break in different places than German ones — and which style of **quotation mark** your straight `"..."` curls into. Set `lang: "de"` and the same typed quotes come out as German low-and-high marks; set `lang: "fr"` and you get guillemets. You met smart quotes back in Chapter 3; this is the knob that decides *which* smart quotes.

Hyphenation itself is `hyphenate`. Left at its default of `auto`, Typst hyphenates only when a paragraph is justified — which is the sensible time to, since justification opens the ugly gaps that hyphenation exists to close. Force it on with `hyphenate: true` or off with `hyphenate: false`:

```
#set text(hyphenate: false)
```

`examples/017-language-and-hyphenation/` sets a deliberately narrow, justified column both ways so you can watch hyphenation earn its keep: with it on, word spacing stays even; with it off, the same lines have to stretch to fill the width, and the gaps yawn.

TIP

Always set `lang` on a document, even an English one. It's the difference between correct hyphenation and none, and between real quotation marks and a guess. It costs one line and it's the most under-used setting in Typst.

4.8 What you've got

You can now reach every visible property of type:

- **The `text` function** — `#text(...)[span]` for a stretch, `#set text(...)` for everything after — as the single home of text styling.
- **Typefaces** with `font`, fallback lists with `font: ("A", "B")`, and the discipline of sticking to Typst's **bundled fonts** (Libertinus Serif, New Computer Modern, DejaVu Sans Mono) so documents render the same everywhere and never warn.
- **Size** in `pt` and `em`, **weight** from 100–900 or by name, and **style** — `normal`, `italic`, `oblique`.
- **Color** through `fill: rgb`, named colors, `luma` grays, and `cmk` for print.
- **Decorations**: `underline`, `overline`, `strike`, `highlight`, `small caps`, and their options.
- **Figures**: `lining` vs `old-style`, `tabular` vs `proportional`, plus `ligatures`, a glance at OpenType `features`, and `letter tracking`.
- **Language** with `lang / region` driving quotes and hyphenation, and `hyphenate` for the final say.

Between this chapter and the markup of Chapter 3, you can write a document and dress it exactly as you like. What you can't do yet is style the *page* it sits on — its size, its margins, its columns, and the spacing between lines and paragraphs. That's Chapter 5.

4.9 Exercises

Solutions are in Appendix A.

4.1. Set a whole document to twelve-point New Computer Modern with a single set rule, then make a single word eighteen-point and bold using `#text`. Compile it and confirm that only that one word changed.

4.2. Run `typst fonts` and read the list. Pick a family that is *not* one of Typst's bundled fonts, set your document to it, and compile. Note the warning Typst prints and what the text falls back to. Then switch to a bundled font and confirm the warning disappears. (The point is to feel the reproducibility trap once, on purpose, so you recognize it later.)

4.3. Typeset a line of chemistry and a line of physics: water and carbon dioxide with subscripts, and $E = mc^2$ with a superscript. Then set a country's name in small capitals.

4.4. Make two stacked lines (or a two-column table) of the numbers 1,024, 512, and 64. Give one version tabular figures and the other proportional, right-align both, and write one sentence on what's different about how the digits line up.

4.5. (*Stretch.*) Drop `#lorem(60)` into a narrow justified column, about seven centimetres wide. Set its language to French and compile, noting where French hyphenation and quotation marks differ from English. Then turn hyphenation off and compare the word spacing. Which do you prefer for that column, and why?

Pages and layout

So far you've been pouring words onto whatever page Typst handed you. It's a good page — A4, sensible margins, a clean single column — but it was chosen for you, and sooner or later you'll want a different one. A letter wants US paper. A conference handout wants two columns. A thesis wants a running header with the chapter title and a page number down in the corner. A draft wants the word DRAFT ghosted across it so nobody circulates it by accident.

All of that is the *page*, and in Typst the page is a thing you configure with a single function. No package to load, no geometry to import, no preamble. You set it once and every page obeys, or you set it again halfway through and the rest of the document changes its mind. This chapter is about that function — and about the smaller layout controls that shape the paragraphs sitting inside it.

5.1 The page is one function

Everything about the sheet itself lives in `#set page(...)`. Put it near the top of your file and it governs the whole document:

```
#set page(paper: "us-letter", margin: 2.5cm)
```

The `paper` argument takes a name — "a4" (the default), "a3", "a5", "us-letter", "us-legal", and a long list of others, right down to business cards and ISO envelopes. You'll almost never need the exotic ones, but they're there.

When no named size fits, give the dimensions directly:

```
#set page(width: 16cm, height: 24cm)
```

That's a small book block, the kind of trim size a novel might use. You can mix the two — name a paper and override just the height — but usually it's one or the other.

NOTE

A set rule takes effect from where you write it onward. Put `#set page` at the top and it shapes the entire document, including page one. Put a second one in the middle and every page after it switches — handy for a landscape spread in the middle of a portrait report.

5.1.1 Margins

Margins are where Typst quietly saves you an afternoon. In many systems the margin is four separate settings you have to keep in sync. Here it's one argument that accepts however much detail you care to give it.

A single length sets all four sides at once:

```
#set page(margin: 2cm)
```

A dictionary sets them in pairs — `x` for left and right, `y` for top and bottom:

```
#set page(margin: (x: 2cm, y: 3cm))
```

Or name each side on its own when you need real asymmetry — a wide inner margin for binding, say:

```
#set page(margin: (top: 2cm, bottom: 3cm, left: 3cm, right: 2cm))
```

You can even set some sides in the dictionary and let the rest default. See [examples/018-paper-and-margins/](#) for the paired form on a small page, where the generous top and bottom are obvious at a glance.

5.1.2 Landscape

To turn the page on its side, don't swap width and height by hand and hope you got it right. Set `flipped`:

```
#set page(paper: "a4", flipped: true)
```

Typst exchanges the two dimensions for you, so an A4 page becomes 297mm wide by 210mm tall. It works with named papers and with explicit `width / height` alike.

Coming from LaTeX. There is no `geometry` package, because there's nothing to import. Where you'd write `\usepackage[a4paper,margin=2.5cm]{geometry}` and hope the options don't fight your document class, Typst gives you one `#set page(paper: "a4", margin: 2.5cm)` and no document class to fight. Margins are an argument, not a package.

5.2 Headers and footers

A header and a footer are just content that Typst reprints at the top and bottom of every page. You hand them over as arguments:

```
#set page(
  header: [_Field notes_],
  footer: [Draft – do not circulate],
)
```

Whatever you put in the brackets is ordinary markup: emphasis, a date, a small image, anything. The interesting part is placing pieces of it left, center, and right, and for that Typst gives you a lovely little tool: `#h(1fr)`, a *stretchy* horizontal space that swells to eat all the room available.

One `1fr` between two things shoves them to opposite edges:

```
header: [_Field notes_ #h(1fr) Chapter 5]
```

The title sits left, the chapter sits right, and the stretch fills the gap. Wrap a piece in two of them and it lands dead center:

```
footer: [#h(1fr) middle #h(1fr)]
```

You'll use `1fr` constantly once you meet it. It's the same idea as a spring: give it room and it expands to fill. `examples/019-header-and-footer/` runs a title-and-chapter header over a centered footer across two pages.

5.3 Page numbers

You could type a page number into the footer by hand, but you'd be wrong on every page but one. Typst counts for you. Set `numbering` on the page:

```
#set page(numbering: "1")
```

and Typst prints the number in the footer automatically — you don't supply a footer at all. The string is a *pattern*, and the digit you put in it chooses the style:

- `"1"` — plain digits: 1, 2, 3.
- `"i"` — lowercase roman: i, ii, iii. (`"I"` gives uppercase.)
- `"a"` — letters: a, b, c.
- `"1 / 1"` — the current page and the total, e.g. "3 / 12". The surrounding text (`/` here) is printed verbatim; only the counter symbols advance.

`number-align` decides where the number sits — `center` by default, or `left`, `right`, and you can add a vertical part like `right + bottom`. `examples/020-page-numbering/` sets `"1 / 1"` across a three-page document, so the footer counts "1 / 3", "2 / 3", "3 / 3".

WARNING

The automatic page number appears only while Typst is drawing the footer *for you*. The moment you supply your own `footer:`, that automatic number disappears — your footer replaces it wholesale. If you want the number inside a custom footer, you have to place it there yourself, which is the next thing.

5.3.1 The number, by hand

Behind the automatic number is a *counter* named `page`, and you can print it anywhere with `counter(page).display()`. That's how you fold a page number into a footer you've designed yourself:

```
footer: context [#h(1fr) #counter(page).display() #h(1fr)]
```

Two things are new there. `counter(page)` is the running tally of pages; `.display()` renders it, optionally taking a pattern string just like `numbering` (so `.display("i")` gives roman). And the `context` keyword in front is Typst's way of saying "this value depends on *where* we are in the document" — the page number is different on every page, so Typst can only work it out once it knows which page it's laying out. You met `#context` in passing before; it gets its proper introduction, along with counters and state, in Chapter 17. For now, treat `context [ ... #counter(page).display() ... ]` as the incantation that puts the live page number into content you control, and move on.

The short version: set `numbering` when you just want a page number in the usual place, reach for `counter(page).display()` inside `context` when you're building a header or footer by hand and want the number to be one ingredient among several.

5.4 Columns

Two columns, newspaper style, is one word:

```
#set page(columns: 2)
```

Text now flows down the left column and continues at the top of the right, spilling onto the next page when both are full. Everything on the page — body, headings, the lot — obeys the column layout. `examples/021-two-columns/` fills two columns with `#lorem` so you can see the flow and the gap down the middle.

That gap is the *gutter*, and it comes from the same `columns` machinery. For a whole-page layout the default gutter is fine; when you want to set it, or when you only want *part* of a page in columns, use the `columns` function instead:

```
#columns(2, gutter: 1cm)[
  #lorem(60)
]
```

This makes a multi-column block right where you put it, without disturbing the single-column text above and below. It's the tool for an abstract that spans the full width followed by a two-column body, or for one boxed aside set in narrow measure. A `#colbreak()` inside the block forces the flow to jump to the next column early, the way `#pagebreak()` jumps to the next page.

TIP

Full-page `#set page(columns: 2)` and a local `#columns(2)[...]` block are two tools for two jobs. Reach for the page setting when the *whole* document is multi-column; reach for the block when only a stretch of it is, and you want the surrounding text left alone.

5.5 The shape of a paragraph

The page is the container; the paragraph is what fills it, and `#set par(...)` tunes how. A handful of knobs cover most of what you'll ever want.

`justify` stretches each line to reach both margins, giving you the clean block-of-text edges of a printed book:

```
#set par(justify: true)
```

By default paragraphs are ragged-right, which reads well on screen and in narrow measure; justification looks best once lines are long enough not to gap awkwardly.

`leading` is the space *between lines within* a paragraph — what typesetters call leading (it rhymes with "heading", after the strips of lead they once slid between lines of metal type). Tighten or loosen it in em units:

```
#set par(leading: 0.8em)
```

`spacing` is the different, larger space *between* one paragraph and the next. The two are easy to confuse; `leading` is inside a paragraph, `spacing` is the gap to the following one.

`first-line-indent` indents the opening line of each paragraph, the traditional book alternative to a blank line between them:

```
#set par(first-line-indent: (amount: 1.2em, all: true))
```

Written as a plain length — `first-line-indent: 1.2em` — Typst indents every paragraph *except* the first one after a heading, which is the usual typographic convention (you don't indent a paragraph that nothing precedes). The dictionary form with `all: true` overrides that and indents every paragraph, which some house styles prefer. `examples/022-paragraph-settings/` combines justification, tighter leading, and a first-line indent into the familiar book-page look.

One more, occasionally useful: `linebreaks` chooses how Typst decides where lines wrap. The default, `auto`, runs an optimizing algorithm when text is justified (weighing the whole paragraph to avoid rivers and cramped lines) and a simple greedy pass otherwise. You can force `"optimized"` or `"simple"` yourself, but the default is usually the right call.

Coming from Word. The line-spacing dropdown, the "space before/after paragraph" boxes, the ruler with its indent markers — all of it collapses into `#set par`. And you set it once, in one place, instead of selecting text and clicking. Change `leading` at the top of the file and every paragraph in a three-hundred-page document reflows to match. No "apply to all," no Format Painter.

5.6 Spacing and breaks by hand

Most spacing should come from your rules, so it stays consistent. But now and then you want to reach in and nudge one spot, and Typst has blunt instruments for exactly that.

`#v(length)` inserts vertical space between blocks — a centimeter of air above a signature line, a bit of breathing room before a section:

```
Yours sincerely,  
#v(2cm)  
Lars De Richter
```

`#h(length)` does the same horizontally, inline. You've already met its stretchy cousin `#h(1fr)`; a fixed `#h(1cm)` just inserts that much space and stops.

To force a new page, `#pagebreak()`:

```
#pagebreak()  
= A chapter that must start fresh
```

That one always breaks. Its gentler variant, `#pagebreak(weak: true)`, breaks *only if the current page already has content* — if a page break just happened, a weak one does nothing rather than leaving a blank page behind. This matters when a break lives inside a rule that might fire right after another break; the weak version keeps you from stacking two and emptying a page. `#v` also has a `weak: true` form, for the same reason: weak space at the very top of a page collapses instead of pushing your text down. `examples/023-spacing-and-breaks/` shows `#v`, `#h(1fr)`, and both kinds of break side by side.

TIP

If you find yourself sprinkling `#v` and `#pagebreak` everywhere to fix spacing, stop and ask whether a set or show rule should be doing it instead. Manual nudges are for one-off exceptions. When *every* section needs space above it, that's a rule's job (Chapter 9), not a hundred hand-placed `#v` s.

5.7 Fills and watermarks

The page has a background you can paint. `fill` floods the whole sheet with a color:

```
#set page(fill: rgb("#faf6ec"))
```

That's a faint warm paper tint, the sort of thing that makes a cover feel less clinical. Any color works; the default is white (well, no fill, which prints as white).

For something behind the text but not the whole sheet — a watermark, a faint logo, a "SPECIMEN" stamp — use `background`. It takes content and draws it behind everything else, sized to the full page:

```
#set page(
  background: align(center + horizon,
    rotate(-30deg, text(52pt, fill: luma(80%))[*DRAFT*])),
)
```

Reading that from the inside out: some big, light-gray text, rotated thirty degrees, centered vertically and horizontally in the page box. Because it's the *background*, your actual document sits on top of it and stays readable. *foreground* is the mirror image — content drawn *over* the text — for the rarer case where you want the stamp to sit on top. [examples/024-page-background/](#) combines a paper tint with a ghosted DRAFT watermark.

NOTE

A watermark built this way repeats on every page, because it's part of the page setup. That's usually what you want for a draft stamp. For a mark on one page only, scope it — but scoping page settings to a range is a Chapter 21 topic, once you've met the tools for it.

5.8 What you've got

You can now shape the page itself, not just fill it:

- **Page size** with `paper:` (named) or explicit `width: / height:`, and **landscape** with `flipped: true`.
- **Margins** as one argument: a single length, an `(x:, y:)` pair, or a full per-side dictionary.
- **Headers and footers** as content, with `#h(1fr)` to align pieces left, center, and right.
- **Page numbers** via `numbering:` (the `"1"`, `"i"`, `"1 / 1"` patterns) and `number-align`, plus `counter(page).display()` inside `context` when you build a footer by hand.
- **Columns** for the whole page (`columns: 2`) or a local block (`#columns(2)[...]`), with a `#colbreak()` to jump early.
- **Paragraph shape** with `#set par: justify`, `leading`, `spacing`, `first-line-indent`, and `linebreaks`.
- **Manual spacing and breaks:** `#v`, `#h`, and `#pagebreak()` — hard and weak: `true`.
- **Backgrounds:** a `fill` color, and `background: / foreground:` for watermarks.

That's a properly furnished page. What's still missing is the good stuff you put *on* it — figures and images come next in Chapter 6, tables in Chapter 7 — and the deeper machinery behind counters and page-dependent content waits for you in Chapter 17.

5.9 Exercises

Solutions are in Appendix A.

- 5.1. Set up a US-letter page with a 3cm left margin and a 2cm right margin (top and bottom your choice). Fill it with `#lorem(80)` and confirm in the PDF that the text block sits left of center, hugging the wider margin.
- 5.2. Give a two-page document a running header that reads your name on the left and the date on the right, using `#h(1fr)` to separate them. Add a footer that centers the page number. (Remember: a custom footer means placing the number yourself.)
- 5.3. Take three paragraphs of `#lorem` and set them justified, with a first-line indent of `1.5em` and no blank-line spacing between paragraphs — the classic novel look. Then switch `first-line-indent` between its plain-length and `all: true` forms and describe, in one sentence, what changed about the first paragraph.
- 5.4. Lay out a page in two columns with a 1.5cm gutter, then add a single heading at the very top that spans the *full* width above both columns. (Hint: one of the two column tools leaves the surrounding text alone.)
- 5.5. (*Stretch.*) Build a one-page "confidential memo" that uses everything: a tinted page fill, a faint rotated watermark in the background, a header with a title and a right-aligned classification ("CONFIDENTIAL"), a footer with a "page 1 / 1" style number, justified body text with tightened leading, and a `#v` of a couple of centimeters above a signature line at the bottom. Compile it and adjust until it looks like something you'd be mildly nervous to leave on a printer.

Figures and images

Here is the thing word processors get wrong more reliably than anything else. You drop an image into a page, and for the rest of that document's life it behaves like a cat you are trying to photograph: never quite where you left it, sliding two paragraphs down the instant you add a sentence above, now and then leaping onto a blank page of its own out of sheer spite. Placing a picture ought to be the easy part. Somehow it becomes the part that eats the afternoon.

Typst places an image with one line. Better than that, it turns a bare image into a proper *figure* — a numbered, captioned thing the rest of your document can point at by name — with barely more effort. This chapter is about both: getting a picture onto the page, and then dressing it up so a sentence three pages later can say "see Figure 4" and always be telling the truth, even after you have shuffled everything around.

Most of our pictures will be drawn with Typst's own shapes, so the examples need no files you don't already have. But we start with a real image, because that is what you came for.

6.1 Placing an image

The function is `image`, and in its shortest form you hand it a filename:

```
#image("logo.svg")
```

That is the whole thing. Typst reads the file, works out the format from its contents, and sets it on the page at its natural size. It understands the formats you'd expect: PNG and JPG for photographs and screenshots, GIF, and SVG for vector art that stays crisp at any size.

The file in `examples/025-basic-image/` is an SVG, and it's worth a look because SVG is just text — a few lines of tags describing a rounded blue square, a white disc, and an orange triangle. You can open it in any editor, read it, and see it in a diff. That is a small superpower: an image asset you can actually inspect, rather than an opaque blob of pixels.

The filename is resolved **relative to the file doing the loading**. If your `main.typ` sits next to `logo.svg`, then `"logo.svg"` finds it; a picture in a subfolder is `"images/logo.svg"`. A path that starts with a slash is treated as absolute *from the project root* instead — the root being the folder you point Typst at with `--root` (Chapter 2). So `"/assets/logo.svg"` means "the `assets` folder at the top of the project," wherever the current file lives.

6.1.1 Sizing it

Natural size is rarely the size you want. Give the image a `width`, a `height`, or both:

```
#image("logo.svg", width: 3cm)
```

Hand it just one dimension and Typst scales the other to match, so the image keeps its proportions and nothing looks squashed. You can use any length Typst understands, including a percentage — `width: 60%` means 60% of the space the image sits in, which is the usual way to say "about two-thirds of the text width" without doing arithmetic. We'll come back to what happens when you pin *both* dimensions, because that is where `fit` earns its keep.

6.1.2 A word for screen readers

One argument you should get into the habit of adding is `alt`, a short text description of the picture:

```
#image("logo.svg", width: 3cm, alt: "The company logo: a blue square with a white disc")
```

Nobody sees `alt` on the page. It's written into the PDF for screen readers and other assistive tools, and it's the difference between a blind reader hearing "the company logo" and hearing nothing at all. It costs you a few seconds. Spend them.

NOTE

Typst infers the format from the file's *contents*, not its extension, so a PNG that someone mischievously named `photo.jpg` still loads correctly. The extension is for humans; the bytes are for Typst.

6.2 Wrapping it in a figure

A loose image is fine for a logo in a header. But most pictures in a serious document want three things a bare `#image` doesn't give you: a number, a caption, and the ability to be referenced from the text. That bundle is a *figure*, and you make one by wrapping your content in `#figure`:

```
#figure(
  image("logo.svg", width: 4cm),
  caption: [The project logo, in its natural habitat.],
)
```

Notice that `image` inside `figure` has no `#` in front of it. The `#` is what drops you from markup into code; once you're already inside the parentheses of `#figure(...)`, you are in code, so the inner call is plain `image(...)`. (If that sounds like a rule you'll forget, you will, and Typst's error message will remind you, kindly. Chapter 13 makes the whole markup-versus-code idea click.)

Compile that and you get your image, centered, with a line beneath it reading something like **Figure 1: The project logo, in its natural habitat.** You typed no number. Typst counts figures for you, in order, and if you insert a new one at the top tomorrow, everything below rennumbers itself without you lifting a finger. The word "Figure" is the *supplement* — the label Typst puts before the number — and it, too, is automatic.

The body of a figure doesn't have to be an image at all. It can be any content: a shape, a block of text, a table. `examples/026-figure-with-caption/` wraps a plain drawn rectangle and a circle, purely to show that the caption and numbering machinery cares nothing for what's inside — it frames whatever you give it and counts.

Coming from LaTeX. No `\begin{figure}`, no `\caption{}`, no `\label{}` — and, the part that surprises people, no `float` package, no `[htbp]` incantation, no `\usepackage{graphicx}` at the top. A figure is one built-in function. Captions, numbering, and cross-references come in the box, and the figure only floats if you ask it to (we'll get there).

6.2.1 Captions above instead of below

By default the caption sits below the figure, which is conventional for images. Tables usually want their caption on top. You move it with a set rule:

```
#set figure.caption(position: top)
```

Set rules are Chapter 9's whole topic, so don't worry about the mechanics yet. The shape to notice is that the caption's position is a setting you change once, in one place, rather than a decision you re-make at every figure.

6.3 Referencing figures

Automatic numbers are only half the win. The other half is never having to *read* those numbers yourself. You attach a label to a figure and refer to it by that label; Typst fills in the number, keeps it correct forever, and makes it a clickable link into the bargain.

A label is a name in angle brackets, written right after the figure:

```
#figure(
  image("chart.svg"),
  caption: [Quarterly revenue.],
) <fig:revenue>
```

Then, anywhere in your prose, an `@` and the label name pull in a reference:

```
Revenue climbed all year (@fig:revenue), then dipped in December.
```

That `@fig:revenue` renders as "Figure 3" (or whatever number the figure ends up with), linked to the figure itself. Move the figure, add ten figures before it, reorder the lot — the reference follows. `examples/027-referencing-figures/` has two labelled figures and points at both from the surrounding prose, so you can watch the numbers and the text stay in lockstep.

The `fig:` at the front of the label is a **convention, not a requirement**. Typst is perfectly happy with `<revenue>` and `@revenue`. But labels are shared across your whole document — headings, equations, tables, and figures all draw from the same pool — so a short prefix like `fig:`, `tbl:`, or `eq:` keeps them sorted and keeps you from accidentally

naming two different things the same. The number Typst prints comes from *what the label is attached to*, not from the prefix: point `@fig:revenue` at a figure and you get "Figure 3" because it's a figure, prefix or no prefix.

TIP

A reference can carry its own wording: `#ref(<fig:revenue>, supplement: [Fig.])` prints "Fig. 3" instead of "Figure 3" for that one mention — handy inside a parenthetical where the full word would be fussy. Chapter 11 gives references the full tour; this is just a taste.

6.4 Figures aren't only for images

We've said it twice now, so let's prove it. When the body of a figure is a **table**, Typst notices, and quietly does the right thing: it labels it "Table" instead of "Figure" and counts tables on their own separate track. Figures and tables don't share a number, because a reader flipping to "Table 2" shouldn't land on a photograph.

```
#figure(
  table(
    columns: 2,
    [*Quarter*], [*Sales*],
    [Q1], [120],
    [Q2], [148],
  ),
  caption: [Quarterly sales, in thousands.],
) <tbl:sales>
```

That auto-detection covers tables and code listings. For anything else — a diagram, a photograph you specifically want counted apart, a musical example — you tell Typst what it is with `kind`, and give it a word to print with `supplement`:

```
#figure(
  my-diagram,
  caption: [The data pipeline, end to end.],
  kind: "diagram",
  supplement: [Diagram],
) <dia:pipeline>
```

Every figure sharing a `kind` shares a counter. So a document can run "Figure 1, Figure 2" down one track, "Table 1, Table 2" down another, and "Diagram 1, Diagram 2" down a third, all at once, each numbered independently and each referenced by its labels. `examples/029-figure-of-a-table-or-diagram/` puts a table figure and two custom "Diagram" figures side by side so you can watch the three counters march in parallel.

NOTE

The `kind` is what separates the counters, and `supplement` is only the word printed on the page and in references. Give two figures the same `kind` and they count together even if you spell their supplements differently — so keep the `kind` consistent for things that belong to one sequence.

6.5 Sizing and fit

Back to that promise about pinning both dimensions. Give an image only a width and there's no conflict — the height follows along to preserve the shape. But the moment you fix *both* width and height, you've described a box that may not match the image's proportions, and Typst needs a policy for the mismatch. That policy is `fit`, and it has three settings:

- `"contain"` (fit the whole image inside the box, adding blank space on two sides if the shapes don't match — the "letterbox" you see on films),
- `"cover"` (fill the box completely and crop whatever overflows), and
- `"stretch"` (fill the box by distorting the image, squashing or stretching it to fit exactly).

The default is `"cover"`, which is usually what you want for a background or a uniform grid of thumbnails, where a little cropping is fine but empty gaps look sloppy. Use `"contain"` when you must see the whole image and can tolerate the margins. Reach for `"stretch"` almost never; distortion is rarely a good look, though it has its uses for deliberate effect.

`examples/028-sizing-and-fit/` scales the logo with a relative `width: 60%` and then drops the same image into three identical boxes, one per fit mode, so the difference is impossible to miss: one letterboxed, one cropped, one squashed.

TIP

A percentage width is the friend of a resilient layout. Say `width: 80%` instead of `width: 13.2cm`, and your figure keeps its proportions relative to the text no matter what page size or margins you switch to later. Absolute lengths pin an image to one specific page; relative ones let it adapt.

6.6 Letting figures float

So far our figures have stayed exactly where we wrote them, wedged between two paragraphs. That's the sensible default, but sometimes you'd rather a big figure drift to the top or bottom of the page and let the text close ranks around the gap it left — the behaviour LaTeX users know, and dread, as *floating*.

Typst floats on request, through the `placement` argument:

```
#figure(
  image("wide-chart.svg"),
  caption: [A chart too tall to sit mid-paragraph.],
  placement: top,
)
```

`placement: top` sends the figure to the top of a page; `bottom` sends it to the foot. `placement: auto` lets Typst pick whichever is nearer, which is the low-effort choice when you just want the thing out of the text's way. Leave `placement` off entirely (its default) and the figure stays put, in the flow, right where you typed it.

There's a lower-level cousin worth knowing by name: `#place`. Where a floating figure politely asks to move to the top or bottom, `#place(top + right, ...)` drops content at an exact spot — a corner, a fixed offset — and pulls it out of the normal flow so the surrounding text ignores it entirely. It's the tool for a watermark, a logo pinned to a corner, a caption glued beside a shape. It's also easy to overuse; most of the time a figure with a sensible `placement` is the better-behaved choice. Chapter 21 covers `#place` and manual positioning in depth.

WARNING

A floating figure needs somewhere to float *to*. On a single short page with nothing above or below it, `placement: top` has no room to work and the figure simply stays where it is — no error, just no visible effect. Floats earn their keep in long, flowing documents, not one-page examples.

6.7 What you've got

You can now put pictures on the page and make them behave:

- **Load an image** with `#image("file")`, in PNG, JPG, GIF, or SVG, with paths relative to the current file (or from the project root with a leading `/`).
 - **Size it** with `width`, `height`, or a percentage, and describe it for screen readers with `alt`.
 - **Control the fit** when both dimensions are pinned — `"contain"`, `"cover"`, or `"stretch"`.
 - **Wrap it in a figure** for automatic numbering, a caption above or below, and a "Figure N" supplement you never type by hand.
 - **Reference figures** by attaching a `<fig:name>` label and writing `@fig:name` — a live, clickable number that stays correct as the document shifts.
 - **Figure anything**, not just images: tables and diagrams too, with `kind` and `supplement` splitting them onto independent counters.
 - **Float** a figure to the top or bottom of the page with `placement`, and know that `#place` exists for exact positioning.
- That's the whole everyday toolkit for illustrations. Next, in Chapter 7, we build the tables those figures sometimes wrap — Typst's grid system, which turns out to be one of its quietly best features.

6.8 Exercises

Solutions are in Appendix A.

6.1. Grab any small PNG or JPG you have lying around (a screenshot will do), put it in a folder next to a `main.typ`, and place it with `#image` at a width of `5cm`. Then add an `alt` description. Compile and confirm it appears at the size you asked for.

6.2. Wrap that same image in a `#figure` with a caption. Compile it and check that Typst prints "Figure 1" and your caption beneath the picture — without you typing the number.

6.3. Add a second figure below the first, give both labels (`<fig:one>` and `<fig:two>`), and write a sentence that refers to each with `@fig:one` and `@fig:two` . Now cut the second figure and paste it *above* the first. Recompile and confirm the numbers in your sentence swapped to match, all on their own.

6.4. Take one image and place it three times inside identical `4cm -by- 2cm` boxes, once with each `fit` value. Line them up and describe, in a sentence under each, what happened to the image and why.

6.5. (*Stretch.*) Build a one-page "figure zoo": a drawn-shape figure, a table figure, and a custom `kind: "diagram"` figure, each captioned and labelled. Reference all three in a short paragraph. Confirm that the figure, table, and diagram each carry their own independent number (Figure 1, Table 1, Diagram 1), and that your references point at the right ones.

Tables and grids

The table is where word processors go to die. You have four columns of tidy numbers, you paste in a fifth, and the whole thing lurches sideways off the page. You drag a border to make room; a different border moves. You get it looking right, add one row, and it isn't right any more. Somewhere in that struggle you stop thinking about your data and start thinking about the software, which is exactly backwards.

Typst takes the opposite deal. You describe the table — these are the columns, here are the cells, this row is the header — and the engine does the arranging. When you add a row, you type a row; nothing lurches. When you want the numbers right-aligned, you say so once, in one place, for the whole column. The table stops being a thing you fight and goes back to being a thing you write.

And there's a bonus hiding in here. The machine that lays out tables is the same one that can place two blocks of text side by side, or set up a poster, or split a page into panels. It's called `grid`, and a `table` is really just a `grid` that also knows it's holding data. Learn one and you've most of the way learned the other. We'll start with tables, because they're the part you'll reach for first, and meet the bare `grid` at the end.

7.1 The shape of a table

Here is a whole table, start to finish ([examples/030-basic-table/](#)):

```
#table(
  columns: 3,
  [*Course*], [*Credits*], [*Exam*],
  [Typesetting], [6], [Project],
  [Statistics], [4], [Written],
  [Pedagogy], [5], [Oral],
)
```

That renders a neat three-column grid with light borders: a header-ish top row of bold labels, then three courses under it.

Two things are doing all the work. `columns: 3` says the table is three columns wide. Everything after it is a flat list of *cells*, and Typst pours them into the grid the way you'd read them — left to right, then down to the next row. After every third cell it wraps to a new line. There are no row brackets, no `\\` at the end of each line, no `&` between columns. You just list the cells in reading order and tell Typst how wide the grid is; it works out where each one lands.

Each cell here is a piece of *content* in square brackets — `[*Course*]`, `[6]` — the same `[...]` content blocks you've been writing since Chapter 3. A cell can hold anything content can hold: a word, a number, a whole paragraph, an image, another table if you're feeling reckless. The bold labels are just `*strong*` markup inside the cell, nothing table-specific.

NOTE

The cells are one long list, so the commas matter and the line breaks don't. Laying each row on its own source line, as above, is purely for your eyes — Typst counts cells, not lines. But do lay them out that way. A table whose source rows match its printed rows is a table you can still edit next month.

Coming from a spreadsheet. A Typst table isn't a live grid you click into; it's a description you write out. That sounds like more work, and for three cells it is. The payoff arrives at scale: the description is plain text, so you can diff it, search it, paste a column from somewhere else, or generate the whole thing from a loop (Part IV). No cell references, no hidden formulas — what you see is what's there.

7.2 Sizing the columns

`columns: 3` is shorthand. What Typst actually wants is a list of *column tracks*, and `3` is a friendly way of saying "three tracks, each `auto`". Spell the list out and you control every column's width. There are three kinds of track, and you'll mix them constantly ([examples/031-column-sizes/](#)):

```
#table(
  columns: (auto, 1fr, 2.5cm),
  [*Feature*], [*What it does*], [*Since*],
  [Preview], [Recompiles as you type, usually faster than you can notice.], [v0.1],
  [Packages], [A central registry you pull templates and tools from.], [v0.6],
  [HTML export], [An experimental second output format beside PDF.], [v0.11],
)
```

- **auto** sizes the column to fit its widest cell and no more. The first column is exactly as wide as "HTML export" needs and stops there.
- A **length** like `2.5cm` (or `2cm`, `40pt`, `1in`) pins the column to a fixed width, come what may. The "Since" column is 2.5 cm whether it holds "v0.1" or nothing.
- A **fraction** like `1fr` claims a share of whatever width is *left over* after the `auto` and fixed columns have taken theirs. Here one column has `1fr`, so it swallows all the remaining space — which is why the long descriptions wrap neatly inside it instead of shoving the table off the page.

The `fr` unit is the quietly brilliant one. It's short for *fraction*, and fractions split the leftover space in proportion. Two columns at `(1fr, 2fr)` divide the remaining width one-third to two-thirds. `(1fr, 1fr, 1fr)` gives three equal columns no matter what's in them. `fr` is how you say "share this space fairly" without doing the arithmetic yourself, and it's the reason a Typst table adapts when the page width changes instead of overflowing.

Rows work the same way, through a `rows:` argument, though you'll set it far less often — cells are usually happy to be as tall as their content. When you do want control, the same three track kinds apply: `rows: 1cm` makes *every* row exactly one centimetre tall; `rows: (auto, 2cm)` sets the first row to fit and the second to a fixed height. If you give fewer track sizes than there are columns or rows, Typst cycles through the list and reuses it, which is how a single value like `rows: 1cm` ends up governing all of them.

7.3 Alignment

By default everything sits top-left. To move it, reach for `align`, and `align` takes three shapes depending on how much control you want.

The simplest is one value for the whole table — `align: center` centres every cell. One step up is a list, one entry per column:

```
#table(
  columns: 3,
  align: (left, center, right),
  ...
)
```

Now the first column is left-aligned, the second centred, the third right — the classic layout for a table of labels and numbers, where you want the figures to line up on their right edge. Like the column tracks, a short list cycles, so `align: (left, right)` on a four-column table alternates left, right, left, right.

The most powerful shape is a *function*. Instead of a fixed list, you hand `align` a little rule that receives a cell's column `x` and row `y` (both counting from zero) and returns an alignment for it. That sounds abstract until you see what it's for ([examples/032-alignment/](#)):

```
#table(
  columns: (1fr, auto, auto),
  align: (x, y) => if y == 0 { center } else if x == 0 { left } else { right },
  [*Item*], [*Qty*], [*Price*],
  [Notebook], [3], [€4.50],
  [Fountain pen], [1], [€28.00],
  [Ink cartridges], [12], [€6.20],
)
```

Read the function out loud: *if we're in the top row, centre it; otherwise, if we're in the first column, left-align; otherwise, right-align*. That single line centres the header, left-aligns the item names, and right-aligns every quantity and price so the euro amounts line up cleanly — three alignment rules that no per-column list could express, because the header row wants different treatment from the body. The `=>` makes an anonymous function; if it looks strange now, Chapter 14 gives functions the full introduction. For tables you only need the pattern: `(x, y) => something-that-depends-on-x-and-y`.

7.4 Fills and zebra striping

A `fill` colours the background of cells, and it takes exactly the same three shapes as `align`: one colour for everything, a list per column, or a function of `(x, y)`. The function is where it earns its keep, because it's how you get *zebra striping* — that alternating shaded-row look that makes a wide table readable across ([examples/033-striped-table/](#)):

```
#table(
  columns: (lfr, auto, auto),
  stroke: none,
  inset: 8pt,
  fill: (x, y) => if calc.odd(y) { luma(240) },
  [*Region*], [*Q1*], [*Q2*],
  [North], [120], [138],
  [South], [98], [102],
  [East], [156], [161],
  [West], [143], [149],
)
```

The fill function says: for a cell in an odd-numbered row, paint it `luma(240)`, a very light gray; otherwise, return nothing. `luma` builds a gray from a single number — `luma(0)` is black, `luma(255)` white, so 240 is a whisper of gray. `calc.odd(y)` is true for rows 1, 3, 5, and so on. The trick is what happens on the even rows: the `if` has no `else`, so the function returns `none`, and a cell with no fill just stays transparent. Odd rows get shaded, even rows don't, and the eye gets a rail to follow.

Turning `stroke: none` off the borders is what makes striping look modern rather than boxed-in — the shading does the row-separating work that lines used to. More on strokes in a moment.

You can also paint a single cell, without any function, by wrapping it in `table.cell` and handing it a `fill` — `table.cell(fill: yellow)[Overdue]` for one flagged cell. `table.cell` is the general "this cell is special" wrapper, and we're about to lean on it hard.

7.5 Header and footer rows

Marking your top row as a *header* does two useful things: it tells Typst (and anyone reading the PDF's structure, including screen readers) that this row labels the columns, and it makes that row *repeat automatically at the top of every page* when a long table breaks across several. You get it by wrapping the first row's cells in `table.header` ([examples/034-header-row/](#)):

```
#table(
  columns: (lfr, auto, auto),
  align: (left, right, right),
  fill: (x, y) => if y == 0 { luma(230) },
  table.header(
    [*Region*], [*Q1*], [*Q2*],
  ),
  [North], [120], [138],
  [South], [98], [102],
  [East], [156], [161],
  table.footer(
    [*Total*], [374], [401],
  ),
)
```

The header repeats by default; if for some reason you want it to appear only once, pass `table.header(repeat: false, ...)`. Here the fill function shades just row zero, giving the header a subtle gray bar — combine that with the bold labels and you have a header that reads as a header without a single border.

`table.footer` is the mirror image: a row pinned to the bottom, repeating on each page, the natural home for a totals line. In a table short enough to fit one page, as in the preview, header and footer just sit top and bottom as you'd expect; their powers show up the moment the table grows past a page.

TIP

A table usually wants a caption and a number ("Table 3") so you can refer to it from your prose. That's not the table's job — it's `figure`'s, which wraps any block, numbers it, and lets you cross-reference it. We met `figure` with images back in Chapter 6; it works exactly the same around a `table`.

7.6 Spanning cells

Real tables aren't always perfect grids. A title stretches across the top; a category label covers three rows beneath it. Those are *spans*, and they're two arguments on `table.cell`: `colspan` to reach across columns, `rowspan` to reach down rows ([examples/035-spanning-cells/](#)):

```
#table(
  columns: 3,
  align: (x, y) => if x == 0 { left } else { center },
  table.cell(colspan: 3, align: center)[*Autumn timetable*],
  [Time], [Mon], [Tue],
  table.cell(rowspan: 2)[Morning], [Maths], [Biology],
  [English], [Chemistry],
  table.cell(rowspan: 2)[Afternoon], [Art], [Music],
  [History], [Drama],
)
```

The `colspan: 3` cell eats the entire first row as a title. Then come three ordinary cells for the "Time / Mon / Tue" row. The `rowspan: 2` on "Morning" is the interesting one: that cell now occupies its column for *two* rows, so on the next source line you only supply the two cells that go beside its lower half — `[English]`, `[Chemistry]` — and Typst slots them into the columns the span didn't claim. You never leave gaps for spanned cells; you just stop providing cells for the space they already fill, and the flow accounts for it. Miscalculate and you'll get a row with the wrong number of cells, which Typst will place somewhere you didn't intend — so when a spanning table looks scrambled, count your cells per row first.

`table.cell` carries more than spans. Any single cell can override the table's defaults for itself: `table.cell(align: right)[...]` to nudge one value, `table.cell(fill: red.lighten(70%))[...]` to flag one entry, `table.cell(inset: 12pt)[...]` to give one cell extra breathing room. The table-wide settings are the defaults; `table.cell` is how one cell disagrees.

7.7 Strokes, lines, and the booktabs look

`stroke` controls the lines. Like `fill` and `align`, it scales from blunt to surgical.

At the blunt end, one value covers every border: `stroke: 0.5pt` for hairlines, `stroke: 2pt + blue` for something louder (a stroke carries a thickness *and* a colour, added together), and `stroke: none` to switch borders off entirely, as the striped table did. A dictionary sets the four sides independently:

```
stroke: (left: 1pt, bottom: 0.5pt, top: none, right: none)
```

And, inevitably, a function of `(x, y)` styles each cell's borders by position — `stroke: (x, y) => if y == 0 { (bottom: 1pt) }` draws a rule under the header row and nowhere else.

But for the single most common look — the clean, ruled, no-vertical-lines style that LaTeX users know as *booktabs* — there's a more direct tool. Instead of reasoning about which cell owns which border, you place horizontal and vertical rules yourself with `table.hline` and `table.vline`, dropped straight into the cell list at the point where you want the line drawn ([examples/036-custom-strokes/](#)):

```
#table(
  columns: 3,
  stroke: none,
  inset: (x: 10pt, y: 6pt),
  align: (left, right, right),
  table.hline(stroke: 1pt),
```

```

table.header([*Element*], [*Symbol*], [*Z*]),
table.hline(stroke: 0.6pt),
[Hydrogen], [H], [1],
[Helium], [He], [2],
[Lithium], [Li], [3],
[Beryllium], [Be], [4],
table.hline(stroke: 1pt),
)

```

Borders are off, and three `table.hline`s do everything: a firm one on top, a lighter one under the header, a firm one to close. That's the whole `booktabs` recipe — thick top and bottom, thin mid-rule, no verticals — and it comes out looking like it was set for print. An `hline` between two cells lands on the row boundary at that point in the list; a `table.vline` works the same way for a vertical rule between columns. Both accept the same `stroke`: you'd give the table, so you can make any single line as thick, coloured, or dashed as you like.

Coming from LaTeX. This whole section is where the relief hits hardest. There is no column-spec mini-language — no `\begin{tabular}{|l|c|r|}` to decode, no counting pipes and letters. Columns are described by an ordinary argument (`columns: (auto, 1fr, 2.5cm)`), alignment by another (`align:`), lines by another (`stroke:`). You don't need the `booktabs` package for `\toprule` and `\midrule`, because `table.hline` is built in and you place it exactly where you want it. And `\multicolumn` / `\multirow`, that pair of incantations, are just `colspan` and `rowspan` — two named arguments on a cell. Everything that was a special syntax in `tabular` is, in Typst, a plain value you pass to a function.

7.8 Spacing: inset and gutters

Two settings govern the whitespace, and they're easy to confuse because both add room but in different places.

`inset` is the padding *inside* each cell — the gap between the cell's content and its border. The default is a sensible few points; bump it up for an airier table (`inset: 10pt`) or shrink it for a dense one. You can make it asymmetric with a dictionary, exactly as example 036 does: `inset: (x: 10pt, y: 6pt)` gives wide horizontal padding and tighter vertical, a common tweak that stops columns feeling cramped without making rows too tall.

Gutters are the space *between* cells — gaps that separate the tracks instead of padding within them. `gutter: 6pt` spaces every row and column apart; `column-gutter` and `row-gutter` control the two axes separately. On a table you reach for gutters rarely (borders and inset usually do the job), but on a bare `grid` used for layout they're the main event, which is where we're headed.

7.9 grid: the same engine, minus the meaning

Everything so far — the column tracks, the `fr` units, `align`, `fill`, the spans, the gutters — belongs to a lower-level function called `grid`. A `table` is a `grid` with two things added: default borders, and the *semantics* that this is tabular data (headers, the accessibility structure a screen reader announces as a table). Strip those away and you get `grid`, which lays content out on a grid and draws nothing and claims nothing about what the content *means*. It's Typst's tool for pure page layout (`examples/037-grid-for-layout/`):

```

#grid(
  columns: (1fr, 1fr),
  column-gutter: 1cm,
  row-gutter: 0.8cm,
  grid.cell(colspan: 2)[
    == Typst, weighed
  ],
  [
    *For* \
    Instant preview, readable
    source, one language for
    content and code.
  ],
  [
    *Against* \
    A younger ecosystem and

```

```

    fewer packages than LaTeX's
    forty-year head start.
  ],
)

```

Two equal `1fr` columns put the two blocks side by side; a `column-gutter` opens a one-centimetre channel between them so they don't touch. The `grid.cell(colspan: 2)` spanning the top holds a heading across both columns — `grid.cell` is the grid's version of `table.cell`, spans and all. There isn't a border in sight, and that's the point: this is layout, not data. The API is the one you already know; only the defaults and the meaning have changed.

So which do you reach for? The rule is about *meaning*, not appearance:

- Use **table** when the content is a table — rows and columns of data where the structure carries information. It's the honest, accessible choice: the PDF records it as a table, headers and all, so tools and screen readers understand it. Even a borderless table (`stroke: none`) should stay a **table** if it's really data.
- Use **grid** when you're only positioning things on the page — two columns of a newsletter, a row of logos, a caption beside a figure, a dashboard of panels. There's no data here, so claiming there is would only mislead anything reading the document's structure.

Put plainly: if you'd call it a table in conversation, use `table`; if you'd call it a layout, use `grid`. When it's a genuine toss-up, prefer `table` — the extra semantics cost you nothing and help someone.

7.10 What you've got

You can now build essentially any table Typst can produce:

- **The core** — `#table(columns: ..., cell, cell, ...)`, cells flowing left to right and wrapping by column count.
- **Sized columns and rows** — `auto` to fit, a length like `2.5cm` to fix, and `fr` units to share the leftover space; the same three kinds for `rows`.
- **Alignment** three ways — one value, a per-column list, or an `(x, y) =>` function for rules that depend on position.
- **Fills**, including the `fill: (x, y) => if calc.odd(y) { luma(240) }` idiom for zebra striping, and per-cell `fill` for flagging one entry.
- **Headers and footers** with `table.header` / `table.footer`, which repeat across page breaks and carry the table's accessible structure.
- **Spanning cells** — `colspan` and `rowspan` on `table.cell`, plus per-cell `align`, `fill`, and `inset` overrides.
- **Strokes** from a single `stroke: 0.5pt`, through `stroke: none`, a per-side dictionary, and an `(x, y)` function, to hand-placed `table.hline` / `table.vline` for a booktabs-clean result.
- **Spacing** — `inset` inside cells, `gutter` / `column-gutter` / `row-gutter` between them.
- **grid** — the same layout engine without the table semantics, for placing blocks side by side, and a clear sense of when to use which.

Tables and grids round out Part II's toolkit for everyday documents. Next, in Chapter 8, Typst shows off the feature it's arguably proudest of: mathematics.

7.11 Exercises

Solutions are in Appendix A.

7.1. Build a four-column table of five films: title, director, year, and a one-word rating. Give it a header row in bold. Get the years and ratings to sit in sensibly sized columns while the title column takes up the slack — think about which track kind each column wants.

7.2. Take the table from 7.1 and right-align the year column, keeping the title left-aligned, using a single `align` list. Then add zebra striping with a `fill` function and switch the borders off. Decide for yourself whether it reads better striped or ruled.

7.3. Make a small weekly planner: a title spanning the full width, a header row of weekdays, and at least one cell that spans two rows (a "lunch" block covering two time slots, say). Count your cells per row carefully — this is the exercise where miscounting bites.

7.4. Reproduce the booktabs look from `examples/036-custom-strokes/` on data of your own: no vertical lines, a thick rule top and bottom, a thin rule under the header, and comfortable `inset`. Then add one `table.vline` to separate a group of columns from the rest and judge whether it improves or clutters the result.

7.5. (*Stretch.*) Using `grid` rather than `table`, lay out a simple "card": a full-width title across the top, then two columns underneath — a small fixed column for a label like "Price" or "Rating" and a `1fr` column for a paragraph of description (`#lorem(30)` will do). Add a gutter between the columns. Then write one sentence in a comment explaining why `grid` is the right choice here and a `table` would be the wrong one.

Math and equations

Here is the thing every other writing tool gets wrong. You are typing along happily, describing a result, and then you need to say that the area of a circle is πr^2 . In a word processor you stop, hunt through an equation editor, click a fraction template, fight the cursor out of the little box, emerge two minutes later with something that doesn't quite match the surrounding font. The sentence you were writing has gone cold.

Typst was built by people who did their share of mathematics, and it shows. You open math the way you open a parenthesis — with a symbol, mid-sentence, without breaking stride — and Typst sets it beautifully. The circle above? You typed `$\pi r^2$`. That's the whole ceremony.

This chapter is where Typst stops being a nice Markdown and starts being a genuine rival to LaTeX. By the end you'll typeset anything from an inline fraction to a multi-line derivation with aligned equals signs and a numbered, referenceable equation. Read it near a keyboard, because math is the one topic that rewards fiddling more than reading.

8.1 Entering math mode

A dollar sign opens math; the next dollar sign closes it. What you write between them lives in a little world with its own rules, where letters lean into italics and `^` means "superscript."

There are two flavors, and the gap between them is almost comically small. Write the math snug against the dollar signs and it sets *inline*, right in the run of your text, sized to match the words around it:

The area of a circle is `$\pi r^2$`, and `$a^2 + b^2 = c^2$` is Pythagoras.

Put a space just inside each dollar sign and the same content lifts out of the paragraph and centers itself on its own line as a *block* (or *display*) equation:

`$ a^2 + b^2 = c^2 $`

That single space is the whole switch. `$x$` is inline; `$ x $` is a display block. It looks like a typo the first time you see it, but it reads naturally once your fingers learn it: a little breathing room inside the dollars means a little breathing room on the page. Example `examples/038-inline-vs-block/` sets the very same equation both ways so you can see the two side by side.

NOTE

The space only has to be *there*, not measured. One space or five, a space or a newline — Typst just checks whether whitespace touches the inside of each dollar sign. That's why you can spread a long block equation across several lines of source (as we will, soon) and it stays a block.

8.2 Symbols have names, not backslashes

Type `$\alpha$` and you get α . Type `$\pi$` and you get π . This is the first big idea of Typst math and the one that trips up everyone arriving from LaTeX: symbols are spelled out by *name*, as plain words, with no backslash in front.

`$\alpha$`, `$\beta$`, `$\gamma$`, `$\theta$`, `$\pi$`, and the capital `$\Omega$`.

Capitalize the name and you get the capital letter — `$\Omega$` is Ω , `$\Gamma$` is Γ . The same naming scheme covers relations, operators, and arrows, and here Typst is kind enough to let you type the ones you already know as symbols:

`$x \leq y$`, `$x \geq y$`, `$x \neq y$`, `$a \times b$`, `$a \cdot b$`,
`$x \in A$`, `$A \subset B$`, `$x \rightarrow y$`, `$n \rightarrow \infty$`.

So `\leq` becomes $\leq$, `\neq` becomes $\neq$, `\rightarrow` becomes $\rightarrow$, and `\infty` — two little circles — becomes the infinity sign ∞ (its long name, *infinity*, works too). `\times` is $\times$, `\cdot` is a centered multiplication dot, `\in` is $\in$, `\subset` is $\subset$. There

are hundreds more, all discoverable in the symbol reference. Many reach a variant with a dot: `arrow.double` for $\Rightarrow$, `in.not` for $\notin$, `eq.not` for $\neq$. Example [examples/039-symbols-and-scripts/](#) gathers a working set of them.

Coming from LaTeX. The *ideas* map over almost unchanged. Math still lives between dollar signs, `^` is still superscript, `_` still subscript, and the `\\` you used to break a line becomes a single `\`. What changes is the spelling: there are **no backslash commands**. `\alpha` becomes `alpha`, `\infty` becomes `oo`, `\times` becomes `times`, and — the one that saves the most keystrokes — `\frac{a}{b}` becomes just `a/b`. Once the backslashes fall away, most equations get *shorter*.

Here's a wrinkle worth knowing early, because the error message is friendly but surprising. In Typst math, a run of letters is a *single* name. Typing `$ab$` doesn't mean "a times b" — it means "the symbol called `ab`," which doesn't exist, so Typst stops and says so (and suggests putting a space between the letters). To multiply two variables, separate them with a space: `$a b$`. That space also explains why `sin`, `log`, and friends work the way they do — which is the next stop.

8.3 Superscripts and subscripts

A caret raises the thing after it; an underscore lowers it. This part is just like LaTeX:

```
$x^2$, $x_i$, $x_i^2$
```

A single caret or underscore grabs one token — one letter, one digit, one symbol. The moment you want more than that, wrap it in parentheses, which group without printing anything:

```
$x^{(n + 1)}$, $a_{(i j)}$, $e^{(-x^2 / 2)}$
```

Without the parentheses, `$x^n + 1$` would raise only the `n` and leave `+ 1` on the baseline — occasionally what you want, usually not. The parentheses are the fix, and you'll reach for them constantly. Note `$a_{(i j)}$`: the space keeps `i` and `j` as two subscript letters sitting together, rather than a nonexistent symbol named `ij`.

8.4 Fractions and roots

The slash is a fraction. Whatever sits to its left becomes the numerator and whatever sits to its right becomes the denominator, and Typst is smart about how far each side reaches:

```
$1/2$, $(a + b)/(c + d)$, $(dif y)/(dif x)$
```

A bare `1/2` stacks one over two. Parentheses decide where each half stops, so `(a + b)/(c + d)` puts the whole sum on each level. There's also a spelled-out `frac(a, b)` for when a slash would read ambiguously, and `binom(n, k)` for a binomial coefficient — two things stacked in parentheses with no rule between them:

```
$frac(1, 1 + 1/x)$, $binom(n, k) = n! / (k! (n - k)!)$
```

Roots come in two shapes. `sqrt(x)` is a square root over whatever's inside the parentheses; `root(3, x)` is a cube root, and in general `root(n, x)` sets the index `n` in the crook of the radical:

```
$sqrt(2)$, $sqrt(x^2 + y^2)$, $root(3, x)$, $root(n, x)$
```

The radical stretches to cover its whole argument automatically — one of those small kindnesses you stop noticing until you go back to a tool that doesn't do it. Example [examples/040-fractions-and-roots/](#) collects fractions, the binomial, and both roots.

8.5 Functions and operators

Type `$sin x$` and Typst sets "sin" upright and "x" italic, with a hair of space between them — exactly the convention you learned in school. It manages this because `sin` is a known operator. So are `cos`, `tan`, `log`, `ln`, `exp`, `lim`, `max`, `min`, `det`, and a long list of others: they render upright, the way function names should, while single-letter variables stay italic.

Why does `sin` come out upright when `x` comes out italic? Because `sin` is one identifier and a *known* one — Typst defines it, set upright the way an operator should be. Single letters have no such definition, so they stay italic variables. (An unknown multi-letter run, recall, is an error, not silent italics.) When you want an arbitrary word upright — a subscript label, say — put it in quotes:

```
$V_"max"$ and $k_"cat"$ set "max" and "cat" as upright labels.
```

When you need an operator Typst doesn't already know, build one with `op` and a quoted name. Give it `limits: #true` if its sub- and superscripts should stack above and below in display mode, the way `lim` and `sum` do. (The `#` is needed because inside math a bare `true` would read as a variable name; `#true` drops into code for the actual boolean.)

```
$ op("argmax", limits: #true)_(x in X) f(x) $
```

If you use the same operator often, define it once in code and reuse the name:

```
#let argmin = math.op("argmin", limits: true)
$ argmin_theta L(theta) $
```

8.6 Sums, integrals, and other big operators

Big operators are where display mode earns its keep. Write the bounds as an ordinary subscript and superscript, and Typst places them wherever the context wants. In a block, the bounds climb above and below the operator:

```
$ sum_(i=1)^n i = (n (n + 1)) / 2 $
$ integral_a^b f(x) dif x $
$ product_(k=1)^n k = n! $
$ lim_(x -> 0) (sin x) / x = 1 $
```

Those four — `sum`, `integral`, `product`, `lim` — set as a fat summation sign with `i=1` tucked underneath and `n` on top, an integral with its limits on the right, and so on. Two small things to notice. `dif` is the upright "d" of a differential (`dif x` for dx); writing a plain italic `d` would technically work but reads worse. And `lim_(x -> 0)` puts the whole arrow expression under the word "lim," because `lim` is one of those `limits: true` operators.

The clever part is that the *same source* behaves differently inline. Drop a sum into a sentence and Typst moves the bounds to the side to keep the line from growing tall: the series $\sum_{n=0}^{\infty} \frac{1}{2^n} = 2$ sits politely on one line, its `n=0` and `∞` shrunk to script size beside the sign. Example `examples/041-sums-and-integrals/` shows both placements.

TIP

Want a block-style stacked sum *inline* anyway, or side-set bounds in a block? Wrap the operator in `limits(...)` to stack or `scripts(...)` to force the side. You'll rarely need to, but it's there when a line looks cramped.

8.7 Matrices, vectors, and cases

Three functions cover most of the structured math you'll write, and they share a tidy convention: a comma separates entries across a row, a semicolon ends the row.

`mat` builds a matrix:

```
$ A = mat(1, 2; 3, 4) $
```

`vec` stacks its arguments into a column vector — no semicolons needed, since a column is one-per-row by definition:

```
$ v = vec(x, y, z) $
```

Both come wrapped in parentheses by default. Change the fence with `delim`: square brackets for a plain matrix, vertical bars for a determinant, or any delimiter you like:

```
$ mat(delim: "[", 1, 0; 0, 1) $
$ mat(delim: "|", a, b; c, d) $
```

`cases` sets a brace-delimited list of alternatives — the shape you want for a piecewise definition. Inside it, an `&` marks where the conditions should line up, so the "if" clauses form a neat column:

```
$ abs(x) = cases(
  x & "if" x >= 0,
  -x & "if" x < 0,
) $
```


The `"if"` in quotes is literal text, set upright — more on that shortly. All three functions, plus the custom delimiters, live in `examples/042-matrices-and-vectors/`.

8.8 Lining equations up

A real derivation is several lines that want to agree on a vertical line — every equals sign stacked, so the eye runs straight down the middle. Typst gives you two characters for this, both usable only inside a display block. A backslash `\` starts a new line. An ampersand `&` marks the alignment column: everything before the `&` on each line is pushed right up to it, everything after flows left from it.

Put the `&` just before each equals sign and the whole derivation snaps into column:

```
$ (a + b)^2 &= (a + b)(a + b) \
    &= a^2 + a b + b a + b^2 \
    &= a^2 + 2 a b + b^2 $
```

Only the first line carries the `(a + b)^2`; the later lines start with `&`, so their left side is empty and they hang from the equals sign. It sets exactly the way a careful teacher writes it on a board. The same move keeps the steps of a solved equation aligned:

```
$ 3 x + 6 &= 21 \
    3 x &= 15 \
        x &= 5 $
```

Example `examples/043-aligned-equations/` has both. Use more than one `&` per line to make several alignment columns (for a system of equations, say), but one is enough to cover the common case, and it's the one to learn first.

WARNING

The `\` line break and the `&` alignment marker are math-mode citizens. They only mean this inside a `$ ... $` block. Out in ordinary markup a `\` is still a hard line break, but there is no `&` alignment there — that's what tables and grids are for (Chapter 7).

8.9 Text, styles, and dropping into code

Sooner or later you need a real word inside an equation — an "if," a "where," a short label. Bare letters won't do it, because Typst reads them as multiplied variables. Wrap the words in double quotes and they set as upright text at the right size:

```
$ f(x) = x^2 quad "for all" x in RR $
```

For emphasis and weight there are functions that mirror the markup ones: `upright(x)` forces upright, `bold(x)` sets bold (heavier than the default math italic), and `italic(x)` forces italic. Accents ride on top of a letter as functions too — `hat(x)` for $\hat{x}$, `arrow(v)` for a vector arrow, `bar(x)`, `dot(x)`, `tilde(a)`.

When the value you want isn't a symbol but a *computed number*, escape out of math and into Typst's code with `#`, exactly as you do in markup:

```
$ pi approx #calc.round(calc.pi, digits: 4) $
```

That drops in `3.1416`, computed at compile time. The `#` is the same doorway into code you met in Chapter 3; it works inside math just as it does outside.

One more practical matter: spacing. Inside math, Typst decides the spacing for you based on what things *are* — a bit of space around a `+`, a hair between juxtaposed variables — and it ignores how many spaces you typed in the source. `$x=y$` and `$x = y$` come out identical. When you genuinely want to *insert* space, ask for it by name: `quad` is a wide gap (for separating two formulas on one line), `space` is a normal word space, and `thin` is a thin sliver. For anything bespoke, `#h(1em)` reaches for the horizontal-space function markup uses.

8.10 Numbering and cross-references

By default, equations aren't numbered — a display block just sits there, anonymous. Turn numbering on with a single set rule, and from that point every block equation gets a number on the right margin:

```
#set math.equation(numbering: "(1)")
```

The "(1)" is a numbering *pattern*: the digit 1 is the counter and the parentheses are literal, so you get (1), (2), (3). (The same pattern language runs headings and lists; Chapter 9 gives it the full treatment.) Inline math is left alone — only display blocks are numbered — so your prose doesn't sprout numbers mid-sentence.

A number is only useful if you can point at it. Pin a label on an equation by writing it in angle brackets right after the closing dollar sign, then refer to it from anywhere with @:

```
$ e^(i pi) + 1 = 0 $ <eq:euler>
```

Euler's identity, @eq:euler, is often called the most beautiful equation in mathematics.

Typst fills in @eq:euler as "Equation 1," rendered as a live link to the equation, and — this is the payoff — the number stays correct no matter how you shuffle the document. Add three equations above this one and it quietly becomes Equation 4, reference and all. The eq: part of the name is just a convention to keep equation labels distinct from labels on figures or headings; Typst doesn't require it, but your future self will thank you. Example [examples/044-numbered-equations/](#) turns numbering on, labels two equations, and references both.

NOTE

A reference reads "Equation 1" because that's the default supplement for an equation. You can change the wording, or drop it, when you tune references in Chapter 11. For now, plain @eq:euler gives a correct, clickable number — which is most of what you wanted.

8.11 What you've got

You can now typeset mathematics, not just decorate a page with it:

- **Two ways in** — inline $\$x\$$ for math in the run of text, and $\$ \times \$$ (mind the spaces) for a centered display block.
- **Symbols by name** — alpha, pi, sum, oo, ->, <=, in — no backslashes, and multi-letter runs are single names, so variables need spaces between them.
- **Scripts** with ^ and _, grouped past one token with parentheses: x^(n+1), a_(i j).
- **Fractions and roots** — a/b, frac, binom, sqrt, root(3, x).
- **Functions and operators** — sin, log, lim set upright automatically, plus op(...) and math.op(...) for your own.
- **Big operators** — sum, integral, product, lim with bounds that stack in display and tuck to the side inline.
- **Structures** — mat, vec, and cases, with delim to choose the fence.
- **Alignment** — \ for a new line and & for the column — derivations that line up on the equals sign.
- **Text and code inside math** — "quoted words", upright / bold / italic, accents, and # to drop in a computed value.
- **Numbering and references** — #set math.equation(numbering: "(1)") , a <eq:name> label, and an @eq:name cross-reference that stays correct.

That is enough to set the mathematics in a homework sheet, a physics lab report, or a research paper. The reference documentation lists every symbol and function by name; now that you know the shape of the system, that list reads like a menu instead of a wall.

8.12 Exercises

Solutions are in Appendix A.

8.1. Write one sentence of ordinary prose that contains the inline equation "E = mc²" (use the caret for the square). Then, on the next line, set the quadratic formula $x = (-b \pm \sqrt{b^2 - 4ac}) / 2a$ as a centered display block. Compile it and confirm the first stays in the line and the second stands alone. (The ± is plus.minus.)

8.2. Typeset the definition of the standard normal density, $f(x) = (1/\sqrt{2\pi}) \cdot e^{-(x^2/2)}$, as a display block. You'll need a fraction, a square root, pi, and a grouped exponent — mind where the parentheses go.

8.3. Set the sum of the first n squares as a numbered display equation: $\sum_{i=1}^n i^2 = n(n+1)(2n+1)/6$. Turn on equation numbering, give it the label <eq:squares>, and then, in a following sentence, refer to it with @eq:squares.

8.4. Typeset a 2×2 matrix $M = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}$ (the rotation matrix) using square-bracket delimiters, then a piecewise definition of the sign function $\text{sgn}(x)$ using cases with three branches (−1, 0, 1) whose conditions line up on an &.

8.5. (*Stretch.*) Write a three-line derivation aligned on its equals sign, that starts from $a^2 - b^2$ and ends at $(a - b)(a + b)$, showing at least one intermediate step. Then define a custom operator with `math.op` (for example, `argmax` or `Var`) and use it in a display equation with a subscript. Verify the whole thing compiles with no warnings.

Set rules

You have been using set rules for five chapters. Every time you wrote `#set text(font: "New Computer Modern")` to pick a typeface, or `#set par(justify: true)` to square off the right margin, or `#set page(margin: 2cm)` to give a document room to breathe, you reached for the same tool without anyone naming it properly. That was deliberate. Set rules are so useful that waiting until you had the whole model before writing a single one would have left you mute for a hundred pages.

Now you have earned the model. This chapter is the whole picture: what a set rule actually *is*, exactly how far its reach extends, and the handful of them you will use every day. It is the first of the two big ideas that turn Typst from a markup language into a styling engine. The second — show rules — is the next chapter, and the two are companions. Set rules configure; show rules transform. Get set rules solid here and the next chapter has half its work done already.

9.1 What a set rule actually is

Start with the two ways to style a piece of text, side by side. You met both in Chapter 4:

The word `#text(fill: red)[danger]` stands out.

```
#set text(fill: red)
Now every word is red.
```

These look similar and do entirely different things. The first is a **direct call**: `#text(...)` is a function, you hand it some content in brackets, and it gives that content — and *only* that content — a red fill. The word "danger" turns red; the rest of the sentence does not. When the brackets close, the effect is over.

The second is a **set rule**. It has no content in brackets, and that absence is the whole point. A set rule does not style anything directly. Instead it changes a *default*. Read `#set text(fill: red)` as a sentence: "from here on, the default fill for text is red." Every stretch of text that comes after it inherits that new default, until something says otherwise. You are not colouring one word; you are reaching into the machine and turning the "text colour" dial, and every character printed after the turn comes out in the new colour.

That distinction — one-shot versus reconfigure-the-default — is the thing to hold onto. A direct call is a verb: *do this, now, to this content*. A set rule is a setting: *change what "normal" means from this point forward*. [examples/045-a-first-set-rule/](#) puts two set rules at the top of a short document and lets the whole thing repaint itself, without a single word styled by hand.

IMPORTANT

A set rule affects everything *after* it, never anything before it, and never the set rule's own line. If a paragraph looks wrong, the rule that governs it is somewhere *above* it in the file. Read upward.

9.1.1 The anatomy

Every set rule has the same shape:

```
#set element(parameter: value)
```

- `#set` is the keyword that says "I am configuring a default, not calling a function."
- `element` is the *function* whose defaults you are changing — `text`, `par`, `page`, `heading`, `list`, and so on. These are the same functions you can call directly; a set rule just changes their defaults instead of invoking them.
- Inside the parentheses go the named parameters you want to change, and only those. Parameters you leave out keep whatever value they already had.

That last point matters more than it looks. Set rules are *additive per parameter*. Two set rules on the same element don't wipe each other out; each one only touches the parameters it names:

```
#set text(size: 12pt)
#set text(fill: navy)
```

After both lines, text is twelve-point *and* navy. The second rule changed the fill and left the size alone. You can also fold them into one call — `#set text(size: 12pt, fill: navy)` — which is tidier when you're setting several things at once. Both styles are common; pick whichever reads better in the moment.

9.2 Where a set rule reaches

A set rule applies from where you write it to the end of the *enclosing block*. What counts as "the enclosing block" is the entire trick, and it's simpler than it sounds.

At the **top level** of your file — not inside any brackets or braces — a set rule reaches to the end of the document. This is the common case. A `#set text(...)` on line 1 governs the whole file, which is exactly why the advice is to configure defaults near the top.

Inside a **content block** `[...]` or a **code block** `{...}`, a set rule is *local*: it reaches only to the closing bracket. The moment the block ends, the default snaps back to whatever it was outside. This is what lets you change a default for just one section and no further:

```
This line uses the document defaults.

#[
  #set text(fill: blue, style: "italic")
  This block is blue and italic.
]

And this line is back to the defaults.
```

The set rule lives inside the square brackets, so it governs only what's inside them. The text before is untouched (the rule hadn't happened yet); the text after is untouched (the block has closed and the default has reverted). `examples/048-scope-of-a-set-rule/` is exactly this, with plain lines top and bottom to prove the middle block is the only thing that changed.

This scoping is not a limitation to work around — it's the mechanism that makes localized styling clean. Want one appendix in a smaller font, or one quoted passage in italics, without disturbing the rest? Wrap it in a content block, set the default inside, and the change is sealed in.

TIP

If you want a run of content styled locally but you're already deep in markup, remember that `[...]` is just a content block you can drop in anywhere. Open a bracket, put your `#set` on the first line, write the passage, close the bracket. The style is contained.

9.3 A tour of the set rules you'll actually use

There are many settable elements, but a working writer leans on a small core. Here is that core, with a pointer back to the chapter where each element got its proper treatment. Nothing below is new — it's the same functions you already know, seen now through the set-rule lens.

`#set text(...)` — the typeface, size, weight, colour, language, and every other property of type (Chapter 4). The workhorse. A document almost always opens with one.

```
#set text(font: "New Computer Modern", size: 11pt, lang: "en")
```

`#set par(...)` — paragraph shape: justification, line spacing (`leading`), first-line indent, and the space *between* paragraphs (`spacing`) (Chapter 5).

```
#set par(justify: true, leading: 0.75em, first-line-indent: 1em)
```

`#set page(...)` — paper size, margins, columns, headers and footers, page numbering (Chapter 5). A set-page rule takes effect from the current page onward, so like the others it belongs near the top.

```
#set page(paper: "a4", margin: 2.5cm, numbering: "1")
```

#set heading(numbering: ...) — switch on automatic heading numbers (Chapter 3 introduced headings; the numbering is here). One rule and Typst counts your sections for you — 1, 1.1, 2, 2.1 — updating every number when you reorder or insert:

```
#set heading(numbering: "1.")
```

examples/046-heading-numbering/ shows the numbers appearing on a nested set of headings, with no counting by hand.

#set list(marker: ...) and **#set enum(numbering: ...)** — the bullet on an unordered list and the counter style on an ordered one (Chapter 3). Give `list` one marker or a tuple that alternates by nesting depth; give `enum` a numbering pattern:

```
#set list(marker: ([--], [.]))
#set enum(numbering: "a")
```

examples/047-list-and-enum-markers/ runs both, including a nested list so you can watch the two markers alternate.

#set table(...) — default strokes, cell padding (`inset`), fills, and column behaviour for every table below (Chapter 7). Set the house style once and stop repeating it on each table:

```
#set table(stroke: 0.5pt + gray, inset: 8pt)
```

#set figure(...) and **#set figure.caption(...)** — figure numbering and caption styling (Chapter 6). Note the dotted name: `figure.caption` is an element in its own right, reachable by a set rule, so you can style captions separately from the figures they sit under:

```
#set figure(numbering: "1")
#set figure.caption(separator: [ --- ])
```

#set block(...) and **#set par(spacing: ...)** — vertical rhythm. `block` governs the space around block-level things (headings, figures, lists); `par(spacing:)` governs the space between paragraphs specifically. When a document feels cramped or loose, these two are the dials.

You do not need to memorize this list. You need to recognize the pattern: *whatever you can call as a function, you can also configure with #set*. Find the element, find its parameters in the reference, set the ones you care about.

9.4 Numbering patterns, in one place

Headings, enumerations, figures, equations, and page numbers all share one notation for *how to count*, so it's worth learning once. A numbering pattern is usually a **string**, and inside that string a few characters are magic — they're the counting symbols — while everything else is printed literally.

The counting symbols:

| Symbol | Counts as | Sequence |
|--------|-------------------|-----------------|
| 1 | Arabic numerals | 1, 2, 3, ... |
| a | lowercase letters | a, b, c, ... |
| A | uppercase letters | A, B, C, ... |
| i | lowercase Roman | i, ii, iii, ... |
| I | uppercase Roman | I, II, III, ... |

Any other character in the pattern — a dot, a parenthesis, a space — is just punctuation that gets printed around the count. So:

- "1." gives 1., 2., 3. — the plainest choice, and the usual one for headings.
- "1.1" gives 1, then 1.1, 1.2 for the level below, 2, 2.1, and so on. The pattern describes *one full level's worth* of separators, and Typst extends it down the nesting.
- "i." gives i., ii., iii. — lowercase Roman, the traditional look for front-matter pages or sub-points.
- "A." gives A., B., C. — uppercase letters, common for appendices.
- "(1)" gives (1), (2), (3) — the parentheses are literal; only the 1 counts. This is the classic look for numbered equations (Chapter 8).

For anything a string can't express, the numbering can be a **function** instead — you receive the level numbers and return whatever content you like:

```
#set heading(numbering: (..nums) => {
  let n = nums.pos()
  numbering("1.1", ..n) + [ · ]
})
```

The `(..nums)` collects however many level-counters Typst hands you; `nums.pos()` turns them into an array; and you build the label from there. Most days a plain string is all you want, but when a house style demands something no string can say — Roman parts with Arabic chapters, a custom separator — the function form is the escape hatch. We return to functions properly in Chapter 14; for now, just know the door exists.

NOTE

The same patterns feed the `numbering` function directly: `#numbering("I.", 4)` prints IV. . That's occasionally handy on its own, but mostly you'll meet these patterns as the `numbering:` argument to a set rule.

9.5 Setting a default only when a condition holds

Sometimes you want a default to apply *conditionally* — draft copies double-spaced and stamped, final copies clean; a handout in a larger face than the same content printed as a book. Typst has a compact form for this: append `if` and a condition to a set rule.

```
#let draft = true

#set text(fill: red) if draft
#set par(leading: 1.5em) if draft

= Chapter one
#lorem(20)
```

Read it as: "make red text the default, *but only if* `draft` is true." Flip the `draft` variable to `false` at the top and both rules simply don't fire — the document reverts to ordinary black, single-ish spacing — without you deleting or commenting out a thing. One flag, one switch, two behaviours.

The condition is any expression that evaluates to `true` or `false`, so this composes with everything the language offers. A common pattern is a single configuration flag near the top of the file that several conditional set rules watch, so that toggling one value reshapes the whole document. That is precisely the kind of switch a template exposes to its users — which is where we're headed in Chapter 19.

WARNING

The condition is evaluated once, where the rule sits, using the variables in scope at that point. `#set text(...)` if `draft` needs `draft` to already be defined above it. If Typst complains about an unknown variable, you've put the flag below the rule that reads it.

9.6 The mental model to keep

Everything in this chapter collapses into one habit:

Configure your defaults once, near the top. Reach for a direct call only for one-offs.

If two-thirds of your headings should be one colour, don't colour two-thirds of them by hand — set the default and override the odd exception with a direct call. If your whole document is eleven-point, say so once at the top, not paragraph by paragraph. The direct call `#text(...)[...]` is for the single word that must break the pattern; the set rule is for the pattern itself. `examples/049-configuring-a-document/` is that habit in miniature: a stack of set rules at the top, then a body of plain markup that inherits the lot.

This is also the right way to *read* someone else's Typst. The set rules near the top are the document's design brief — its typeface, its margins, its numbering scheme, its list style — all declared before the content starts. Skim those and you know how the document will look before you've read a line of it.

Two forward pointers. Set rules can only change values that a parameter already accepts — a colour, a size, a boolean. When you need to change the *structure* of an element — wrap every heading in a coloured box, print figures with a rule above the caption, replace a term with a link throughout — you want a **show rule**, Chapter 10. Show rules transform;

set rules configure. And when you've assembled a satisfying stack of set rules and want to hand it to someone else — or reuse it across a dozen documents — you bundle it into a **template**, Chapter 19. A template is, to a first approximation, a well-chosen pile of set rules with a nice front door.

Coming from LaTeX. A set rule is the everyday job you'd reach for `\renewcommand`, `\setlength`, or a class-option for — changing a default like the section font, the list label, or the paragraph spacing. But there's no `\makeatletter`, no digging into a `.cls` file, no fear of clobbering an internal macro. You don't *redefine* the element; you hand its existing parameters new values, in the same syntax you'd use to call it. And scope is lexical and obvious: a rule inside `[...]` is local, the way a LaTeX group `{...}` scopes `\bfseries`, but without the ambient surprise of which commands happen to respect grouping.

Coming from Word. A set rule is Word's *styles* done right. When you modify the "Normal" or "Heading 1" style in Word, every paragraph tagged with it updates — that's the good idea Word had. A set rule is the same idea, except the definition lives in your document as one readable line instead of buried in a dialog box, it applies by *element* (every heading, every list) rather than by a label you have to remember to click, and it changes precisely at the point in the text where you wrote it.

9.7 What you've got

You now understand the first half of Typst's styling model:

- **What a set rule is** — `#set element(param: value)` changes the *default* for every following instance of that element, as opposed to a direct call `#element(...)[...]`, which styles only its own content.
- **Additivity** — set rules stack per parameter; a later rule overrides only the fields it names and leaves the rest intact.
- **Scope** — a set rule reaches from where it's written to the end of its enclosing block: document-wide at the top level, local inside `[...]` or `{...}` — which is how you restyle one section without touching the others.
- **The core set rules** — `text`, `par`, `page`, `heading`, `list`, `enum`, `table`, `figure` / `figure.caption`, and `block`, each tied back to the chapter that taught the element.
- **Numbering patterns** — the shared notation `("1.", "1.1", "i.", "A.", "(1)")` for headings, enums, figures, and equations, plus the function form for when a string won't do.
- **Conditional set rules** — `#set ... if <condition>` to make a default fire only when a flag is set, the seed of a configurable template.
- **The habit** — configure defaults once, near the top; use direct calls for one-offs; read a document's set rules as its design brief.

Next, show rules: the companion idea that lets you *transform* an element's appearance and structure, not just adjust its settings.

9.8 Exercises

Solutions are in Appendix A.

9.1. Write a short document (a heading and two paragraphs of `#lorem`) and, with two set rules at the top, make the whole thing twelve-point New Computer Modern with justified paragraphs. Confirm that adding a third paragraph inherits the same styling with no extra work.

9.2. Take the document from 9.1 and turn on heading numbers with a single set rule so the heading reads "1." before its title. Then add a sub-heading and check that it numbers itself "1.1" automatically.

9.3. Set a custom bullet marker for lists and a lettered numbering `(a)`, `(b)`, `(c)` for enumerations, each with one set rule. Write one of each list to prove they took effect.

9.4. Demonstrate scope. Write three lines: a normal line, then a content block `[...]` containing a `#set text` rule that makes its text italic and coloured, then another normal line. Compile it and confirm that only the middle block changed — the first and third lines are untouched.

9.5. (*Stretch.*) Put a `draft` flag at the top of a document (`#let draft = true`). Using conditional set rules, make the text larger and coloured *only* when `draft` is true — and, as a second effect, loosen the line spacing. Compile once with the flag `true` and once with it `false`, and confirm the document snaps between a marked-up draft and a clean final with a single edit. (Optional: add a page header that reads "DRAFT" only in draft mode — you'll need a show rule or a direct call for the header content, so treat this as a bridge to Chapter 10.)

Show rules

You want your section headings to have a thin colored line under them. It's a common look — you've seen it in a hundred reports and textbooks — and you go looking for the parameter that does it. You read the whole `heading` reference. There's `numbering`, `outlined`, `depth`, `supplement`. There is no `underline-with-a-colored-rule`.

Of course there isn't. No sane function would offer a parameter for every decorative flourish anyone might ever want; the list would never end. Set rules, from Chapter 9, can only reach the knobs the element's designer chose to expose. A colored rule under every heading isn't a knob. It's a different *shape*.

Show rules give you that shape. Where a set rule adjusts an element's settings, a show rule intercepts the element on its way to the page and lets you rebuild it into anything you like. It is the more powerful of Typst's two styling ideas, and once it clicks, the question stops being "does Typst support this look?" and becomes "how do I want to build it?"

10.1 Set tunes, show transforms

Hold the two ideas side by side, because the whole chapter lives in the difference.

A **set rule** changes an element's *parameters* — the named arguments its function accepts. `#set heading(numbering: "1.")` reaches into every heading and flips the `numbering` argument. You are configuring the element. You can only touch what the element chose to expose.

A **show rule** changes an element's *appearance* wholesale. It says: whenever you're about to display one of these, stop, hand it to me, and use what I give back instead. You are no longer configuring the heading. You are deciding, from scratch, what a heading *is* on the page — and you can wrap it, recolor it, add a line under it, box it, or replace it with something else entirely.

Set rules are a dial. Show rules are a workshop.

10.2 The function form: rebuilding an element

The most powerful form of show rule hands you the element as a value and asks for replacement content in return:

```
#show heading: it => {
  // build something using `it`, return it
}
```

Read that as: "Show every `heading` by running this function." The `it` is the heading itself, caught mid-flight. Off it you can read the element's fields — the same names you'd pass to the `heading` function — and use them to build whatever you want. The most useful heading fields are:

- `it.body` — the heading's text (as content).
- `it.level` — 1 for `=`, 2 for `==`, and so on.
- `it.numbering` — the numbering pattern, or `none` if unset.

So the colored-rule look you couldn't find a parameter for is four lines (`examples/050-show-heading-function/`):

```
#show heading: it => block(below: 0.8em)[
  #text(fill: rgb("#2b6cb0"), weight: "bold")[#it.body]
  #v(-6pt)
  #line(length: 100%, stroke: 0.6pt + rgb("#2b6cb0"))
]

= Introduction
= Method
```

Each heading now renders as a blue bold title with a thin blue rule tucked underneath. We read the heading's own text from `it.body`, wrap it in a colored `text`, nudge upward with a negative `v`, and draw a `line` the full width

of the column. Typst runs this for every heading in the document, so `Introduction` and `Method` come out matching without a second thought.

The function can do anything a normal chunk of Typst can. Branch on `it.level` to make top-level headings look different from subsections; check `it.numbering` and add a marker only when a heading is numbered; measure, box, indent, whatever the design wants. The element is just a value now, and you have the whole language.

IMPORTANT

A show rule returns the element's *entire* appearance — including whether it's a block. A heading is normally block-level: it sits on its own line with space around it. If your function returns bare inline content, you hand that layout back, and the "heading" runs straight into the paragraph after it. When you rebuild a block element, wrap your result in `block[...]` (as above) unless you deliberately want a run-in heading.

Coming from LaTeX. This is `\renewcommand` and friends, minus the danger. Redefining `\section` in LaTeX means overwriting a global macro and praying nothing else depended on the old one; reaching inside an existing command's behavior often means `\usepackage{etoolbox}` and patching it with `\patchcmd`, a genuinely fiddly business. A Typst show rule doesn't overwrite anything. It's a scoped instruction — "display headings like this, from here" — that you can even confine to one part of the document. No package, no patching, no global state to break.

10.3 The show-set form: set rules, but selective

The function form is the powerful one, and often more power than you need. Plenty of the time you don't want to rebuild an element — you just want a set rule to apply to *some* elements and not others. That's the **show-set** form, and it's the one you'll reach for most:

```
#show heading.where(level: 1): set text(size: 18pt, fill: navy)
```

Read it right to left: "apply `set text(...)` — but only to headings where the level is 1." A plain `#set text(size: 18pt)` would enlarge the entire document; a plain `#set heading(...)` can't change text size at all, because size isn't a heading parameter. The show-set form threads the needle: it aims a set rule at exactly the elements a selector picks out.

It's the safest show rule you can write. You're not returning new content, so there's nothing to accidentally break — no lost block layout, no recursion (more on that later), no forgotten field. You're just saying "these specific elements get these settings." `examples/051-show-set/` gives level-1 headings a large navy style and, with a second show-set, paints every inline `raw` snippet purple:

```
#show heading.where(level: 1): set text(size: 18pt, fill: navy)
#show raw: set text(fill: rgb("#7b2d8b"))
```

Notice the level-1 heading keeps its numbering, its spacing, its bold weight — everything you *didn't* mention. Show-set adjusts; it doesn't replace. When a design need is really "make these elements a bit different," reach for show-set before you reach for a full rebuild.

10.4 Selectors: what a rule can match

Every show rule starts with a selector — the part before the colon that decides which elements the rule catches. You've seen two already. Here's the full everyday set.

A **bare element function** matches every instance of that element:

```
#show heading: ...      // every heading
#show raw: ...          // every code snippet, inline and block
#show emph: ...         // every _emphasized_ span
#show strong: ...       // every *strong* span
#show link: ...         // every link
#show figure: ...       // every figure
#show list: ...         // every bullet list
```

Any element function can be a selector: `par`, `table`, `quote`, `ref`, `outline`, and the rest. If you can create it, you can show it.

A `.where(...)` **selector** narrows a bare element to instances whose fields match. This is how you distinguish a heading level, a specific figure kind, or a particular list:

```
#show heading.where(level: 1): ...           // only top-level headings
#show figure.where(kind: image): ...         // only image figures
#show raw.where(block: true): ...            // only block code, not inline
```

`.where` matches on the element's fields — the same names you read off `it` — so `heading.where(level: 2)` and reading `it.level == 2` inside a function are two routes to the same targets.

Put those together and one document can style each heading level on its own terms ([examples/054-show-where/](#)):

```
#show heading.where(level: 1): set text(size: 18pt, fill: navy)
#show heading.where(level: 2): it => block(above: 1em, below: 0.6em)[
  #text(fill: maroon, weight: "bold")> #it.body
]
```

Level-1 headings come out large and navy through a `show-set`; level-2 headings get a full rebuild — maroon, bold, with a `>` marker in front. The two rules coexist without interfering, because each selector catches only its own level. Mixing a `show-set` and a function-form rule in the same document is completely normal; use whichever fits each target.

10.5 Matching text, not elements

Selectors don't stop at elements. You can also match raw *text* — literal strings or regular-expression patterns — and Typst will restyle every occurrence wherever it falls.

A **string selector** matches a literal word or phrase anywhere in the document:

```
#show "TODO": text(fill: red, weight: "bold")[TODO]
```

Every `TODO` you type from then on comes out bold and red, no matter where it appears ([examples/052-show-string/](#)). The right-hand side can be content, as here, or a function that receives the matched text. That second option is handy when you want to *reuse* the matched string rather than retype it:

```
#show "Acme": smallcaps
```

Here `smallcaps` is handed the matched content and returns it in small capitals, so every mention of the product name is set consistently without you writing it out twice.

A **regex selector** matches a pattern instead of a fixed string, which is where this gets genuinely useful. Wrap a regular expression in `regex(...)` and give a function that transforms each match — the matched text is on `it.text` :

```
#show regex("[A-Z]{2,}"): it => smallcaps(lower(it.text))
```

That one line sets every acronym in the document — any run of two or more capitals — in small caps, the classic typographic treatment for `HTML`, `CSS`, `NASA`, and friends. It lowercases the match first (`lower`) and hands the result to `smallcaps`, which draws lowercase letters as small capitals. You typed the acronyms in ordinary capitals; the rule does the styling — [examples/053-show-regex/](#) runs it alongside a second rule that tints any four-digit year:

```
#show regex("[A-Z]{2,}"): it => smallcaps(lower(it.text))
#show regex("\\d{4}"): it => text(fill: rgb("#c05621"))[#it.text]
```

The result reads "the html spec of 1991" with the acronym in small caps and the year in warm orange — and it keeps working as you write, catching every new acronym and year automatically. Anywhere you'd otherwise reach for `find-and-replace`, a string or regex show rule does the job once and for good.

TIP

Regex show rules are perfect for house-style rules you'd never remember to apply by hand: small-capping acronyms, formatting phone numbers or version tags, flagging placeholder text like `XXX`, coloring a product's name. Write the rule once at the top of the file and forget about it.

10.6 The document-wide show rule

Leave the selector off entirely and something bigger happens. A show rule with no element in front of it — just `#show: ...` — matches *the entire rest of the document*:

```
#show: it => block(fill: luma(240), inset: 8pt, it)
```

Everything after that line is gathered up, handed to the function as `it`, and replaced by whatever comes back. Here the whole document lands in a gray box. The rule doesn't target headings or strong text or anything specific; it wraps the lot.

That sounds like a party trick until you realize it's the mechanism behind every template in Typst. Instead of an inline function, you point the show rule at a named one (`examples/055-document-wide-show/`):

```
#let report(body) = {
  set page(width: 12cm, height: auto, margin: 1.5cm)
  set text(font: "New Computer Modern", size: 11pt)
  set par(justify: true)
  show heading: set text(fill: navy)
  body
}

#show: report

= A tiny template
#lorem(28)
```

`report` is an ordinary function that takes the document body, applies a stack of set and show rules, and returns the body dressed in them. The line `#show: report` feeds the entire document below it into `report` as `body`. From that point the document is New Computer Modern, justified, with navy headings — a complete change of clothes from one line.

That's a template in miniature. Real ones do far more — title pages, headers, multi-argument configuration — but the engine is exactly this: a function that wraps the document, invoked by a selector-less show rule. Chapter 19 builds one properly. For now, note the shape, because you'll write `#show: something` at the top of nearly every serious document you make.

10.7 Two things that bite

Show rules are powerful, and power comes with a couple of sharp edges. Both are easy to avoid once you've seen them.

10.7.1 Recursion

A show rule watches for a kind of element. What if the rule's own output contains that same kind of element? It matches again. And again.

```
#show strong: it => strong[!#it.body!] // don't
```

The rule catches strong text and returns... new strong text, which the rule catches, which returns more strong text. Typst doesn't hang — it notices the runaway and stops with a clear error:

```
error: maximum show rule depth exceeded
= hint: maybe a show rule matches its own output
```

The fix is to not re-create the element you're matching. If you want strong text to also be red, don't build a fresh `strong`; wrap the element you were *given*:

```
#show strong: it => text(fill: red, it) // fine
```

That works because `it` is the original element, and Typst marks it as already shown — passing it back through doesn't re-trigger the rule. Building a brand-new `strong[...]` does. The rule of thumb: reuse `it` freely, but think twice before your function constructs the very element its selector is watching for.

10.7.2 Order and scope

A show rule affects only the content that comes *after* it, within the same scope. Put it at the top of the file and it governs the whole document; put it halfway down and the first half is untouched; put it inside a `{ }` block or a function body and it evaporates at the closing brace. This is the same "from here on, within here" behavior set rules have, and it's what makes the template pattern work — the rules inside `report` apply to the `body` in that function and nowhere else.

The practical consequence: if a show rule doesn't seem to be firing, check *where* it sits. A rule below the content it was meant to catch will sit there doing nothing, wondering why nobody calls.

10.8 What you've got

You can now change not just an element's settings but its whole appearance:

- **The distinction** — set rules tune an element's *parameters*; show rules replace or wrap its *appearance*. When no parameter does what you want, that's the show rule's cue.
- **The function form** — `#show heading: it => ...` catches the element as `it`, reads its fields (`it.body`, `it.level`, `it.numbering`), and returns rebuilt content. The powerful one.
- **The show-set form** — `#show heading.where(level: 1): set text(...)` aims a set rule at just the matched elements. The safest and most common one.
- **Selectors** — a bare element (`heading`, `raw`, `emph`, `link`, `figure`, `list`), `.where(field: value)` to narrow by field, a literal `"string"`, or a `regex(...)` pattern with the match on `it.text`.
- **The document-wide form** — `#show: it => ...` or `#show: template-fn` wraps the entire rest of the document. This is how templates work (Chapter 19).
- **The two traps** — a rule that re-creates the element it matches recurses (reuse `it` instead of rebuilding), and a rule only ever affects content after it, within its scope.

Between set rules and show rules you now hold both halves of Typst's styling system. Everything else — references, bibliographies, your own reusable templates — is built on this pair.

10.9 Exercises

Solutions are in Appendix A.

10.1. Write a function-form show rule that makes every level-1 heading dark green and draws a horizontal `line` above it (not below). Give the document two or three headings and confirm they all match.

10.2. Using the show-set form, make block quotes (`quote(block: true)[...]`) italic and inline `raw` code red — two separate rules. Check that ordinary body text is unaffected.

10.3. Write a string show rule that replaces every occurrence of your project's name with the name in bold small caps. Then add a regex rule that sets any all-caps acronym of two or more letters in small caps. Type a sentence that contains both and confirm each is styled correctly.

10.4. You write `#show strong: it => strong(it.body)` and Typst refuses to compile. Explain in one sentence what goes wrong, then rewrite the rule so that strong text also becomes underlined *without* triggering the error.

10.5. (*Stretch.*) Write a `#let` function `note(body)` and a selector-less `#show: note` rule that restyles the whole document: a fixed small page, a justified paragraph, a distinct heading color, and every acronym (regex) in small caps. You've just written the skeleton of a template — keep it around, you can grow it into a real one in Chapter 19.

References and cross-references

Open any book you wrote in a word processor and go hunting for the phrase "see Figure 4 above." Odds are decent that Figure 4 is now Figure 6, sitting below the sentence that points up at it, and that nobody noticed for two drafts. Every hand-typed number in a long document is a small promise to keep it up to date, and every one of those promises eventually gets broken, usually at the worst moment — the reviewer's copy, the printed proof.

Typst does not make you type the numbers. You already saw the trick twice: in Chapter 6 you labelled a figure and pointed at it with `@fig:name`, and in Chapter 8 you did the same with an equation. This chapter is where that becomes a full toolkit. You'll label *anything* worth pointing at — headings, figures, tables, equations — reference it from anywhere, bend the wording to taste, and then let Typst build a table of contents and a list of figures from the same information, all of it correct by construction and all of it in one compile.

11.1 Labels: naming a thing

A **label** is a name you pin to a piece of your document so you can refer back to it later. You write it in angle brackets, and it attaches to whatever comes *immediately before* it:

```
= Introduction <sec:intro>
```

That's a heading called `sec:intro`. The label itself prints nothing; it's an invisible handle. Attach one to a figure and you write it after the closing parenthesis; attach one to a display equation and it goes after the closing dollar sign:

```
#figure(image("chart.svg"), caption: [Revenue.]) <fig:revenue>

$ e^{i \pi} + 1 = 0 $ <eq:euler>
```

Four kinds of thing can be labelled *and* usefully referenced: headings, figures, tables (which are figures — Chapter 6), and equations. Those are the elements Typst knows how to number, and a reference is really just a way of printing a number you didn't have to type.

All your labels live in one shared namespace. A heading, a figure, and an equation cannot all be called `<results>` — the second one is a collision. That is why you keep seeing those little prefixes: `sec:` for a section heading, `fig:` for a figure, `tbl:` for a table, `eq:` for an equation. They are pure convention. Typst attaches no meaning to them and would take `<banana>` just as happily. But they keep your names unique and they tell *you*, six months later, what a label points at. Use them.

IMPORTANT

The prefix is decoration; the number comes from what the label is attached to. Pin `<fig:x>` onto an equation and a reference to it prints "Equation 3," not "Figure" anything. The label's spelling never overrides the thing's real nature.

11.2 Referencing: `@name` and `#ref`

To point at a label, put an `@` in front of its name:

```
The measurements are collected in @fig:revenue.
```

That renders as "Figure 3" (or whatever number the figure lands on), typeset as a live, clickable link straight to the figure. Two ingredients go into it: a **supplement** — the word "Figure" — and the **number**, both supplied for you.

`@name` is shorthand. The full function is `ref`, and the two are the same thing:

```
... in @fig:revenue.
... in #ref(<fig:revenue>).
```

The `@` form is what you'll use ninety-nine times out of a hundred, because it reads cleanly in the middle of a sentence. You reach for `#ref(...)` when you're already inside code, or — the real reason it exists — when you want to pass it an argument, which is the next section.

WARNING

Reference a name you never defined and Typst stops with `label <y> does not exist in the document`. That's a feature: a broken cross-reference is a compile error you fix now, not a stale "Figure ??" a reader finds later. Mind the spelling, including the prefix.

11.3 Referencing headings (numbering required)

Try to reference a heading and you may hit a wall:

```
= Introduction <sec:intro>
See @sec:intro.
```

```
error: cannot reference heading without numbering
hint: you can enable heading numbering with
`#set heading(numbering: "1.")`
```

The reason is simple once you say it out loud: a reference prints a *number*, and by default headings don't have one. Figures and equations come pre-numbered; headings don't, until you ask. So you ask, once, at the top of the file:

```
#set heading(numbering: "1.")
```

That's the set rule from Chapter 9. The `"1."` is a numbering pattern — top-level headings become 1, 2, 3; their subheadings 1.1, 1.2; and so on down. With numbering switched on, `@sec:intro` springs to life and prints "Section 1," a link to the heading. Nest a subheading under it and `@sec:background` reads "Section 1.1," tracking the outline structure exactly.

Why "Section" and not "Heading"? Because "Section" is the default *supplement* for headings, the same way "Figure" is the default for figures. Which brings us to how you change it.

`examples/056-labels-and-references/` puts all of this in one short document: numbered headings at two levels, each labelled and referenced, plus a figure reference for good measure, and the long `#ref(<sec:results>)` form alongside the `@` shorthand.

11.4 Supplements: the word before the number

The supplement is the label Typst prints ahead of the number — "Figure," "Table," "Equation," "Section." You bend it in two places, depending on how wide you want the change to reach.

For **one reference**, pass `supplement` to `ref`. This is the argument that `@` can't take, so you drop to the function form:

```
See #ref(<fig:revenue>, supplement: [Fig.]).
```

That single mention prints "Fig. 3"; every other reference to the same figure is untouched. Handy inside a tight parenthetical where the full word would crowd the line.

For **every reference to a kind of element**, and for the captions too, set the supplement on the element itself:

```
#set heading(supplement: [Chapter])
#set figure(supplement: [Illustration])
```

Now all heading references read "Chapter 1," and every figure — in its caption *and* in every reference — reads "Illustration 1." The set rule changes the element's identity across the whole document; the per-reference `supplement` overrides it for a single mention. A local override always beats the global setting, so you can set "Illustration" everywhere and still shorten one stubborn reference to "Fig."

`examples/057-supplements/` sets both globals and then overrides one figure reference back to "Fig.," so you can watch the layers stack: caption, default reference, and the one-off override, side by side.

TIP

The supplement takes content, not just a string, so it can hold anything a caption can: `supplement: [Fig.]`, `supplement: [Abb.]` for a German document, even `supplement: [§]` if you want references that read "§ 1." Whatever you put there, it appears before the number.

11.5 Customizing every reference

Setting the supplement changes the *word*. Sometimes you want to change the whole *look* — colour every reference, box it, drop the supplement and keep only the number. For that you write a show rule on `ref`, exactly the way Chapter 10 showed you show rules on headings:

```
#show ref: it => {
  let el = it.element
  if el != none and el.func() == figure {
    box(fill: luma(235), inset: (x: 4pt), radius: 2pt, it)
  } else {
    text(fill: blue, weight: "bold", it)
  }
}
```

Two things earn their keep here. The parameter `it` is the reference, and inside the rule `it.element` hands you the actual thing being referenced — the figure, the heading — so you can look at it and decide. Here `el.func() == figure` asks "is this a reference to a figure?" and styles those differently from everything else. The rest of the reference machinery still runs: because each branch ends by rendering `it`, the supplement and number you'd normally get are produced for you, and you're only wrapping them in a box or a colour. You change the packaging, not the contents.

`examples/060-customizing-references/` runs this rule over a document with two heading references and a figure reference, so the grey figure chip and the blue heading links show up together.

NOTE

`it.element` can be `none` — for the odd reference that resolves to something without an element behind it — which is why the rule guards with `el != none` before poking at it. Inspecting `it.element` is your window into *what* is being referenced; leaning on it too hard drifts into Chapter 17's territory of context and locations, but a simple `func()` check like this stays well inside what you know now.

11.6 Outlines: the table of contents

A cross-reference points at one thing. An **outline** points at all of them at once — it's how Typst builds a table of contents. One function call, no arguments required:

```
#outline()
```

Drop that near the top of a document with numbered headings and you get a contents list: every heading, in order, with its number, a row of dots, and the page it's on. It reads the same heading structure your references do, so the numbers in the table of contents and the numbers in the text always agree.

Three arguments cover almost everything you'll want to adjust:

```
#outline(
  title: [Contents], // the heading over the list (default: "Contents")
  depth: 2,          // include headings down to level 2 only
  indent: 2em,       // how far each level is indented from the last
)
```

`title` sets the words at the top — pass `title: none` to drop it entirely. `depth` limits how deep the outline goes: `depth: 2` lists parts and chapters but leaves the level-3 subsections out, which keeps a contents page from sprawling. `indent` controls the staircase; give it a length like `2em` for a fixed step, and each level of heading sits that much further right than its parent.

IMPORTANT

The outline reflects heading numbering. Switch numbering off and the entries lose their 1., 1.1. prefixes — the list still builds, but flat and unnumbered. If your contents page looks bare, check whether `#set heading(numbering: "1.")` is present.

`examples/058-table-of-contents/` sets a small page and pours in `#lorem` filler so the body runs across four pages, with `depth: 2` trimming a lone level-3 heading out of the contents. It's the whole loop in miniature: numbered headings, an outline that finds them, and page numbers that point where they should.

11.7 Lists of figures and tables

Here's the part that surprises people. `#outline()` isn't wired specifically to headings — headings are just its *default target*. Point it at a different kind of element and it collects those instead. Aim it at figures and you get a list of figures:

```
#outline(title: [List of figures], target: figure)
```

That walks the document, gathers every figure, and lists each one by its supplement, number, and caption — exactly the "List of Figures" page you'd flip to in a printed book. `examples/059-list-of-figures/` builds one from three captioned shapes.

Tables are figures too, so a list of *tables* is the same call with a narrower target. `figure.where(kind: table)` means "figures whose kind is table," which is precisely the tables and none of the images:

```
#outline(title: [List of tables], target: figure.where(kind: table))
```

Swap `table` for `image` and you'd get only the picture figures; swap in a custom `kind` you defined back in Chapter 6 and you'd get a list of just those. One function, `retargeted`, produces every "list of ..." a document could want.

11.8 Coming from LaTeX

Coming from LaTeX. The vocabulary maps almost one to one, minus the paperwork. `\label{x}` becomes `<x>`, and it attaches to the preceding element instead of needing to sit inside the float. `\ref{x}` becomes `@x`, and it already carries its supplement, so you rarely write the "Figure" by hand the way you did with `Figure-\ref{x}`. `\tableofcontents` becomes `#outline()`, `\listoffigures` becomes `#outline(target: figure)`.

The part that feels like magic is what's *gone*. No `.aux` file. No "Rerun to get cross-references right" warning. No compiling twice — or three times, when the page numbers shift the layout, which shifts the page numbers. Typst resolves every reference and every outline in a single pass, so the numbers are right the first time, every time. `\pageref`, if you're wondering, lives inside `it.element`'s location and belongs to Chapter 17; for now, the number and the link cover what `\ref` and `\pageref` did between them.

11.9 The whole point: they never lie

Step back and notice what you no longer do. You don't type figure numbers. You don't type section numbers. You don't type page numbers into a contents list. You attach names, you point at names, and Typst counts.

So when you move Section 3 above Section 1, every reference to it rennumbers, the table of contents reorders, the page numbers update, and the list of figures re-sorts — because all of them were computed from the document's real structure, not copied out of it by a tired human at 2 a.m. A cross-reference in Typst is not a note about where something *was*. It's a live question, asked fresh at every compile: *where is this now?* The answer is always current, because there's no cached copy to fall out of date.

That is the entire pitch of automated cross-referencing, and you now have all of it.

11.10 What you've got

You can now point at anything and trust the numbers:

- **Label** any heading, figure, table, or equation with `<name>`, attached to the element just before it, and keep names unique with `sec: / fig: / tbl: / eq:` prefixes drawn from one shared namespace.
- **Reference** it with `@name` in prose, or `#ref(<name>)` in code — a live, clickable supplement-plus-number that errors loudly if the label is missing.
- **Number headings** with `#set heading(numbering: "1.")`, which references to headings require, and which drives their "Section N" numbers.

- **Change the supplement** per reference with `#ref(<name>, supplement: [Fig.])` or globally with `#set figure(supplement: [...])` — local beats global.
- **Restyle every reference** with `#show ref: it => {...}`, inspecting `it.element` to treat different kinds differently while keeping the automatic number.
- **Build a table of contents** with `#outline()`, tuned by `title`, `depth`, and `indent`, reflecting your heading numbering.
- **Build a list of figures or tables** by retargeting the same function: `#outline(target: figure)` and `target: figure.where(kind: table)`.
- **Reorganize fearlessly**, knowing every number, link, and list recomputes in one compile, with no `.aux` files and no second pass.

Next, in Chapter 12, we point references outward: citations and bibliographies, where the same "name it, cite it, let the tool format it" idea handles the literature instead of your own figures.

11.11 Exercises

Solutions are in Appendix A.

11.1. Write a document with three top-level headings. Try to reference the first one with `@sec:one` *before* turning on numbering, and read the error Typst gives you. Then add `#set heading(numbering: "1.")`, recompile, and confirm the reference now prints "Section 1."

11.2. Give one of those headings a subheading and label it `<sec:sub>`. Reference it and confirm you get "Section 1.1." Then reorder the two top-level headings (cut one, paste it above the other) and recompile — verify the numbers in your references followed the move on their own.

11.3. Add a figure (a drawn shape is fine) labelled `<fig:demo>` and reference it twice: once as a plain `@fig:demo`, and once as `#ref(<fig:demo>, supplement: [Fig.])`. Confirm the first reads "Figure 1" and the second "Fig. 1," and that changing `#set figure(supplement: [Diagram])` at the top changes the first but leaves your "Fig." override alone.

11.4. Take a document with at least four headings across two levels and add `#outline()`. Then constrain it with `depth: 1` and confirm the subheadings drop out of the contents. Add a couple of captioned figures and a second outline, `#outline(title: [Figures], target: figure)`, and confirm both lists build from the same source.

11.5. (*Stretch.*) Write a `#show ref: it => {...}` rule that renders heading references and figure references differently — for instance, headings in one colour and figures wrapped in a small box — by branching on `it.element.func()`. Keep the automatic number working by ending each branch with `it`. Confirm the rule leaves the numbers correct while changing only how they look.

Citations and bibliographies

You have a paper due, and it leans on other people's work — a book here, three journal articles there, a web page you'd rather not admit you're citing. Two jobs come with that territory. In the running text you have to name your sources the way your field expects: (Sweller, 1988) if you write APA, a terse [1] if you write IEEE. And at the end you owe the reader a reference list, alphabetized or numbered, every title and volume and page in exactly the right place, italics and commas included.

Done by hand, this is a special kind of tedium. Change one source and every number after it shifts. Switch from APA to IEEE for a different journal and you reformat the lot. Anyone who has done a literature review in a word processor has felt the particular dread of the manually maintained bibliography.

Typst does the whole thing for you. You keep your sources in one small data file, you cite them by a short key, and a single line renders the list — in whatever style you name, correct down to the last comma, kept in sync automatically. This chapter gets you citing, gets your reference list looking right, and spends most of its time on APA, because that's the style this book's readers ask about most and the one it's easiest to get subtly wrong.

12.1 The shortest possible bibliography

Two ingredients: a file of sources and a call to render them. Here is the whole thing, start to finish (it's `examples/061-first-citation/`).

First, a sources file. Typst's native format is called Hayagriva, and it's plain YAML — a name, then indented fields. Save this as `refs.yml`:

```
sweller1988:
  type: article
  title: "Cognitive load during problem solving: Effects on learning"
  author: Sweller, John
  date: 1988
  parent:
    type: periodical
    title: Cognitive Science
    volume: 12
    issue: 2
  page-range: 257-285
  serial-number:
    doi: "10.1207/s15516709cog1202_4"

mcluhan1964:
  type: book
  title: "Understanding media: The extensions of man"
  author: McLuhan, Marshall
  date: 1964
  publisher: McGraw-Hill
  location: New York
```

The word at the top of each entry — `sweller1988`, `mcluhan1964` — is the *key*. It's yours to choose; a lowercase surname plus year is a common habit. That key is how you'll refer to the source from your document.

Now the document. You cite with an `@` followed by a key, exactly the way you referenced a figure or an equation in Chapter 11 — the same little symbol, doing double duty. Then one `#bibliography` call renders the list:

= A first citation

Working memory is a bottleneck for learning `@sweller1988`. The idea that

```
media themselves shape thought is older still @mcluhan1964.
```

```
#bibliography("refs.yml", style: "apa")
```

Compile that and you get a tidy page. In the prose, each `@key` becomes an author–date citation in parentheses: "...a bottleneck for learning (Sweller, 1988)." At the bottom, under a heading, sits the reference list:

Bibliography

McLuhan, M. (1964). Understanding media: The extensions of man.
McGraw-Hill.

Sweller, J. (1988). Cognitive load during problem solving: Effects on
learning. Cognitive Science, 12(2), 257–285.
https://doi.org/10.1207/s15516709cog1202_4

That's a complete, correctly formatted, alphabetized APA reference list from six lines of markup. Notice what you did *not* do: you didn't format the citation, didn't alphabetize the list, didn't italicize the journal name, didn't run a second program. You named your sources and named a style. Typst did the rest.

NOTE

The `#bibliography` call renders the list wherever you put it. That's almost always the end of the document — but it's an ordinary piece of content, so if you want a reference list halfway through, put it halfway through. One document gets one bibliography.

12.2 Where the sources live: YAML or BibTeX

Typst reads two source formats, and it doesn't care which you pick.

The first is the Hayagriva `.yaml` you just saw — Typst's own format, designed for it, and generally the cleaner of the two to write by hand. Each entry is a `type`, a `title`, an `author` (or a list of them, ["Deci, Edward L.", "Ryan, Richard M."]), a `date`, and whatever else the work needs. A journal article nests its journal under `parent`; a book names a `publisher`. The full field list lives in Hayagriva's documentation, but you can get a long way by copying an entry that resembles what you have and changing the words.

The second is BibTeX's `.bib` — the format LaTeX has used for decades. If you've ever written a paper in LaTeX, you have `.bib` files already, probably exported straight from Zotero or Mendeley. The same two sources look like this (`examples/064-bibtex-source/`):

```
@book{mcluhan1964,
  title   = {Understanding media: The extensions of man},
  author  = {McLuhan, Marshall},
  year    = {1964},
  publisher = {McGraw-Hill},
  address = {New York},
}

@article{sweller1988,
  title   = {Cognitive load during problem solving: Effects on learning},
  author  = {Sweller, John},
  year    = {1988},
  journal = {Cognitive Science},
  volume  = {12},
  number  = {2},
  pages   = {257--285},
  doi     = {10.1207/s15516709cog1202_4},
}
```

To use it, you point `#bibliography` at the `.bib` instead of the `.yaml`. Nothing else changes — the `@key` and `#cite` syntax is identical:

```
#bibliography("refs.bib", style: "apa")
```

Which should you use? If you already have `.bib` files, use them; there's no reason to convert. If you're starting fresh, the YAML tends to be less fiddly to edit by hand. Either way the output is the same, because Typst normalizes both into the same internal shape before it formats anything.

Coming from LaTeX. No `bibtex` or `biber` run. No `.aux` or `.bbl` files to shuffle, no "compile, run bibtex, compile twice more" ritual to get the numbers to settle. `#bibliography(...)` resolves in the single compile pass that produces your PDF, every time. And the `.bib` files you already own work as they are — you point at them and go.

WARNING

One YAML trap bites everyone once. A colon has special meaning in YAML, so a title with a colon in it — and academic titles love a colon — must be **quoted**: `title: "Cognitive load: Effects on learning"`. Leave the quotes off and Typst stops with a "failed to parse YAML" error pointing at the colon. When in doubt, quote the title. It never hurts.

12.3 Getting APA right

APA is the worked example of this chapter, because it's what most social-science and education writers need and because its rules are just intricate enough to be worth having automated. You ask for it with `style: "apa"` — the short name works, and so does the full "american-psychological-association" if you enjoy typing. Everything below is one file, `examples/062-apa-reference-list/`.

Give it a spread of source types — a book, journal articles with one, two, and three authors, and a web page — and cite them in a mix of prose and parentheses:

```
#cite(<sweller1988>, form: "prose") showed that instruction which
overloads working memory hurts learning rather than helping it. Feedback
is one of the strongest levers a teacher has, but only when it lands on a
manageable load @hattie2007. Motivation matters too: learners persist
when the work meets their needs for competence and autonomy @deci2000.

#bibliography("refs.yml", title: [References], style: "apa")
```

APA's in-text rules come out of that exactly as the manual prescribes:

- `#cite(<sweller1988>, form: "prose")` puts the name in the sentence and the year in parentheses: "Sweller (1988) showed...". More on `form` in a moment.
- `@hattie2007` — a source with three authors — collapses to "(Hattie et al., 2007)". APA abbreviates three-or-more-author works to the first author plus *et al.* on every in-text mention, and Typst knows the rule.
- `@deci2000` — two authors — keeps both, joined with an ampersand inside the parentheses: "(Deci & Ryan, 2000)".

The reference list obeys the same manual. It's alphabetized by first author, each entry hangs with the second line indented, journal titles and volume numbers are italic, and DOIs render as full `https://doi.org/...` links, all of which is current APA 7th edition:

References

Deci, E. L., & Ryan, R. M. (2000). The "what" and "why" of goal pursuits: Human needs and the self-determination of behavior. *Psychological Inquiry*, 11(4), 227–268. https://doi.org/10.1207/S15327965PLI1104_01

Hattie, J., Timperley, H., & Clarke, S. (2007). The power of feedback. *Review of Educational Research*, 77(1), 81–112. <https://doi.org/10.3102/003465430298487>

Notice the heading says `References`, not `Bibliography`. That's the one place APA and Typst's default disagree: Typst titles the list "Bibliography" out of the box, and APA wants "References". The fix is the `title:` argument you saw above — `title: [References]` — and it takes any content you like, so `title: [Works cited]` or a differently styled

heading is a keystroke away. Pass `title: none` to drop the heading entirely (handy when your template supplies its own).

TIP

APA expects article titles in *sentence case* — only the first word (and the word after a colon, and proper nouns) capitalized. Hayagriva prints your title as you store it, so store it in sentence case. The same goes for a `.bib` file: write the title in the case you want to see. Typst won't second-guess your capitalization, which is a feature the day you cite a work with an unusual name in it.

12.4 The many shapes of a citation

A citation isn't one thing. Sometimes the author's name belongs in your sentence ("Sweller (1988) argued..."), sometimes the whole reference belongs in parentheses at the end. Sometimes you want only the year, because you just named the author yourself. Typst calls these *forms*, and you select one with the `form` argument to `#cite`. Example `examples/063-citation-forms/` lays them all out side by side.

```
The load matters @sweller1988.
#cite(<sweller1988>, form: "prose") made the case directly.
The framework is due to #cite(<deci2000>, form: "author").
It was published in #cite(<deci2000>, form: "year").
```

Under APA, those produce, in order:

```
The load matters (Sweller, 1988).
Sweller (1988) made the case directly.
The framework is due to Deci & Ryan.
It was published in 2000.
```

The plain `@sweller1988` is the *normal* form — the parenthetical citation, and the one you'll use most. `form: "prose"` weaves the name into the sentence with the year trailing in parentheses. `form: "author"` and `form: "year"` give you just that one piece, for when you're building the citation into your own prose by hand. (There's a fifth, `form: "full"`, that drops a whole bibliography-style entry inline; you'll rarely reach for it.)

`@sweller1988` and `#cite(<sweller1988>)` are the same thing — the `@` form is just shorthand for the function with the normal form. Reach for the full `#cite(...)` when you need to pass an argument, like `form` or the page number we're about to add.

12.4.1 Pointing at a page

Cite a specific passage and you need a locator — "p. 262", "pp. 230–231". Typst calls this a *supplement*, and there are two ways to attach one.

In the `@` shorthand, put the locator in square brackets right after the key:

```
One clear statement is on @sweller1988[p.~262].
```

That renders "...on (Sweller, 1988, p. 262)". The `~` is a non-breaking space (it keeps "p." glued to "262" across a line break, a small courtesy from Chapter 3). With the function form, the same thing goes in the `supplement` argument:

```
#cite(<deci2000>, supplement: [pp.~230--231]) narrows it further.
```

giving "(Deci & Ryan, 2000, pp. 230–231)". The `--` becomes an en dash, the correct connector for a page range. Use whichever form is handier: brackets when you're writing `@key` inline, `supplement:` when you're already inside a `#cite(...)` for its `form`.

12.5 Only what you cite

By default, the reference list contains exactly the works you cited and nothing else. Add a source to your `.yaml` but never cite it, and it stays out of the list — which is what you want, most of the time, and matches what APA asks for.

Now and then you want the opposite: a complete list of everything in the file, a "further reading" section or an annotated bibliography where nothing is cited in the text at all. Pass `full: true`:

```
#bibliography("refs.yaml", title: [Further reading], full: true, style: "apa")
```


and every entry in the file appears, cited or not.

12.6 Choosing a style

The `style` argument is the whole personality of your citations. Feed it the name of a built-in style and Typst reformat every citation and the entire list to match — no other edits. The names are short strings:

- `"apa"` — American Psychological Association, author–date. Our worked example.
- `"ieee"` — numbered [1], [2]; the engineering default (and Typst's default if you name no style at all).
- `"chicago-author-date"` — Chicago's author–date variant.
- `"mla"` — Modern Language Association, common in the humanities.
- `"vancouver"` — numbered, common in medicine.

That's a handful of the ninety-odd styles Typst ships. To feel the switch, take one source file and render it two ways (examples/065-switching-styles/). With `style: "apa"` a citation reads "(Sweller, 1988)" and the list is alphabetical. Change that single string to `style: "ieee"` and the *same document* turns into "[1]", "[2]" with a numbered list, authors initials-first, titles in quotes — the full IEEE look, from one edited word. Nothing else in the file moves.

Need a style that isn't built in — a specific journal's house format, say? Instead of a name, hand `style` a path to a `.csl` file, one of the thousands of Citation Style Language definitions the academic world already maintains:

```
#bibliography("refs.yml", style: "chem-journal.csl")
```

Typst speaks CSL natively, so if your journal publishes a `.csl` (most do), you drop it next to your document and point at it.

NOTE

Style names occasionally get renamed between Typst versions. If you compile and see a *deprecation warning* naming a replacement — for instance, `"council-of-science-editors"` was superseded by a longer, version-specific name — the message tells you exactly what to switch to. Take its advice; the old name may vanish in a later release.

12.7 What you've got

You can now handle sources the way a serious document needs:

- **A sources file** in either format — Hayagriva `.yaml` (Typst's own) or BibTeX `.bib` (LaTeX's) — each source a keyed entry with a `type`, `title`, `author`, and `date`.
- **Citing by key** — `@key` in prose for the natural case, `#cite(<key>)` when you need arguments. Both resolve against the bibliography you render with `#bibliography("refs.yml", style: "apa")`.
- **APA, correctly** — author–date in-text, *et al.* for three-plus authors, an alphabetized hanging-indent reference list, retitled to "References" with `title: .`
- **Citation forms** — parenthetical (the default), `form: "prose"`, and the `"author"` / `"year"` pieces for hand-built citations.
- **Page locators** — `@key[p.~5]` in brackets or `supplement: [p.~5]` in the function.
- **Scope control** — cited works only by default, or `full: true` for the whole file.
- **Restyling in one word** — any built-in name (`"apa"`, `"ieee"`, `"chicago-author-date"`, `"mla"`, `"vancouver"`, ...) or a path to a `.csl` file, applied to citations and list alike with no other edits.

That covers the citation needs of an essay, a thesis, or a journal submission. The reference documentation for `bibliography` and `cite` lists every argument and every built-in style; now that you know the shape of the system, it reads like a menu.

12.8 Exercises

Solutions are in Appendix A.

12.1. Create a `refs.yml` with two sources of your choice — a book and a journal article. Write a short paragraph that cites each with `@key`, end it with `#bibliography("refs.yml", style: "apa")`, and compile. Confirm the in-text citations read "(Author, Year)" and the reference list is alphabetized.

12.2. Take the same document and change the bibliography's title to "References" so it matches APA. Then compile it again with `style: "ieee"` instead of `"apa"`, without touching anything else, and describe in one sentence what changed in both the in-text citations and the list.

12.3. Add a third source with *three* authors to your `refs.yml` and cite it with `@key` under `style: "apa"`. Confirm the in-text citation abbreviates to "(FirstAuthor et al., Year)" while the reference list still names all three.

12.4. Cite one of your sources in prose (name in the sentence) using `#cite(<key>, form: "prose")`, and cite another with a specific page using the bracket locator `@key[p.~10]`. Compile and check that the first reads "Author (Year)..." and the second ends "... (Author, Year, p. 10)".

12.5. (*Stretch.*) Export a `.bib` file from a reference manager (or write one by hand) with at least three real sources, including one web page. Render it with `style: "apa"` and again with `full: true` so uncited entries appear. Find a `.csl` file for a journal you care about and render the same sources through it. Note anything the CSL style does that the built-in APA style does not.

From markup to code

You have been writing Typst for twelve chapters, and the whole time a second language has been hiding behind a single character. Every `#set text(...)`, every `#lorem(30)`, every `#let draft = true` from the last chapter reached across a border into it. You used the crossings without being told they were crossings. Time to look at the map.

That border separates Typst's two halves. On one side is **markup** — the mode you have written all book, where `=` makes a heading and `*word*` makes it bold and everything you type lands on the page as content. On the other side is **code** — a small, real programming language with variables, values, arithmetic, and types. The `#` is the door between them, and it swings both ways.

This is the first chapter of Part IV, and its whole job is to make that language stop feeling like magic. If you have never programmed, good: we start from nothing, and by the end you will read `$(seats * price)` the way you now read `*bold*`. If you have programmed, this will go quickly, but skim it anyway — Typst has a couple of ideas that no other language does, and the round-trip between markup and code is one of them.

13.1 The two worlds

Here is the border in a single file (`examples/066-into-code-and-back/`):

```
This sentence is markup. Type a hash and Typst reads what follows
as one code expression: two plus two is $(2 + 2), and
#upper("shouting") comes back in capitals.
```

The plain text is markup, set as-is. Then `$(2 + 2)` happens: the hash tells Typst "the next thing is code, not words," Typst computes `4`, and drops the `4` back onto the page. `#upper("shouting")` is the same move — `upper` is a function that uppercases a string, and its result, the text `SHOUTING`, lands in the sentence. From the reader's side it is one smooth line. Underneath, you crossed into code twice and came back.

That is the first direction: **# in markup drops you into a single code expression**. Just one. The hash reaches forward exactly as far as one complete expression and no further — `#upper("shouting")` is one call, so the whole call comes along, but the space after it is markup again.

The second direction is the mirror image. Inside code, square brackets open a **content block** — a patch of markup you can write right in the middle of the language:

```
{
  let mood = "sunny"
  [The forecast is *#mood* today.]
}
```

The `{ ... }` is a block of code. The `let` line is code. But `[The forecast is *#mood* today.]` is markup again — headings, `*bold*`, the lot — with one more `#` splicing the code value `mood` back in. So the trip reads: markup → code → markup → code → markup, all in four lines. This round-trip is the mental model for the entire part. Hold onto it:

IMPORTANT

`#` takes you from markup into code. `[...]` takes you from code back into markup. Content blocks nest inside code, code nests inside content blocks, as deep as you like. Nothing about Typst's scripting makes sense until this click into place — so if it hasn't yet, reread the two snippets above before moving on.

13.2 Naming a value with `#let`

The most useful thing you can do in code is give a value a name. Typst's word for that is `let` (you met it in passing last chapter). Write a name, an equals sign, and a value:

```
#let pi = 3.14159
```

That is a **binding**: the name `pi` is now bound to the value `3.14159`. From this line onward you can write `#pi` in your markup and get the number back:

```
A circle's circumference is #pi times its diameter.
```

Notice there are no parentheses around `#pi`. The rule is simple: `#` followed by a single token — a name, a number, a function call — needs nothing extra. It is only when the expression is built from several pieces that you wrap it in parentheses so Typst knows where it ends. `#pi` is fine; `#pi * 2` would confuse the markup parser (is that `*` the start of bold text?), so you write `#(pi * 2)`. More on that in a moment.

Names earn their keep the instant a value appears twice. Example `examples/067-let-bindings/` runs a tiny course flyer on three of them:

```
#let course = "Typst for teachers"
#let seats = 24
#let price = 15

= #course

Enrolment is capped at #seats seats, at #price euro each. If every
seat sells, that is #(seats * price) euro.
```

`#(seats * price)` computes `360` from the two numbers above it. The payoff is not saving a multiplication — it is that the `360` cannot lie. Change `seats` to `30` and the total becomes `450` on the next compile, with no second edit and no chance of the heading and the arithmetic disagreeing. A value written once and referred to by name is a value that stays consistent with itself.

13.2.1 Rebinding, and where a name reaches

A binding is not carved in stone. Write `let` again with the same name and you **rebind** it — the name points at the new value from that line forward:

```
#let price = 15
Early price: #price euro.

#let price = 12
Discounted price: #price euro.
```

The first sentence prints `15`, the second prints `12`. This is the second half of example 067. Nothing above the second `let` changes; the new value takes over only from where you wrote it.

That "from where you wrote it" is the whole rule for a binding's reach. A name is visible from its definition down to the end of the block it lives in — for a top-level `let`, that means the rest of the document. Ask for a name *before* you have bound it and Typst stops with an error, because at that point in the file the name genuinely does not exist yet. If you have read Chapter 9 on set rules, this will feel familiar: like a set rule, a binding reaches forward, never backward.

TIP

Put the bindings a document depends on — a colour, a client's name, a tax rate — in a little cluster near the top of the file. Then the values that steer everything are in one place, and tuning the document is a matter of editing that block, not hunting through the prose.

13.3 Values have types

Every value in Typst has a **type** — a category that says what kind of thing it is and what you can do with it. `3.14159` is a number; `"sunny"` is a piece of text; `true` is a yes-or-no. Types are not bureaucracy; they are how Typst knows that `2 + 2` is arithmetic but `"foo" + "bar"` is gluing text together. You will rarely name a type out loud, but knowing the cast of characters makes every error message and every reference-manual page readable.

You can ask any value its type with the `type` function, and see how a value is written with `repr` (short for *representation*, a debug view). Example `examples/068-values-and-types/` lays out one literal of each everyday type beside the type Typst reports:

```
- #repr(42) --- #type(42)
- #repr(3.14) --- #type(3.14)
- #repr("hello") --- #type("hello")
- #repr(true) --- #type(true)
- #repr(12pt) --- #type(12pt)
- #repr(50%) --- #type(50%)
- #repr([bold]) --- #type([bold])
- #repr((1, 2, 3)) --- #type((1, 2, 3))
- #repr((name: "Ada")) --- #type((name: "Ada"))
```

which prints:

```
42 — int
3.14 — float
"hello" — str
true — bool
12pt — length
50% — ratio
[bold] — content
(1, 2, 3) — array
(name: "Ada") — dictionary
```

Read down that list — it is the working vocabulary of the whole language:

- **Integer** (`int`) — a whole number: `42` , `-7` , `0` .
- **Float** (`float`) — a number with a fractional part: `3.14` , `0.5` . ("Float" is short for *floating-point*, the standard way computers store such numbers.)
- **String** (`str`) — text in double quotes: `"hello"` . Note the quotes: in code, text must be quoted, because unquoted words are names. This is the opposite of markup, where text is just text.
- **Boolean** (`bool`) — one of exactly two values, `true` or `false` . The stuff of yes/no decisions, and of every `if` condition (Chapter 15).
- **Length** — a measurement: `12pt` , `2cm` , `1em` . These carry a unit, so they know that `1em` scales with the font while `12pt` does not.
- **Ratio** — a percentage: `50%` , `100%` . A fraction of something, resolved once Typst knows what that something is.
- **Content** — a piece of typeset material: the result of `[bold]` , a heading, an image, a whole chapter. Every content block `[...]` is a value of this type, which is exactly why you can bind one to a name.
- **Array** — an ordered list of values: `(1, 2, 3)` . Chapter 16.
- **Dictionary** — values stored under named keys: `(name: "Ada")` . Also Chapter 16.

Two more values round out the set, and they are special enough to name on their own: `none` and `auto` . `none` is deliberately nothing — the absence of a value, what a set rule uses to mean "no stroke," "no fill," "no header." `auto` means "decide for me" — the default many parameters take, letting Typst pick a sensible value from context. Each is the only value of its own type, a singleton. You will meet both constantly; for now just recognise them as legitimate values, not mistakes.

NOTE

`type` and `repr` are your two flashlights. When a value surprises you — a function hands back something you did not expect, or an expression won't compile — drop a `#repr(mystery)` into the page and *look* at it. Half of learning a language is learning to interrogate it, and these are the two questions that always get an answer.

13.4 Operators

With values and types in hand, you can compute. Typst's operators are the ones you would guess, and they respect the arithmetic you learned in school ([examples/069-operators/](#)):

```
- #(3 + 4 * 2)    // 11 — times binds tighter than plus
- #((3 + 4) * 2)  // 14 — parentheses override it
- #repr(6 / 2)    // 3.0 — division always gives a float
```

The four arithmetic operators are `+` , `-` , `*` , `/` , with `*` and `/` binding tighter than `+` and `-` , and parentheses to override the order when you need to. One quirk worth filing away: **division always produces a float**, even

when it comes out even. `6 / 2` is `3.0`, not `3`. If you specifically need whole-number division or a remainder, `calc.div-euclid` and `calc.rem` give them to you; most of the time the float is exactly what you want.

Comparisons ask a question and answer with a boolean:

```
- #(3 < 5)      // true
- #(3 == 3.0)   // true
- #(4 != 4)     // false
```

The full set is `==` (equal), `!=` (not equal), `<`, `>`, `<=`, `>=`. Note `==`, two equals signs, for *comparison* — a single `=` is the binding from `#let`, an entirely different job. And `3 == 3.0` is `true` because Typst compares the numbers, not their spelling.

Booleans combine with three plain-English operators — `and`, `or`, `not` — rather than symbols:

```
- #(true and false) // false
- #(true or false)  // true
- #(not true)       // false
```

Finally, the operator that does the most surprising double duty. `+` adds numbers, yes, but it is really a **joiner**, and it will join anything joinable — strings end to end, arrays into a longer array, and even content into more content:

```
- #("Lear" + "ning")      // Learning
- #repr((1, 2) + (3, 4))  // (1, 2, 3, 4)
- #([Hello ] + [world])   // Hello world
```

That last one is quietly powerful. Because content is a value and `+` joins it, you can build up a document by adding pieces together — a fact that pays off the moment you start writing your own functions in Chapter 14.

13.5 Two kinds of block

You have now seen both of Typst's blocks in passing; here they are side by side, because knowing which to reach for is half of writing clean code ([examples/070-code-vs-content-blocks/](#)).

A **content block** `[...]` produces content. Whatever markup you put between the brackets becomes a content value — one you can drop on the page immediately, or bind to a name and place later:

```
#let reminder = [Remember to *save* first.]

#reminder
```

`reminder` holds a bold-bearing sentence as a value. Writing `#reminder` sets it. You could place it three times, or hand it to a function; it is just content, sitting in a variable.

A **code block** `{...}` runs a sequence of statements and evaluates to their joined result. Reach for it when you need to do a few things — bind some names, compute — before you produce output:

```
{
  let w = 8
  let h = 5
  [The rectangle is #w by #h, so its area is ]
  [#(w * h).]
}
```

Inside the braces, the two `let` lines do their work quietly (a binding produces no content), and the two content lines are joined into a single piece of content, which the block hands back to the page: `The rectangle is 8 by 5, so its area is 40.` That is the rule for a code block's value in one sentence: **everything the block produces, joined together** — and since `let` statements produce nothing, what is left is your actual result.

So: `[...]` when you want content, `{...}` when you want to run several steps. In markup you prefix either with a hash — `#[...]`, `#{...}` — to drop into it. In code you are already inside the language, so the bare `[...]` and `{...}` are enough.

13.6 Putting values back: interpolation

Computing a value is only useful if you can get it onto the page, and you already know how: write `#` and the expression. Splicing a computed value back into the document like this is called **interpolation**, and it is the punchline of the whole chapter. `#pi`, `#{2 + 2}`, `#upper("hi")`, `#{seats * price}` — every one of these computes something and drops the result into the flowing text.

The one rule to remember is the parentheses rule from earlier, now with a reason attached. A bare `#name` or `#function(...)` is unambiguous, so it needs no parentheses. But an expression with an operator in it — `#{a + b}`, `#{price * 1.21}` — must be wrapped, because otherwise the markup parser sees the `+` or `*` and wonders whether you meant text formatting. The parentheses say "all of this is one code expression; read it as arithmetic."

Example `examples/071-interpolation/` puts every idea in this chapter to work on something you might actually print — a café receipt whose figures compute themselves:

```
#let espresso = 3
#let sandwich = 8
#let cookie = 2
#let subtotal = espresso + sandwich + cookie
#let vat = calc.round(subtotal * 0.21, digits: 2)
#let total = subtotal + vat

= Café Turing --- receipt

#table(
  columns: (lfr, auto),
  align: (left, right),
  stroke: none,
  [Espresso], [#espresso],
  [Sandwich], [#sandwich],
  [Cookie], [#cookie],
  table.hline(),
  [Subtotal], [#subtotal],
  [VAT (21%)], [#vat],
  [*Total*], [*#total*],
)
```

The prices are named once at the top. `subtotal` adds them; `vat` takes 21% and rounds to two decimals with `calc.round`; `total` adds the two. The table's right column is nothing but interpolations — `#espresso`, `#subtotal`, `#total` — so not a single number is typed twice. Change the price of a sandwich and the subtotal, the VAT, and the total all move on the next compile. That is the difference between a document you *type* and a document you *compute*, and it is the reason the rest of Part IV exists.

13.7 A first taste of methods

One last thing to plant, because you will trip over it soon. Many values carry their own little functions, called **methods**, that you call with a dot after the value:

```
- #("hello".len()) // 5
- #((1, 2, 3).len()) // 3
```

`"hello".len()` asks the string how long it is; `(1, 2, 3).len()` asks the array how many items it holds. The dot means "call this value's own function." Strings, arrays, and dictionaries come loaded with these — ways to slice, search, sort, reverse, and reshape — and they get a whole chapter of their own (Chapter 16). For now, just recognise the shape: `value.method(...)` is a value answering a question about itself.

Coming from LaTeX. TeX has "programming" too, but of a peculiar kind: it is macro *expansion*, textual substitution where `\newcommand` pastes tokens and you count expansion passes, wrestle `\expandafter`, and discover that a "variable" is really a stretch of replacement text. There are no first-class values, no types, no `2 + 2` that simply evaluates to `4`. Typst throws all of that out and gives you an ordinary expression language: values you can name, types you can inspect, operators that mean what they say, and functions that take arguments and return results. The mental weight lifts. You stop thinking about *when* things expand and start thinking about *what* they are.

13.8 What you've got

You can now read and write Typst's scripting language at a beginner's level, which is most of the way to fluent:

- **The round-trip** — `#` drops from markup into a single code expression; `[...]` drops from code back into a content block. The two nest inside each other without limit.
- **Bindings** — `#let name = value` names a value; `#name` uses it. A binding reaches from its line to the end of its block, and rebinding with a second `let` replaces it from that point on.
- **Types** — the everyday cast: `int`, `float`, `str`, `bool`, `length`, `ratio`, `content`, `array`, `dictionary`, plus the singletons `none` and `auto`. Inspect any value with `#type(x)` and see it plainly with `#repr(x)`.
- **Operators** — arithmetic (`+`, `-`, `*`, `/`, with `/` always yielding a float), comparison (`==`, `!=`, `<`, `>`, `<=`, `>=`), logic (`and` or `not`), and `+` as a universal joiner for strings, arrays, and content.
- **Blocks** — `[...]` produces content; `{...}` runs statements and evaluates to their joined result. Prefix either with `#` to enter it from markup.
- **Interpolation** — computing values and splicing them back with `#`, parenthesising any expression that is more than a single token.
- **Methods** — the `value.method(...)` shape, a taste of the machinery Chapter 16 opens up.

Next chapter turns the corner from *using* values to *making machines that produce them*: functions, the single most powerful idea in the language, and the thing every `#set`, `#table`, and `#upper` you have written was secretly one of all along.

13.9 Exercises

Solutions are in Appendix A.

13.1. Bind three values at the top of a file: your name as a string, your age as an integer, and a favourite colour as a string. Then write one sentence of markup that interpolates all three, so the sentence is assembled entirely from the bindings rather than typed out.

13.2. Predict, *without compiling*, what each of these prints, then check yourself: `#(10 - 2 * 3)`, `#repr(9 / 4)`, `#("na" + "na" + " batman")`, `#(5 <= 5)`, `#(not (3 > 4))`. For the second one, explain why the answer has a decimal point.

13.3. Write a line that uses `#type` and `#repr` to inspect three values of your own choosing, one of them a content block `[...]`. Read the reported types back and make sure each matches the value you wrote.

13.4. Build a mini invoice like example 071, but for three items of your choosing with your own prices. Compute a subtotal, add a 6% tax rounded to two decimals with `calc.round`, and a total, and interpolate all three into a small table. Then change one price and recompile to confirm every derived figure updates on its own.

13.5. (*Stretch.*) Using a single code block `#{ ... }`, bind two numbers, then produce a piece of content that states both numbers and their sum and their product in one sentence — for example, "8 and 5 sum to 13 and multiply to 40." Everything the reader sees should come out of the one block. (Hint: the block's value is its joined content, so a `[...]` line at the end carries your result out; interpolate the arithmetic inside it.)

Functions and closures

By now you've called functions in nearly every chapter of this book. `#text(fill: red)[...]`, `#image("cat.jpg")`, `#figure(...)`, `#table(...)` — every one of those is a *function*: a named thing you hand some values to, and that hands something back. You never had to think about it, because Typst wrote them for you.

This chapter is where you start writing your own.

The reason you'd want to is the oldest reason in programming: you're repeating yourself. Say you're building a worksheet, and every answer sits in the same little green box —

```
#box(fill: green, inset: 6pt, radius: 3pt)[#text(fill: white)[42]]
```

— and there are forty of them. Type that out forty times and two things happen. It's tedious, and somewhere around box twenty-three you'll fat-finger `insert` for `inset`, and one box will quietly come out wrong. The cure is to describe the box *once*, give it a name, and use the name forty times.

That's a function.

14.1 Defining a function

You already know the keyword. In Chapter 13 you used `#let` to give a name to a value:

```
#let pi = 3.14159
```

Add a parameter list in parentheses and the very same `#let` defines a function instead:

```
#let greet(name) = [Hello, #name! Welcome aboard.]
```

Read it left to right: define `greet`, which takes one thing called `name`, and whose body is the content to the right of the `=`. That `name` is a *parameter* — a stand-in for whatever you pass in when you call the function. And you call it exactly the way you've been calling built-ins all along, with a value in parentheses:

```
#greet("Ada")
#greet("Grace")
```

Each call drops its argument into the body where `name` sits and produces the finished content: "Hello, Ada! Welcome aboard." and the same for Grace. Write the recipe once, serve it as often as you like ([examples/072-defining-a-function/](#)).

A one-line body is fine when the whole function is a single expression. When it needs to do a few things first, give it a *block body* — curly braces, several statements, and the last expression as the result:

```
#let shout(word) = {
  let loud = upper(word)
  text(weight: "bold")[#loud!]
}
```

Here the function upper-cases the word, stashes it in a local variable, then returns it as bold content. There's no `return` keyword in sight, and you don't need one: a block hands back whatever its last expression evaluates to, the same rule you met for code blocks in Chapter 13. `return` does exist, and it's handy when you want to bail out early —

```
#let sign(x) = {
  if x < 0 { return "negative" }
  "zero or positive"
}
```

— but for a function that runs straight to the end, the last-expression rule is all you need, and most of the functions you'll write never say `return` at all.

14.2 Returning content, returning a value

Look again at those two functions. `greet` ends in a bracketed `[...]`, so it hands back *content* — the same kind of thing a heading or a paragraph is, something you can drop straight into your markup. `shout` does too. But a function can just as easily return a plain value:

```
#let area(width, height) = width * height
```

`area(4, 3)` gives you the number `12` — not content to display, but a value to carry on computing with. The difference isn't a special mode you switch on; it's simply *what the last expression happens to be*. End on content and you get content; end on a number, a string, or a colour and you get that. A function is just a name for "evaluate this body with these inputs," and the body decides.

14.3 Named arguments and defaults

`greet` had one positional parameter — positional because Typst matches it *by position*, first argument to first parameter. That's perfect when a function has one or two obvious inputs. It gets miserable when it has six, and you're left counting commas to work out which one is the corner radius.

The answer is *named* parameters, and you've been using them since Chapter 4 without a second thought. `#text(fill: red, size: 12pt)[...]` passes `fill` and `size` by name. You write your own the same way, by giving a parameter a default value with a colon:

```
#let tag(body, color: gray, size: 9pt) = box(
  fill: color,
  inset: (x: 6pt, y: 3pt),
  radius: 4pt,
)[#text(fill: white, size: size, weight: "bold")[#body]]
```

`body` is positional; `color` and `size` are named, and each carries a default. Now the caller picks exactly what to override and leaves the rest alone ([examples/073-named-and-default-args/](#)):

```
#tag[draft]                // gray, 9pt — every default in place
#tag(color: green)[approved] // just the colour changed
#tag(size: 12pt, color: red)[urgent] // both, in any order
```

That last line is the point of names: because Typst matches a named argument by its name and not by its position, you can order the named ones however reads best. The positional arguments are still matched by position — first positional to the first parameter, and so on — so those keep their order among themselves. By convention you write the positionals first and the named ones after, which is what nearly all Typst code does and what keeps a call easy to scan at a glance. (Typst will actually let you interleave the two, but there's little reason to make a reader hunt for which argument is which.)

Coming from LaTeX. A LaTeX custom command is `\newcommand{\tag}[1]{...}`, where the arguments are numbered — `#1`, `#2`, `#3` — and you address them by position and nothing else. Optional arguments exist, but only one, only in brackets, and defining more than a couple turns into `xparse` and a small adventure. A Typst function has as many named, defaulted parameters as you like, each with a readable name instead of a number, and no package to load. The gap only widens from here.

14.4 Passing a whole block: the content-argument trick

Notice something about that `tag` function: you called it `#tag[draft]`, with the text in square brackets, not `#tag("draft")`. That's not a special feature of `tag`. It's a piece of syntax sugar that runs through the entire language, and once you see it, a lot of Typst stops looking like magic.

Here's the rule. A content block in square brackets, written right after a function call, is passed as that call's **last positional argument**. So:

```
#greet[Ada]          is exactly #greet([Ada])
#tag(color: red)[urgent] is exactly #tag(color: red, [urgent])
```

That's the whole trick, and it's why `#figure[...]`, `#text(...)[...]`, and `#box[...]` have worked all book. `figure` is a plain function whose last parameter is the figure's body; the bracket sugar just hands it over without the parentheses and quotes. Every one of those "special" markup forms is an ordinary function call wearing comfortable clothes.

Which means the moment you write a function whose last parameter is a `body`, you get a first-class Typst component — one that takes a block of marked-up content and wraps it in whatever you want. A callout box, say (`examples/074-content-argument/`):

```
#let callout(body, title: "Note", color: blue) = block(
  fill: color.lighten(85%),
  stroke: (left: 3pt + color),
  inset: 10pt,
  radius: (right: 4pt),
  width: 100%,
)[
  #text(fill: color, weight: "bold")[#title] \
  #body
]
```

Now you have a reusable box that reads like something built in:

```
#callout[
  An ordinary note. The whole bracketed block became the `body` argument.
]

#callout(title: "Tip", color: green)[
  Give it a different title and colour with named arguments.
]
```

The content can be paragraphs, lists, images, other callouts — anything, because it's just content, and it rode in through the last positional slot. This pattern — a `body` parameter plus a few named options with defaults — is the backbone of almost every component and template you'll build in Part V. Learn it once here.

TIP

Put the `body` parameter *last* in the list and the bracket sugar works automatically. If you ever need the content in the middle, you can still pass it the long way, `#callout([text], title: "Note")`, but there's rarely a reason to. Last position, every time.

14.5 However many arguments you like

Sometimes you don't know in advance how many arguments there'll be. A function that builds a breadcrumb trail — `Home > Docs > Functions` — should take three items today and five tomorrow without you rewriting its signature. For that, Typst gives you a *variadic* parameter: prefix a parameter with `..` and it becomes a sink that swallows every leftover argument.

```
#let breadcrumbs(..items) = items.pos().join(text(fill: gray)[ > ])
```

Whatever you pass, `items` collects it. But `items` isn't an array — it's an `arguments` value, which keeps positional and named arguments in separate compartments. You pull the positional ones out with `.pos()` (an array) and the named ones with `.named()` (a dictionary). Here `.pos()` gives us the list of crumbs, and `join` stitches them together with a grey separator between each (`examples/075-variadic/`):

```
#breadcrumbs("Home", "Docs", "Functions") // Home > Docs > Functions
```

The same idea powers a function that adds up however many numbers you throw at it:

```
#let total(..nums) = nums.pos().sum()

#total(2, 4, 6, 8)    // 20
```

Variadics have a mirror image, and it's just as useful. If `..` in a *parameter* list gathers loose arguments up, `..` at a *call site* spreads a collection back out into separate arguments. Say the breadcrumbs already live in an array:

```
#let path = ("Users", "ada", "notes.typ")
#breadcrumbs(..path)    // Users > ada > notes.typ
```

Without the `..`, you'd be passing one argument — the whole array — and `breadcrumbs` would draw a single crumb containing a list. With it, the three elements arrive as three separate arguments, exactly as if you'd typed them out. Spreading works for named arguments too: spread a dictionary and each key becomes a named argument. It's the natural way to forward a bundle of options from one function into another.

14.6 Functions are values

Here's the idea that quietly makes everything above possible, and a good deal more besides: in Typst, a function is a value like any other. A number can live in a variable, get passed to a function, or be returned from one. So can a function. There is no separate, lesser category for them — `greet` is a value sitting in a variable named `greet`, no different in kind from `pi` sitting in a variable named `pi`.

Three things follow, and each is genuinely useful.

You can store a function in a variable. You've been doing it all along — `#let greet(name) = ...` is really "make a function and name it `greet`." There is an even barer syntax for writing one inline, the *arrow* form, with the parameters on the left of `=>` and the body on the right:

```
#let double = x => x * 2
#let add = (a, b) => a + b
```

`double` and `add` are ordinary functions; they just skipped the parameter-list parentheses on the way in. One parameter can go bare; two or more need parentheses around them. These are the same functions you'd get from the `#let double(x) = ...` form — two spellings, one idea.

You can pass a function to another function. If a function is a value, it can be an argument:

```
#let twice(f, x) = f(f(x))

#twice(double, 5)    // 20 — double(double(5))
```

`twice` doesn't know or care what `f` does; it just calls it twice. Hand it `double` and you get quadrupling. This is how you write a function that takes *behaviour* as a parameter — sort by a key you supply, map an operation over a list, filter by a test. You'll lean on it hard in the next few chapters.

You can return a function from a function. Which brings us to the good part.

14.7 Closures: functions that remember

A function defined inside another function can see the outer function's variables — and it keeps seeing them even after the outer function has finished and returned. A function that captures its surroundings like this is called a *closure*, and it turns an ordinary function into a factory for building specialised ones ([examples/076-functions-as-values/](#)):

```
#let multiplier(n) = x => x * n

#let triple = multiplier(3)
#let tenfold = multiplier(10)

#triple(4)    // 12
#tenfold(4)   // 40
```

Follow the `n`. When you call `multiplier(3)`, the arrow function `x => x * n` is created with `n` set to `3`, and it *remembers* that `3` from then on. The outer call is long over, but the little function it produced carries its own private copy of `n` around with it. `triple` and `tenfold` are two independent functions baked from the same mould, each holding a different captured value.

That's a closure: a function plus the slice of the world it was born into. It sounds abstract until you notice you've already brushed against the pattern. In Chapter 10 a `report` function wrapped an entire document and applied a stack of styling choices to it; a real template is that same idea grown up — a function that captures a whole design and hands it to any body you give it. In Part V you'll build them on purpose.

14.8 Pre-filling arguments with `.with()`

Closures give you factories when you need real logic. For the common case — "the same function, but with a couple of arguments already filled in" — there's a shortcut that's less typing and reads better. Every function has a `.with` method that pre-fills some of its arguments and returns a *new* function with those baked in. It's called partial application, and it's tidier than it sounds.

Start from one general component and stamp out configured variants (examples/077-with-partial-application/):

```
#let badge(body, color: gray) = box(
  fill: color, inset: (x: 6pt, y: 3pt), radius: 4pt,
)[#text(fill: white, weight: "bold")[#body]]

#let ok = badge.with(color: green)
#let danger = badge.with(color: red)

#ok[passed] #danger[failed] #badge[neutral]
```

`ok` is `badge` with `color: green` locked in; call it and you still supply the `body`, but the colour's already decided. You could write `ok` as a closure — `#let ok = body => badge(body, color: green)` — and get the identical result. `.with` is just the short, clear way to say the same thing.

It works on built-ins too, which is where it really earns its keep. Tired of retyping the same `text` settings for every title?

```
#let title = text.with(size: 16pt, weight: "bold", fill: navy)
#title[A pre-styled title]
```

One named function, defined once, and every title in the document stays consistent because they all come from the same pre-filled call. You'll find `.with` all over real Typst code for exactly this reason.

14.9 What you've got

You can now write functions, which means you can stop repeating yourself and start building your own vocabulary on top of Typst's:

- **Defining them** — `#let f(x) = [...]` for a one-line body, and `#let f(x) = { ... }` for a block whose last expression is the result. `return` bails out early; most functions don't need it.
- **What comes back** — whatever the last expression is. End on content and you get content to display; end on a number or string and you get a value to compute with.
- **Parameters** — positional (matched by order) and named-with-defaults (matched by name, optional at the call). By convention, write positionals first and named ones after; among the named ones, any order goes.
- **The content-block trick** — `#f[body]` is `#f([body])`, and content in brackets is passed as the last positional argument. This is why `#figure[...]` works, and how you build components that take a block of content.
- **Variadics** — `..args` sinks any number of arguments into an `arguments` value (`.pos()`, `.named()`); `..collection` at a call site spreads one back into separate arguments.
- **Functions as values** — store them, pass them, return them. Arrow syntax (`x => x * 2`) writes one inline.
- **Closures** — a function that captures variables from where it was defined; a factory for producing configured functions.
- **.with** — pre-fill some of a function's arguments and get back a new, specialised function.

Set and show rules gave you Typst's styling. Functions give you Typst's *language*. Everything in Part V — your own reusable components, templates, and packages — is functions, all the way down.

14.10 Exercises

Solutions are in Appendix A.

14.1. Write a function `kbd(key)` that renders a keyboard key: its argument set in bold monospace inside a small bordered box (a `box` with a `stroke`, a little `inset`, and a `radius`). Use it three times in a sentence — `#kbd("Ctrl")`, `#kbd("C")`, and so on — and confirm all three come out identical.

14.2. Give `kbd` a named parameter `fill` that defaults to `luma(240)` (a light grey) so most keys are grey, but you can override one to highlight it. Call it once with the default and once with `fill: yellow`.

14.3. Write a `callout(body, title: "Note", color: blue)` like the one in the chapter, then use the trailing content-block sugar to call it with a multi-line body containing a bullet list. Confirm the whole list arrives as `body`. Now call it a second time as `#callout([short one], title: "Aside")` — the long-hand form — and check you get the same kind of result.

14.4. Write `avg(..nums)` that returns the average of however many numbers it's given (sum divided by count — `.pos()` gives you the array; an array has both `.sum()` and `.len()`). Test it with `#avg(4, 8, 12)`, then build an array `#let scores = (10, 7, 9, 8)` and call `#avg(..scores)` by spreading it.

14.5. (*Stretch.*) Write a closure factory `heading-style(color)` that returns a function `it => ...` suitable for a `#show heading: ... rule` — one that colours the heading and draws a rule under it in the same colour. Make two of them, `heading-style(navy)` and `heading-style(maroon)`, and swap between them by changing a single line. You've just written a configurable style as a value — which is most of what a template is.

Control flow

Back in Chapter 1 there was a five-line loop that wrote its own list — the powers of two, from 2 up to 1024, without a single row typed by hand. It was a party trick at the time. You weren't meant to understand it; you were meant to notice that a Typst document can compute part of itself and file that away as interesting.

The bill for that trick comes due now, and we can afford it. You have values and types and `let` from Chapter 13. You have functions from Chapter 14. The two ideas in this chapter — *choosing* and *repeating* — are the last pieces that turn "a document with a bit of code in it" into "a document that builds itself from data." By the end you'll be able to write that powers-of-two loop yourself, and rather more besides.

There are only three things to learn: `if` for choosing, `for` and `while` for repeating, and a couple of small words (`break`, `continue`) for steering a loop mid-flight. That's the whole toolkit. It's less than most languages give you, and it's enough.

15.1 Choosing with `if`

Suppose an inventory count sits in a variable and you want the page to say one thing when the item is in stock and another when it isn't. You don't want to edit the document every time the number changes; you want it to decide. That's an `if`.

```
#let stock = 3

#if stock > 0 [
  In stock — #stock left. Ships today.
] else [
  Out of stock. We will email you when it returns.
]
```

Read it in plain English: *if* `stock` is greater than zero, print the first block; *else*, print the second. Exactly one of the two content blocks reaches the page. Change `stock` to 0 and the sentence flips, with nothing else touched. That's [examples/078-if-else/](#).

The `[ ... ]` brackets are the content blocks you already know. The `else` and its block are optional — an `if` with no `else` simply prints nothing when the condition is false, which is often precisely what you want (a "draft" watermark that appears only in draft mode, say).

15.1.1 `if` is also an expression

Here's the part that surprises people coming from other tools. In Typst, `if` doesn't just *do* something — it *is* something. It evaluates to a value, the same way `2 + 2` evaluates to 4. So you can hand its result straight to a `let`:

```
#let badge = if stock == 0 {
  "SOLD OUT"
} else if stock < 5 {
  "LOW STOCK"
} else {
  "AVAILABLE"
}
```

Whichever branch runs, its last expression becomes the value of the whole `if`, and that value lands in `badge`. No branch was "printed"; a string was *chosen*. The `else if` in the middle chains as many as you like — Typst tries each condition top to bottom and takes the first that holds, falling through to the final `else` if none do.

Notice the two flavours side by side. In markup you reach for the `[ ... ]` form and get *content*. In code — inside `{ ... }`, feeding a `let` — you get a *value*. Same keyword, same logic; the difference is whether you're building the document or computing a value to use later. Example 078 shows both in one short file.

NOTE

The braces `{ ... }` around each branch are code blocks, and a code block's value is its last expression. That's why `else { "AVAILABLE" }` yields the string — it's the last (and only) thing in the block. If you find yourself writing `{ "AVAILABLE" }` and wondering where the `return` went, there isn't one; the block just is its final value.

15.1.2 The conditions themselves

The thing between `if` and its block is any expression that comes out `true` or `false` — a **boolean**, from Chapter 13. You build those with the comparison operators, which do what they look like:

| Operator | True when |
|---------------------------------------|----------------------------------|
| <code>==</code> | the two sides are equal |
| <code>!=</code> | they are <i>not</i> equal |
| <code><</code> <code>></code> | less than / greater than |
| <code><=</code> <code>>=</code> | less-or-equal / greater-or-equal |

And you glue conditions together with three plain words — `and`, `or`, `not` — rather than the cryptic symbols some languages use:

```
#if stock > 0 and stock < 5 [ Running low. ]
#if not draft [ Final copy. ]
```

`a and b` is true only when both are; `a or b` when at least one is; `not a` flips a boolean. That's the entire logic vocabulary, and it reads aloud like the sentence it stands for.

WARNING

`=` and `==` are different animals. A single `=` assigns (`stock = 0` puts zero into `stock`); a double `==` compares (`stock == 0` asks whether it's zero). Reach for the wrong one inside an `if` and Typst will object — but the two-second confusion catches everyone at least once.

15.2 Repeating with `for`

A conditional runs a block at most once. A loop runs it once *per item* in a collection. This is where documents start writing themselves.

The shape is `for variable in collection`, then a block. Each pass binds the variable to the next element and runs the block. Here it turns an array of titles into a bullet list:

```
#let books = (
  "The Left Hand of Darkness",
  "Piranesi",
  "A Canticle for Leibowitz",
)

#for title in books [
  - #emph[#title]
]
```

Three elements, three passes, three bullets. Add a fourth book and a fourth bullet appears — you never touch the list markup again. The `[ ... ]` block is emitted once per pass, and Typst stacks them into your document. That's the first half of `examples/079-for-over-data/`.

15.2.1 Looping over a dictionary

Arrays aren't the only thing you can walk. A dictionary hands you its entries as `(key, value)` pairs, and you can unpack each pair right in the loop header:

```
#let hours = (
  Monday: "9–11",
  Wednesday: "14–16",
  Friday: "10–12",
)
```

```
)

#for (day, slot) in hours [
  / #day: #slot
]
```

The `(day, slot)` destructures each pair — `day` gets the key, `slot` gets the value — so the body reads naturally. This produces a term list (that's what `/` starts) mapping each day to its hours. Same example, second half.

15.2.2 Markup form and code form

Like `if`, a `for` comes in two flavours. The markup form uses `[ ... ]` and emits content, as above. The code form uses `{ ... }` and is where you go when a pass needs to *do* something more than print — update a running total, decide whether to skip an item, build up an array:

```
#for n in (1, 2, 3) {
  [Item #n is here. ]
}
```

Inside the `{ ... }` you're in code, but `[ ... ]` still drops you back into content whenever you want to emit some. Both forms loop identically; pick the one that fits what the body needs to do. Most of the time in a document you'll want the markup form. The moment a loop has to compute as well as print, switch to code.

15.3 Counting with `range`

Often you don't have a collection to loop over — you just want to do something a certain number of times, or count. That's what `range` is for: it manufactures a sequence of integers on demand.

It comes in three settings:

- `range(5)` — the numbers `0, 1, 2, 3, 4`. One argument means "start at zero, stop before five."
- `range(2, 6)` — `2, 3, 4, 5`. Two arguments give an explicit start and stop.
- `range(0, 10, step: 2)` — `0, 2, 4, 6, 8`. A `step:` counts in strides. It can be negative (`range(10, 0, step: -1)` counts down).

```
#for n in range(5, 55, step: 5) [#n #h(0.8em)]
```

That prints `5 10 15 ... 50` — multiples of five to fifty. Which brings us to the one thing about `range` you must not forget.

IMPORTANT

Ranges are **half-open**: they include the start and *exclude* the end. `range(5, 55, step: 5)` stops at 50, never reaching 55. This is why the Chapter 1 loop wrote `range(1, 11)` for the powers of two through *ten* — it stops just before 11. It trips up newcomers constantly. When a loop runs one pass shy of what you expected, this is nearly always why: bump the end by one.

Note that `step:` is a *named* argument. `range(0, 10, 2)` is an error — Typst won't read the bare `2` as a step. It's always `step: 2`.

15.4 Repeating with `while`

A `for` loop runs a known number of times: once per item, once per number in a range. Sometimes you don't know the count in advance — you want to keep going *until some condition changes*. That's a `while`.

```
#let x = 1
#while x <= 1000 {
  [#x #h(0.8em)]
  x = x * 2
}
```

Before each pass, Typst checks the condition. While `x` is at most 1000, it prints `x` and doubles it: `1 2 4 8 ... 512`. Once `x` reaches 1024, the condition is false and the loop stops. There are your powers of two again — this time *computed* by a loop with a running variable, not counted out by hand. [examples/080-range-and-while/](#) has both the range and the `while`.

WARNING

A `while` loop repeats until its condition is false, so *something inside the loop must make that eventually happen*. Forget the `x = x * 2` line and `x` stays `1` forever, the condition holds, and Typst spins until it gives up with an error. If a compile hangs on a `while`, look first at whether the loop can ever end.

15.5 Steering a loop: `break` and `continue`

Two small keywords let a loop change its mind partway through.

`continue` abandons the *current* pass and jumps to the next item. It's how you filter — skip the elements you don't want and keep going:

```
#let scores = (88, 42, 95, 67, 30, 71, 100, 55)

#for s in scores {
  if s < 60 { continue }
  [#s #h(0.8em)]
}
```

Any score below 60 hits `continue`, so nothing prints for it; the passing scores sail through to the `[#s]`. `break` is the bigger hammer: it abandons the loop *entirely*, no more passes. Swap the condition for `if s == 100 { break }` and the loop halts the instant it meets a perfect score. Both live in `examples/081-break-and-continue/`. The pattern — an `if` inside the loop that decides whether to `continue` past this item or `break` out of the whole thing — is how you turn a raw list into just the part you care about.

15.6 A note on scope

A `let` binding inside a block — a loop body, an `if` branch, any `{ ... }` or `[ ... ]` — lives and dies inside that block. It's invisible outside, and it's fresh on every pass of a loop:

```
#let label = "outer"
#{
  let label = "inner" // a different, local label
  [Inside the block, label is #label.]
}
Outside, label is still #label.
```

The inner `label` shadows the outer one only within the braces; the moment the block closes, the outer `label` is back, untouched. This is the same scoping you met with set rules in Chapter 9 (`#set` inside `[ ... ]` is local to it) and with function parameters in Chapter 14 — one consistent rule across the whole language. It's what lets a loop use a scratch variable per pass without those passes stepping on each other.

15.7 The payoff: data in, document out

Everything so far has been building toward one idea, and it's worth stating plainly: **you keep your data in one place, and let a loop typeset it**. The document stops being a thing you write and becomes a thing you *generate* from a small structure you can read, check, and edit at a glance.

Here's the shape of it — a workshop schedule built from a single array (`examples/082-a-self-building-document/`):

```
#let sessions = (
  ("09:00", "Welcome & setup", 30),
  ("09:30", "Markup basics", 60),
  ("10:30", "Coffee break", 15),
  ("10:45", "Styling with rules", 75),
  ("12:00", "Lunch", 60),
)

#let minutes = 0
#for (time, session, length) in sessions {
  minutes = minutes + length
}
```

```
#table(
  columns: (auto, lfr, auto),
  align: (left, left, right),
  table.header([*Time*], [*Session*], [*Min*]),
  ..for (time, session, length) in sessions {
    (time, session, [#length])
  }
)

Total scheduled: *#minutes minutes* across #sessions.len() sessions.
```

Two loops do all the work. The first walks the array once and adds up the minutes into a running total — accumulation, the same move as the doubling `while`. The second is the interesting one. That `..for` in front of the loop is the *spread* operator: the loop produces an array of cells — three per session — and the `..` unpacks them straight into the `table` call as if you'd typed every cell yourself. One row of the table per row of your data.

Now sit with what you can do. Add a session to the array and a table row appears *and* the total updates. Reorder them and the table reorders. Delete the coffee break and it's gone from both. The table markup never changes, because it isn't really markup — it's a rule for turning `sessions` into a table, applied to whatever `sessions` holds. Point it at ten sessions or a hundred and it just works. That is the whole promise from Chapter 1, delivered: describe the document once, let the machine elaborate it.

NOTE

The `#sessions.len()` at the end asks the array how many elements it has — a taste of the collection methods that are Chapter 16's whole subject. Arrays, dictionaries, and strings have a deep bag of tricks (mapping, filtering, sorting) that make "data in, document out" even tidier than a hand-written loop. This chapter gave the loops; the next gives you richer things to loop over and cleaner ways to reshape them.

Coming from LaTeX. You have almost certainly fought `\ifthenelse` from the `ifthen` package, hand-rolled `\newif\if@draft` incantations, or reached for `\foreach` from `pgffor` and discovered it doesn't quite behave like a loop in a language. Control flow in LaTeX is a patchwork bolted onto a macro expander, and it shows: expansion timing bites, and a stray brace derails everything. Typst has none of that baggage. `if` and `for` and `while` are ordinary parts of one language, they *return content* like everything else, and they nest and compose without ceremony. The powers-of-two loop is four lines because it's a loop, not a macro pretending to be one.

15.8 What you've got

You can now make a document decide and repeat:

- **if / else if / else** — choose which block runs. In markup it emits content (`#if cond [ ... ] else [ ... ]`); in code it's an *expression* whose value you can store in a `let`.
- **Conditions** — boolean expressions built from `==`, `!=`, `<`, `<=`, `>`, `>=` and joined with the plain words `and`, `or`, `not`.
- **for ... in** — run a block once per element, over an array, over a dictionary's `(key, value)` pairs, or over a `range`. Markup form emits content; code form computes.
- **range** — `range(end)`, `range(start, end)`, `range(start, end, step: n)`; **half-open**, so it stops *before* the end.
- **while** — repeat as long as a condition holds; make sure something in the loop eventually ends it.
- **break and continue** — bail out of the whole loop, or skip to the next item; the basis of filtering.
- **Scope** — a `let` inside a block is local to that block and fresh each pass.
- **The big idea** — keep data in one structure and generate the document from it with a loop; a spread `..for` feeds rows straight into a `table` or any other function that takes many arguments.

Next, in Chapter 16, we go deep on the things you've been looping over — arrays, dictionaries, and strings — and the methods that let you transform them without writing the loop by hand at all.

15.9 Exercises

Solutions are in Appendix A.

15.1. Put `#let temperature = 28` at the top of a document. With a single `#if ... else` in markup, print "Warm — leave the coat at home." when the temperature is 20 or above, and "Bring a jacket." otherwise. Change the number and confirm the sentence flips.

15.2. Rewrite the powers-of-two loop from Chapter 1 yourself: use `#for n in range(1, 11)` and, for each `n`, print a line reading `2^n =` followed by `calc.pow(2, n)`. Then predict, without running it, what `range(1, 4)` would produce instead — and check yourself.

15.3. Given `#let cart = (("Pen", 2), ("Notebook", 5), ("Stapler", 8))`, use a `for` loop over the pairs to print each item and its price as a bullet list. Then, with a second loop and a running variable, compute and print the total price. (This is Example 082 in miniature.)

15.4. Loop over `range(1, 21)` and, using `continue`, print only the numbers that are *not* divisible by 3. (Hint: `calc.rem(n, 3)` gives the remainder; a number is divisible by 3 when that remainder is `0`.)

15.5. (*Stretch.*) Build a small multiplication table as a grid. Loop the rows 1 to 5 and, inside each, loop the columns 1 to 5, printing `row * col` in each cell. Feed the cells into a `#table(columns: 5, ...)` with a spread `..for`. When it works, change one `5` to a `10` and watch the table grow in both directions with no other edit.

Arrays, dictionaries, and strings

Suppose you have a list of twelve students and you want a certificate for each. Or six products with prices, and you'd like the total printed at the bottom. Or a reading list that should always come out alphabetized, whatever order you typed it in. Until now you'd have done all of that by hand: typed each row, added the numbers on a calculator, sorted the list in your head.

This is the chapter where that stops. Chapter 15 gave you loops — a way to *repeat* work. This chapter gives you the things worth looping over. There are three of them, and they are the data half of Typst's language:

- **Arrays** — ordered lists of values: `(2, 3, 5, 7)`.
- **Dictionaries** — values you look up by name: `(name: "Ada", born: 1815)`.
- **Strings** — text, which you've used since page one and which turns out to be quietly programmable.

Once a document can hold data and reshape it, it can build itself from a single source of truth instead of from your patience. We'll end by turning a list of records into a formatted table, which is the move behind nearly every data-driven document you'll ever make.

16.1 Arrays: values in a row

An array is a sequence of values in a fixed order, written with parentheses and separated by commas. The values can be anything — numbers, strings, content, even other arrays:

```
#let primes = (2, 3, 5, 7, 11)
#let week = ("Mon", "Tue", "Wed", "Thu", "Fri")
```

You reach a value by its **position**, counting from zero, with `.at()`:

```
#week.at(0) // "Mon" — the first one
#week.at(2) // "Wed"
```

Counting from zero trips everyone up once. The first element is at position `0`, so the *fifth* is at position `4`. When you want the end of the array, though, you don't have to know how long it is: negative indices count backwards from the tail.

```
#week.at(-1) // "Fri" — the last one
#week.at(-2) // "Thu" — one before the last
```

`.at(-1)` is worth committing to muscle memory; "the last element" comes up constantly and you rarely know or care how many there are. For the two ends there are also named shortcuts, `.first()` and `.last()`, and `.len()` tells you how many elements the array holds:

```
#week.first() // "Mon"
#week.last()  // "Fri"
#week.len()   // 5
```

A **slice** pulls out a run of consecutive elements. `.slice(start, end)` takes everything from `start` up to *but not including* `end` — the same half-open convention `range` uses (Chapter 15), so the two compose without surprises:

```
#week.slice(1, 4) // ("Tue", "Wed", "Thu") — positions 1, 2, 3
```

Two arrays join with `+`, exactly like strings:

```
#(1, 2) + (3, 4) // (1, 2, 3, 4)
```

To ask whether a value is present, use `.contains()` or the `in` operator — they mean the same thing, and `in` reads more like English:

```
#week.contains("Wed") // true
#("Wed" in week)      // true
```

Two more you'll want daily: `.rev()` returns the array reversed, and `.sorted()` returns it in order. Both hand back a *new* array and leave the original alone.

```
#(3, 1, 4, 1, 5).sorted() // (1, 1, 3, 4, 5)
```

`.sorted()` orders numbers numerically and strings alphabetically. When you want to sort by something *derived* from each element — length, a particular field, the absolute value — pass a `key` function that maps each element to the thing to compare on:

```
#("kiwi", "fig", "cherry").sorted(key: w => w.len())
// ("fig", "kiwi", "cherry") — shortest word first
```

That `w => w.len()` is an arrow function (Chapter 14): given a word, it returns its length, and `sorted` compares those lengths instead of the words themselves. Keep the shape in mind — half the methods in this chapter take a little function like it.

Example `examples/083-arrays/` runs all of this on a week's worth of days.

WARNING

A one-element array **must** have a trailing comma: `("solo",)`. Without it, `("solo")` is just the string `"solo"` in ordinary parentheses — Typst can't tell the two apart otherwise. It's the one piece of array syntax that bites people, usually when an array happens to shrink to a single item.

16.1.1 Growing and shrinking an array

Arrays are values, and values in Typst don't change underneath you. But a *variable* that holds an array can be reassigned, and two methods do that conveniently: `.push()` adds an element to the end, `.pop()` removes the last one and hands it back.

```
#{
  let stack = ("wash", "rinse")
  stack.push("dry") // stack is now ("wash", "rinse", "dry")
  let done = stack.pop() // done = "dry"; stack is back to two
}
```

These only work on something you can assign to — a `let` binding, a loop variable — because they rewrite the variable. Reach for them when you're *building* an array a piece at a time inside a loop, which is a common and readable pattern:

```
#{
  let evens = ()
  for n in range(1, 20) {
    if calc.rem(n, 2) == 0 { evens.push(n) }
  }
  // evens is now (2, 4, 6, 8, 10, 12, 14, 16, 18)
}
```

That works, and you'll see it in the wild. But building an array by hand with a loop and a `push` is often the long way round — which brings us to the heart of the chapter.

16.2 The transformer trio

Three methods do most of the real work with arrays, and they're the reason data-driven Typst is pleasant rather than tedious. Each one takes a function and applies it across the whole array for you. Learn these three and you'll reach for hand-written loops far less often.

`.map(f)` runs a function over every element and collects the results into a new array. "Transform each one":


```
#(1, 2, 3, 4).map(n => n * n) // (1, 4, 9, 16)
```

`.filter(pred)` keeps only the elements for which a test returns `true`, and drops the rest. "Keep the ones that pass":

```
#(1, 2, 3, 4, 5, 6).filter(n => calc.rem(n, 2) == 0) // (2, 4, 6)
```

`.fold(init, f)` collapses the whole array down to a single value. You give it a starting value and a function of two arguments — the running result so far, and the next element — and it walks the array accumulating as it goes. "Boil it down to one thing":

```
#(1, 2, 3, 4).fold(0, (running, n) => running + n) // 10
```

Read that fold as: start the running total at `0`; for each `n`, the new total is the old total plus `n`. Summing is so common that there's a dedicated `.sum()` for exactly this case, and a `.product()` for multiplying — but `fold` is the general tool, and it does things `sum` can't, like building a string or finding a maximum.

The three compose. Because each returns an array (well, `fold` returns whatever you accumulate), you can chain them left to right and read the pipeline like a sentence:

```
#scores.map(s => s + 5).filter(s => s >= 60).sum()
// curve every score, keep the passing ones, add them up
```

Example `examples/084-map-filter-fold/` runs the trio on a set of exam scores — a five-point curve with `map`, the passing marks with `filter`, the total with `fold` and then `sum`, and the whole pipeline chained into one line.

16.2.1 A few more that pull their weight

Four supporting methods round out the toolkit:

- `.join(separator)` glues an array of strings or content into one, with a separator between: `("a", "b", "c").join(", ")` gives `"a, b, c"`. It even takes a different last separator for the Oxford-comma crowd: `.join(", ", last: " and ")` gives `"a, b and c"`.
- `.enumerate()` pairs each element with its index, handing back an array of `(index, value)` pairs — perfect when a loop needs a row number.
- `.zip(other)` stitches two arrays together element by element: `(1, 2, 3).zip(("a", "b", "c"))` gives `((1, "a"), (2, "b"), (3, "c"))`. If the arrays differ in length, it stops at the shorter one.
- `.sum()` and `.product()`, already met, are the common folds named.

WARNING

`.join()` is for strings and content, not raw numbers — `(1, 2, 3).join(", ")` is an error, because you can't glue an integer to a comma. Convert first: `(1, 2, 3).map(str).join(", ")`. The `str` function turns any value into its text, and you'll pair it with `join` constantly.

Coming from a spreadsheet or LaTeX. In a spreadsheet, `map` is dragging a formula down a column, `filter` is a filter view, and `fold` is `SUM` (or `PRODUCT`, or a pivot). The difference is that here the operation lives in your document as one readable line, versioned in plain text, instead of hidden in a grid of cells you have to click to inspect. And in LaTeX, where looping over a list means `\foreach` from PGF and a good deal of expansion-order folklore, `map` / `filter` / `fold` are ordinary method calls that return ordinary values. No package, no `\edef`, no surprises.

16.3 Dictionaries: values with names

An array indexes by position: element `0`, element `1`. That's perfect when order is the point and awkward when it isn't. If you're describing a *person*, "field number 1" tells you nothing; you want to say "their birth year." A **dictionary** stores values under names instead of positions.

```
#let ada = (name: "Ada Lovelace", born: 1815, field: "computing")
```

Same parentheses as an array, but each entry is a `key: value` pair. Read a value back with a dot and the key's name:

```
#ada.name // "Ada Lovelace"
#ada.born // 1815
```

The keys are **strings** — `name`, `born`, and `field` are stored as `"name"`, `"born"`, `"field"`. The dot form `ada.name` is the tidy shorthand, and it works whenever the key is a plain word. When the key isn't a simple identifier, or when you don't know it until the document runs — it's sitting in a variable — use `.at()` with the key as a string:

```
#ada.at("field") // "computing" — same as ada.field
#let k = "born"
#ada.at(k) // 1815 — the key came from a variable
```

To see everything a dictionary holds, three methods take it apart:

```
#ada.keys() // ("name", "born", "field")
#ada.values() // ("Ada Lovelace", 1815, "computing")
#ada.pairs() // (("name", "Ada Lovelace"), ("born", 1815), ("field", "computing"))
```

`.pairs()` is the one that pairs beautifully with a loop — each item is a `(key, value)` array you can destructure:

```
#for (key, value) in ada.pairs() [
  / #key: #value
]
```

You grow and shrink a dictionary the same way you did an array, on a variable you can assign to. `.insert(key, value)` adds or overwrites an entry; `.remove(key)` deletes one and returns the value it held:

```
{
  let record = ada
  record.insert("died", 1852)
  let year = record.remove("field") // year = "computing"
}
```

And to check whether a key exists before you reach for it, `in` works on dictionaries too — it tests keys:

```
#("died" in ada) // false
#("born" in ada) // true
```

Example `examples/085-dictionaries/` builds a record, reads it both ways, lists its keys and values, and inserts and removes fields. It also shows a small thing worth internalizing: assigning `let record = ada` and mutating `record` leaves `ada` untouched, because values are copied, not shared. No spooky action at a distance.

TIP

Reaching for a key that doesn't exist is an error that stops compilation. When a field might be missing, give `.at()` a fallback: `ada.at("died", default: "-")` returns the dash instead of blowing up. That one habit prevents most dictionary-related crashes when your data is uneven.

16.4 Strings: sequences of characters

You've written strings in every chapter — every `"a4"`, every font name, every label. They're more capable than they've let on. A string is a *sequence of characters*, and it comes with a toolkit that overlaps deliberately with the array one.

`.len()` gives the length, `.upper()` / `.lower()` (as the functions `upper()` and `lower()`) flip the case, and `.contains()` tests for a substring:

```
#upper("quiet") // "QUIET"
#"typesetting".contains("set") // true
```

`.split(separator)` is the workhorse: it breaks a string into an *array* of pieces, which is how raw text becomes data you can loop over.

```
#"Ada,Grace,Alan".split(",") // ("Ada", "Grace", "Alan")
```

`.trim()` shaves whitespace off both ends — indispensable when your data arrives with stray spaces — and `.replace(old, new)` swaps every occurrence of one substring for another:

```
#" hello ".trim()           // "hello"
#"a-b-c".replace("-", " / ") // "a / b / c"
```

`.slice(start, end)` cuts out a run of characters, just like the array version, and `.find(pattern)` returns the matched text if it's there and `none` if it isn't:

```
#"Typst".slice(0, 3)        // "Typ"
#"hello world".find("world") // "world"
```

Because a string is a sequence, you can loop straight over it, one character at a time:

```
#for c in "hi" [ #upper(c) ] // H, then I
```

Example `examples/086-strings/` puts these to work: it takes a scruffy slug `" the-wind-up-bird "`, trims it, splits it on hyphens into words, and rebuilds those words into a title-cased heading using `slice` and `upper`.

16.4.1 Strings and regex

`.replace()` and `.find()` (and its cousin `.match()`) accept more than plain text. Hand them a `regex(...)` and they match by *pattern* — the regular expressions you met in the show rules of Chapter 10. To collapse any run of spaces down to one:

```
#"too many spaces".replace(regex(" +"), " ")
// "too many spaces"
```

`.match(regex(...))` returns a dictionary describing the first match — its `text`, its `start` and `end` positions, and any captured groups — or `none` if the pattern doesn't match. It's the tool when you need to *pull* structure out of text rather than replace it: a year out of a citation, a code out of a filename.

NOTE

`.len()` and `.slice()` count *bytes*, not characters, and for plain English text those are the same number. They diverge on accented or non-Latin characters: `"café".len()` is 5, because `é` takes two bytes. When you truly need to count or walk *characters*, `"café".clusters()` gives you the array `( "c", "a", "f", "é" )` — four elements — to work with instead.

16.5 Putting it together: an array of dictionaries

Here's the shape that runs the data world, and it's just the two collections nested. An **array of dictionaries** is a list of records, each record a dictionary with the same keys — a table, essentially, one row per element:

```
#let pioneers = (
  (name: "Ada Lovelace", born: 1815, field: "computing"),
  (name: "Grace Hopper", born: 1906, field: "computing"),
  (name: "Alan Turing", born: 1912, field: "logic"),
)
```

Every tool in this chapter now points at it. Loop over the array to get one record at a time, and reach into each with the dictionary syntax:

```
#for p in pioneers [
  - *#p.name* (b. #p.born) - #p.field
]
```

Or work on the whole dataset at once. `map` a single field out of every record; `filter` to the records that match a condition; `sorted` by any field with a key function. They chain exactly as before:

```
#pioneers.map(p => p.name)           // every name
#pioneers.filter(p => p.field == "computing") // just the computer scientists
#pioneers.sorted(key: p => p.born)    // eldest first
```

Example `examples/087-array-of-dictionaries/` does all three on a slightly larger roster.

16.5.1 The payoff: data to table

Now the move the whole chapter was pointing at. The `table` function (Chapter 7) wants a flat run of cells. `map` can turn each record into a row of cells, and the **spread operator** `..` (Chapter 14) unpacks that list of rows straight into the table's arguments:

```
#table(
  columns: 3,
  table.header([Name], [Born], [Field]),
  ..pioneers.map(p => (p.name, [#p.born], p.field)).flatten()
)
```

Three things happen in that last line. `map` turns each record into a three-element array — its cells. `.flatten()` dissolves the array-of-arrays into one long array of cells, because `table` wants cells, not rows-of-cells. And `..` spreads that array into the argument list, as if you'd typed every cell by hand. Add a column, and it's one more expression in the `map`; add a row, and it's one more dictionary in the data. The table never changes.

Example `examples/088-data-to-table/` is this, fleshed out: the records sorted by birth year, a computed "century" column derived with a touch of arithmetic, grey strokes, and a `show` rule that holds the header. It is, in miniature, every report you'll ever generate from data — a spreadsheet export, a class roster, a price list — and it's maybe a dozen lines.

That's the arc: **read data into arrays and dictionaries, reshape it with `map` / `filter` / `fold`, and render the result**. Change the data, recompile, and the document redraws itself. This is the same instinct as the self-building documents in Chapter 15, now aimed at real records instead of number ranges.

One limit worth naming before you go. Everything here is computed from the data *in front of it* — the values you wrote, transformed. Some values depend instead on *where the element lands in the finished document*: the page a heading falls on, the current section number, a running total that accumulates down the pages. Those can't be known while Typst is still deciding the layout, and they need a different mechanism — `context`, the subject of Chapter 17.

16.6 What you've got

You've picked up the data half of Typst's language:

- **Arrays** — ordered lists: indexing with `.at()` (including `.at(-1)` from the end), `.first()` / `.last()` / `.len()`, `.slice()`, `+`, `.contains()` / `in`, `.rev()`, `.sorted(key: ...)`, and `.push()` / `.pop()` for building on a variable. Plus the trailing-comma rule for single-element arrays.
- **The transformer trio** — `.map()` to reshape every element, `.filter()` to keep the ones that pass, `.fold()` to collapse to one value, all driven by arrow functions and chainable end to end. With `.sum()`, `.join()`, `.enumerate()`, and `.zip()` alongside.
- **Dictionaries** — records keyed by string: `.name` and `.at("name")` access, `.keys()` / `.values()` / `.pairs()`, `.insert()` / `.remove()`, "key" in dict, and a `default:` for keys that might be missing.
- **Strings** — as programmable sequences: `.len()`, `.split()`, `.replace()`, `.trim()`, `.contains()`, `.slice()`, `.find()`, `upper()` / `lower()`, looping over characters, and `regex(...)` for pattern work.
- **The array of dictionaries** — the workhorse dataset, turned into a formatted table with `map` + `flatten` + the `.. spread`.

16.7 Exercises

Solutions are in Appendix A.

16.1. Start with `#let nums = (8, 3, 11, 6, 3, 9)`. Print its length, its first and last elements, the last element again using a negative index, and the middle two elements with a slice. Then print the array sorted, and confirm `nums` itself is unchanged afterwards.

16.2. Given `#let prices = (4.50, 12.00, 3.25, 8.75)`, use `map` to add 21% VAT to each price, `filter` the original list down to the items over 5.00, and `.sum()` to print the grand total of all four. Do it in three separate expressions, then write one chained expression that prints the total VAT-inclusive cost of just the items over 5.00.

16.3. Build a dictionary describing a book (`title`, `author`, `year`). Print a sentence like "Title by Author (year)" using field access. Then `insert` a `pages` field, `remove` the `year` field, and print the remaining keys. Finally, use `.at()` with a `default:` to print a `publisher` field that you never added, without causing an error.

16.4. Take the string `"lastname, firstname"` (for a real name of your choice). Use `.split(", ")` to break it into parts, then reassemble and print it as "Firstname Lastname" — first name first, each part capitalized. (Hint:

`.slice(0, 1)` and `upper()` capitalize a first letter; the two parts come out of `split` reversed from the order you want.)

16.5. (*Stretch.*) Write an array of at least four dictionaries describing songs, each with `title`, `artist`, and `seconds` (the duration). Render them as a table sorted by duration, with a header row, using `map` + `flatten` + the `..` spread. For extra credit, add a computed final column that shows each duration as `m:ss` (minutes and seconds) — you'll need integer division and `calc.rem`, and a little care so that a nine-second song reads `0:09`, not `0:9`.

Context, state, and counters

Here is a footer that lies:

```
#set page(footer: [Page 3 of 12])
```

It might be right. On page three of a twelve-page document, briefly, it is even true. Then you add a paragraph, the document spills onto a thirteenth page, and every footer in the book is quietly wrong. The number 12 was a promise you had no business making, because when you typed it the document wasn't finished, and a document's page count is a fact about the *finished* thing.

This is the knot the whole chapter unties. Some values simply cannot be known while you're writing the line that needs them. What page am I on? How many figures come before this one? How many pages are there in total? Each answer depends on the layout — on where things actually land once Typst has done the fussy work of filling pages. And the layout doesn't exist yet when Typst is reading your source top to bottom.

Typst's escape from this is almost cheeky: it lays the whole document out with placeholders where those values go, reads the real numbers off the finished layout, and runs the layout *again* with the numbers filled in. If the second pass shifts anything, it goes round once more. It keeps going until the numbers stop changing. You met the front door of this machinery back in Chapter 5, where a hand-built footer used `context counter(page).display()` and we promised the full story later. This is later.

17.1 The value you can't have yet

Try to print the current page number the obvious way and Typst stops you:

```
Page #counter(page).display()
```

```
error: can only be used when context is known
= hint: try wrapping this in a `context` expression
```

`counter(page)` is the running tally of pages, and `.display()` renders it — but *which* page? The answer depends on where this text lands, and at the moment Typst reads the line, that isn't settled. So it refuses to guess.

The fix is the `context` keyword:

```
Page #context counter(page).display()
```

`#context <expression>` means "don't evaluate this yet — wait until you know where we are, then run it." Inside that expression you may read layout-dependent values; outside it you may not. That's the entire rule. The error above is just Typst telling you that you stepped outside the fence.

You can wrap a single expression, as above, or a whole block of content:

```
#context [
  This is page #counter(page).display(), and there's more
  ordinary markup in here too.
]
```

Everything inside is evaluated once the context is known, so anything in there can ask position-dependent questions.

The most basic question of all is "where am I?", and `here()` answers it. It hands back a *location* — an opaque token marking this exact spot — and you can ask that location things:

```
#context [We're on page #here().page(),
  #here().position().y down the page.]
```

`here().page()` gives the page number as an integer; `here().position()` gives a dictionary with `page`, `x`, and `y`, the last two being real lengths from the top-left corner. `examples/089-context-basics/` runs this over two pages, and the same source line prints "page 1" on the first and "page 2" on the second — proof that the expression really is evaluated fresh at each spot.

NOTE

`context` is not a loop or a delay you can feel. Typst still compiles in milliseconds. Think of it less as "run this later" and more as "run this where it belongs" — the difference between reading a thermometer and reading a thermometer *in a specific room*.

17.2 Counters

A counter is a number Typst maintains for you as it walks the document. You've been benefiting from several without knowing their names. `counter(page)` counts pages. `counter(heading)` counts headings — it's what puts the "2.1" in front of a numbered subsection. `counter(figure)` is why figures say "Figure 4". Each is summoned by handing `counter` the element you want counted.

Every counter answers the same small vocabulary of questions, all of which need `context` because all of them depend on position:

- `.display()` renders the current value. Give it a numbering pattern to choose the style — `.display("i")` for roman numerals, `.display("A")` for uppercase letters, just like the `numbering` patterns from Chapter 5.
- `.get()` returns the current value as an array of numbers (an array, because a counter can have levels, like `1.2.3`). For a simple counter, `.get()` is `(3,)` and `.get().first()` is `3`.
- `.at(loc)` returns the value at some *other* location, not just "here".
- `.final()` returns the value at the very end of the document.

And two that *change* the counter rather than read it — these don't need `context`, because setting a value doesn't depend on layout:

- `.step()` bumps the counter by one.
- `.update(n)` sets it to `n`; `.update(x => x + 2)` sets it via a function of the old value. (That `x => x + 2` is a closure, from Chapter 14.)

17.2.1 Page X of Y

Now the honest version of the footer we started with. The current page is `counter(page).display()`; the total is `counter(page).final()`, the value at the end of the document. `.final()` returns an array, so `.first()` pulls the number out:

```
#set page(footer: context [
  #set align(center)
  Page #counter(page).display() of #counter(page).final().first()
])
```

That footer is correct on every page, forever, because there's no `12` hard-coded anywhere — Typst lays the document out, discovers the real total, and fills it in on the next pass. `examples/090-page-x-of-y/` runs it over three pages, and the footers read "Page 1 of 3", "2 of 3", "3 of 3", exactly as they should.

TIP

There's a shorthand for this. `counter(page).display("1 of 1", both: true)` prints the current page *and* the final total in one call, substituting both into the pattern — so "1 of 1" becomes "3 of 3". Handy, though the spelled-out version above shows what's actually happening.

17.2.2 Your own counter

The built-in counters count Typst's own elements. But you can invent one by passing a *name* — any string you like — instead of an element:

```
#let thm = counter("theorem")
```

`counter("theorem")` is now a counter that nobody but you touches. Wrap a `step` and a `display` in a small helper and every theorem numbers itself:


```
#let theorem(body) = {
  thm.step()
  block(inset: 8pt, stroke: 0.5pt + gray, radius: 3pt)[
    *Theorem #context thm.display().* #body
  ]
}

#theorem[The angles of a triangle sum to a straight angle.]
#theorem[There are infinitely many prime numbers.]
```

The first renders "Theorem 1", the second "Theorem 2", and if you slip a new one in between them tomorrow, the rest renumber without you lifting a finger. That's the whole point of a counter over a hand-typed number: it's a single source of truth that stays true. `examples/091-custom-counter/` sets three of them in bordered boxes.

NOTE

`.step()` and `.update()` sit *outside* the `context` in that helper, and `.display()` sits *inside* one. That split is the pattern to remember: changing a counter is a plain instruction Typst can carry out immediately; reading a counter is a question about position and needs a context to answer.

17.2.3 The end, from the beginning

`.final()` earns a second look, because it does something that ought to be impossible. It reads a value from the end of the document and lets you use it at the *start*:

```
#context [This worksheet has #ex.final().first() exercises.]
```

Put that line above your exercises, before a single one has been written in the source, and it still prints the right total. Typst can't know the number on the first pass — so it leaves a blank, finishes laying out, counts the exercises it found, and comes back to fill in the blank on the next pass. `examples/094-final-total/` prints "This worksheet has 4 exercises" up top, above four exercises that only appear afterward. Reading the future is just iteration wearing a disguise.

17.3 State

A counter is a number that only ever goes up by one. Sometimes you want a value that can be anything and can change to anything — a title, a color, a running subtotal. That's `state`:

```
#let part = state("part", "Part I – Basics")
```

`state` takes a name and an initial value. From then on it behaves like a counter's more flexible cousin: `.update(...)` changes it (to a value, or via a function of the old value), and inside `context`, `.get()` reads it, `.at(loc)` reads it elsewhere, and `.final()` reads it at the end.

The classic use is a running header that names the current section. Here's the shape of it:

```
#set page(header: context [
  #emph(part.get()) #h(1fr) #counter(page).display()
])

= Warm-up
#lorem(30)

#part.update("Part II – Practice")
#pagebreak()

= Drills
```

The header reads `part.get()` and prints whatever the state currently holds. On page one that's the initial "Part I — Basics"; after the update it's "Part II — Practice". `examples/092-state-basics/` runs it, and the header changes from one part to the next right where you'd want it to.

There's a subtlety hiding in that example, and it's worth seeing clearly because it trips people up.

WARNING

A header reads state as of the **top** of its page, not the bottom. If you put `#part.update(...)` in the *middle* of a page, that page's header won't see it yet — the update happens below where the header was drawn. That's why the example updates the state and *then* breaks the page: the change lands at the very end of page one, so page two's header, drawn at the top, already has the new value. Update-then-break is the reliable order. (For a header that must track sections starting mid-page, you'll want `query`, a Chapter 21 tool.)

17.3.1 When it won't settle

The iterate-until-stable trick has an obvious failure mode, and Typst is candid about it. If a value depends on itself in a way that never stops changing, the passes never agree, and after five tries Typst gives up:

```
#let s = state("s", 1)
#context s.update(s.final() + 1) // don't do this
```

Read that update: set `s` to one more than its own final value. But changing `s` changes its final value, which changes the update, which changes the final value... Each pass makes the number bigger, forever. Typst runs five rounds, watches 1, 2, 3, 4, 5 march past without settling, and prints:

```
warning: document did not converge within five attempts
warning: value of `state("s")` did not converge
```

The document still compiles — you get *a* number — but it's meaningless, and the warning is telling you so. The cure is always the same: make sure a value never depends, however indirectly, on its own final result. Counters and states are for reading layout, not for feeding themselves.

17.4 measure and layout

The contextual values so far all answer "where am I in the document?" Two more tools answer a different question — "how big is this, and how much room do I have?" — and they round out the chapter.

`measure` tells you a piece of content's size *before* it's placed:

```
#context {
  let size = measure[A considerably longer heading]
  size.width // a length, e.g. 118pt
}
```

It hands back a dictionary with `.width` and `.height`. That lets you size something to fit its own contents — for instance, draw a rule exactly as wide as the text above it:

```
#let ruled(body) = context {
  let size = measure(body)
  stack(spacing: 3pt,
    body,
    line(length: size.width, stroke: 1.5pt + orange))
}
```

Short text gets a short rule, long text a long one, each underline matched to its own words. [examples/093-measure-and-layout/](#) shows two of them stacked.

`layout` comes at it from the other side. Instead of measuring your content, it tells you the size of the *region* your content is about to fill, and lets the content adapt:

```
#layout(size => {
  if size.width < 5cm {
    // narrow: stack the tags vertically
  } else {
    // wide: lay them in a row
  }
})
```

`layout` takes a function; Typst calls it with the available `size` and uses whatever content comes back. The same block of tags in the example sits in a tidy row at full page width but stacks into a column inside a four-centimeter box — responsive layout, decided at compile time by the room actually available.

NOTE

`layout` establishes its own context, so you don't write `context` in front of it — the `size` it hands you is already a resolved, contextual value. `measure`, by contrast, reads the surrounding context, so it needs a `context` around it like everything else in this chapter.

17.5 Tying it together

Step back and the whole of Typst's "magic" is this one idea wearing different hats. The running header that knows its chapter: `state`. "Figure 7" and "Table 3": counters, stepped by show rules you'll write in Chapter 10's spirit. The table of contents with correct page numbers: counters read with `.at()`. "Page 4 of 20": a counter and its `.final()`. A box that hugs its caption: `measure`. A template that reflows for a narrow column: `layout`. All of it rests on the same bargain — Typst lays the document out, reads the truth off the result, and goes round again until the truth stops moving.

You've been living inside that bargain since Chapter 1. Now you can write your own clauses.

Coming from LaTeX. This chapter quietly replaces a whole shelf of LaTeX machinery. `\thepage` and friends, `\newcounter` / `\stepcounter` / `\the...`, the `\label` – `\ref` – `\pageref` trio, and above all the notorious two-pass dance where LaTeX writes numbers into an `.aux` file on one run and reads them back on the next (so you compile twice, or three times, and learn to ignore "Rerun to get cross-references right"). Typst folds every bit of that into `context` plus counters and state, and does the iterating itself, invisibly, in a single command. No `.aux`, no rerun, no guessing how many passes you need. You just ask for the value inside a `context` and Typst makes sure it's the right one.

17.6 What you've got

You can now reach the values that depend on the finished page:

- **context** — `#context <expr>` defers a read until Typst knows *where* it is. Layout-dependent values live inside it; outside it, they're an error.
- **here()** — the current location, with `.page()` and `.position()` for the page number and coordinates.
- **Counters** — `counter(page)`, `counter(heading)`, and custom `counter("name")`, with `.step()`, `.update()`, `.display()`, `.get()`, `.at(loc)`, and `.final()`.
- **"Page X of Y"** — `counter(page).display()` with `counter(page).final()`, or the `.display("1 of 1", both: true)` shorthand.
- **State** — `state("name", initial)` for values that vary through the document, with `.update()`, `.get()`, `.at()`, and `.final()`, and the update-then-break rule for headers.
- **Convergence** — Typst iterates until values settle, and warns when they never do; never let a value depend on its own final result.
- **measure and layout** — the size of a piece of content, and the size of the room it's landing in, for content that fits and adapts.

That's the last hard idea in Part IV, and the engine behind an enormous amount of what Typst does for you. Part V puts it to work: your own functions (Chapter 18), templates (Chapter 19), and packages (Chapter 20) all lean on this chapter to build things that number, label, and lay themselves out.

17.7 Exercises

Solutions are in Appendix A.

17.1. Make a two-page document with a small fixed page size. In the body of each page, print a sentence that names the current page number using `#context counter(page).display()`. Confirm the first page says "1" and the second says "2" — from the same source line if you can arrange it.

17.2. Build a footer that reads "p. X / Y" — current page, a slash, total pages — centered, using `counter(page).display()` and `counter(page).final()`. Give the document at least three pages and check every footer.

17.3. Define a custom counter `counter("figure-note")` and a helper `note(body)` that steps it and prints "Note 1", "Note 2", ... in bold ahead of the body. Set four notes and confirm they number 1 through 4. Then move the third note above the first and confirm they renumber themselves.

17.4. Add a summary line at the *top* of the document from 17.3 that reads "This page has N notes", where N is filled in with `.final()`. Explain, in one sentence, how a number from the end of the document can appear at the top.

17.5. (*Stretch.*) Give a document a running header showing the current section title via `state`. Use two sections, each starting on its own page, and get the header right on both pages — including the subtlety about *when* the state update has to happen relative to the page break. Then, for extra credit, deliberately break it: write a `state` whose update depends on its own `.final()`, compile, and read the convergence warning Typst gives you.

Your own functions

You wrote a `callout` in Chapter 14. It was good. You used it, liked it, and moved on. Two weeks later you start a new document — a different report, a different folder — and you want that same callout. So you open the old file, scroll past three hundred lines of prose to find it, copy the twelve lines of the definition, paste them into the top of the new file, and carry on.

Do that four times and you have four copies of `callout` in four documents. Then you improve one of them — a nicer colour, a bit more padding — and now you have three documents that are subtly wrong and one that's right, and no way to tell which is which without reading all four.

This is the moment a function wants to leave home. Chapter 14 taught you to *write* functions; this chapter is about where to *keep* them. The answer is a file of their own, and a couple of small keywords — `import` and `include` — for wiring that file into any document that needs it. By the end you'll have a little personal library that every future document can borrow from, and a single place to fix the callout when you finally get the padding right.

18.1 Machinery in one file, content in another

Start with the smallest possible split. Here is a function in a file all by itself, `helpers.typ` :

```
// helpers.typ
#let callout(body, title: "Note", color: blue) = block(
  fill: color.lighten(85%),
  stroke: (left: 3pt + color),
  inset: 10pt,
  radius: (right: 4pt),
  width: 100%,
)[
  #text(fill: color, weight: "bold")[#title] \
  #body
]
```

That's the whole file. Compile it on its own and you get a blank page, which is exactly right: `helpers.typ` doesn't say anything, it only *defines* something. It's machinery, not content.

Next to it sits `main.typ`, the actual document, which borrows the function:

```
// main.typ
#import "helpers.typ": callout

= Release notes

#callout[
  The `callout` function isn't defined anywhere in this file — it was
  imported from `helpers.typ`, which sits right beside it.
]
```

The `#import` line reads the other file, finds the name `callout` in it, and makes that name available here as if you'd defined it locally. Everything below uses `callout` without knowing or caring that it lives next door. That's Example 095 (`examples/095-splitting-into-files/`), and it is the entire idea of this chapter in eight lines. The rest is refinement.

The path in quotes is relative to the file doing the importing. Because `main.typ` and `helpers.typ` are in the same folder, `"helpers.typ"` is all you need. A helper a folder up would be `"../helpers.typ"`; one in a subfolder, `"lib/helpers.typ"`. The same rules as any file path.

NOTE

A `.typ` file that you import is called a *module*. That's the proper Typst word for it, and you'll see it in error messages. A module is just a file whose top-level `#let` definitions can be pulled into other files.

18.2 The three ways to import

`#import` has three forms, and the difference between them is entirely about *which names land in your document*. Take a small library with two public names — an accent colour and a keycap function:

```
// lib.typ
#let accent = rgb("#2b6cb0")
#let kbd(key) = box(
  fill: luma(245), stroke: 0.5pt + luma(160),
  inset: (x: 4pt, y: 1pt), radius: 3pt, baseline: 2pt,
)[#text(font: "DejaVu Sans Mono", size: 8pt)[#key]]
```

Example 096 (`examples/096-import-forms/`) imports this same file all three ways so you can see them side by side.

18.2.1 Named: take exactly what you ask for

List the names you want after a colon:

```
#import "lib.typ": accent, kbd
```

Now `accent` and `kbd` exist in your document, and nothing else does. This is the everyday form. It's explicit — a reader can see at a glance precisely what came from where — and it keeps your namespace uncluttered. When you know which handful of things you need, name them.

18.2.2 Star: take everything

A `*` means "import every public name in the module":

```
#import "lib.typ": *
```

After this, `accent` and `kbd` are both available, along with anything else `lib.typ` happens to define now or in the future. It's convenient when you're using most of a library and don't want to maintain a list. The cost is that you no longer control what enters your namespace. If `lib.typ` defines a `total` and your document already has a `total`, one silently shadows the other, and you get to spend an afternoon finding out why. Reach for `*` with libraries you know well; be wary of it with large or unfamiliar ones.

18.2.3 Namespaced: take the module, keep the dot

Leave off the colon entirely and you import the module *itself*, bound to a name taken from the filename:

```
#import "lib.typ"

#lib.kbd("Esc")
#text(fill: lib.accent)[accented]
```

Nothing from inside the module lands in your namespace — only the name `lib` does. You reach through it with a dot: `lib.kbd`, `lib.accent`. This is the safest form, because it can't collide with anything: your `total` and the library's `total` are your `total` and `lib.total`, unmistakably different. The small tax is a `lib.` in front of every use, which for a library you lean on heavily can get wordy. For a document that pulls from several libraries at once, that little prefix is often worth it — you can always tell which house a function came from.

TIP

No single form is correct. Named for the few things you use often; namespaced when clarity or clash-avoidance matters; star when you're deep inside one library and the convenience wins. Most real documents mix named and namespaced and almost never need star.

NOTE

You can rename on the way in with `as`: `#import "lib.typ": kbd as key` brings `kbd` in under the name `key`. Handy when two libraries both export a `callout` and you need them both in one file.

18.3 Building a small style library

One function in a file is a helper. A handful of functions plus the constants they share is a *library*, and that's where this pays off. The pattern is always the same: constants at the top, components below, all in one file you import from.

Here's `lib.typ` from Example 097 ([examples/097-a-style-library/](#)), the centerpiece of the chapter. First the theme — the values every component draws from:

```
// lib.typ - theme constants
#let brand = rgb("#2b6cb0") // headings, rules, accents
#let muted = luma(110)      // captions, secondary text
#let gap   = 0.9em         // standard vertical breathing room
```

These are the knobs. Put them at the top, name them well, and the entire look of every document you build on this library is decided in three lines. Want a green house style instead of blue? Change `brand` once; everything downstream follows.

Then the components, each one leaning on those constants rather than hard-coding a colour:

```
// lib.typ - components
#let callout(body, title: "Note", color: brand) = block(
  fill: color.lighten(85%),
  stroke: (left: 3pt + color),
  inset: 10pt, radius: (right: 4pt), width: 100%,
)[#text(fill: color, weight: "bold")[#title] \ #body]

#let kbd(key) = box(
  fill: luma(245), stroke: 0.5pt + luma(160),
  inset: (x: 4pt, y: 1pt), radius: 3pt, baseline: 2pt,
)[#text(font: "DejaVu Sans Mono", size: 8pt)[#key]]

#let framed(body, caption: none) = figure(
  block(stroke: 0.5pt + muted, inset: 10pt, radius: 4pt, width: 100%)[#body],
  caption: caption,
)
```

Three components — a titled note box, a keycap, and a labeled figure — and each defaults to a theme constant. `callout`'s colour defaults to `brand`; `framed`'s frame uses `muted`. A caller who overrides nothing gets the house style for free; a caller who wants a one-off green callout still can, with `callout(color: green)[...]`.

Now `main.typ` is nothing but content and one import:

```
// main.typ
#import "lib.typ": brand, callout, kbd, framed

#show heading: set text(fill: brand) // a set rule using a library constant

= Keyboard shortcuts

#callout[
  Everything on this page — the heading colour, this box, the keycaps
  below — comes from `lib.typ`. Nothing is styled twice.
]

To copy, press #kbd("Ctrl") + #kbd("C").

#framed(caption: [A framed panel from the same library.])[
  #callout(title: "Tip", color: green)[Components nest happily.]
]
```

Notice the imported `brand` doing double duty: it colours a callout by default *and* feeds a `set` rule for headings, right there in the document. A library constant is an ordinary value — you can use it anywhere a value goes, not only inside the library's own functions.

That's the whole arc of the chapter in one example. The machinery lives in `lib.typ`; the document imports the few names it needs; the look is decided once, centrally, and every document built on the library inherits it. Improve the callout tomorrow and all of them improve together — the copy-paste drift from the opening of this chapter simply cannot happen, because there's only ever one copy.

18.4 Bundling a whole look into one function

The library above hands you components you place by hand. Sometimes you want the opposite: apply an entire look to a document in one stroke — page size, fonts, heading style, the lot — without touching any of it individually. You already have the tool for this. It's a function whose body is a stack of `set` and `show` rules, wrapped around a `body` parameter.

Put it in its own file, `look.typ` (Example 099, `examples/099-a-reusable-look/`):

```
// look.typ
#let brand = rgb("#7c3aed")

#let report(body) = {
  set page(width: 13cm, height: auto, margin: 1.6cm)
  set text(font: "Libertinus Serif", size: 11pt)
  set par(justify: true)

  set heading(numbering: "1.1")
  show heading.where(level: 1): set text(fill: brand, size: 17pt)
  show heading.where(level: 2): set text(fill: brand)

  body // hand the rest of the document back, now that the rules are in force
}
```

`report` takes a `body`, sets up the world it wants — page, text, headings — and returns `body` at the end so it inherits all of it. Every `set` and `show` rule inside the function applies to whatever `body` turns out to be.

The trick is how you apply it. You *could* write `#report[...huge block of the whole document...]`, but there's a cleaner idiom you met in passing with `show` rules:

```
// main.typ
#import "look.typ": report

#show: report

= Field notes
An entire look arrives from one line. Swap in a different look function and
the document reskins without touching a word of content.

== Method
...
```

`#show: report` means "take everything from here to the end of the document, and pass it through `report`." The rest of the file becomes the `body` argument, and all of `report`'s rules take hold across the whole document. One line at the top and the document is styled.

This is the seam where a library of components becomes a *template*, and it's exactly where Chapter 19 picks up. A template is this same `#show: function` move, grown up: one function that captures an entire design and applies it to any content you feed it. You've now built the small version by hand.

IMPORTANT

The `set` rules only take effect from where the function's body runs. With `#show: report` at the very top of the file, that's the whole document — which is what you want. Put the `#show:` line halfway down and only the second half gets the treatment. For a whole-document look, `#show: report` goes first, above your content.

18.5 `import` brings definitions; `include` brings content

There's a second keyword that looks like `import` and does something almost opposite, and mixing them up is a classic early stumble. So let's draw the line sharply.

- `#import "file.typ"`: `name` reaches into another file and pulls out its **definitions** — the `#let`-bound names — so you can use them here. It does *not* put any of that file's *content* on the page. Importing `helpers.typ` earlier added nothing visible to the document; it only made `callout` available.
- `#include "file.typ"` does the reverse. It runs the other file and splices its **rendered content** into the document at the point of the call — its headings, its paragraphs, its images, all of it appears right there. It's the Typst equivalent of pasting the file's output in place.

Example 098 (`examples/098-include-vs-import/`) puts them side by side. One side file holds a definition:

```
// defs.typ
#let signoff(name) = text(style: "italic", fill: luma(90))[- #name, editor]
```

The other holds content — actual words meant for the page:

```
// section.typ
== From the archives
This whole section lives in its own file. `#include` drops its rendered
content into the document at the point of the call.
```

And `main.typ` uses each for what it's good at:

```
// main.typ
#import "defs.typ": signoff      // a definition – usable, prints nothing yet

= The Weekly
An opening paragraph, typed right here in main.typ.

#include "section.typ"          // content – the heading and paragraph appear here

#signoff("Ada Lovelace")       // now use the imported definition
```

Read the output top to bottom and the two roles are plain: the `#import` line adds nothing where it sits but makes `signoff` work later; the `#include` line is where the archived section physically shows up. Definitions come in through `import`; words come in through `include`.

The obvious use for `include` is splitting a long document into a file per chapter — `main.typ` becomes a table of contents of `#include` lines, one per `chapter-01.typ`, `chapter-02.typ`, and so on, each file editable on its own. We'll build exactly that when we assemble a whole book in Part VI.

WARNING

A file you `#include` runs in the document's context, but the names it defines do *not* leak back into `main.typ`. If `section.typ` defines a helper you want to call from `main.typ`, you must `#import` it as well — `include` alone won't expose it. Content flows out; definitions don't.

18.6 Keeping the library tidy

A library is code other files depend on, which raises the stakes on a few small habits. None of them are hard; all of them save a future headache.

Give it default arguments. Every component in `lib.typ` had defaults — `title: "Note"`, `color: brand`. A caller who wants the common case types nothing; a caller who wants a variation overrides one argument. Defaults are what make a library pleasant instead of demanding. (Chapter 14 covers the mechanics; a library is just where they earn their keep.)

Document each function with a one-line comment. A `// callout(body, title:, color:)` — a titled note box. above the definition costs you five seconds and tells the next reader — usually you, months later — what the thing is for and how to call it, without reverse-engineering the body.

Keep the public names deliberate. Everything you `#let` at the top level of a module becomes importable, whether you meant it to or not. If a component needs a little internal helper that no document should call directly, that's fine — just know it's exposed, and consider a naming convention (a leading underscore, `_clamp`, is a common signal for "internal, don't rely on this") so a `#import: *` reader knows what's really public.

Group constants at the top. The theme block — colours, sizes, spacing — is the part you'll tweak most. Putting it together at the top of the file, above the components, means you never hunt through function bodies to change a colour. Machinery below, knobs on top.

Coming from LaTeX. This is the job a `.sty` package file does: you write `\usepackage{mystyle}` in the preamble and `\input{chapter1}` to splice a chapter in. Typst folds both jobs into plain `.typ` files and two keywords — `#import` is your `\usepackage`, `#include` is your `\input` — with no separate package format, no `.sty` versus `.cls` distinction, and no `TEXINPUTS` path to configure. Your own reusable code is just a Typst file sitting next to the document. (Publishing a library for *other people* to install by name is a further step, and that's Chapter 20; for your own use, a file beside your document is the whole story.)

18.7 What you've got

You can now spread work across files instead of piling it into one:

- **A file of your own** — put `#let` definitions in a `.typ` module and it becomes machinery any document can borrow. A module you compile alone renders blank, and that's correct: it defines, it doesn't say.
- **Three import forms** — `named` (`#import "lib.typ": a, b`) for the few things you use; `star` (`( : * )`) for everything, at the cost of namespace control; `namespaced` (no colon, then `lib.a`) for the safest, clash-proof access. Rename on the way in with `as`.
- **A style library** — theme constants at the top, components below, each defaulting to a constant. One file decides the look; every document that imports it inherits the look and every future fix.
- **A bundled look** — a `report(body)` function wrapping a stack of set and show rules, applied to a whole document with `#show: report`. This is a template in embryo (Chapter 19).
- **import vs include** — `#import` pulls in a file's *definitions*; `#include` splices in a file's *content*. Definitions come through one, words through the other, and they don't cross over.
- **Good habits** — default arguments, a one-line comment per function, deliberate public names, constants grouped on top.

Part V is about reuse, and this is its foundation. A template (Chapter 19) is a library with a `#show:` function at its heart; a package (Chapter 20) is a library you've addressed so others can install it by name. Both are the file you just learned to write, taken one step further.

18.8 Exercises

Solutions are in Appendix A.

18.1. Take the `kbd` function you wrote for Exercise 14.1 and move it into a file called `keys.typ`. From a separate `main.typ`, import it with the named form and use it in a sentence — `Press #kbd("Ctrl") + #kbd("C")`. Confirm that compiling `keys.typ` on its own produces a blank page, and that `main.typ` renders the keycaps.

18.2. Add a second name to `keys.typ` — a constant `#let key-color = luma(245)` — and have `kbd` use it as its fill. Now import `keys.typ` all three ways in turn (named, star, and namespaced) from `main.typ`, and for each write the one line that uses `kbd` and `key-color`. For the namespaced form, that's `keys.kbd(...)` and `keys.key-color`.

18.3. Build a two-component library, `lib.typ`, containing a `brand` colour constant and two components that both default their colour to `brand`: a `callout` and a `tag` (a small inline label — a `box` with a fill and white text). In `main.typ`, import all three names and produce a short document that uses each component twice, once with the default colour and once overridden. Then change `brand` in `lib.typ` and confirm every default-coloured element moves together.

18.4. Split a short document across files with `#include`. Write `intro.typ` and `body.typ`, each holding a heading and a paragraph of `#lorem` text, and a `main.typ` that sets the page and then includes them in order. Confirm the two sections appear in the document exactly where the `#include` lines sit.

18.5. (*Stretch.*) Turn the reusable-look idea into a switchable one. Write `looks.typ` defining two whole-document functions — `formal(body)` and `playful(body)` — that differ in font, heading colour, and page margin. In `main.typ`, apply one with `#show: formal`, then change that single line to `#show: playful` and watch the entire document change character without a word of content moving. You've just written two templates and swapped between them in one line — which is most of what Chapter 19 is about.

Templates

You've typed the same fifteen lines at the top of every document for a month. A `set page` with your margins, a `set text` with your font, a `set par(justify: true)`, a show rule to color the headings, and a little title block you paste in and edit by hand. Every new report starts with a pilgrimage to the last one, to copy the preamble across. Then one afternoon you change the font in a single document, forget to change it in the others, and now none of them match — and you can't remember which one holds the good version.

That itch has a name, and the cure is a template. A template is the entire look of a document — every set rule, every show rule, the title block, the running headers, the lot — packed into one function that you apply with a single line. Write it once. Apply it to a hundred documents. Change it in one place and all hundred follow.

You've already met the mechanism, in miniature, at the very end of Chapter 10. This chapter grows it into the real thing.

19.1 A template is a function

Back in Chapter 10 you wrote a `report` function that took the document `body`, applied a stack of rules, and handed the body back wearing them. Then a selector-less `#show: report` fed the whole document into that function.

That was a template. There is no deeper secret. A template is an ordinary function that takes your content as its last argument, dresses it in set and show rules and any scaffolding you like, and returns the dressed result — invoked by a show rule with no selector in front of it.

Everything else in this chapter is making that function *do more*: take configuration, build a title block, live in its own file. The shape never changes. Here's the smallest version that still earns the name (`examples/100-a-first-template/`):

```
#let report(body) = {
  set page(width: 12cm, height: auto, margin: 1.6cm)
  set text(font: "Libertinus Serif", size: 11pt)
  set par(justify: true, leading: 0.7em)
  set heading(numbering: "1.")
  body
}

#show: report

= Field notes
#lorem(30)
```

`report` is a plain function with one parameter. Inside, four set rules establish the page, the font, justified paragraphs, and numbered headings. The last line is just `body` — the function's result is the content it was handed, now sitting *after* all those rules, so the rules apply to it. (Remember from Chapter 10 that set and show rules only affect what comes after them, within their scope. The rules run first; `body` comes last; the body is styled.)

Then `#show: report` does the feeding. It has no selector, so it catches the entire rest of the document, hands that content to `report` as `body`, and replaces it with what comes back. From that one line down, your document is justified Libertinus with numbered headings, and you never touched the content to make it so.

19.2 Giving the template something to configure

The `report` above dresses the body but knows nothing *about* it. Real documents have a title, an author, a date — things the template should lay out for you rather than make you build by hand every time. That means parameters: the named, defaulted parameters you met in Chapter 14.

Add a couple in front of `body` and build a title block out of them (`examples/101-configurable-template/`):

```
#let report(title: none, author: none, body) = {
  set page(width: 12cm, height: auto, margin: 1.6cm)
  set text(font: "Libertinus Serif", size: 11pt)
  set par(justify: true)
  set heading(numbering: "1.")

  align(center)[
    #text(size: 18pt, weight: "bold")[#title] \
    #text(style: "italic")[#author]
  ]
  v(1.2em)

  body
}
```

Now the function has three parameters. `title` and `author` are named, with a default of `none`; `body` is positional and comes *last*. Inside, before the body, we render a centered title block: the title big and bold, the author in italics below it, a little vertical breathing room, and then the body.

Applying it looks like this:

```
#show: report.with(
  title: "Weeknotes",
  author: "A. Reader",
)

= Monday
#lorem(28)
```

You get a centered "Weeknotes" title over an italic byline, followed by your numbered sections — a real front matter from two lines of configuration.

19.2.1 Why `.with` is the magic word

Look hard at that `#show: report.with(...)`, because it's the hinge the whole pattern turns on, and it's two ideas you already know clicking together.

The first is `.with`, from Chapter 14. `report.with(title: "Weeknotes", author: "A. Reader")` doesn't *call* `report` — it pre-fills those two arguments and returns a *new function* with them baked in. That new function has exactly one parameter still waiting to be supplied: `body`.

The second is the selector-less show rule, from Chapter 10. `#show: someFunction` gathers up the entire rest of the document and passes it to `someFunction` as its argument.

Put them side by side and it clicks. `.with` fills the *configuration* and leaves a single hole named `body`. The show rule then drops the whole document into that hole. The two halves meet in the middle: you supply the config, Typst supplies the content, and `report` gets a complete set of arguments. That is the entire idiom, and you'll write it at the top of nearly every serious document you make.

IMPORTANT

The `body` parameter must come **last** in the function's parameter list. The selector-less show rule always feeds the document in as the final positional argument (the same content-block sugar from Chapter 14: `#show: f` behaves like handing `f` one trailing block of content). If `body` sat in the middle, that content would land in the wrong slot and the whole thing would misbehave. Configuration first, `body` last, every time.

19.3 Configuration versus content

It's worth being clear about the two kinds of thing a template takes, because they feel different and behave differently.

The **configuration** is data *about* the document: a title string, an author, a date, whether to show an abstract. Small values, set once, at the top. You pass them by name through `.with`, and inside the template you place, style, or branch on them.

The **body** is the document itself — every heading and paragraph below the show line. You never pass it explicitly; the selector-less show rule feeds it in for you.

A good habit: give every configuration parameter a default, usually `none`. Two reasons. The template still compiles if you forget one, instead of erroring on a missing argument. And a default of `none` lets the template *notice* an omission — “if no date was given, don't draw the date line” — which is exactly how you'll make the abstract optional in a moment.

19.4 Moving the template into its own file

Once a template is more than a handful of lines, you don't want it squatting on top of every document that uses it. Give it its own file and import it, exactly as you would any multi-file library from Chapter 18.

Put the function in `template.typ` (`examples/102-template-in-a-file/`):

```
// template.typ
#let article(title: none, author: none, body) = {
  set page(width: 12cm, height: auto, margin: 1.6cm)
  set text(font: "Libertinus Serif", size: 11pt)
  set par(justify: true)
  set heading(numbering: "1.")

  align(center)[
    #text(size: 18pt, weight: "bold")[#title] \
    #text(style: "italic")[#author]
  ]
  v(1em)

  body
}
```

And the document that uses it shrinks to nothing but configuration and content:

```
// main.typ
#import "template.typ": article

#show: article.with(
  title: "On importing templates",
  author: "A. Reader",
)

= Why split the file?
#lorem(30)
```

`#import "template.typ": article` pulls the `article` function into `main.typ`, and from there it's used identically to before. The payoff is separation: the styling lives in one file, the writing in another. Ten documents in a folder can all `#import` the same `template.typ`, and fixing the font once fixes it for all ten. That is the difference between a template and a preamble you keep pasting.

19.5 A real article template

Time to build the thing you'd reach for. The template below adds a date, an *optional* abstract, and a more considered title block, and it styles the headings with a show rule on top of the set rules. It is the centerpiece of the chapter (`examples/103-an-article-template/`).

Start with the signature and the set rules:

```
#let article(
  title: none,
  author: none,
  date: none,
  abstract: none,
  body,
) = {
  set page(
    width: 14cm, height: auto,
    margin: (x: 2cm, top: 2cm, bottom: 1.6cm),
  )
```

```

set text(font: "Libertinus Serif", size: 10.5pt)
set par(justify: true, leading: 0.65em, first-line-indent: 1.2em)
set heading(numbering: "1.1")
show heading.where(level: 1): set text(size: 13pt)

```

Four configuration parameters, then `body` last, just as the rule demands. The set rules do the everyday styling; the lone show rule bumps up the size of top-level headings without disturbing anything else about them — the show-set form from Chapter 10, doing quiet work.

Next, the title block, assembled from the configuration:

```

align(center)[
  #block(text(size: 20pt, weight: "bold")[#title])
  #v(0.2em)
  #if author != none {
    text(size: 11pt)[#author]
  }
  #if date != none {
    linebreak()
    text(size: 9.5pt, fill: luma(40%))[#date]
  }
]

v(1em)

```

The title is always drawn; the author and date are drawn only if they were supplied. That's the `none`-default habit paying off — `if author != none` guards the byline, so an untitled draft or a dateless memo still lays out cleanly instead of printing an empty line.

Then the optional abstract, and finally the body:

```

if abstract != none {
  block(width: 100%, inset: (x: 1.4em))[
    #align(center)[#text(weight: "bold", size: 9.5pt)[Abstract]]
    #set text(size: 9.5pt)
    #set par(first-line-indent: 0pt)
    #emph(abstract)
  ]
  v(1em)
}

body
}

```

The whole abstract is wrapped in `if abstract != none`, so a document that skips it pays no price. Inside the block, two set rules — a smaller text size and no first-line indent — apply *only* within those brackets, because set rules are scoped (Chapter 9). The abstract comes out indented on both sides, smaller, italic, under a centered "Abstract" label; the body below it is unaffected.

The document that drives all this is, once again, just a show line and content:

```

#show: article.with(
  title: "A Field Guide to Templates",
  author: "A. Reader",
  date: "July 2026",
  abstract: [
    Templates package a document's entire look into one function, so a
    single line dresses the whole thing.
  ],
)

= Introduction
#lorem(40)

```



```
== Background and motivation
#lorem(30)
```

Notice the abstract is passed as *content* — square brackets, not a string — because it's a paragraph you might want to emphasize or format, not a label. Configuration doesn't have to be plain text; a parameter can take a whole block of content when that's what the slot deserves.

The result is a tidy article: a bold centered title, author and gray date beneath, a set-off italic abstract, and numbered sections in Libertinus. Every document that applies `article` comes out matching, because the design lives in one function instead of in your muscle memory. In practice this template belongs in its own file, imported exactly like `examples/102-template-in-a-file/` — it's shown here in one piece only so you can read the whole thing at a sitting.

Coming from LaTeX. This is your `\documentclass`, and then some. In LaTeX the class deciding what an "article" looks like is a `.cls` file — often an opaque wall of `\def`s and `\RequirePackage`s you inherit and rarely dare to open. A Typst template is just a function, written in the same language as the rest of your document, sitting in a file you can read top to bottom in a minute. Want to know what it does? Read it. Want to change a margin? Edit the `set page`. There's no separate class-writing dialect and no ceremony for switching: `#show: article.with(...)` swaps the entire look, and swapping to a different template is a one-line edit.

19.6 Templates you can install

Everything so far has been a template *you* wrote, living in your own folder. Typst also has a way to share templates with the world: the package registry at Typst Universe. Many of the packages there are templates — for a thesis, a CV, or a conference paper — and you pull one in the same way you'd start a new project from it:

```
typst init @preview/name:version
```

You copy the exact name and version from the package's page on Universe. That command scaffolds a new document wired to the chosen template, ready for you to fill in the configuration and write. Under the hood it's the same `#show: template.with(...)` idiom you've been using all chapter — the package just ships the function and a starter `main.typ` so you don't write either by hand. We take packages properly in Chapter 20, including how to publish one of your own; for now, know that the template you build for yourself and the template you install from Universe are the very same kind of thing.

19.7 What you've got

You can now bottle an entire document design and pour it over any content:

- **What a template is** — an ordinary function that takes the document `body` last, applies set and show rules plus any scaffolding, and returns the dressed document. Nothing more exotic than a function that wraps your content.
- **How it's applied** — a selector-less `#show: template` feeds the whole document in as `body`. With configuration, `#show: template.with(title: ...)` pre-fills the settings (`.with`, from Chapter 14) and leaves the show rule to supply the body.
- **The one hard rule** — `body` is the *last* parameter, because the show rule always hands the document in as the final positional argument.
- **Configuration versus content** — named, defaulted parameters (`title`, `author`, `date`, `abstract`) carry data about the document; the body is the document. Default configuration to `none` so you can make parts optional with `if x != none`.
- **A real title block** — build it from configuration, guard optional pieces with `if`, and scope any local set rules inside their own block.
- **Templates in their own file** — `#import "template.typ": article` (Chapter 18) keeps styling and writing apart, so one template backs many documents.
- **Installable templates** — Typst Universe ships templates as packages; `typst init @preview/...` scaffolds a document from one. Full story in Chapter 20.

A template is where set rules, show rules, functions, and imports all come together and start earning their keep. It's also the direct ancestor of the book template we build in Part VI — the same idea, scaled up until it can typeset an entire book, this one included.

19.8 Exercises

Solutions are in Appendix A.

19.1. Take the `report` from `examples/100-a-first-template/` and give it a second personality: add a `show heading: set text(fill: navy)` inside the function so every heading turns navy. Apply it with `#show: report` and confirm all headings change from the one edit.

19.2. Add a `date` parameter (defaulting to `none`) to the configurable `report` in `examples/101-configurable-template/`, and render it under the author in a smaller, gray text. Apply it once *with* a date and once *without*, and confirm the dateless version doesn't leave a blank line (hint: guard it with `if date != none`).

19.3. Move a template into its own file. Take your answer to 19.2, put the function in `template.typ`, and `#import` it into a `main.typ` that contains only the show line and some content. Confirm the output is identical to the single-file version.

19.4. Extend the article template in `examples/103-an-article-template/` with a `keywords` parameter that takes an array of strings (e.g. `( "typst", "templates" )`) and prints them, comma-separated, in italics under the abstract. Make it optional: nothing prints if `keywords` is left at its default. (Hint: `( "a", "b" ).join(", ")`.)

19.5. (*Stretch.*) Write a second template, `memo(to: none, from: none, body)`, with a completely different front matter — a left-aligned "To / From / Date" block instead of a centered title. Keep it in the same `template.typ` as your `article` (export both). Now write one `main.typ` that imports both and, by changing a single line, switches the same body between two looks. You've just proven the payoff: the design and the writing are genuinely separate.

Packages

You need a diagram. Not a photograph of one — a real, sharp, editable diagram: a couple of boxes, an arrow, a label or two. You could build it out of Typst's `line` and `rect` primitives and a lot of patience, nudging coordinates until it looks right. Or you could type one line at the top of your file and borrow a drawing library that someone spent two years perfecting.

That someone is a stranger on the internet, and their work is one import away.

In Chapter 18 you learned to pull a function out of your own file with `#import "helpers.typ": callout`. In Chapter 19 you wrapped a whole look into a template. This chapter is the same `#import`, reaching further: past your own folder, past your own machine, all the way to a community registry of ready-made Typst code. You'll learn to use packages, and — because your templates from the last chapter deserve an audience — to publish your own.

20.1 Typst Universe

The registry is called **Typst Universe**, and it lives at `typst.app/universe`. It's a searchable catalogue of community packages: drawing kits, table helpers, unit typesetters, slide systems, whole document templates. Everything there is open source, and pulling one in takes a single line.

That line names three things — a *namespace*, a *name*, and a *version*:

```
#import "@preview/cetz:0.3.4": *
```

The `@preview` namespace is the public registry: every package on Universe lives there. (There's a second namespace, `@local`, for packages you keep privately on your own machine — handy for testing, and we'll get to it.) After the namespace comes the package's name, `cezt`, and after the colon its version, `0.3.4`.

That version is not optional. Leave it off and Typst refuses to compile: it will not guess which release you meant. This feels pedantic for about a day and then you're grateful for it — more on why in a moment.

The first time you compile a file with that import, Typst reaches out to the network, downloads the package, and tucks it into a cache in your user directory. Every compile after that reads from the cache. So you pay the network cost once, per package, per version; from then on the package is *yours*, offline, as fast as any local file. Get on a plane right after the first build and everything still works.

NOTE

The cache lives in a system-dependent spot — on macOS it's `~/Library/Caches/typst/packages/`, on Linux `~/.cache/typst/packages/`. You rarely need to look, but if you ever want to force a fresh download, that's the folder to clear.

20.1.1 A package in action

CeTZ (a pun on the German pronunciation of "TikZ", LaTeX's famous drawing package) is a library for drawing with coordinates instead of a mouse. Here's the whole of `examples/104-using-a-package/`:

```
#import "@preview/cetz:0.3.4"
#set page(width: 7cm, height: auto, margin: 0.8cm)

= A figure, drawn in code

#align(center, cetz.canvas({
  import cetz.draw: *
  line((0, 0), (3, 0), (3, 2), close: true)
  rect((2.6, 0), (3, 0.4), stroke: 0.5pt)
  content((1.5, -0.35), $a$)
```

```
content((3.4, 1), $b$)
content((1.2, 1.25), $c$)
}))
```

The import brings in the `cetz` module; `cetz.canvas` opens a drawing surface, and inside it `line`, `rect`, and `content` place a labelled right triangle by coordinate. The result is a crisp little figure that stays sharp at any zoom, because every stroke is an instruction rather than a pixel. You didn't write the drawing engine. You just described the triangle.

IMPORTANT

Because this example downloads a package, it needs the network the first time you compile it. In this book's repository it carries a `.skip-build` marker so the offline build script leaves it alone — but on your own machine, connected, `typst compile main.typ out.pdf` works exactly as advertised.

20.2 Why the version is pinned

Now, that pedantic required version. Here's the reason, in the form of a package that will not compile.

`codly` is a popular package for prettifying code listings. Ask for an old release under today's Typst and watch:

```
#import "@preview/codly:1.2.0": *
```

```
error: unknown variable: pattern
└─ @preview/codly:1.2.0/src/args.typ:10:29
```

Nothing is wrong with your document. The package itself uses a feature — `pattern` — that a later Typst version renamed. The code that was correct when `codly 1.2.0` shipped is now, against a newer compiler, broken. This is not `codly`'s fault or Typst's; it's simply what happens when a living language and a living package drift out of step.

Package versions follow **semantic versioning** — that `major.minor.patch` triple, like `0.3.4`. The convention is a promise about compatibility: a bump in the last number (`0.3.4` → `0.3.5`) is a fix that shouldn't change how things behave; the middle number adds features; the first number is reserved for changes that *break* existing code. (Packages still at major version zero, like `cetz 0.3.4`, are declaring themselves pre-1.0, where even minor bumps may break. Most of Universe still lives here — the ecosystem is young.)

Pinning the version is how you opt out of surprise. Your document says "build me with exactly `cetz 0.3.4`", and it means it, forever. A newer, subtly different release can appear on Universe next week and your file won't notice or care. Come back in two years and it still compiles the same way — the pinned version is cached, frozen, exactly the code you tested against.

WARNING

The flip side: a package pinned to an old version can fall behind the Typst compiler itself, exactly as `codly 1.2.0` did. If an import suddenly errors after you upgrade Typst, check Universe for a newer release of the package and bump your pin to match. A well-behaved package can also declare the minimum Typst version it needs (a `compiler` field in its manifest), so you get a clear message instead of a cryptic one.

20.3 A short tour of Universe

Universe is large and uneven — some packages are polished and maintained, others are a weekend experiment someone never came back to. Rather than a catalogue, here is an honest map of the neighbourhoods worth knowing, by what you'd reach for them to do. Names, not endorsements; check the version compatibility and the last-updated date before you commit.

- **Drawing and diagrams.** `cetz` for general vector drawing by coordinate; `fletcher` (built on `cetz`) for node-and-arrow diagrams — flowcharts, commutative diagrams, state machines.
- **Tables.** Typst's native tables (Chapter 7) already go far, but `tablem` lets you write tables in a Markdown-like shorthand, and `tablex` predates some native features if you meet it in older documents.
- **Code listings.** `codly` and `zebraw` both turn fenced code blocks into properly styled listings with line numbers and highlighting.
- **Units and science.** `unify` typesets numbers and physical units correctly (`#qty("299792458", "m/s")`); `physica` gives you compact operators for calculus and vectors, which we'll use in a moment.

- **Glossaries.** *glossarium* manages a glossary and acronym list, expanding each term on first use — the sort of bookkeeping you never want to do by hand.
- **Running headers.** *hydra* puts the current section's title in the page header automatically, tracking where you are in the document.
- **Slides.** *polylux* and *touying* turn Typst into a presentation tool, with slides, overlays, and speaker notes. Yes, you can replace PowerPoint with a text file.

For one worked example, take *physica*. It exists because physicists and engineers write the same operators constantly and would rather not spell each one out. Here's `examples/105-a-useful-package/`:

```
#import "@preview/physica:0.9.4": *

The gradient of a scalar field $phi$:
$ grad phi = pdv(phi, x) vu(x) + pdv(phi, y) vu(y) + pdv(phi, z) vu(z) $

$ dv(f, t) quad div vb(E) = rho / epsilon_0 $
```

`grad` is the gradient, `pdv(phi, x)` a partial derivative, `dv` a total derivative, `div` a divergence, `vu(x)` a unit vector, `vb(E)` a bold vector. Six short names for notation that would otherwise be a fiddly pile of subscripts and upright *s*. The package did the fiddling once so you don't repeat it.

TIP

Both packages in this chapter — `cetz:0.3.4` and `physica:0.9.4` — are verified to compile cleanly with Typst 0.15.0. When you go shopping on Universe yourself, the surest test is the one you already know: pin a version, import it, and compile a two-line document. If it builds clean, it's good.

20.4 Starting from a template package

Chapter 19 taught you to build a template — a function that wraps a document in a consistent look. Templates travel on Universe too, and they get a special convenience: you can start a brand-new project *from* one.

```
typst init @preview/charged-ieee
```

That command downloads the `charged-ieee` template package and scaffolds a fresh project from it — a starter `main.typ` already wired to the template, ready for you to fill in your title and start typing. It's the fastest possible way from "I want an IEEE-style paper" to a blank page that already looks like one. (As with `#import`, you can pin a version by appending `:0.1.0`; unlike `#import`, `typst init` will happily default to the latest release if you don't.)

The web app has the same thing behind its "Start from template" button. Either way, the template author did the layout; you do the writing.

20.5 Publishing your own package

So how does a package get onto Universe in the first place? The surprise is how little there is to it. A package is an ordinary `.typ` file with one extra thing beside it: a manifest.

The manifest is a file named `typst.toml`, and here's a complete, minimal one (from `examples/106-publishing-a-package/`):

```
[package]
name = "callout-box"
version = "0.1.0"
entrypoint = "lib.typ"
authors = ["Your Name <you@example.com>"]
license = "MIT"
description = "A tiny titled callout box."
```

Six fields. The `name` and `version` are how the world will refer to your package; `entrypoint` names the `.typ` file Typst loads when someone imports it. The other three — `authors`, `license`, `description` — are required for publishing, not for the package to work locally. The `license` must be a valid SPDX identifier (MIT, Apache-2.0, MIT-0, and so on); Universe only accepts licenses that let people actually use what you shared.

The entrypoint is just code you've already written. Ours, `lib.typ`, exports one function:

```
#let callout(title: none, body) = block(
  fill: luma(240),
  stroke: (left: 2pt + rgb("#3b6ea5")),
  inset: 10pt, radius: 2pt, width: 100%,
  {
    if title != none [ #strong(title) \ ]
    body
  },
)
```

20.5.1 Test it before you ship it

Before submitting anything, you want to *use* your package and confirm it works. You don't need to publish for that — you can import the entrypoint by a plain relative path, exactly the trick from Chapter 18:

```
#import "lib.typ": callout

#callout(title: "Before you publish") [
  Publish these two files and this import line becomes
  `#import "@preview/callout-box:0.1.0": callout`.
]
```

That's the entire local smoke test in `examples/106-publishing-a-package/`: a manifest, an entrypoint, and a `main.typ` that imports it by path. When you're ready to test it the way a *user* would — via the `@preview`-style machinery — copy the package into the `@local` namespace's folder on your machine and import it as `@local/callout-box:0.1.0`. Same mechanism as `@preview`, no network, no publishing.

20.5.2 Getting onto Universe

The registry is a single public GitHub repository: `github.com/typst/packages`. Publishing is a **pull request** to it. You add your package's files under a path that encodes its name and version —

```
packages/preview/callout-box/0.1.0/
```

— containing your `typst.toml`, your entrypoint, and any supporting files. Open the PR, a maintainer reviews it (there's an automated check plus a human skim for sanity), and once merged, `@preview/callout-box:0.1.0` is live for everyone. Publishing a version is permanent: you never overwrite `0.1.0`, you release `0.1.1` next to it. That permanence is the whole reason your pinned imports keep working.

20.5.3 Template packages

A template package is the same idea with one extra section in the manifest. Our `examples/107-template-package/` ships a `report` function of the Chapter 19 shape, plus a `[template]` block:

```
[package]
name = "tidy-report"
version = "0.1.0"
entrypoint = "lib.typ"
authors = ["Your Name <you@example.com>"]
license = "MIT-0"
description = "A minimal, no-fuss report template."

[template]
path = "template" # folder copied into the new project
entrypoint = "main.typ" # file the user starts editing
thumbnail = "thumbnail.png"
```

The `[template]` section is exactly what makes `typst init @preview/tidy-report` work. `path` is a folder of starter files that get copied into the user's new project; `entrypoint` is the file they open first; `thumbnail` is the preview image shown on Universe. The starter's own `main.typ` imports the published package by name (`#import "@preview/tidy-report:0.1.0": report`) — because once it's copied into someone else's project, your

`lib.typ` isn't next door anymore; it's in the registry. That one-line difference between the local test and the shipped starter is the whole trick, and both files sit side by side in the example folder so you can compare them.

Coming from LaTeX. Universe is Typst's CTAN, and `#import "@preview/..."` is its `\usepackage` — but with two differences you'll feel immediately. First, the version is pinned *per project*, right there in your source, instead of whatever TeX Live installed system-wide; two documents on your disk can use two different versions of the same package without a fight. Second, there's no `tlmgr`, no multi-gigabyte distribution to update, no "package not found" after a fresh install. The first compile fetches exactly what this document needs and caches it. Nothing else, nowhere else.

20.6 What you've got

You can now:

- **Pull in community packages** from Typst Universe with `#import "@preview/name:version"`, remembering the version is required and that the first compile downloads, then caches for offline use.
- **Read a version number** as a compatibility promise, and understand why pinning protects your document from both the package's future and the compiler's.
- **Diagnose a broken import** — an old package on a new Typst — and know to check Universe for a newer release.
- **Scaffold a project from a template package** with `typst init @preview/...`.
- **Assemble your own package:** a `typst.toml` manifest, an entrypoint, a local relative-import test, and the shape of a pull request to `typst/packages`.
- **Turn a Chapter 19 template into a package** by adding a `[template]` section to the manifest.

20.7 Exercises

Solutions are in Appendix A.

20.1. Add `#import "@preview/cetz:0.3.4": *` to a fresh document and, inside a `cetz.canvas`, draw a single circle with `circle((0, 0), radius: 1)`. Compile it (you'll need the network the first time). Then compile it again with the network off — confirm the second build is fast and offline.

20.2. Take the import from 20.1 and delete the `:0.3.4`, leaving `#import "@preview/cetz": *`. Compile. Read the error. In one sentence, say why Typst refuses rather than fetching the newest `cetz` for you.

20.3. Browse `typst.app/universe` for three minutes. Find one package that would help with a document *you* actually write, note its name and current version, and write the exact one-line import you'd use.

20.4. Write a `typst.toml` for a package called `my-notes` at version `0.2.0`, with entrypoint `lib.typ`, yourself as author, an SPDX license of your choice, and a one-line description. Put a trivial exported function in `lib.typ` and a `main.typ` that imports it by relative path and uses it. Confirm the whole thing compiles with no network.

20.5. (*Stretch.*) Take the template you built for the Chapter 19 exercises and turn it into a template package: give it a `typst.toml` with a `[template]` section, a `template/` folder containing a starter `main.typ` that imports your template by its (hypothetical) `@preview` name, and a local `main.typ` that imports the entrypoint by relative path to prove the wrapper works. You don't have to publish it — just build the folder that *could* be published, and make the local test compile.

Advanced layout

Most of the time you want Typst to make the layout decisions. You pour in headings and paragraphs and figures, and the engine breaks the lines, fills the columns, and floats the pictures somewhere sensible. That's the whole pleasure of the thing: you say *what*, and it works out *where*.

But now and then you know exactly where. You want a "DRAFT" stamp glued to the top-right corner, a label rotated up the spine of a page, a little logo you drew yourself, a dotted leader marching from a chapter title to its page number. For those moments Typst keeps a drawer of manual controls — tools that let you place content by hand, wrap it in a coloured box, transform it, draw with it, or reach into the finished document and ask what's inside.

This chapter is that drawer. None of it is exotic; it's just the layer below the automatic one, the escape hatches you reach for when the defaults won't do. We've been dropping promissory notes about it since Chapter 5 ("scoping a watermark is a Chapter 21 topic"), Chapter 6 ("`#place` gets covered later"), and Chapter 17 ("for mid-page sections you'll want `query`"). Time to pay them off.

21.1 Placing things exactly

You met `#place` by name in Chapter 6, as the lower-level cousin of a floating figure. Here's what it actually does: it lifts a piece of content clean out of the text flow and pins it to a position on the page.

```
#place(top + right, box(fill: yellow, inset: 6pt)(*DRAFT*))
```

The first argument is an alignment — `top + right`, `bottom + center`, `horizon + left`, any horizontal-plus-vertical pair. That's the anchor. The content goes wherever that alignment points, measured against the page's text area.

Two offsets fine-tune it. `dx` shifts horizontally, `dy` vertically, both taking a length that can be negative:

```
#place(top + right, dx: -0.2cm, dy: 0.2cm, [...])
```

Positive `dx` moves right, positive `dy` moves down — the usual screen convention, with the origin at the top-left. So to tuck a corner tag *inward* from the top-right you pull it left (negative `dx`) and push it down (positive `dy`), which is exactly what the yellow tag in `examples/108-place-and-float/` does.

The crucial part — the thing that surprises people once and then becomes the whole point — is that placed content **reserves no space**. The flow behaves as though it isn't there. In example 108 the body paragraph starts at the very top of the page, right underneath the tag, because as far as the text is concerned the tag doesn't exist. That's a feature: it's how a watermark can sit *behind* your words without shoving them around.

WARNING

Because placed content reserves no space, it will happily overlap your body text if you aim it into an occupied spot. That's not a bug for a faint background stamp, but it will bite you if you `#place` something opaque in the middle of a paragraph. Placement is for the margins, corners, and backgrounds — the parts of the page the flow isn't using.

21.1.1 Floating, the manual way

Add `float: true` and `#place` stops pinning to a fixed corner and starts *floating*: it sends the block to the top or bottom of the page and lets the body text close up the gap it left behind.

```
#place(bottom, float: true, clearance: 10pt, block(...))
```

This is the same machinery a figure uses when you write `placement: bottom` (Chapter 6) — in fact a figure's float *is* this, one layer down. The difference is that `#place(float: true)` floats *any* content, not just a figure, and doesn't add a caption or a number. Use it for a callout box you want at the foot of the page, or a full-width banner at the top, without dressing it up as a figure. `clearance` sets the gap between the float and the body text it displaces.

TIP

Remember Chapter 5's watermark, the one built with `#set page(background: ...)`? That mark repeats on *every* page, because it's part of the page setup. To stamp a mark on **one** page only, don't touch the page settings — just `#place` the mark directly on the page you want. Placement happens once, where you write it, so a single placed stamp stays on a single page. That's the scoping trick Chapter 5 promised.

21.2 Boxes and blocks

Almost everything decorative in Typst — a coloured highlight, a bordered callout, a fixed-size frame — is one of two containers: a `#box` or a `#block`. They take the same styling arguments and differ in exactly one way.

A `#box` is **inline**. It sits inside a line of text, like a single word, and the line flows around it:

```
Here is an inline #box(fill: rgb("#fff2b0"), inset: (x: 3pt),
  outset: (y: 3pt), radius: 2pt)[highlight] in a sentence.
```

The words before and after stay on the same line; the box just rides along with them. That's the tool for a highlighted term, an inline coloured chip, a bit of content you want boxed *without* breaking the paragraph.

A `#block` is **block-level**. It takes its own horizontal band of the page, with the text above and below it, the way a paragraph or a heading does:

```
#block(fill: rgb("#eef4ff"), stroke: 1pt + rgb("#3b6fb0"),
  radius: 6pt, inset: 10pt, width: 100%)[
  A callout that takes its own slab of the page.
]
```

Both share the same styling vocabulary, and it's worth learning once:

- `fill` — a background colour.
- `stroke` — a border; `1pt + blue` means one-point blue, or pass a dictionary for per-side control.
- `radius` — rounded corners.
- `inset` — padding *inside* the box, between the border and the content.
- `outset` — the mirror image, growing the box *outward* past its content (handy on an inline box so a highlight extends a hair above and below the text, as in the example).
- `width` and `height` — a fixed size, instead of shrinking to fit.
- `clip: true` — cut off anything that overflows the box's bounds.

That last pair earns its own demonstration. Give a block a fixed `height` and turn on `clip`, and content that overruns is sliced at the edge rather than spilling out:

```
#block(width: 4.5cm, height: 1.8cm, clip: true, stroke: 1pt + gray)[
  #lorem(30)
]
```

`examples/109-box-and-block/` runs all three side by side: the inline highlight, the bordered callout, and the clipped frame. The rule of thumb: reach for a `box` when the thing belongs *in* a line, a `block` when it wants a line of its own.

21.3 Stacking

`#stack` is the simplest way to line several things up along one axis. You give it a direction and a single spacing value, then the items:

```
#stack(dir: ttb, spacing: 8pt,
  rect(width: 100%)[first],
  rect(width: 100%)[second],
  rect(width: 100%)[third],
)
```

`dir: ttb` stacks top-to-bottom — a column. `dir: ltr` stacks left-to-right — a row. (There's `btt` and `rtl` too, if you ever want to build upward or lay out a right-to-left script.) The `spacing` goes evenly between every pair of items. `examples/110-stack/` shows a column of rectangles over a row of circles.

This overlaps with `grid` (Chapter 7), and the difference is worth keeping straight. A `grid` gives you a two-dimensional lattice: aligned rows *and* columns, with cells that line up across both. A `stack` is one-dimensional — a

single run of items along one direction, each taking exactly the space it needs. When you're placing things in a row or a column and you don't need anything to line up in the other axis, `stack` is the lighter tool. The moment you need a real table of alignment, go back to `grid`.

21.4 Nudging within the flow

`#place` and the transforms take content out of the flow. Two smaller tools work *inside* it, for when you just want to shift something a little without uprooting it.

`#pad` adds space around content, like an inset that lives outside the box rather than inside it:

```
#pad(x: 1cm, [This paragraph is indented on both sides.])
```

It takes the same side arguments as a margin — `x`, `y`, `left`, `top`, and so on — and it's the clean way to inset a block quote or push one paragraph in from both edges.

`#align` positions content within the width it's given:

```
#align(center)[Centred.]
#align(right)[Flushed right.]
```

You've been using `align` since the early chapters without much ceremony; it's worth naming here as the in-flow counterpart to `#place`. `align` moves content within the space the flow *has* allotted it; `place` ignores the flow and pins to the page. Same verb, different altitude.

21.5 Transforming content

Four functions distort content geometrically without changing what it *is* — the letters stay selectable text, the shapes stay shapes.

```
#rotate(-20deg, [rotated])
#scale(x: 140%, y: 70%, [scaled])
#move(dx: 0pt, dy: -8pt, [nudged up])
#skew(ax: 20deg, [slanted])
```

`#rotate` turns content by an angle. `#scale` stretches it — pass `x` and `y` separately to squash or elongate, or a single factor for both. `#move` shifts it by an offset, the way `#place`'s `dx` / `dy` do but *without* leaving the flow. And yes, `#skew` exists in Typst 0.15: it slants content, `ax` shearing horizontally and `ay` vertically, the effect you'd use for a hand-faked italic or a parallelogram.

There's a catch with all four, and it's the same catch. By default a transform is purely visual: the surrounding layout reserves the content's *original* footprint and ignores whatever the transform did to its shape. Rotate a tag inside a line of text and it'll poke out above and below, because the line still thinks the tag is its old upright self. Pass `reflow: true` and the layout re-measures the transformed result and makes room for it:

```
before #rotate(30deg, reflow: true)[X] after
```

[examples/111-transforms/](#) shows the whole quartet in a row, then the same rotation with and without `reflow` so you can watch the line grow.

Coming from LaTeX. All those little positioning macros you kept looking up — `\raisebox{2pt}{...}` to nudge something up, `\hspace{1cm}` to shove it sideways, `\rotatebox`, `\scalebox`, and the whole `\node{...} at (...)` apparatus from TikZ for pinning things to absolute coordinates — collapse into a handful of ordinary functions here. `\raisebox` is `#move`. `\hspace` is `#h`. `\rotatebox` is `#rotate`. TikZ's absolute node placement is `#place`. No package to load, no `\usepackage{graphicx}`, no picture environment. They're built in, they nest, and they take content like anything else.

21.6 Filling space with `#repeat`

`#repeat` tiles a piece of content across the width it's given, as many times as fit. On its own that sounds pointless. Combine it with a stretchy `1fr` box and it becomes the dotted leader line every table of contents wants:

```
Chapter 3 #box(width: 1fr, repeat[.]) 12
```

The `box(width: 1fr, ...)` swells to eat all the horizontal slack between the title and the page number (that's the same `1fr` spring from Chapter 5), and `repeat[.]` fills it with dots. Change the title's length and the dots restretch to match. `examples/112-repeat-leaders/` builds a little mock contents page this way, and the lines stay aligned no matter how long the titles run.

This isn't a party trick — it's exactly how Typst's own `#outline` draws its dotted leaders. Now you know the machinery underneath. `repeat` also takes a `gap` to space the tiles out and a `justify` to spread them edge to edge (on by default), if plain packed dots aren't the look you want.

21.7 Reserving space without drawing

`#hide` is the odd one out: it lays content out completely, taking up its full space, and then draws nothing. The content is there — it occupies its box, it pushes its neighbours — it's just invisible.

```
The capital of France is #hide[Paris].
```

That renders the sentence with a Paris-shaped gap where the answer would be. It's the tool for a worksheet with fill-in blanks, an answer key printed in the same layout as the questions, or a form where you want the spacing to match a filled-in version exactly. Because the hidden content still measures, everything around it lines up as if it were visible — which a blank space of guessed width never quite manages.

21.8 Asking the document about itself

Everything so far has *put things on* the page. `query` does the opposite: it reaches into the finished document and hands you back the elements that are already in it.

Because the answer depends on the whole laid-out document, `query` only works inside `#context` (the keyword from Chapter 17 that means "compute this once you know where we are"). Give it a selector and it returns an array of the matching elements:

```
#context {
  let headings = query(heading)
  [This document has #headings.len() headings.]
}
```

From each returned element you can read its fields (`h.body`, `h.level`) and, via `h.location()`, look up *where* it landed — including its page number, through `counter(page).at(h.location())`. String those together and you've built a table of contents by hand:

```
#context {
  for h in query(heading) {
    let page-no = counter(page).at(h.location()).first()
    [#h.body #box(width: 1fr, repeat[.]) #page-no \ ]
  }
}
```

`examples/113-query/` runs exactly this, dotted leaders and all. You'd normally just call `#outline` for a contents page, but the same technique builds things `#outline` can't: a list of every figure over a certain size, a custom index, or a running header that names the current section.

That last one closes a loop from Chapter 17. Remember the warning there: a header built from *state* reads the state as of the top of its page, so a section that starts halfway down the page won't show up in that page's header. `query` sidesteps the problem, because it sees the *entire* document at once, not just what came before the current point. A header that queries all the headings and matches them against the current page number can find a section that began mid-page — something *state* alone can't do.

NOTE

`query` runs after layout, so it can see everything, but it also means the document may need an extra compilation pass to settle (Typst handles that for you). Keep queries simple and specific — `query(heading.where(level: 1))` rather than fishing through everything — and they stay fast.

21.9 Drawing your own shapes

Sometimes the content you want isn't text at all — it's a small graphic: an icon, a badge, a divider, a logo. Typst has a modest set of drawing tools that compose like everything else.

The primitives are `#rect`, `#circle`, `#line`, and `#polygon`. You've seen the first two dressed up as boxes; here they're shapes, taking a `fill` and a `stroke`:

```
#circle(radius: 20pt, fill: green)
#polygon(fill: aqua, (0pt, 0pt), (30pt, 0pt), (15pt, 25pt))
```

For anything with a curve, reach for `#curve`. This is the general path tool: you feed it a sequence of moves and segments, and it traces them out.

IMPORTANT

If you find older Typst code or answers online using `#path` to draw curves, note that it's gone. In Typst 0.15 the drawing function is `#curve`; the name `path` now refers to an unrelated file-path type. Anywhere a tutorial says `#path`, write `#curve` instead — the segment syntax is a little different but the idea is the same.

A `#curve` is built from segment functions. `curve.move(pt)` lifts the pen to a point; `curve.line(pt)` draws a straight segment to a point; `curve.cubic` and `curve.quad` draw Bézier curves (the smooth kind), with control points that pull the curve into shape; and `curve.close()` joins back to the start. Points are `(x, y)` pairs, with `y` growing downward.

Here's a checkmark — two straight segments:

```
#curve(
  stroke: (paint: white, thickness: 5pt, cap: "round", join: "round"),
  curve.move((-10pt, 1pt)),
  curve.line((-2pt, 9pt)),
  curve.line((11pt, -9pt)),
)
```

Drop that on a green `#circle` and you've got a "verified" badge. `examples/114-a-custom-shape/` assembles one: a `circle` for the disk, a `curve` of straight segments for the tick, `#place` to stack the tick over the disk, and a single `curve.cubic` for a smooth flourish underneath — the straight-line and Bézier halves of `curve` in one small picture. It's not a drawing program, and for serious diagrams you'll want the CeTZ package (Chapter 20). But for a logo or an icon, the built-ins are enough.

21.10 A door to native code

One last tool, mentioned rather than demonstrated, because it needs a compiled binary this book can't ship: `plugin`.

```
#let mymod = plugin("mymodule.wasm")
```

`plugin` loads a WebAssembly module — a chunk of pre-compiled, fast, native code — and lets your Typst document call into it as if its functions were ordinary Typst functions. The point is speed and reach: some jobs (parsing a tricky data format, running a heavy computation, syntax-highlighting code) are far faster in compiled Rust or C than in Typst's scripting language, and a plugin lets a package author write that part in a real systems language, compile it to WebAssembly, and call it from Typst. Several packages you install (Chapter 20) ship a `.wasm` plugin inside them and use it quietly on your behalf; you benefit without ever writing `plugin` yourself.

Writing your own is a genuine programming project — you compile a module against Typst's plugin protocol — and well past the scope of a layout chapter. If the idea grabs you, the reference at <https://typst.app/docs/reference/foundations/plugin/> is the place to start. For now, file it away: when a package feels suspiciously fast at something that ought to be slow, a plugin is often why.

21.11 What you've got

You can now overrule the automatic layout when you need to:

- **Place content by hand** with `#place(align, dx:, dy:)`, out of the flow and reserving no space, and float any block with `float: true` — the manual version of a figure's placement, and the way to stamp a single page.
- **Wrap content in a container:** an inline `#box` or a block-level `#block`, both with `fill`, `stroke`, `radius`, `inset`, `outset`, `width` / `height`, and `clip`.
- **Line items up on one axis** with `#stack(dir:, spacing:, ...)`, the one-dimensional lighter sibling of `grid`.

- **Nudge within the flow** using `#pad` for surrounding space and `#align` for positioning.
- **Transform content** — `#rotate`, `#scale`, `#move`, `#skew` — visually by default, or claiming layout space with `reflow: true`.
- **Fill space with `#repeat`** for dotted leaders and the like.
- **Reserve space without drawing** using `#hide`.
- **Interrogate the document** with `context query(...)` to find its elements and build custom contents lists and headers — including ones that track sections starting mid-page.
- **Draw your own shapes** with `#curve` (the modern replacement for the removed `#path`), plus `#rect`, `#circle`, `#line`, and `#polygon`.
- **Know that plugin exists** as the door to native WebAssembly code, even if you never open it.

That's the manual-controls drawer, and it's the end of Part V. You can build reusable functions, templates, and packages, and now you can reach past the automatic layout when a design demands it. Part VI puts all of it to work: we design a real book template and then wire up the pipeline that typesets this very book.

21.12 Exercises

Solutions are in Appendix A.

21.1. Pin a small "CONFIDENTIAL" tag to the top-right corner of a page, inset a couple of millimetres from the edge with `dx / dy`. Fill the page with `#lorem(60)` and confirm the body text starts at the very top, ignoring the tag. Then move the tag to `bottom + center` and watch where it lands.

21.2. Build a `#block` with a coloured fill, a coloured border, rounded corners, and 10pt of inset — a callout box. Then give it a fixed `height` shorter than its content and add `clip: true`. Describe, in one sentence, what `clip` did.

21.3. Recreate a single table-of-contents line by hand: the text "Chapter 1" on the left, "7" on the right, and a dotted leader stretching between them. (Hint: a `1fr`-wide box and `#repeat`.) Make the chapter title longer and confirm the dots restretch.

21.4. Take the four transforms — `#rotate`, `#scale`, `#move`, `#skew` — and apply each to the same short word inside a sentence. First leave them as they are and note how the line's height ignores them; then add `reflow: true` to the rotation and describe what changes.

21.5. (*Stretch.*) Use `context query(heading)` to build your own miniature table of contents for a short multi-page document: list each heading with its page number and a dotted leader, indenting sub-headings one em per level. (Hint: `h.level`, `h.body`, and `counter(page).at(h.location())`.) Compare your result to what a plain `#outline()` produces — and notice that you could now tweak yours in ways `#outline` won't let you.

Designing a book template

You have three chapters. Each lives in its own file, each is full of headings and code and the occasional boxed aside, and each compiles on its own into a perfectly nice little PDF. What you do not have is a *book*. A book is the thing that wraps those three files in a shared design — a title page, a table of contents, running heads that name the current chapter, part dividers, page numbers that agree with the contents, an index at the back — so that the three files stop being three documents and become Chapter 1, Chapter 2, and Chapter 3 of one.

That wrapper is a template. You built a small one in Chapter 19: a function that takes the document `body`, dresses it in set and show rules, and hands it back, applied with a single `#show: article.with(...)`. Everything in this chapter is that same function, grown until it can typeset a book — this book, in fact. We are going to design a real, reusable, O'Reilly-style book template, walk through it a file at a time, and by the end you will understand how every book-length document is assembled from parts.

The whole thing lives in `examples/115-oreilly-book-template/`. It compiles to an eleven-page book with a cover, a contents page, two part dividers, three short chapters, and a working index. Open it alongside this chapter; we will read it piece by piece, and it is worth having the finished PDF in view while we do.

22.1 The shape of the project

A book is too big to live in one file, so the first design decision is how to split it up. Here is the whole project:

```
main.typ          the entry point: metadata + content
template/
  theme.typ       colors, fonts, sizes — the knobs
  book.typ        the book() template and part() divider
  admonitions.typ the note / tip / warning boxes
  code.typ        show rules that theme code blocks
  index.typ       idx() and make-index()
chapters/
  01-getting-started.typ
  02-first-document.typ
  03-automating.typ
```

The split is not arbitrary. Everything under `template/` is *machinery* — the design, written once, that you would reuse for the next book without changing a line. Everything under `chapters/` is *content* — the actual writing, which changes with every book. And `main.typ` is the thin seam between them: it names the book, applies the template, and lists the chapters. This is the same division you drew in Chapter 18 between a library and the document that imports it, and the same one Chapter 19 drew between a template file and the document that applies it. A book is just where the division finally earns its keep, because there is enough of both kinds of thing that keeping them apart actually matters.

We will build it from the inside out: constants first, then the template that uses them, then the components, then the index, and finally the `main.typ` that ties it all together.

22.2 The theme: every knob in one place

Open `template/theme.typ` and there is not a single function in it — only values:

```
#let font-body = "Libertinus Serif"    // body text
#let font-head = "New Computer Modern" // titles, headings, labels
#let font-mono = "DejaVu Sans Mono"    // code

#let ink = luma(25)                    // body text
#let accent = rgb("#0b7285")          // headings, rules, the house colour
#let muted = luma(120)                 // page numbers, captions
```



```
#let hairline = luma(205)      // thin rules

#let size-body = 10pt
#let size-code = 8.5pt
```

This is the "constants at the top" habit from Chapter 18, pulled out into a file of its own. Every other file in the template imports these names and refers to `accent` and `font-head` rather than hard-coding `rgb("#0b7285")` and "New Computer Modern" in fifteen places. The payoff is the one Chapter 18 promised: to reskin the entire book — a green house colour, a different heading font — you edit `theme.typ` and nothing else. The rest of the template does not know or care what the values are, only what they are *called*.

All three fonts are ones Typst bundles, so the book compiles to the same pages on anyone's machine. That is not a small thing for a template you intend to share; a template that depends on a font the recipient does not have is a template that greets them with a warning.

22.3 The template function

Now `template/book.typ`, the heart of it. Strip away the details and `book()` has exactly the shape Chapter 19 taught:

```
#let book(title: none, subtitle: none, author: none, body) = {
  set page(width: 13cm, height: 19cm, margin: (x: 1.8cm, top: 2cm, bottom: 1.8cm),
    header: none, footer: none)
  set text(font: font-body, size: size-body, fill: ink, lang: "en")
  set par(justify: true, leading: 0.62em,
    first-line-indent: (amount: 1.2em, all: false))
  set heading(numbering: "1.1")

  // ... show rules for code and headings ...
  // ... front matter: title page and contents ...

  body
}
```

Configuration parameters first — `title`, `subtitle`, `author` — then `body` last, because the selector-less `show` rule always feeds the document in as the final positional argument. That is the one hard rule from Chapter 19, and it does not relax just because the function got bigger. The `set` rules establish the page (a deliberately small one, 13 by 19 centimetres, so the multipage preview stays legible), the body font, justified paragraphs, and numbered headings.

The one new touch is `first-line-indent: (amount: 1.2em, all: false)`. Passing a dictionary instead of a bare length turns on the book convention: indent the first line of every paragraph *except* the one right after a heading. It is the kind of detail you never notice when it is right and cannot stop noticing when it is wrong.

You apply `book()` exactly the way you applied the article template, from the top of `main.typ`:

```
#import "template/book.typ": book, part

#show: book.with(
  title: "Learning Typst",
  subtitle: "A field guide, in miniature",
  author: "Typeset with the Chapter 22 template",
)
```

`book.with(...)` pre-fills the three configuration arguments and leaves `body` open (that is Chapter 14's partial application); the selector-less `#show:` then drops the entire rest of the document into that opening (that is Chapter 10's document-wide `show` rule). The two halves meet in the middle, precisely as they did for the small template. Nothing about the idiom changed. What changed is how much the function does once it has the body.

22.3.1 Front matter, drawn once

Before the body appears, `book()` lays down the front matter — the title page and the contents — by calling two small helpers. The title page is an isolated `#page`, the manual-layout trick from Chapter 21:


```
#let _title-page(title, subtitle, author) = page(header: none, footer: none)[
  #set align(center)
  #v(3fr)
  #text(font: font-head, size: 30pt, weight: "bold", fill: ink)[#title]
  #if subtitle != none {
    v(0.7em)
    text(font: font-head, size: 14pt, fill: accent, style: "italic")[#subtitle]
  }
  #v(1.2em)
  #line(length: 35%, stroke: 1pt + accent)
  #v(1fr)
  #if author != none { text(font: font-head, size: 13pt)[#author] }
  #v(2fr)
]
```

Calling the `page` function (rather than the `set page` rule) produces a single self-contained page with its own settings — no running head, no footer — and whatever comes after it resumes on a new page under the outer settings. Inside, the `v(3fr)`, `v(1fr)`, `v(2fr)` springs are the fractional spacing from Chapter 5: they divide the leftover vertical space in a 3:1:2 ratio, floating the title into the lower-middle of the page and the author toward the foot, without a single hand-measured margin. And the `if subtitle != none` guards are the `none`-default habit from Chapter 19 — leave out a subtitle and the line simply does not appear, rather than leaving a gap.

22.4 Chapters and parts

In this template a chapter is a level-1 heading. You write `= Installing the tools` at the top of a chapter file and the template turns it into a full chapter opener — a page break, a small "CHAPTER 1" kicker, the big title, and a rule beneath. That transformation is a show rule on `heading.where(level: 1)`, the function form from Chapter 10:

```
show heading.where(level: 1): it => {
  pagebreak(weak: true)
  v(1.4cm)
  block(spacing: 0pt)[
    #text(font: font-head, fill: muted, weight: "bold", size: 10pt, tracking: 2pt)[
      #upper[Chapter] #context counter(heading).display("1")
    ]
    #v(0.35em)
    #text(font: font-head, fill: ink, weight: "bold", size: 23pt)[#it.body]
    #v(0.45em)
    #line(length: 100%, stroke: 0.8pt + accent)
  ]
  v(0.9em)
}
```

The heading arrives as `it`, we read its text off `it.body`, and we rebuild it into whatever we like — here, a two-line masthead. The chapter number comes from `counter(heading).display("1")`, read inside a `#context` because, as Chapter 17 drilled in, a counter's value depends on where you are and can only be read once Typst knows. The `pagebreak(weak: true)` starts each chapter on a fresh page; `weak` means it collapses to nothing if we are already at the top of one, so no blank pages pile up. The two deeper levels get the gentler show-set treatment — recoloured and set in the heading font, keeping their automatic 1.1 numbering:

```
show heading.where(level: 2): set text(font: font-head, fill: accent, size: 12.5pt)
show heading.where(level: 3): set text(font: font-head, fill: ink, size: 10.5pt)
```

Parts are the level above chapters, and they need a different mechanism. Chapters number continuously through the whole book — Chapter 1, 2, 3, across every part — so a part cannot simply be a higher heading level, because the heading counter would reset the chapter number in each part. Instead, `part()` is a plain function with a counter of its own:

```
#let part(title) = {
  counter("part").step()
  page(header: none, footer: none)[
```

```

#set align(center + horizon)
#text(font: font-head, size: 11pt, fill: muted, weight: "bold", tracking: 3pt)[
  #upper[Part] #context counter("part").display("I")
]
#v(0.5em)
#line(length: 22%, stroke: 1pt + accent)
#v(0.7em)
#text(font: font-head, size: 26pt, weight: "bold", fill: ink)[#title]
]
}

```

`counter("part")` is a named counter you invent — the custom-counter idea from Chapter 17 — untouched by anything Typst counts on its own. Each call steps it and prints the value as a roman numeral with `.display("I")`, on its own vertically centred page (`align(center + horizon)` inside an isolated `#page`). Because parts live outside the heading counter, they stay out of the chapter numbering entirely, which is exactly what we want.

NOTE

The trade-off of keeping parts out of the heading system is that `#outline` does not list them, since it collects headings. Adding the parts to the contents page is a genuinely instructive exercise — you would query the part markers and merge them with the headings by page order — and it is waiting for you at the end of the chapter.

22.5 The running head, from query

Flip to any body page and the top shows the book title on the left and the current chapter on the right, over a hairline rule. Getting the *current chapter* onto every page is the problem Chapter 17 warned you about and Chapter 21 solved. A running head drawn from `state` reads the state as of the top of its page, which fails for a chapter that starts halfway down. The fix is `query`, which sees the whole finished document at once:

```

set page(header: context {
  let n = here().page()
  let openers = query(heading.where(level: 1)).map(h => h.location().page())
  if n not in openers {
    let current = none
    for h in query(heading.where(level: 1)) {
      if h.location().page() <= n { current = h }
    }
    let label = if current != none {
      [#numbering("I", ..counter(heading).at(current.location())) - #current.body]
    } else { [] }
    grid(columns: (1fr, auto),
      text(font: font-head, size: size-small, fill: muted, style: "italic")[#title],
      text(font: font-head, size: size-small, fill: muted)[#label])
    v(3pt)
    line(length: 100%, stroke: 0.5pt + hairline)
  }
})

```

Read it from the top. `here().page()` is the number of the page we are drawing the header for. `query(heading.where(level: 1))` returns every chapter heading in the book; we walk them and keep the last one whose page is at or before this one — that is the chapter this page belongs to. From that heading we read its number with `counter(heading).at(current.location())` (a counter read at *some other* location, Chapter 17 again) and its title from `current.body`, and set them at the outer edge. The book title on the other side is just `title`, the template's own parameter, which the header closure can see because it was defined inside `book()`.

The little flourish: if the current page is itself a chapter opener — `if n not in openers` — we draw no header at all, because a running head on a chapter's first page is a thing careful books do not do. That check is only possible because `query` already told us which pages the chapter openers landed on.

The footer is the easy sibling, a centred page number read with the same counter you met in "Page X of Y":

```
set page(footer: context {
  set align(center)
  text(font: font-head, size: size-small, fill: muted)[#counter(page).display("1")]
})
```

Coming from LaTeX. This is `\pagestyle{headings}` and the `fancyhdr` package, minus the package and minus the incantations (`\leftmark`, `\rightmark`, `\markboth`, and the mystery of which one is which). You do not configure a mark-passing mechanism; you ask the finished document what chapter a page is in and print the answer. And there is no second compile to get it right — Typst iterates until the numbers settle, as Chapter 17 explained, so the head is correct the first time.

22.6 The contents page

The table of contents is the one place the template does *less* than you might expect, because Typst does most of it. The whole contents helper is:

```
#let _contents-page() = {
  block(above: 0pt, below: 1em)[
    #text(font: font-head, size: 22pt, weight: "bold", fill: ink)[Contents]
    #v(2pt)
    #line(length: 100%, stroke: 0.6pt + accent)
  ]
  show outline.entry.where(level: 1): set text(font: font-head, weight: "bold")
  outline(title: none, depth: 2, indent: 1.2em)
}
```

`#outline()` from Chapter 11 walks the headings, reads each one's page number, and draws the list with dotted leaders — and because it reads the same heading structure the running head does, the numbers in the contents and the numbers in the text can never disagree. We pass `title: none` and draw the "Contents" heading by hand, so that the outline's title does not get caught by the chapter-opener show rule and grow a giant "CHAPTER" masthead of its own. The `show outline.entry.where(level: 1)` rule bolds the chapter lines so they stand out from their subsections — the same `.where` selector logic, now aimed at outline entries. `depth: 2` keeps the contents to chapters and their sections; deeper subsections stay out.

22.7 The admonition boxes

`template/admonitions.typ` is a component library in the Chapter 18 sense: a handful of related functions built on the shared theme. There is one general box and five named variants. The general one is the content-argument pattern from Chapter 14 — `body` first, options named with defaults:

```
#let admonition(body, kind: "Note", color: note-color) = block(
  fill: color.lighten(90%),
  stroke: (left: 3pt + color),
  inset: (x: 10pt, y: 8pt),
  radius: (right: 3pt),
  width: 100%,
)[
  #text(font: font-head, fill: color, weight: "bold", size: size-small,
    tracking: 0.6pt)[#upper(kind)]
  #v(-0.35em)
  #set par(first-line-indent: 0pt)
  #body
]
```

A block, tinted with a very light version of the box's colour (`color.lighten(90%)`), a solid rule of that colour down the left edge, a bold uppercase label, and then the body. Because `body` is the last parameter, the trailing-content sugar works and you call it `#note[...]` with the text in brackets, exactly like a built-in — that is the whole reason Chapter 14 told you to put `body` last.

The five variants are not five more functions. They are one function with its arguments pre-filled, using `.with` from Chapter 14:

```
#let note = admonition.with(kind: "Note", color: note-color)
#let tip = admonition.with(kind: "Tip", color: tip-color)
#let important = admonition.with(kind: "Important", color: important-color)
#let warning = admonition.with(kind: "Warning", color: warning-color)
#let caution = admonition.with(kind: "Caution", color: caution-color)
```

Each is `admonition` with its label and colour baked in and the `body` still open. A chapter file imports the lot with `#import "../template/admonitions.typ": *` and writes `#warning[...]`; you get an orange box with a "WARNING" label. Improve the box once — more padding, a rounder corner — and all five improve together, because there is only ever one box.

22.8 Theming the code blocks

`template/code.typ` is a single function that installs show rules on `raw` — the code element — and returns the body wearing them, the whole-document `show: shape` from Chapter 10:

```
#let code-rules(body) = {
  show raw: set text(font: font-mono)
  show raw.where(block: false): box.with(
    fill: code-bg, inset: (x: 3pt), outset: (y: 3pt), radius: 2pt,
  )
  show raw.where(block: true): it => block(
    fill: code-bg, stroke: (left: 2pt + accent),
    inset: (x: 9pt, y: 8pt), radius: (right: 3pt), width: 100%,
    text(size: size-code, it),
  )
  body
}
```

Three rules, each aimed with the `.where` selectors from Chapter 10. Every code span, inline or block, gets the mono font. Inline code — `raw.where(block: false)` — is wrapped in a small tinted `box.with(...)` so a term like `main.typ` reads as a chip in the middle of a sentence. Block code — `raw.where(block: true)` — gets the full O'Reilly listing: a light fill, a coloured rule down the left, generous padding. Notice `box.with(...)` on the right of a `show` rule: the matched element becomes the box's final argument, so the rule wraps each inline snippet in a configured box. `book()` switches the whole thing on with one line, `show: code-rules`, and Typst's own syntax highlighter colours the keywords inside for free.

22.9 The index

The index is the piece that shows off the most machinery, because a back-of-the-book index is exactly the kind of thing you cannot type by hand and keep correct. It comes in two halves, in `template/index.typ`.

The first half is a marker you scatter through the prose. `#idx("compiler")` prints the word *and* records it:

```
#let idx(term) = [#metadata(term)<index-entry>#term]
```

`metadata` places an invisible value into the document — the term string — and the `<index-entry>` label tags it so we can find it again. The `#term` after it prints the word in the running text. That is all a marker is: a labelled, invisible note that says "the topic *compiler* is discussed right here."

The second half runs at the back of the book and builds the page:

```
#let make-index() = {
  pagebreak(weak: true)
  // ... draw the "Index" title by hand ...
  context {
    let entries = query(<index-entry>)
    let pages = (:)
    for e in entries {
      let term = e.value
      let p = counter(page).at(e.location()).first()
      let seen = pages.at(term, default: ())
      if p not in seen { seen.push(p) }
```

```

    pages.insert(term, seen)
  }
  // ... lay out sorted terms with their page lists ...
}
}

```

This is Chapter 21's `query` doing the thing Chapter 21 said it could do that `#outline` cannot. `query(<index-entry>)` returns every marker in the finished book. For each one, `e.value` is the term we stored and `counter(page).at(e.location())` is the page it landed on — a counter read at another element's location, one more time (Chapter 17). We group the pages under each term in a dictionary, dropping duplicates, then sort the terms alphabetically and lay them out in two columns with dotted leaders. The result, on the last page of the example, reads `compiler ... 3, 4` and `template ... 6, 9`, and every one of those numbers was discovered, not typed. Move a chapter and they all move with it.

22.10 Tying it together in `main.typ`

Everything above is machinery. `main.typ` is the book, and it is short:

```

#import "template/book.typ": book, part
#import "template/index.typ": make-index

#show: book.with(
  title: "Learning Typst",
  subtitle: "A field guide, in miniature",
  author: "Typeset with the Chapter 22 template",
)

#part[Getting started]
#include "chapters/01-getting-started.typ"
#include "chapters/02-first-document.typ"

#part[Going further]
#include "chapters/03-automating.typ"

#make-index()

```

Read it and the whole architecture is visible at a glance. The `#import` lines pull in the template's functions (Chapter 18). The `#show: book.with(...)` applies the entire design (Chapter 19). Each `#part[...]` drops a divider; each `#include` splices in a chapter file's rendered content — `include` for content, not `import` for definitions, the distinction Chapter 18 drew a sharp line under. And `#make-index()` at the end builds the index from the markers the chapters left behind. The design is in one place, the writing in another, and `main.typ` is the table of contents of the project itself.

The chapter files, in turn, are almost pure content. Each opens with a couple of imports for the components it uses and then just writes:

```

#import "../template/admonitions.typ": *
#import "../template/index.typ": idx

= Installing the tools
Before you can typeset a single page you need the #idx("compiler") ...
#note[ You do not have to install anything at all to try Typst. ... ]

```

That is the point of the whole exercise. The person writing Chapter 1 thinks about Chapter 1. They write `= Installing the tools` and `#note[...]` and `#idx("compiler")`, and the template turns those into a chapter opener, an O'Reilly note box, and an index entry — without the author touching a colour, a font, or a page number. The design was decided once, in `template/`, and now it just happens.

Coming from LaTeX. You have just written a document class and its packages — a `.cls` and a clutch of `.sty` files — except every one of them is a plain Typst file in the same language as the book, readable top to bottom,

with no `\makeatletter`, no `\RequirePackage`, and no separate class-writing dialect. The book's `main.typ` is your `\documentclass{book}` plus `\begin{document}`, and it is about as long.

22.11 What you've got

You have built a real book template, and in doing so you have seen every major idea in the book working at once:

- **A layered project** — constants in `theme.typ`, the template in `book.typ`, components in their own files, content in `chapters/`, all wired together by a short `main.typ`. Machinery and writing, kept apart (Chapters 18, 19).
- **A template that does front matter** — `book()` takes configuration first and `body` last, applied with `#show: book.with(...)`, and lays down a title page and a `#outline` table of contents before the body (Chapters 19, 11).
- **Chapter and part openers** — a function-form `show rule` rebuilds level-1 headings into chapter openers reading their number from the heading counter; a custom `counter("part")` numbers full-page part dividers (Chapters 10, 17).
- **A running head that never lies** — built with `context` and `query`, it names the chapter a page belongs to even when the chapter began mid-page, and it stays off the opener pages (Chapters 17, 21).
- **Reusable components** — admonition boxes from the content-argument pattern and `.with` variants, and code blocks themed by `raw show rules` (Chapters 14, 10).
- **A generated index** — `idx()` records terms with `metadata` and a label; `make-index()` queries them, reads each page with `counter(page).at(...)`, and builds the page by hand (Chapters 21, 17).

This is the payoff the whole book was building toward. Set rules, show rules, functions, closures, `.with`, imports, includes, context, counters, state, query, `#place`, `#outline` — none of them were the point on their own. The point was this: enough of them, arranged well, and you can bottle the entire design of a book and pour it over any content you like.

In Chapter 23 we take this template and assemble a fuller book with it — more chapters, real front and back matter, the works — and shake out the problems that only appear at length. Then in Chapter 24 we close the loop the preface promised: a pipeline that runs Markdown through Pandoc into Typst, feeds it to a template like this one, and lets a book — this book — typeset itself.

22.12 Exercises

Solutions are in Appendix A.

22.1. Reskin the whole book by editing one file. In `theme.typ`, change `accent` to a colour of your choosing and `font-head` to `"Libertinus Serif"`. Recompile and confirm that the headings, rules, part and chapter openers, admonition labels, and index letters all follow — from two edits, in one place.

22.2. Add a sixth admonition. In `admonitions.typ`, define `#let example = admonition.with(kind: "Example", color: accent)`, export it, and use it in a chapter file. Confirm it comes out styled like the others with no new box code written — the whole box already exists, you are only pre-filling it.

22.3. Give the running footer a "page X of Y" instead of a bare number, using `counter(page).final()` from Chapter 17 (remember `.first()` to pull the integer out of the array). Check that the total is correct on every page, including the first and last.

22.4. Add a `date` parameter to `book()` (defaulting to `none`) and print it in small muted text under the author on the title page — but only when a date is supplied, guarded with `if date != none`. Apply the template once with a date and once without, and confirm the dateless cover leaves no empty line.

22.5. (*Stretch.*) Put the parts into the contents page. Have `part()` record each divider with a `metadata` marker and a label (the way `idx` does), then build a custom contents page with `context : query` both the part markers and the level-1 headings, sort the combined list by page location, and lay it out so each part heading sits above the chapters below it. Compare it to the plain `#outline` version and notice that you have just built something `#outline` could not — which is the entire reason `query` exists.

Building the book

The template is done. Chapter 22 spent its whole length building one: a `book()` function that draws a title page and a table of contents, a `part()` divider, running heads that never lie, admonition boxes, themed code, a generated index — the entire look of a book, decided once and bottled up in a folder called `template/`. It compiles to a handsome eleven-page sampler.

A sampler is not a book. A book is fifteen chapters, a preface, a copyright page, two appendices and an index; it is page numbers that turn from roman to arabic somewhere near the start; it is a diagram that has to sit beside the chapter that mentions it, and a heading that keeps trying to fall off the bottom of a page. None of that is design work — the design is finished. It is *assembly*, and assembly has its own problems, most of which only show up once the document gets long enough to have them.

This chapter takes the Chapter 22 template and builds an actual book with it. The whole thing is `examples/116-assembling-a-book/`, and the first thing to notice is that we do not open the template folder even once. We import it and use it. That is the entire promise of a template, finally cashed.

23.1 The book as a manifest

Here is the complete `main.typ` for our little book, an almanac of weather. Read it top to bottom; there is nothing hidden.

```
#import "../115-oreilly-book-template/template/book.typ": book, part
#import "../115-oreilly-book-template/template/index.typ": make-index

#show: book.with(
  title: "The Weather Almanac",
  subtitle: "A very small book, assembled from parts",
  author: "M. Cumulus",
)

#part[Fair weather]
#include "chapters/01-a-clear-morning.typ"
#include "chapters/02-the-noon-sun.typ"

#part[Rough weather]
#include "chapters/03-the-gathering-storm.typ"

#make-index()
```

(That is the shape of it — we will add the front and back matter in a moment.) The first two lines reach *across* to another example folder and pull in the template by a relative path: `../115-oreilly-book-template/template/book.typ`. The template is not copied, not vendored, not reimplemented — it is imported, exactly the way Chapter 18 imported a library. A different book would import the same file and get the same design. That is what "reusable" means when it is true and not just claimed.

Everything after the imports is content, and there is almost none of it. A `#show: book.with(...)` applies the whole design (Chapter 19). Then the file is nothing but `#part[...]` dividers and `#include` lines. It reads like a table of contents because that is precisely what it is: a manifest that names the parts of the book and the order they go in. The writing lives elsewhere; `main.typ` just says how the pieces fit.

NOTE

Because `main.typ` imports a file *outside* its own folder, you compile it from the repository root and tell Typst where the root is: `typst compile --root . examples/116-assembling-a-book/main.typ book.pdf`. The `--root`

flag sets the boundary that relative paths may cross. Without it, Typst refuses to read files above the input's own directory — a safety fence, not a bug.

23.2 One file per chapter

The alternative to this is a single enormous `main.typ` with all fifteen chapters pasted into it, and you do not want it. A thousand-line file is slow to scroll, murder to search, and a nightmare the moment two people edit it at once. Splitting the book into one file per chapter is the same instinct that put the template's machinery in separate files — keep each thing small enough to hold in your head — applied to the writing instead of the design.

The tool for the splice is `#include`, and its difference from `#import` matters. `#import` pulls *definitions* — functions, variables — out of a file so you can call them. `#include` pulls the *rendered content* of a file and drops it into the document right there, as if you had typed it. You import the template because you want its `book()` function; you include a chapter because you want its words on the page. Chapter 18 drew that line; here is where it earns its keep.

A chapter file, then, is almost pure content. It imports the components it happens to use and then just writes:

```
#import "../115-oreilly-book-template/template/admonitions.typ": *
#import "../115-oreilly-book-template/template/index.typ": idx

= A clear morning

#lorem(45) The technical name for the calm is an #idx("anticyclone"): a broad
region of sinking air where the sky stays clear for days.

#note[ #lorem(24) ]
```

The `=` becomes a full chapter opener, the `#note[...]` becomes an O'Reilly box, the `#idx("anticyclone")` records an index entry — and the person writing this chapter thought about none of it. They wrote a heading, a note, and a marked term. The template did the rest, three folders away.

23.2.1 Assets live next to the chapter that needs them

Chapter 2 of the almanac has a little diagram of the sun, and the image file sits right beside the chapter that uses it, in `chapters/`:

```
#figure(
  image("sun-diagram.svg", width: 3.5cm),
  caption: [A schematic noon sun. The image file lives beside this chapter.],
)
```

The path `"sun-diagram.svg"` has no folders in it, and that is the point. In Typst a file path is resolved relative to the file it is *written in*, not relative to `main.typ` and not relative to wherever the content ends up after `#include`. So `image("sun-diagram.svg")` inside `chapters/02-the-noon-sun.typ` looks in `chapters/`, finds the file next to it, and renders it — even though the figure will land pages away in the finished book. Keep a chapter's images in its own folder and you can move, rename, or delete the whole chapter without hunting down assets scattered across the project.

23.3 Front matter, body, and back matter

Open any real book to the first pages and the page numbers are lowercase roman: i, ii, iii. That run — title page, copyright, preface, contents — is the *front matter*, and it is numbered separately so that the publisher can add or drop a foreword at the last minute without renumbering the whole book. Then the body begins and the numbers restart at a proud arabic 1. At the end comes the *back matter*: appendices, the index, a colophon, numbered right along with the body.

Our template already draws the title page and contents, and it already restarts the page counter at 1 before the body. What it does not do is give us a roman front matter of our own — a copyright page, a preface. Adding that is an assembly job, and it is the one genuinely fiddly thing in this chapter, so we will take it slowly.

The mechanism is two page settings and a counter reset. To number a run of pages in roman, you set the page's numbering pattern to `"i"`; to switch to arabic and start over, you set it to `"1"` and reset the page counter:

```
#set page(numbering: "i")      // front matter: i, ii, iii ...
// ... copyright, preface ...
```

```
#counter(page).update(1)    // restart the count
#set page(numbering: "1")    // body: 1, 2, 3 ...
```

That is the whole idea, and in a book built from scratch it is all you need. Our book has one wrinkle: the Chapter 22 template installed an *explicit* footer for the body — it prints the number itself rather than letting the page's `numbering` field do it. So for the front matter we hand those pages their own automatic footer with `footer: auto`, which does read the `numbering` field and prints our roman numeral. We wrap the whole front matter in a `#[...]` block so that the moment it closes, the template's own header and footer snap back for the body:

```
#show: book.with( /* ... title, subtitle, author ... */ )

// FRONT MATTER — roman, its own footer, no running head
#[
  #set page(header: none, footer: auto, numbering: "i")
  #counter(page).update(1)
  #include "front/copyright.typ"
  #include "front/preface.typ"
]

// BODY — restart at arabic 1
#set page(numbering: "1")

#part[Fair weather]
#counter(page).update(1)
#include "chapters/01-a-clear-morning.typ"
```

Scoping the front-matter settings inside `#[...]` is the trick that keeps this honest: a `set` rule lasts only to the end of the block it lives in (Chapter 9), so the template's running head and arabic footer return untouched afterward, and we never had to reach into the template to change them. The counter reset just before Chapter 1 makes the first chapter's page read 1 even though a part divider sits in front of it — the divider is unnumbered, so the reader's eye lands on a clean 1 at the top of the body.

Compile it and the numbers behave: the copyright page is *i*, the preface *ii*, and Chapter 1 opens on 1. The front-matter pages carry no running head; the body pages carry the chapter name, drawn by the template's `query`-based header from Chapter 22. The transition is invisible, which is exactly how a reader should experience it.

The front-matter files themselves are deliberately plain. A preface is *not* a chapter, so it must not be a `=` heading — that would trip the chapter-opener show rule and grow a giant "CHAPTER" masthead. Instead it draws its own modest title by hand, the same move the template uses for "Contents" and "Index":

```
#text(font: font-head, size: 20pt, weight: "bold", fill: ink)[Preface]
#line(length: 100%, stroke: 0.6pt + accent)
#lorem(55)
```

The back matter follows the same rule. Appendix A draws an "APPENDIX A" kicker and its title by hand rather than as a heading, so it stays out of the chapter numbering, and then `#make-index()` — imported from the template — queries every `#idx(...)` marker the chapters left behind and builds the index page. In the finished almanac the index reads `anticyclone ... 1`, `insolation ... 2`, `barometer ... 5`, and every one of those numbers was discovered by the compiler, not typed by us.

Coming from LaTeX. You have met all of this before under other names. `#include` is `\include`, and if you want to compile just two chapters while you work, you would reach for `\includeonly{...}` — in Typst you comment out the `#include` lines you are not using, no separate command needed. The roman-then-arabic dance is `\frontmatter`, `\mainmatter`, and `\backmatter` from the `book` class, which flip the page numbering and (for `\frontmatter`) unnumber the chapters. The difference is that those are opaque commands you trust; here the same behavior is three lines of `set page` and a counter reset, sitting in plain sight in your own `main.typ`, doing exactly and only what they say.

23.4 Building it

The build is one command:

```
typst compile --root . examples/116-assembling-a-book/main.typ book.pdf
```

Point Typst at the manifest, name the output, and a few milliseconds later `book.pdf` exists. That speed is not a party trick; it is the thing that makes a long document bearable to work on. But you do not want to retype that command after every edit, so for the actual writing loop you use `watch`:

```
typst watch --root . examples/116-assembling-a-book/main.typ book.pdf
```

Now Typst sits in the background and rebuilds the moment you save any file the book depends on — a chapter, an image, the template, `main.typ` itself. Keep the PDF open in a viewer that reloads on change and you have a live preview of the whole book beside your editor.

Here is where Typst's incremental compilation, promised back in Chapter 1, pays off at scale. When you change one word in Chapter 9, Typst does not re-typeset Chapters 1 through 8; it recomputes only what your edit touched and reuses the rest. A three-hundred-page book stays as responsive to edit as a three-page one. Anyone who has waited out a LaTeX build on a long document, gone to make tea, and come back to a warning about an underfull box knows exactly how large a mercy this is.

23.5 Problems that only appear at length

A one-page example never strands a heading or lands a chapter on the wrong side of a spread, because it has no length for those problems to hide in. A real book has plenty. Here are the ones you will actually hit, and the one-line fixes.

Chapters should start on a right-hand page. Open a printed book and every chapter begins on a *recto* — the right-hand page, odd-numbered — even when that leaves the previous left-hand page half empty. To force it, break to the next odd page before the chapter:

```
#pagebreak(to: "odd")
= The gathering storm
```

`pagebreak(to: "odd")` skips to the next odd page number, inserting a blank left-hand page if it has to. In a `main.typ` you would put it before the `#include` of any chapter you want on a recto, or fold it into the template's chapter-opener show rule to make it automatic for every chapter. (For a screen-only PDF that nobody will print double-sided, leave it off — recto starts are a courtesy to the binding, and they waste paper you do not have.)

Headings must not strand at the foot of a page. A heading that prints as the last line of a page, with its section starting overleaf, is an orphan, and it looks like a mistake because it is one. Typst prevents it by default: a heading is *sticky*, meaning it refuses to be separated from the content beneath it and drags itself to the next page rather than sit alone at the bottom. The catch is that a show rule which rebuilds a heading into a plain `block` — as the template does for chapter openers — can quietly drop that stickiness. If you ever see a rebuilt heading strand, put the stickiness back explicitly:

```
show heading: it => block(sticky: true, it)
```

A `sticky` `block` clings to whatever block follows it, which is the whole of what "keep this heading with its text" means.

The table of contents grows too long. The template's contents lists headings down to level 2 (`depth: 2`). That is fine for three chapters and overwhelming for thirty — a contents page that runs to four pages of sub-sub-sections is a contents page nobody reads. The fix is one number: set the outline's `depth` to `1` and list chapters only.

```
#outline(title: none, depth: 1)
```

It is worth knowing this lives in the template's contents helper, so changing it is a one-word edit in one place, and every book built from the template inherits the shorter contents.

Assets multiply. We covered the mechanism above — paths resolve relative to the file that names them — but the discipline is what matters at length. With forty figures across fifteen chapters, a flat `images/` folder becomes a swamp of `figure1.png`, `figure1-final.png`, `figure1-final-v2.png`. Keep each chapter's assets in the chapter's folder, named for what they are, and the swamp never forms.

23.6 Where this is going

Notice what we did *not* do in this chapter: write any prose. The almanac's chapters are `#lorem` filler, because the point was the scaffolding — the manifest, the includes, the numbering, the build. In a real project you would now sit down and write, in Typst markup, chapter by chapter, and the book would grow.

That is one honest way to make a book, and for a book conceived in Typst it is the right one. But you might already have your chapters written — in Markdown, say, because that is where the words started, or because a co-author will only touch Markdown, or because the book has to publish to the web as well as to print. Retyping all of it into Typst markup would be daft.

You do not have to. In Chapter 24 we close the loop the preface promised: a pipeline that runs Markdown through Pandoc, converts it to Typst, and pours it into a template exactly like this one — `#include`, front matter, index and all. It is how *this* book is built. The assembly you just learned is the target that pipeline aims at; now we automate the aiming.

23.7 What you've got

You can now take a finished template and build a real book on top of it:

- **A book as a manifest** — a `main.typ` that imports the template by relative path and then does nothing but list `#part[...]` dividers and `#include` chapter files, so the structure of the book is visible at a glance.
- **One file per chapter** — content split into small files spliced with `#include` (rendered content) rather than `#import` (definitions), each chapter's images living in the chapter's own folder.
- **Roman front matter and an arabic body** — `set page(numbering: "i")` for the front matter, a `counter(page).update(1)` and `set page(numbering: "1")` to restart the body, all scoped in a block so the template's own header and footer return by themselves.
- **Back matter** — an appendix drawn by hand to stay out of the chapter numbering, and an index generated from the markers the chapters left behind.
- **A build loop that scales** — `typst compile` for a one-shot build, `typst watch` for live editing, and incremental compilation keeping both fast no matter how long the book gets.
- **The at-length fixes** — `pagebreak(to: "odd")` for recto chapter starts, `block(sticky: true)` to keep a heading with its text, `depth` to tame a long contents, and a folder discipline that keeps assets from breeding.

23.8 Exercises

Solutions are in Appendix A.

23.1. Take `examples/116-assembling-a-book/` and add a fourth chapter. Write a short `chapters/04-the-clearing.typ` with a `= heading` and a paragraph of `#lorem`, then splice it into `main.typ` with one `#include` line under the second `#part`. Recompile and confirm it appears as Chapter 4, in the contents, with the right page number — all from a single added line.

23.2. Add a second front-matter page: a dedication, between the copyright page and the preface. It must be numbered in roman like its neighbours and must *not* be a `= heading` (so it grows no chapter masthead). Confirm the preface's page number shifts from *ii* to *iii* to make room, without you touching any number by hand.

23.3. Make every chapter start on a right-hand page. Add `#pagebreak(to: "odd")` before each chapter's `#include` in `main.typ`, recompile, and read the page numbers: some chapters should now begin on odd pages with a blank left-hand page inserted before them. Then explain, in a sentence, why you might *not* want this for a PDF that will only ever be read on screen.

23.4. Shorten the table of contents. In the template's contents helper (`_contents-page` in `book.typ`), change the outline's `depth` from `2` to `1` so the contents lists chapters but not their sections. Recompile the almanac and confirm the change; then note what you would have had to do instead if the `depth` had been hard-coded in fifteen different places rather than one.

23.5. (*Stretch.*) Give the book a real "half-title then title" opening and a back-matter colophon. Add a front-matter page before the template's title page that shows just the book's title in small caps (you will need to place it inside the front-matter block, and think about whether the template lets you put anything *before* its own title page — if not, say why, and put your half-title first in the roman run instead). Then add a colophon as the very last page, after the index: a short paragraph naming the fonts and the tool, drawn by hand like the appendix. Compile and page through the result; you now have a book with a proper front and back, assembled entirely from parts you imported.

The Pandoc bridge

Here is a confession about the book you are reading. Its chapters are not written in Typst. Every one of them — this one included — is a plain Markdown file, the same `.md` you'd write for a README or a GitHub wiki. You've been reading Typst's output all along, but the *source* is Markdown.

That should sound like a contradiction. This is a book about Typst; why isn't it written in Typst? Because Markdown is a wonderful format to *write* in — light, familiar, readable in a plain editor, painless to review on GitHub — and Typst is a wonderful engine to *typeset* with. This chapter is about having both: you write in Markdown, and a pipeline converts it to Typst and compiles it to a finished PDF. By the end, you'll understand exactly how this book turns itself into the thing in your hands, and you'll be able to do the same to any pile of Markdown you have.

The bridge between the two worlds is a program called **Pandoc**.

24.1 Pandoc, the universal converter

Pandoc is the Swiss Army knife of document conversion. It reads a few dozen formats and writes a few dozen more — Markdown, HTML, LaTeX, Word, EPUB — and, since version 3.0, **Typst**. It works by parsing your input into an internal document model and then rendering that model in the target format. Markdown in, Typst out.

The smallest possible taste, on the command line:

```
echo '# Hello' | pandoc --from gfm --to typst
```

```
= Hello
<hello>
```

Pandoc read a Markdown heading and wrote the Typst equivalent: `= Hello`, plus a `<hello>` label so the heading can be cross-referenced. `--from gfm` says the input is GitHub-Flavored Markdown (the dialect this book uses, with its tables and alerts); `--to typst` picks the output. That's the whole engine. Everything in this chapter is teaching that one command to produce not just correct Typst but *nicely typeset* Typst.

Point Pandoc at a real chapter and it handles the everyday things without help:

- `#`, `##`, `###` headings become `=`, `==`, `===`.
- `*emphasis*` and `**strong**` become `#emph[...]` and `#strong[...]`.
- Bullet and numbered lists become Typst lists.
- Fenced code blocks become Typst raw blocks, language tag and all.
- Markdown tables become `#table(...)` calls.
- Links become `#link(...)[...]`.

If that were all we needed, the pipeline would be one command. But a book wants more than a faithful transcription — it wants a title page, a table of contents, styled admonition boxes, a running design. Getting from "correct Typst" to "a book" is the interesting part, and it takes three steps.

24.2 The three-step pipeline

The whole build lives in `examples/117-pandoc-book-build/` and in the repository's `scripts/build-book.sh`. It does exactly three things:

1. **Convert** every chapter's Markdown into one big Typst *body* with Pandoc.
2. **Assemble** the book by putting a short *head* — one that imports and applies the Chapter 22 book template — in front of that body.
3. **Compile** the combined file to a PDF with Typst.

In shell, stripped to its bones:

```
pandoc book/*.md --from gfm --to typst \
  --lua-filter book-filter.lua \
  --output body.typ

cat head.typ body.typ > learning-typst.typ

typst compile learning-typst.typ learning-typst.pdf
```

The `book/*.md` glob expands in order, because the chapters are named `00-`, `01-`, `02-`, and so on — the numbering scheme earning its keep one last time. Pandoc concatenates them into a single body. We prepend the head with a plain `cat`, and Typst compiles the result.

NOTE

You might expect to use Pandoc's own `--template` feature to wrap the body, and you can — but there's a trap. A Pandoc template treats every `$...$` as a variable to substitute, and Typst uses `$...$` for math. The two notations collide badly. Concatenating a plain Typst head with `cat` sidesteps the whole problem: the head is pure Typst, Pandoc never sees it, and nothing gets confused. Sometimes the blunt tool is the right one.

Two of those steps are trivial (`cat` and `typst compile`). The craft is in the other two: the Lua filter that shapes Pandoc's conversion, and the head that applies the design. Take them in turn.

24.3 Teaching Pandoc new tricks with a Lua filter

Pandoc's default conversion is faithful but literal, and a book needs a few things it won't do on its own. Rather than post-process the output with fragile text substitution, Pandoc lets you insert a **filter**: a small program that runs on the parsed document *between* reading and writing, so you transform the document model itself. Filters can be written in Lua, a tiny language Pandoc embeds, and ours — `book-filter.lua` — is a short program doing a handful of small jobs.

24.3.1 Turning alerts into admonitions

Throughout this book, an admonition looks like this in Markdown:

```
> [!NOTE]
> Something worth pausing on.
```

That's a GitHub alert, and Pandoc's `gfm` reader understands it: it parses the block into a *Div* (a generic container) tagged with the class `note` — or `tip`, `important`, `warning`, `caution`. But Pandoc's Typst writer, left alone, flattens that *Div* into a plain `#block` and throws the flavour away. A note and a warning come out looking identical, which defeats the purpose.

The filter catches the *Div* before the writer sees it and rewrites it into a call to the matching admonition function — `#note`, `#tip`, `#warning`, and the rest — that the Chapter 22 template already exports:

```
local kinds = { note=true, tip=true, important=true,
                warning=true, caution=true }

function Div(el)
  for _, class in ipairs(el.classes) do
    if kinds[class] then
      local out = { pandoc.RawBlock("typst", "#" .. class .. "[") }
      for _, blk in ipairs(el.content) do
        if not (blk.t == "Div" and blk.classes:includes("title")) then
          table.insert(out, blk)
        end
      end
      table.insert(out, pandoc.RawBlock("typst", "]"))
      return out
    end
  end
end
```


Read it as a recipe. When Pandoc hands the filter a `Div`, we check whether it's one of our alert kinds. If so, we build a replacement: a raw chunk of Typst, `#note[`, then the alert's own content (minus the little inner `title Div` that just held the word "Note"), then a closing `]`. The content blocks in the middle are still ordinary Pandoc blocks, so the Typst writer converts *them* normally — we've only wrapped them in a function call. The result in the body is `#note[ ...converted content... ]` — a call to the very `note` function you built in Chapter 22 (a warning becomes `#warning[...]`, and so on). The template draws the coloured box. The alert's flavour survives, because we transformed the document at the structural level instead of patching its text.

24.3.2 Passing math through untouched

The second job is subtler. This book writes its math in Typst's syntax — `$sum_{i=1}^n$`, `$\pi r^2$`, `$a \leq b$` — right in the Markdown prose. But Pandoc, told to read dollar-math, assumes it's *LaTeX* and would try to convert it, which mangles Typst-native spellings like `\pi` (Typst's π) or `\infty` (Typst's ∞). We don't want conversion here; we want the exact text preserved. So the filter intercepts every piece of math and hands it straight back as raw Typst:

```
function Math(el)
  if el.mathtype == "InlineMath" then
    return pandoc.RawInline("typst", "$" .. el.text .. "$")
  else
    return pandoc.RawInline("typst", "$ " .. el.text .. " $")
  end
end
```

Because the source was already valid Typst math, wrapping the raw text back in `$...$` reproduces it perfectly. This is the quiet advantage of writing a book about Typst *in* files destined for Typst: the math needs no translation, only protection from a well-meaning converter.

24.3.3 Numbering the book like a book

A book's Preface shouldn't be "Chapter 1," and its appendices want letters, not numbers — but to Pandoc every `#` heading is just a top-level heading. Left alone, the numbered chapters would start at 2 (the Preface having eaten number 1). The filter fixes this by emitting the Preface and each Appendix as *unnumbered* headings, which Typst does not count:

```
function Header(el)
  if el.level == 1 then
    local text = pandoc.utils.stringify(el)
    if text == "Preface" or text:match("^Appendix") then
      return pandoc.RawBlock("typst",
        "#heading(level: 1, numbering: none)" .. text .. ")")
    end
  end
end
```

With the front and back matter set aside, the chapters in between number 1 through N exactly as they should. It's a small rule that turns a flat sequence of headings into the structure of a real book.

24.3.4 Making the links work

A book is full of links — "see [Appendix A]", "the runnable version is in `examples/003-first-document/`". In the Markdown these are ordinary relative links: `25-appendix-a-solutions.md`, `../examples/003-first-document/`. Left alone, Pandoc turns into a Typst link to that literal path — and in a single PDF those paths lead nowhere, rendering as oddly styled text that clicks into the void. The filter rewrites both kinds into something that works:

```
function Link(el)
  local t = el.target
  if t:match("^https?://") then return nil end      -- external: leave it
  local repo = t:match("^%.%./(examples.*)$")      -- an example folder...
  if repo then
    el.target = "https://github.com/.../tree/main/" .. repo -- ...→ GitHub
    return el
  end
  local md = t:match("^([%w%-]+)%.(md)")          -- another chapter...
```



```

if md then
  return { pandoc.RawInline("typst", "#link(<" .. md .. ">)[",
    table.unpack(el.content),
    pandoc.RawInline("typst", "]" ) }
end
end

```

A link into `examples/` becomes a real GitHub URL, so a reader of the PDF can click straight through to the example in the repository. A link to another chapter becomes `#link(<25-appendix-a-solutions>)[Appendix A]` — an internal jump. For that jump to land somewhere, each chapter heading needs a matching label, so the same heading rule from a moment ago also tags every chapter with one: `#heading(level: 1)[Appendix A] <25-appendix-a-solutions>`. The label is the file's own name — which is exactly what the link points at. That is also why the build converts the chapters *one file at a time*, passing each file's name to the filter: it's the one thing a heading can't otherwise know about itself. Now "see Appendix A" is a live cross-reference inside the PDF, the same kind you built by hand in Chapter 11.

24.3.5 Dropping the author's notes

The last job is a one-liner. Each chapter ends with an HTML comment holding solution notes for the appendix author — content that must never reach a reader. Pandoc reads it as raw HTML, and the filter simply discards all raw HTML:

```

function RawBlock(el)
  if el.format == "html" then return {} end
end

```

A few small functions, and Pandoc's literal conversion becomes a book-aware one.

24.4 The head: the Chapter 22 template, reused

The body that Pandoc produces is just content — headings, paragraphs, the `#note` and `#warning` calls the filter emitted. It has no page, no fonts, no title page, and no definition of `note`. Supplying all that is a template's job, and here is the payoff of the whole book: we do not write a new one. The `head.typ` we `cat` in front *imports the book template you built in Chapter 22* and applies it.

```

// head.typ (import paths shortened for print)
#import "../115-oreilly-book-template/template/book.typ": book
#import "../115-oreilly-book-template/template/admonitions.typ": note, tip, ...

#show: book.with(
  title: "Learning Typst",
  subtitle: "A hands-on guide to the Typst typesetting system",
  author: "Written with AI, under human direction",
  width: 21cm, height: 29.7cm, margin: (x: 2cm, top: 2.2cm, bottom: 2cm),
)

```

That is the entire head. The imports pull in the template's `book` function and its admonition variants — the very `note`, `tip`, and `warning` the Lua filter's output calls. The `#show: book.with(...)` applies the template (the document-wide show rule from Chapter 10, fed a configured function from Chapter 14), and because the Pandoc body is concatenated on *after* this line, that body becomes `book()`'s `body` argument. The template lays down its title page and contents, then styles every chapter that follows with the chapter openers, running heads, themed code, and admonition boxes it was built to produce.

Look at the `width / height / margin` arguments, because they carry a small lesson. Chapter 22's `book()` took a title and an author; it now also takes a page size, so the *one* template can dress both the pocket-sized sampler of Chapter 22 and this full book — which, with its wide code listings, wants an A4. Making the page a parameter cost a single line in the template and turned a demo into something genuinely reusable. A template earns the name the day a second, different document asks it for a favour.

That reuse is the whole point. The Markdown-to-PDF pipeline needed no new styling system: it feeds the converted content into the very template this book spent Chapter 22 building by hand. The design is one Typst template, whether the content arrives written in Typst (Chapter 23) or converted from Markdown here. Same template, two front doors.

24.5 Running it

With Pandoc and Typst installed, the whole book builds with one command from the repository root:

```
scripts/build-book.sh
```

It converts every chapter, assembles them under the template, compiles the result, and writes `build/learning-typst.pdf` — the whole book, with a title page, a linked table of contents, numbered chapters, styled admonitions, themed code, and typeset math, all from a folder of Markdown files. The preview beside this example (`examples/117-pandoc-book-build/out.png`) is its title page.

That is the sentence this whole book has been walking toward: **the book typesets itself**. The document you're holding is the output of running that script on the Markdown that describes it — and, because this chapter is one of those Markdown files, the description includes the description of how it's built. The snake, contentedly, eats its tail.

Coming from LaTeX. If you've used Pandoc to make PDFs before, it was almost certainly through LaTeX: `pandoc thesis.md -o thesis.pdf` quietly shelled out to a LaTeX engine. The Typst writer is the same idea with a faster, friendlier backend — no multi-gigabyte TeX install, compilation in a blink, and legible intermediate output you can actually read and debug. If you have a stack of Markdown and you want a PDF, `--to typst` is now the shortest path to a good-looking one.

24.6 What's simplified, honestly

A pipeline this short makes trade-offs, and it's only fair to name them. Markdown tables come through wrapped in figures, so they pick up "Table N" numbers you may not want. The index the template can generate stays empty, because the Markdown carries no `#idx` markers for it to collect. Fine typography you'd hand-tune in a real Typst document — exact page breaks, widow control around one figure — isn't something a batch conversion can decide for you. None of these are hard to improve; each is another small filter or a template tweak, and adding one is a good exercise. The pipeline is a foundation, not a ceiling.

What it proves is the important thing: Markdown and Typst are not rivals. Markdown is a superb way to *write*, Typst a superb way to *typeset*, and Pandoc is the bridge that lets you keep both. Write in the format that's pleasant to write in; compile with the engine that makes it beautiful.

24.7 What you've got

You can now turn a body of Markdown into a typeset Typst PDF:

- **Pandoc** converts Markdown to Typst with `pandoc --from gfm --to typst`, handling headings, emphasis, lists, code, tables, and links out of the box.
- **The three-step pipeline** — convert each chapter to a body, `cat` a head that applies the template in front, compile — keeps Typst code and Pandoc's `$. $` templating from colliding.
- **A Lua filter** shapes the conversion: GitHub alerts become the template's `#note` / `#warning` boxes, Typst-syntax math is passed through untouched, thematic breaks become rules, cross-chapter links become internal jumps and example links become GitHub URLs, the Preface and appendices (and their subsections) are left unnumbered, and author-only HTML comments are dropped.
- **The head reuses your template** — it imports and applies the Chapter 22 book template (passing an A4 page), so the Markdown-sourced book gets the exact design — title page, contents, chapter openers, running heads, code theme, admonitions — that you built by hand.
- **scripts/build-book.sh** runs the lot and typesets this entire book from its Markdown source.

And with that, the book has closed its own loop: it taught you Typst, and then it showed you the machine that printed it. What remains are the appendices — the solutions to the exercises, migration maps from LaTeX and Word, a quick-reference cheat sheet, and where to go next — reference material for when the reading is done and the writing begins.

24.8 Exercises

Solutions are in Appendix A.

24.1. Install Pandoc (if you haven't) and run the smallest pipeline by hand: write a three-paragraph Markdown file with a heading, a bold word, and a bullet list, and convert it with `pandoc --from gfm --to typst`. Read the output and match each piece of Typst to the Markdown it came from.

24.2. Take that same file and compile it to a PDF two ways: once through Typst (`pandoc ... --to typst -o out.typ` then `typst compile out.typ`) and, if you have a LaTeX install, once the traditional way (`pandoc file.md -o out.pdf`). Compare the speed and the results.

24.3. Add a GitHub alert (`> [!TIP]`) to your Markdown file and convert it to Typst *without* the filter. Confirm it comes out as a plain block with the word "Tip" buried inside, losing its styling — the problem the filter exists to solve.

24.4. Write a two-line Lua filter that drops every `RawBlock` of format `html` , and run your file through it with `--lua-filter` . Put an HTML comment in the Markdown and confirm it vanishes from the output.

24.5. (*Stretch.*) Build a miniature book of your own: two or three Markdown files, a short Typst preamble that sets a page size and defines an `admonition` function, and a shell script that converts the Markdown, concatenates the preamble, and compiles the result. You'll have rebuilt this chapter's pipeline in miniature — and you'll never be afraid of a folder of Markdown again.

is removed. Confirm it's gone from the `-t typst` output. 24.5 - Mirrors examples/117: N markdown files, a preamble.typ defining `#let admonition(kind, body) = ...` and page/text set rules, and a build.sh doing `pandoc(*.md) -> body.typ`, `cat preamble body > out.typ`, `typst compile`. Grade on a working end-to-end build, however minimal. The insight: converter + template + compile = a book from Markdown. -->

Appendix A · Solutions to the exercises

Here are worked solutions to every exercise in the book, chapter by chapter. Where a solution shows code, it's the part that matters — assume the ordinary document around it. The stretch exercises (the last one in most chapters) are open-ended by design, so what follows is one good answer, not the only one. If your version compiles and does the job, count it as correct.

Chapter 1 — Why Typst?

1.1 A word processor fuses content and appearance into a single binary file that you edit by direct manipulation: what you see on screen *is* the document. A typesetting system keeps the two apart — your content lives as marked-up plain text, the appearance lives in rules, and the layout is computed from the structure. The gap matters more as a document grows: with centralized rules one change propagates everywhere, the layout stays predictable, and because the source is plain text you can search it, diff it, and track its history.

1.2 A word processor is the better tool when the job is short, simple, or collaborative in a hands-on-the-page way: a one-page letter or invitation; a document a non-technical colleague has to edit live; anything that calls for real-time co-editing with tracked comments (though Typst's web app narrows that last gap). Any answer works as long as the *why* is coherent — short, simple, or WYSIWYG-collaborative all point at the word processor.

1.3 `range(1, 11)` counts 1 through 10; `range(1, 4)` counts 1, 2, 3 and stops — `range` is half-open, so it never reaches its second number. The loop therefore prints three rows: $2^1 = 2$, $2^2 = 4$, $2^3 = 8$. (Chapters 15 and 16 return to ranges properly.)

1.4 No single right answer; the win is simply having looked. The reference is organized by category — text, layout, math, and so on — and noticing that shape is the whole point of the two minutes.

Chapter 2 — Getting Typst running

2.1 Whatever installs Typst on your machine: `brew install typst` on macOS, `winget install --id Typst.Typst` on Windows, your distro's package or the GitHub binary on Linux, or `cargo install --locked typst-cli` anywhere with Rust. `typst --version` should then print something like `typst 0.15.0`. If you get `command not found`, the binary isn't on your `PATH`: open a fresh terminal, and make sure the install directory is on `PATH`.

2.2 Compile the file, then edit and recompile:

```
typst compile examples/003-first-document/main.typ out.pdf
```

Change the `=` heading line, run the command again, and the PDF changes. The output is regenerated from the source on every compile — you never edit the PDF directly.

2.3 `typst watch` prints `compiling ...` and then `compiled successfully in N ms` on each save. Introduce a deliberate error — delete a closing `]` — and watch prints an error with the file and line number, then keeps watching; fix and save, and it returns to a clean compile. That tight, forgiving loop is the point.

2.4 Export the same file twice, once at each resolution:

```
typst compile examples/004-output-formats/main.typ out.png
typst compile examples/004-output-formats/main.typ out.png --ppi 300
```

The 300-ppi file is larger and stays crisp when you zoom in; the default (144 ppi) is lighter and looks fine on screen. Send the 300-ppi version to a printer and the 144-ppi one to a chat. (ppi is pixels per inch.)

2.5 Any folder with a `main.typ` that sets `#set document(title: ..., author: ...)` and a heading of your choosing. After compiling, the title and author appear in the PDF viewer's document-properties panel (and the browser tab, if you open it there). Compiling with `--open` builds and opens the result in one step. This is your first taste of `#set`, which Chapter 9 covers in full.

Chapter 3 — Markup: the content layer

3.1 A title and two sections:

```
= My title
== First section
Some prose...
== Second section
More prose...
```

= is the title, == a section — and the space after the equals signs is required. Compile it and eyeball the size contrast between the two heading levels; that's the whole exercise.

3.2 Inline markup is `_emphasized_`, `*strong*`, and inline raw code with ``backticks``. A nested list is made purely by indentation:

```
- one
- two
  - two-a
  - two-b
- three
```

The `two-a` and `two-b` items become a sublist because they're indented past `two`; nothing else marks them.

3.3 Each of these does one specific job:

See pages 30--34, then a break---a real one---5~km on, and a literal `\*`.

-- is an en dash (for ranges), --- an em dash (for an aside), ~ a non-breaking space (so the 5 and the km never split across a line), and \ escapes the character after it. Check the PDF to catch, for instance, a single hyphen where you meant an en dash.

3.4 A term list and a link:

```
/ Title one: a note.
/ Title two: a note.
/ Title three: a note.

#link("https://example.com")[borrow it here]
```

A term list starts each line with `/ Term:` ; the link is `#link("url")[label]`. Any working term syntax and any correct link count.

3.5 One small document combining headings, emphasis, both list kinds, a link, and a fenced block nested under a numbered item. The trap flagged in the chapter's Lists warning: the block must be indented *under* its list item, or a flush-left block ends the list and the numbering restarts.

```
= Weeknight risotto

== Ingredients
- Arborio rice
- Stock
- A glass of _something_ white

== Method
+ Toast the rice.
+ Add stock a ladle at a time:

  ```
 stir, wait, repeat
  ```

+ Serve. See #link("https://example.com")[the original].
```

Because the code block is indented under step 2, the list keeps counting to 3 instead of restarting at 1.

Chapter 4 — Text and fonts

4.1 A document-wide set rule, then a local override:

```
#set text(font: "New Computer Modern", size: 12pt)
```

Only the `#text(size: 18pt, weight: "bold")` word changes.

The set rule governs the whole document; the `#text` call overrides just the span it wraps. Everything stays 12 pt regular except the one bold, 18 pt word.

4.2 Name any font Typst doesn't bundle — Helvetica, Arial, Comic Sans — and it still compiles, but prints warning: unknown font family: <name> and renders the text in the fallback (Libertinus Serif). Switch to a bundled family (Libertinus Serif, New Computer Modern, or DejaVu Sans Mono) and the warning disappears. The lesson: a named-but-missing font compiles, but the result isn't reproducible on another machine.

4.3 Subscripts, a superscript, and small caps:

```
Water is H#sub[2]O and carbon dioxide is CO#sub[2].
Einstein: E = mc#super[2].
Set in #smallcaps[Belgium].
```

`#sub[...]` lowers, `#super[...]` raises (the ² could also be typed as the literal character), and `#smallcaps[...]` sets small capitals.

4.4 Right-aligned tabular figures line up digit for digit in a clean column, because every digit is the same width. Proportional figures look ragged even when right-aligned, because a 1 is narrower than the other digits, so the columns of digits don't stack. The switch is one text property:

```
#set text(number-width: "tabular") // vs "proportional"
```

4.5 A narrow, justified French column:

```
#set page(width: 7cm)
#set par(justify: true)
#set text(lang: "fr")
#lorem(60)
```

French switches to guillemets and its own hyphenation patterns, so line breaks fall in different places than English. Turn hyphenation off with `#set text(hyphenate: false)` and the justified lines stretch, opening wider word gaps. Neither is objectively right — prefer whichever trades even spacing against broken words the way you like for that column.

Chapter 5 — Pages and layout

5.1 Asymmetric margins come from a per-side dictionary:

```
#set page(paper: "us-letter", margin: (left: 3cm, right: 2cm, y: 2.5cm))
#lorem(80)
```

The wider left margin pushes the text block rightward, so it sits away from the left edge. The per-side dictionary is the whole tool here.

5.2 A header and a hand-placed page number:

```
#set page(
  header: [Name #h(1fr) 12 July],
  footer: context [#h(1fr) #counter(page).display() #h(1fr)],
)
#lorem(40)
#pagebreak()
#lorem(40)
```

The moment you supply your own `footer:`, Typst's automatic page number vanishes, so you place it by hand with `counter(page).display()` — wrapped in `context` because the number depends on *where* it's drawn. The `#h(1fr)` springs push the header date to the right and centre the footer number.

5.3 The plain-length form does *not* indent the first paragraph after a heading (the usual typographic convention):

```
#set par(justify: true, first-line-indent: 1.5em, spacing: 0em)
```

Switch to the dictionary form to indent that first paragraph too:

```
#set par(first-line-indent: (amount: 1.5em, all: true), spacing: 0em)
```

In one sentence: `all: true` gives the first paragraph its indent; the plain length withholds it. Setting `spacing` small (or `0em`) collapses the gaps between paragraphs for the classic novel look.

5.4 Use the local `#columns` function, not `#set page(columns: 2)`, so the heading can span the full width:

```
= Heading spanning the full width
```

```
#columns(2, gutter: 1.5cm)[
  #lorem(80)
]
```

The page-level setting would force the heading itself into the first column; `#columns(...) [...]` confines the two-column flow to its own block and leaves the heading above it, spanning the width.

5.5 Open-ended. A complete answer combines the chapter's tools — one version:

```
#set page(
  paper: "a4",
  fill: rgb("#fbfbf5"),
  background: align(center + horizon,
    rotate(-30deg, text(size: 90pt, luma(90%)) [DRAFT])),
  header: [Internal memo #h(1fr) CONFIDENTIAL],
  footer: context align(center) [#counter(page).display("1 / 1", both: true)],
)
#set par(justify: true, leading: 0.5em)

#lorem(120)

#v(2cm)
#line(length: 6cm, stroke: 0.5pt)
Signature
```

Grade yourself on combining the pieces coherently, not on exact taste. The `both: true` in `display` gives the `1 / 3` total-pages form.

Chapter 6 — Figures and images

6.1 A sized image with alt text:

```
#image("shot.png", width: 5cm, alt: "The app's main window.")
```

The path is relative to the file's folder. Giving one dimension scales the other proportionally. `alt` is invisible on the page but written into the PDF for screen readers. Any real PNG or JPG works.

6.2 Wrap the image in a figure:

```
#figure(
  image("shot.png", width: 5cm),
  caption: [A caption.],
)
```

Note that the inner `image(...)` has no leading `#`: it's already inside code, as an argument to `figure(...)`. The output reads `Figure 1: A caption.` below the image, with the number supplied automatically.

6.3 Two figures labelled `<fig:one>` and `<fig:two>`, referenced with `@fig:one` and `@fig:two`. Move `fig:two` above `fig:one` in the source and it becomes `Figure 1` — the references renumber themselves on the next compile. The point: a reference tracks its label, not a number you typed. See `examples/027-referencing-figures/`.

6.4 Bake the shared box settings once with `box.with`, then vary `fit`:


```
#let b = box.with(width: 4cm, height: 2cm, stroke: 0.5pt)

#b[#image("photo.jpg", width: 100%, height: 100%, fit: "contain")]
#b[#image("photo.jpg", width: 100%, height: 100%, fit: "cover")]
#b[#image("photo.jpg", width: 100%, height: 100%, fit: "stretch")]
```

`contain` fits the whole image inside the box and letterboxes the spare space; `cover` fills the box and crops the overflow; `stretch` fills the box by distorting the image. Mirrors [examples/028-image-fit/](#).

6.5 Three figures whose `kind` differs, so each gets its own counter:

```
#figure(rect(width: 2cm, height: 1cm), caption: [A drawn shape.]) <fig:shape>
#figure(table(columns: 2, [a], [b]), caption: [A little table.]) <tbl:mini>
#figure(
  rect(width: 2cm, height: 1cm, stroke: (dash: "dashed")),
  kind: "diagram", supplement: [Diagram],
  caption: [A diagram.],
) <dia:flow>
```

See `@fig:shape`, `@tbl:mini`, and `@dia:flow` – each numbered 1.

A drawn shape takes the default kind (Figure 1), a table figure the `table` kind (Table 1), and the last one an explicit kind: `"diagram"` (Diagram 1). The insight: `kind` separates the counters, so all three read 1; `supplement` only sets the printed word. Mirrors [examples/029-figure-kinds/](#).

Chapter 7 — Tables and grids

7.1 A movie table with a flexible title column:

```
#table(
  columns: (1fr, auto, auto, auto),
  [*Title*], [*Director*], [*Year*], [*Rating*],
  [Metropolis], [Lang], [1927], [Great],
  [Rashomon], [Kurosawa], [1950], [Great],
  [Le Trou], [Becker], [1960], [Superb],
  [Stalker], [Tarkovsky], [1979], [Hypnotic],
  [Chungking Express], [Wong], [1994], [Dazzling],
)
```

Making `Title` a `1fr` column lets it absorb the slack while the shorter columns stay `auto`. A plain `columns: 4` works too; choosing the track kind to fit the content is the learning goal.

7.2 Per-column alignment plus zebra stripes:

```
#table(
  columns: 4,
  align: (left, left, right, center),
  stroke: none,
  fill: (x, y) => if calc.odd(y) { luma(240) },
  [*Title*], [*Director*], [*Year*], [*Rating*],
  [Metropolis], [Lang], [1927], [Great],
)
```

`align` takes a per-column list; `stroke: none` drops the rules; the `fill` function tints odd rows. Striped or ruled is an aesthetic call — either is fine with a reason.

7.3 A schedule with spans:

```
#table(
  columns: 3,
  table.cell(colspan: 3)[*Weekly schedule*],
  [Mon], [Tue], [Wed],
  table.cell(rowspan: 2)[Gym], [Email], [Design],
)
```

```
[Reading], [Ship],
)
```

The trap: the row *after* a `rowspan` cell supplies only the cells the span doesn't already cover — here the last row gives just two cells, because `Gym` still occupies the first column. Once you account for the spans, every row's cell count lines up; miscount and Typst shifts everything into the wrong slots.

7.4 The booktabs look: no grid, three explicit horizontal rules.

```
#table(
  columns: 3,
  stroke: none,
  inset: (x: 10pt, y: 6pt),
  table.hline(stroke: 1pt),
  [*Item*], [*Qty*], [*Price*],
  table.hline(stroke: 0.6pt),
  [Pens], [12], [3.00],
  [Ink], [2], [8.50],
  table.hline(stroke: 1pt),
)
```

`stroke: none` clears the default grid; a thick rule on top, a thin one under the header, and a thick one at the foot give the classic three-line table, with a roomy inset. Add `table.vline(x: k, stroke: ...)` for a single vertical rule between columns `k-1` and `k` — whether it helps is a judgement call (booktabs orthodoxy says skip verticals), but the exercise is about placing one on purpose.

7.5 A card built with `grid` rather than `table`:

```
#grid(
  columns: (auto, 1fr),
  column-gutter: 1em,
  row-gutter: 0.6em,
  // grid, not table: this is layout, not tabular data. A table would
  // announce itself as data to a screen reader and imply a grid of
  // records where there is only a card. grid keeps the structure honest
  // and draws no borders by default.
  grid.cell(colspan: 2)[*A fine pen*],
  [Price], [#lorem(30)],
)
```

The full-width title spans both columns; a small `auto` label column sits beside a `1fr` description. A `table` would misrepresent a layout card as data — the comment is the sentence the exercise asks for.

Chapter 8 — Math and equations

8.1 An inline formula and the quadratic formula as a display block:

Einstein showed that $E = mc^2$ ties mass to energy.

$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

The space in mc^2 matters: `mc` would be read as one two-letter identifier and error. Likewise the spaces inside $4ac$ and $2a$ make them products rather than names. That space-inside-`$...$` rule and the multi-letter-identifier trap are the whole lesson.

8.2 The Gaussian:

$$f(x) = \frac{1}{\sqrt{2\pi}} e^{-x^2/2}$$

The exponent must be parenthesised — $e^{-x^2/2}$ — otherwise only $-x^2$ (or less) rides up as the power. An explicit `dot` before `e` is fine too.

8.3 A numbered equation and a reference to it:

```
#set math.equation(numbering: "(1)")

$ sum_{i=1}^n i^2 = (n (n + 1) (2 n + 1)) / 6 $ <eq:squares>

The identity in @eq:squares sums the first $n$ squares.
```

`#set math.equation(numbering: "(1)")` turns on equation numbers; the label and `@eq:squares` do the cross-reference. Mind the spaces in `n (n + 1)` and `2 n`, and group the whole numerator in one paren pair over `6`.

8.4 A matrix and a piecewise definition:

```
$ M = mat(delim: "[", cos theta, -sin theta; sin theta, cos theta) $

$ op("sgn")(x) = cases(
  1 & "if" x > 0,
  0 & "if" x = 0,
  -1 & "if" x < 0,
) $
```

`delim: "["` gives square brackets and `;` ends each matrix row. In `cases`, the commas separate the branches and `&` aligns the conditions; the quoted `"if"` prints as upright text inside `math`.

8.5 A derivation aligned on its equals sign, and a custom operator:

```
$ a^2 - b^2 &= a^2 - a b + a b - b^2 \
           &= a(a - b) + b(a - b) \
           &= (a - b)(a + b) $
```

Each line uses `\` for the newline and `&` before the `=` so the equals signs stack. Any valid intermediate step is fine. For the operator, `math.op` mints one that typesets upright and takes a subscript like a built-in:

```
#let argmax = math.op("argmax", limits: true)

$ hat(theta) = argmax_theta L(theta) $
```

`limits: true` puts the subscript directly under `argmax` in a display equation. The whole thing must build with no warnings — check it.

Chapter 9 — Set rules

9.1 Two document-wide set rules:

```
#set text(font: "New Computer Modern", size: 12pt)
#set par(justify: true)

= A heading
#lorem(30)

#lorem(30)
```

Both rules govern the whole document, so a third paragraph pasted anywhere below inherits the same font, size, and justification with no extra markup.

9.2 Automatic heading numbers:

```
#set heading(numbering: "1.")

= First
== A sub-heading
```

The top-level heading shows `1.`, the `==` sub-heading `1.1`, with no manual counting. The rule has to sit *above* the headings it governs.

9.3 Custom list and enum markers:

```
#set list(marker: [--])
#set enum(numbering: "a")

- one
- two

+ first
+ second
```

Any content works as a list marker (`[>]`, `[·]`, `sym.arrow.r`), and `numbering` takes a pattern string. The marker can even be a tuple that alternates by depth, e.g. `marker: ([--], [·])`.

9.4 A set rule's scope stops at the end of its enclosing block:

```
A normal line.

#[
  #set text(fill: blue, style: "italic")
  Only this block is blue and italic.
]

Back to normal.
```

The `#[...]` content block fences off the styling. The common slip is dropping the `#` before `[`, which makes it plain bracketed markup text.

9.5 A `draft` flag toggling conditional set rules:

```
#let draft = true

#set text(size: 14pt, fill: red) if draft
#set par(leading: 1.5em) if draft

= Report
#lorem(30)
```

Flip `draft` to `false` and both conditional rules skip, snapping the document back to its clean defaults with a one-character edit. The `if draft` conditions must sit below the `#let` that defines the flag. The optional `DRAFT` header genuinely needs a show rule or a direct call to place content, so treat it as a deliberate bridge to Chapter 10.

Chapter 10 — Show rules

10.1 A rule above the body draws a line before each level-1 heading:

```
#show heading.where(level: 1): it => block[
  #line(length: 100%, stroke: 0.6pt + green.darken(30%))
  #text(fill: green.darken(30%))[#it.body]
]

= A styled heading
```

Because the line is emitted before `it.body` inside the block, every level-1 heading gets a rule above it; wrapping in `block` preserves the heading's block layout. Use plain `heading: if` if the document has only one level.

10.2 Two show-set rules that leave body text alone:

```
#show quote.where(block: true): set text(style: "italic")
#show raw: set text(fill: red)
```

Ordinary paragraphs are untouched because neither selector matches them; only block quotes and raw code pick up the new styling.

10.3 A string replacement and a regex:

```
#show "Projectname": strong(smallcaps[Projectname])
#show regex("[A-Z]{2,}"): it => smallcaps(lower(it.text))
```

The first restyles a fixed word; the second small-caps every run of two or more capitals (an acronym). They overlap — the regex would also catch an all-caps project name — so either accept that or keep the project name mixed-case. Spotting the overlap is half the exercise.

10.4 The buggy rule rebuilds a `strong` element inside its own show rule, which the same rule then matches again, forever — Typst stops with `maximum show rule depth exceeded`. The fix is to reuse `it` instead of constructing a new `strong`:

```
#show strong: it => underline(it)
```

`underline(it)` wraps the strong element you were handed, adding an underline without creating anything the rule matches again.

10.5 A whole-document wrapper — the skeleton of a template:

```
#let note(body) = {
  set page(width: 12cm, height: auto, margin: 1.5cm)
  set par(justify: true)
  show heading: set text(fill: navy)
  show regex("[A-Z]{2,}"): it => smallcaps(lower(it.text))
  body
}

#show: note

= A note
NASA and ESA both agree. #lorem(20)
```

A selector-less `#show: note` pipes the whole document through `note`, which applies four effects at once. This is exactly the shape of a template; Chapter 19 grows it into a real one.

Chapter 11 — References and cross-references

11.1 Without numbering, `@sec:one` errors with `cannot reference heading without numbering` (the hint even suggests `#set heading(numbering: "1.")`). Add that set rule and the reference prints `Section 1` as a link — `Section` is the default heading supplement. Mirrors `examples/056-`.

11.2 Label a sub-heading `== Sub <sec:sub>` and `@sec:sub` prints `Section 1.1`. Reorder the two top-level headings and everything rennumbers in one compile: the moved heading and every reference to it update together. References track the label and the structure, never a number you typed.

11.3 A per-reference supplement overrides the default:

```
#figure(rect(width: 3cm), caption: [A rectangle.]) <fig:demo>

@fig:demo prints "Figure 1".
#ref(<fig:demo>, supplement: [Fig.]) prints "Fig. 1".
```

Adding `#set figure(supplement: [Diagram])` makes the plain reference *and* the caption read `Diagram 1`, but the per-reference `supplement: [Fig.]` still wins for that one mention — local beats global. Mirrors `examples/057-`.

11.4 `#outline()` lists every heading; `#outline(depth: 1)` lists only level-1 headings, dropping the sub-headings; `#outline(target: figure)` lists the figures by caption instead. All three read the same document structure, and numbered entries need `#set heading(numbering: "1.")`. Mirrors `examples/058-` and `examples/059-`.

11.5 A show rule that styles heading and figure references differently by branching on the referenced element's kind:

```
#show ref: it => {
  let el = it.element
  if el != none and el.func() == figure {
    box(fill: luma(235), inset: (x: 4pt), radius: 2pt, it)
  } else {
```

```

    text(fill: blue, weight: "bold", it)
  }
}

```

`it.element` is the referenced element and `.func()` identifies its kind, so the rule can box figure references and colour the rest. Ending each branch with `it` keeps Typst's automatic supplement-and-number intact — only the look changes. Guard with `el != none`, since some references have no element. Mirrors [examples/060-](#).

Chapter 12 — Citations and bibliographies

These solutions depend on a bibliography file, so the snippets don't compile on their own — pair each with the `refs.yml` (or `.bib`) it cites.

12.1 Any `refs.yml` with a book (`type: book`, a `publisher`) and an article (`type: article`, a parent periodical, a page range) works:

```

pigeon2019:
  type: book
  title: "Pigeons: an urban history"
  author: "Featherstone, Ada"
  year: 2019
  publisher: Corvid Press

```

As shown by `@pigeon2019`, the birds adapt fast.

```
#bibliography("refs.yml", style: "apa")
```

Check that `@key` produces (Author, Year) under APA and that the list sorts by first author's surname with a hanging indent. The usual mistake is an unquoted title containing a colon, which breaks the YAML parse. Mirrors [examples/061-first-citation/](#).

12.2 `title: [References]` renames the heading (the default is `Bibliography`). Switching `style: "apa"` to `"ieee"` changes in-text citations from (Author, Year) to numbered [1], and the list becomes numbered in order of first citation, authors initials-first, titles in quotes. Only the one string changed. See [examples/065-switching-styles/](#).

12.3 Add a source with three authors, e.g. `author: ["Hattie, John", "Timperley, Helen", "Clarke, Shirley"]`. In-text APA collapses it to (Hattie et al., 2007), while the list entry spells out Hattie, J., Timperley, H., & Clarke, S. (2007). — APA's three-or-more-author rule, applied for you. See [examples/062-apa-reference-list/](#).

12.4 The prose form and a page locator:

```

#cite(<pigeon2019>, form: "prose") // "Featherstone (2019)"
@pigeon2019[p.~10]               // "(Featherstone, 2019, p. 10)"

```

`form: "prose"` gives the running Author (Year) form; the bracket locator `@key[p.~10]` adds a page, printing (Author, Year, p. 10). The `~` is a non-breaking space (a plain space is accepted too); `supplement: [p.~10]` on the function form is equivalent. See [examples/063-citation-forms/](#).

12.5 Open-ended. The core is a valid `.bib` with a book, an article, and a web page (`@book` / `@article` / `@misc` or `@online`), compiled with `style: "apa"`. `full: true` makes uncited entries appear. Point `style: at` at any real `.csl` file by path and Typst renders through it — it speaks CSL natively. Credit yourself for naming one concrete thing the journal's CSL does that built-in APA doesn't (abbreviated journal names, a different author separator, no DOI). If you hit the `council-of-science-editors` deprecation warning, use the replacement name the warning prints.

Chapter 13 — From markup to code

13.1 A sentence with no literal data, only interpolations:

```

#let name = "Ada"
#let age = 36
#let colour = "green"

```

My name is `#name`, I am `#age`, and I like `#colour`.

Bare `#name` and `#age` need no parentheses because each is a single token.

13.2 Reading the expressions off:

```
#(10 - 2 * 3)           // 4    - multiplication first, then 10 - 6
#repr(9 / 4)            // 2.25 - division always yields a float
#("na" + "na" + " batman") // "nana batman" - strings join
#(5 <= 5)               // true
#(not (3 > 4))           // true - 3 > 4 is false, and not false is true
```

The decimal on `9 / 4` is the tell that `/` always produces a float; `#repr(8 / 4)` prints `2.0` for the same reason, even though it divides evenly.

13.3 Any three values, at least one being content:

```
#type(42)      / #repr(42)      // int / 42
#type("hi")    / #repr("hi")    // str / "hi"
#type([*x*])   / #repr([*x*])   // content / strong(body: [x])
```

`repr` of content shows its internal element form — `strong(body: [x])` here. The exact text doesn't matter; the point is that `[...]` is a value of type `content`.

13.4 A single source of truth for a small invoice:

```
#let a = 10
#let b = 4
#let c = 6
#let subtotal = a + b + c           // 20
#let tax = calc.round(subtotal * 0.06, digits: 2) // 1.2
#let total = subtotal + tax         // 21.2

#table(columns: 2,
  [Subtotal], [#subtotal],
  [Tax], [#tax],
  [Total], [#total],
)
```

Change one price, recompile, and the subtotal, tax, and total all follow — the numbers live in one place and everything downstream is computed.

13.5 Everything the reader sees comes out of one code block:

```
{
  let a = 8
  let b = 5
  [#a and #b sum to #(a + b) and multiply to #(a * b).]
}
```

Prints `8` and `5` sum to `13` and multiply to `40`. The `let` lines produce nothing, so the block's value is just the trailing content line, which carries the sentence to the page. The arithmetic is parenthesised inside the brackets so it's computed, not printed literally.

Chapter 14 — Functions and closures

14.1 One definition, reused with no copy-paste drift:

```
#let kbd(key) = box(
  stroke: 0.5pt, inset: (x: 4pt, y: 1pt), radius: 3pt,
)[#raw(key)]
```

Press `#kbd("Ctrl") + #kbd("C")` to copy.

14.2 A named parameter with a default:

```
#let kbd(key, fill: luma(240)) = box(
  fill: fill, stroke: 0.5pt, inset: (x: 4pt, y: 1pt), radius: 3pt,
)[#raw(key)]

#kbd("Esc") and #kbd("Enter", fill: yellow)
```

Most calls stay short; you override `fill` only when you want a different colour.

14.3 A component with a positional body and named options:

```
#let callout(body, title: "Note", color: blue) = block(
  fill: color.lighten(85%), stroke: (left: 3pt + color),
  inset: 10pt, width: 100%,
)[#text(fill: color, weight: "bold")[#title] \ #body]

#callout[
  Intro line. \
  - a
  - b
]

#callout([A short aside.], title: "Aside")
```

`#callout[...]` and `#callout([...])` are the same call — the trailing content block becomes the `body` argument. Everything inside the brackets, list and all, arrives as one content value.

14.4 A variadic average:

```
#let avg(..nums) = {
  let xs = nums.pos()
  xs.sum() / xs.len()
}

#avg(4, 8, 12)           // 8
#let scores = (10, 7, 9, 8)
#avg(..scores)           // 8.5
```

`..nums` collects any number of positional arguments; `.pos()` turns them into an array to sum and count. Spreading `..scores` unpacks the array back into separate arguments, so one function serves both call styles.

14.5 A closure factory that returns a ready-made show-rule function:

```
#let heading-style(color) = it => block(below: 0.8em)[
  #text(fill: color, weight: "bold")[#it.body]
  #v(-6pt)
  #line(length: 100%, stroke: 0.6pt + color)
]

#show heading: heading-style(navy)    // swap navy -> maroon to reskin

= A heading
```

The returned closure captures `color`, so picking a palette is now a one-line change — the embryo of a template (Chapter 19).

Chapter 15 — Control flow

15.1 One condition, two content blocks, exactly one printed:

```
#let temperature = 28
#if temperature >= 20 [Warm — leave the coat at home.] else [Bring a jacket.]
```

Drop the number to 12 and the output flips.

15.2 A table of powers of two:

```
#for n in range(1, 11) [
  $2^#n$ = #calc.pow(2, n) \
]
```

`range(1, 11)` runs `n` from 1 to 10. `range(1, 4)` would give only 1, 2, 3 — three lines, $2^1=2$ through $2^3=8$ — because `range` is half-open and stops before its second number. (That's the payoff of Exercise 1.3.)

15.3 A cart, listed and summed:

```
#let cart = (("Pen", 2), ("Notebook", 5), ("Stapler", 8))

#for (item, price) in cart [- #item: #price]

#let total = 0
#for (item, price) in cart { total = total + price }
Total: #total // 15
```

The loop header destructures each `(item, price)` pair, and a running variable accumulates the total across the second loop.

15.4 Skip the multiples of three with `continue`:

```
#for n in range(1, 21) {
  if calc.rem(n, 3) == 0 { continue }
  [#n ]
}
```

Prints 1 2 4 5 7 8 10 11 13 14 16 17 19 20. `continue` abandons the current iteration whenever `n` divides by 3. `range(1, 21)` reaches 20 (half-open again).

15.5 A multiplication table from nested loops fed to a spread:

```
#table(
  columns: 5,
  ..for r in range(1, 6) {
    for c in range(1, 6) {
      ([#{ r * c }],)
    }
  }
)
```

The inner loop yields one single-element array `([...],)` per cell, and the spread `..` flattens them all into the table's arguments. The trailing comma is what makes `([...],)` an array rather than a parenthesised value. Change both `6`s to `11` and `columns: 5` to `columns: 10` and the table grows in both directions with no other edit. Any working nested-loop version passes; a row-at-a-time build with string concatenation is fine too.

Chapter 16 — Arrays, dictionaries, and strings

16.1 Array basics:

```
#let nums = (8, 3, 11, 6, 3, 9)

#nums.len() // 6
#nums.first() // 8
#nums.last() // 9
#nums.at(-1) // 9 — negative indices count from the end
#nums.slice(2, 4) // (11, 6) — the middle two
#nums.sorted() // (3, 3, 6, 8, 9, 11)
```

`sorted()` returns a *new* array; print `nums` again afterward and it's still `(8, 3, 11, 6, 3, 9)`. Nothing you called mutated the original.

16.2 Transform, keep, and fold with `map`, `filter`, and `sum`:

```
#let prices = (4.50, 12.00, 3.25, 8.75)

#prices.map(p => p * 1.21)           // add VAT to each
#prices.filter(p => p > 5.00)         // (12.00, 8.75)
#prices.sum()                        // 28.50

// VAT-inclusive total of the items over 5.00, chained:
#prices.filter(p => p > 5.00).map(p => p * 1.21).sum() // 25.0075
```

They chain left to right. Round with `calc.round(x, digits: 2)` if you want a tidy 25.01.

16.3 A dictionary you build up and query safely:

```
{
  let book = (title: "Dune", author: "Herbert", year: 1965)
  book.insert("pages", 412)
  let dropped = book.remove("year") // dropped is 1965
  [
    #emph(book.title) by #book.author, #book.pages pages. \
    Keys now: #repr(book.keys()) \
    Publisher: #book.at("publisher", default: "unknown")
  ]
}
```

Keys are strings. `insert` adds a pair, `remove` deletes one and returns its value, and `at(..., default: ...)` reads a possibly-missing key without crashing — here it yields `unknown`. The binding must be a `let` so it can be mutated.

16.4 Split a "Surname, First" string and rebuild it in natural order:

```
#let name = "Lovelace, Ada"
#let parts = name.split(", ") // ("Lovelace", "Ada")
#(parts.at(1) + " " + parts.at(0)) // "Ada Lovelace"

// or, more elegantly:
#name.split(", ").rev().join(" ") // "Ada Lovelace"
```

`split` breaks the string on `", "`; reversing the two parts and joining with a space gives natural order.

16.5 An array of song dictionaries rendered as a sorted table with a computed `m:ss` column:

```
#let songs = (
  (title: "Blackbird", artist: "The Beatles", seconds: 138),
  (title: "Teardrop", artist: "Massive Attack", seconds: 329),
  (title: "Roygbiv", artist: "Boards of Canada", seconds: 161),
  (title: "Svefn-g-englar", artist: "Sigur Ros", seconds: 594),
)

#table(
  columns: 4,
  table.header([Title], [Artist], [Seconds], [m:ss]),
  ..songs.sorted(key: s => s.seconds).map(s => (
    s.title,
    s.artist,
    [#s.seconds],
    {
      let m = calc.div-euclid(s.seconds, 60)
      let r = calc.rem(s.seconds, 60)
      let rr = if r < 10 { "0" + str(r) } else { str(r) }
      [#m:#rr]
    },
  )).flatten(),
)
```

`sorted(key: ...)` orders by duration, `map` turns each dictionary into a four-cell row, and `flatten` unrolls the rows into the flat argument list the spread needs. `calc.div-euclid(seconds, 60)` gives the whole minutes and `calc.rem` the leftover seconds; the `r < 10` pad is what makes a nine-second song read `0:09` and not `0:9`. Add `show table.cell.where(y: 0): strong` for a bold header if you like. Mirrors [examples/088-data-to-table/](#).

Chapter 17 — Context, state, and counters

17.1 The same source line, evaluated per location:

```
#set page(width: 9cm, height: 6cm, margin: 1cm)

Some text. #context [This is page #counter(page).display().]
#pagebreak()
#context [This is page #counter(page).display().]
```

The line prints `1` on the first page and `2` on the second: `context` is evaluated where it lands, so one line yields two answers.

17.2 "Page x of y" in the footer:

```
#set page(width: 9cm, height: 7cm, margin: 1cm, footer: context [
  #set align(center)
  p. #counter(page).display() / #counter(page).final().first()
])

#lorem(40) #pagebreak() #lorem(40) #pagebreak() #lorem(20)
```

The footers read `p. 1 / 3`, `p. 2 / 3`, `p. 3 / 3`. `.final()` returns an array with the counter's last value, and `.first()` pulls the integer out. The shorthand `counter(page).display("1 / 1", both: true)` does the same job.

17.3 A custom counter for numbered notes:

```
#let fn = counter("figure-note")
#let note(body) = { fn.step(); [*Note #context fn.display().* #body] }

#note[First.]
#note[Second.]
#note[Third.]
#note[Fourth.]
```

The four notes number themselves `Note 1` through `Note 4`. `step()` runs outside `context` (it advances the counter in document order); `display()` runs inside `context` (it reads the value where it's printed). Reorder the calls and the numbers follow the new order automatically.

17.4 A number from the end of the document, printed at the top:

```
#context [This page has #fn.final().first() notes.]
```

Placed at the top, it still reports the correct total. Typst lays the whole document out to discover the counter's final value, then makes another pass to fill that value into the line — the "future" number is knowable because the entire document is laid out before any final PDF is emitted.

17.5 A running header driven by `state`:

```
#let sec = state("section", "Introduction")

#set page(header: context emph(sec.get()))

= Introduction
#lorem(30)

#sec.update("Methods")
#pagebreak()
```

```
= Methods
#lorem(30)
```

The header reads the current section from state. The crucial subtlety: the `sec.update("Methods")` must come *before* the `#pagebreak()`, so the new value is already in effect at the top of page two, where the header is drawn. Initialising the state with the first section's title keeps page one right too.

To break it on purpose, give a state an update that depends on its own `.final()`:

```
#let s = state("s", 1)
#context s.update(s.final() + 1)
#context s.get()
```

This still compiles, but Typst prints a warning that the document did not converge within five attempts, listing the values it tried (1, 2, 3, 4, 5, final 6). The moral: a value must never depend on its own final result.

Chapter 18 — Your own functions

18.1 A definition in one file, imported into another. `keys.typ`:

```
#let kbd(key) = box(
  stroke: 0.5pt, inset: (x: 4pt, y: 1pt), radius: 3pt,
)[#raw(key)]
```

`main.typ`:

```
#import "keys.typ": kbd

Press #kbd("Ctrl") + #kbd("C") to copy.
```

Compiling `keys.typ` on its own renders a blank page — it only defines. `main.typ` imports the name and shows the caps. The relative path `"keys.typ"` resolves because the two files sit together.

18.2 Give `keys.typ` a second name, then reach both three ways. In `keys.typ`:

```
#let key-color = luma(245)
#let kbd(key) = box(
  fill: key-color, stroke: 0.5pt, inset: (x: 4pt, y: 1pt), radius: 3pt,
)[#raw(key)]
```

The three imports:

```
#import "keys.typ": kbd, key-color // named: specific names in scope
#import "keys.typ": * // star: everything in scope
#import "keys.typ" // module: reach via keys.kbd, keys.key-color
```

Named and star let you write `kbd(...)` directly; the plain module import needs the `keys.` prefix. Same two names underneath — only the spelling at the call site differs.

18.3 A shared brand colour that both components default to. `lib.typ`:

```
#let brand = rgb("#2b6cb0")

#let callout(body, title: "Note", color: brand) = block(
  fill: color.lighten(85%), stroke: (left: 3pt + color),
  inset: 10pt, width: 100%,
)[#text(fill: color, weight: "bold")[#title] \ #body]

#let tag(body, color: brand) = box(
  fill: color, inset: (x: 6pt, y: 3pt), radius: 4pt,
)[#text(fill: white, weight: "bold")[#body]]
```

`main.typ`:

```
#import "lib.typ": brand, callout, tag

#callout[Default.] #callout(color: green)[Override.]
#tag[Default] #tag(color: red)[Override]
```

Both components default `color: brand`, so editing `brand` in `lib.typ` moves every default-coloured element at once, while explicit overrides stay put.

18.4 `include` splices rendered content; `import` would not. With `intro.typ` and `body.typ` each holding a heading and some prose, `main.typ`:

```
#set page(width: 12cm, height: auto, margin: 1.5cm)
#set text(font: "Libertinus Serif")

#include "intro.typ"
#include "body.typ"
```

`include` drops the rendered content of each file exactly where its line sits, in call order. `import` would have brought names into scope but nothing visible — that's the difference the exercise is after.

18.5 Two whole-document looks, swapped with one line. `looks.typ`:

```
#let formal(body) = {
  set text(font: "Libertinus Serif", size: 11pt)
  set page(margin: 2.2cm)
  show heading: set text(fill: navy)
  body
}

#let playful(body) = {
  set text(font: "New Computer Modern", size: 12pt)
  set page(margin: 1.4cm)
  show heading: set text(fill: fuchsia)
  body
}
```

`main.typ`:

```
#import "looks.typ": formal, playful

#show: formal          // change to `playful` to reskin

= Title
#lorem(40)
```

One `#show:` line swaps the entire look; the content never moves. That's two templates and a one-line switch between them — most of what Chapter 19 formalises.

Chapter 19 — Templates

19.1 Inside `report`'s block body, among the other set rules and before `body`, add one line:

```
show heading: set text(fill: navy)
```

Then `#show: report` over a document with a couple of headings recolours every one. The show rule is scoped to the body the function wraps, so a single line reskins all the headings at once.

19.2 An optional date on the title block:

```
#let report(title: none, author: none, date: none, body) = {
  align(center)[
    #text(size: 18pt, weight: "bold")[#title] \
    #text(style: "italic")[#author]
    #if date != none [ \ #text(size: 9pt, fill: luma(40%))[#date] ]
  ]
  body
}
```

```

    ]
  body
}

```

Apply it once with `date: "July 2026"` and once without. The `none` default plus the `if` guard means the date line — linebreak and all — simply doesn't render when omitted, so you never get a stray blank line.

19.3 Move the `#let report(...)` from 19.2 into `template.typ`, then in `main.typ`:

```

#import "template.typ": report

#show: report.with(title: "Field notes", author: "A. Lovelace", date: "July 2026")

= A heading
#lorem(20)

```

The output is identical to the single-file version. Moving the code into its own file changes where the definition lives, nothing about how it behaves.

19.4 Add `keywords: ()` to the signature (default empty array) and guard on it after the abstract:

```

#if keywords != () {
  block(inset: (x: 1.4em)) [
    #set text(size: 9pt, style: "italic")
    #keywords.join(", ")
  ]
}

```

Default to `()` (or `none`) and check before printing, so nothing shows when it's omitted; `.join(", ")` turns `("typst", "templates")` into `typst, templates`.

19.5 A second template with completely different front matter, kept beside `article` in the same `template.typ`:

```

#let memo(to: none, from: none, date: none, body) = {
  set page(width: 12cm, height: auto, margin: 1.6cm)
  set text(font: "Libertinus Serif", size: 11pt)
  grid(columns: (auto, 1fr), row-gutter: 0.4em,
    [*To:*], [#to], [*From:*], [#from], [*Date:*], [#date])
  line(length: 100%, stroke: 0.5pt)
  v(0.6em)
  body
}

```

Then `main.typ` chooses the look on one line:

```

#import "template.typ": article, memo

#show: article.with(title: "Q3 review", author: "A. Lovelace")
// #show: memo.with(to: "Team", from: "Ada", date: "July 2026")

= Heading
#lorem(30)

```

Swap which `#show:` line is active and the same body switches between a centred article and a left-aligned memo. The design and the writing are genuinely separate — that's the whole payoff.

Chapter 20 — Packages

20.1 Import a Universe package and let the cache do the work:

```

#import "@preview/cetz:0.3.4"

#cetz.canvas({
  import cetz.draw: *

```



```
circle((0, 0), radius: 1)
})
```

The first compile reaches the network to download `cetz`; every compile after that reads it from the local cache, so it's fast even offline. (Import with `: *` instead and the `canvas` / `draw` names land in scope directly — either style is fine.) The lesson is download-once-then-offline.

20.2 A bare `#import "@preview/cetz"` errors: Typst insists on a version. A name without one is ambiguous and would make builds non-reproducible — "newest" could change under you and break the document silently. Pinning `@preview/cetz:0.3.4` guarantees the same build forever. (`typst init` *does* default to the latest version, because scaffolding a new project once is a different situation from reproducibly recompiling an existing one.)

20.3 No single answer; credit yourself for naming a real Typst Universe package, its current version, and a well-formed line like `#import "@preview/NAME:VER"`. Read the version straight off the package's Universe page.

20.4 A local package: `typst.toml`:

```
[package]
name = "my-notes"
version = "0.2.0"
entrypoint = "lib.typ"
authors = ["Ada Lovelace"]
license = "MIT"
description = "Small note helpers."
```

`lib.typ` exports something like `#let hi(name) = [Hello, #name!];` `main.typ` reads it by relative path with `#import "lib.typ": hi` and calls `#hi("world")`. It compiles offline because a relative import never touches the registry — the pattern from `examples/106-a-local-package/`.

20.5 Turn the Chapter 19 template into a template package. The shape (mirrors `examples/107-`):

- `typst.toml` with a `[package]` section and a `[template]` section (`path = "template"`, `entrypoint = "main.typ"`, `thumbnail = "thumbnail.png"`);
- `lib.typ` exporting your Chapter 19 wrapper (the `show-rule` function);
- `template/main.typ` importing the package by its hypothetical `@preview/<name>:<ver>` name — an illustration that won't compile until published;
- root `main.typ` importing `lib.typ` by relative path and applying the wrapper — this is the one that must compile offline.

The graded points are the `[template]` fields, the relative-vs-`@preview` distinction between the two `main.typ` files, and a clean local compile.

Chapter 21 — Advanced layout

21.1 Placed content reserves no space:

```
#place(top + right, dx: -2mm, dy: 2mm,
  box(fill: red, inset: 4pt, text(fill: white)[*CONFIDENTIAL*]))

#lorem(60)
```

The first line of `lorem` starts at the top margin and the tag overlaps it. Move the anchor to `bottom + center` and the tag pins to the foot, centred, while the body text still flows as if it weren't there. See `examples/108-`.

21.2 A fixed-height block that clips its overflow:

```
#block(
  fill: rgb("#eef4ff"), stroke: 1pt + blue, radius: 6pt,
  inset: 10pt, height: 1.5cm, clip: true,
)[#lorem(30)]
```

Without a fixed `height` the block grows to fit its text. Fix the height and the text overflows past the bottom edge; add `clip: true` and the overflow is cut off cleanly at the block's border instead of spilling out. See `examples/109-`.

21.3 A dot leader that fills the slack on a line:

```
Chapter 1 #box(width: 1fr, repeat[.]) 7
```

The `1fr` box absorbs all the free space and `repeat[.]` fills it with dots. Lengthen `Chapter 1` to a real title and the dot run shortens automatically to keep the `7` at the right edge. See `examples/112-`.

21.4 Four inline transforms:

```
#rotate(20deg)[word]
#scale(x: 140%)[word]
#move(dy: -4pt)[word]
#skew(ax: 20deg)[word]
```

By default these don't reflow, so the line keeps the untransformed word's height and the rotated or moved versions poke out of it. Add `reflow: true` to the rotation and the line grows to contain the rotated box. See `examples/111-`.

21.5 A hand-built table of contents from `context query`:

```
#context {
  for h in query(heading) {
    let p = counter(page).at(h.location()).first()
    [
      #pad(left: (h.level - 1) * 1em)[
        #h.body #box(width: 1fr, repeat[.]) #p
      ]
    ]
  }
}
```

`context` gives the whole-document view; `query(heading)` returns every heading, `counter(page).at(h.location())` finds each one's page, and the `1fr + repeat` pair draws the leader. The output looks like `#outline()`, but you own every part of it — filter by level, add section numbers, change the leader — which `#outline` won't let you do. Like Chapter 17's `.final()`, `query` sees the entire document at once. See `examples/113-`.

Chapter 22 — Designing a book template

These solutions edit the multi-file book template from `examples/115-`, so the fragments below are meant to slot into it rather than compile standalone.

22.1 Edit `theme.typ` only — for instance `#let accent = rgb("#a12f5a")` and `#let font-head = "Libertinus Serif"`. Everything that names `accent` moves with it: the heading colour, the opener/part/contents rules, the chapter and part kickers, the index letters, the code block's left rule, the title-page rule. `font-head` re-fonts every heading, label, kicker, and title. The admonition colours are their own constants, so they don't follow `accent` unless you point them at it too. One file is the single source of truth; the rest of the template only ever names the constants.

22.2 One line in `admonitions.typ` mints a new kind by configuring the existing box function:

```
#let example = admonition.with(kind: "Example", color: accent)
```

It's already exported through the module's `*`, so a chapter can call `#example[...]` and get a tinted block with an `EXAMPLE` label and an accent side-rule — no new box code, because `.with` just pre-fills the box that's already written.

22.3 Replace the footer body with a total:

```
Page #counter(page).display("1") of #counter(page).final().first()
```

(or the `display("1 of 1", both: true)` shorthand), still inside the `context` footer. `.final()` returns an array and `.first()` pulls the integer. It's right on every page because Typst iterates until the total settles (Chapter 17). The total counts from wherever the page counter was last reset — here the `counter(page).update(1)` at the start of the body — so "of N" reflects the numbered body pages.

22.4 Give `book` a `date` parameter before `body`, thread it into `_title-page`, and guard it there:

```
#if date != none { v(0.3em); text(size: 8.5pt, fill: muted)[#date] }
```

Apply the template once with `date: "July 2026"` and once without. The `none` default plus the `if` guard is the same optional-line habit from Chapter 19 — no blank gap when the date is omitted.

22.5 Have `part()` record each divider with a `metadata` marker the way `idx` does:

```
#let part(title) = {
  counter("part").step()
  [#metadata(title)<part-entry>]
  page(/* ... existing divider ... */)[/* ... */]
}
```

Then build the contents page by querying both markers and sorting by page location:

```
#context {
  let parts = query(<part-entry>).map(e => (
    kind: "part", loc: e.location(), body: e.value))
  let chaps = query(heading.where(level: 1)).map(h => (
    kind: "chapter", loc: h.location(), body: h.body,
    num: counter(heading).at(h.location())))
  let items = (parts + chaps).sorted(key: it => it.loc.position().page)
  for it in items {
    let p = counter(page).at(it.loc).first()
    if it.kind == "part" {
      strong(it.body)
    } else {
      [#h(1em) #numbering("1", ..it.num) #it.body
      #box(width: 1fr, repeat[.]) #p \ ]
    }
  }
}
```

Sorting the mixed list by page location is the crux — `it.loc.position().page` gives a sortable key. Because `query` sees the whole document, parts (which aren't headings) and chapters (which are) merge into one ordered contents — something `#outline`, which only knows headings, cannot build. Any merged-and-sorted version that puts each part above its chapters passes.

Chapter 23 — Building the book

These solutions edit the assembled book in `examples/116-`, so the snippets are files or lines to add to that project.

23.1 Create `chapters/04-the-clearing.typ`:

```
= The clearing
#lorem(60)
```

Then add one line in `main.typ` under the `#part[Rough weather]` block, after chapter 03:

```
#include "chapters/04-the-clearing.typ"
```

Recompile with `typst compile --root . examples/116-assembling-a-book/main.typ out.pdf`. It appears as Chapter 4 — the heading counter continues, it lands in the outline on its own, and its page number is discovered by query. Adding content is a one-line edit to the manifest.

23.2 Create `front/dedication.typ` with no `=` heading, so it stays out of the chapter numbering:

```
#v(1fr)
#set align(center)
#emph[For everyone who checks the sky before leaving the house.]
#v(1fr)
```

Include it inside the front-matter block, between the copyright and preface includes (add `#pagebreak(weak: true)` if you want it alone on a page). Because roman numbering is automatic, the preface rennumbers from ii to iii by itself — none of the numbers are typed.

23.3 Prefix each chapter include with a break to the next odd page:

```
#pagebreak(to: "odd")
#include "chapters/01-a-clear-morning.typ"
```

Chapters now start on odd (recto) pages, with Typst inserting a blank verso where needed. The reason *not* to bother for a screen-only PDF: read on a phone or laptop there's no left/right spread and no binding, so the inserted blanks are just wasted scroll — recto starts as a print-binding courtesy. You could also fold it into the template's level-1 heading show rule (`pagebreak(weak: true, to: "odd")`) so it's automatic and still collapses when already on an odd page.

23.4 In `book.typ`, `_contents-page()`, change `outline(title: none, depth: 2, indent: 1.2em)` to `depth: 1`. The contents now lists only the level-1 chapter lines, no sections. Because the depth lives in one place, it's a single edit — if it had been repeated across many files you'd have had to find and fix every copy and risk missing one, which is exactly the argument for Chapter 22's theme/constants layering.

23.5 Half-title first: the template's `book()` draws its own full title page as the first thing in the front matter, so you can't inject a page *before* it from `main.typ` without editing the template. The honest answer is to say so and place the half-title as the first page *inside* the roman front-matter block instead — it then follows the template's title page. (Conventionally a half-title precedes the title page, so note that with this template it has to come after unless you edit the template; both are acceptable if reasoned.) Half-title, no heading:

```
#v(2fr)
#set align(center)
#smallcaps(text(size: 18pt)[The Weather Almanac])
#v(3fr)
```

Colophon after `#make-index()`, drawn by hand like the appendix (`muted` is the template's own colour constant):

```
#pagebreak(weak: true)
#v(1fr)
#set align(center)
#set text(size: 9pt, fill: muted)
Set in Libertinus Serif and New Computer Modern. \
Typeset with Typst.
#v(1fr)
```

Front and back matter are just content you place around the body; the template hands you the fonts and colours to make them match. Any version that reasons correctly about ordering and keeps the extra pages out of the chapter numbering (no `=` headings) passes.

Chapter 24 — The Pandoc bridge

24.1 Run any three-paragraph GitHub-Markdown file through `pandoc --from gfm --to typst`. The mapping is one-to-one: `#` becomes `=`, `**bold**` becomes `#strong[...]`, a `-` list becomes Typst list items. Reading the output and seeing that correspondence is the exercise; no compile needed.

24.2 Two ways to a PDF:

```
pandoc x.md --to typst -o x.typ && typst compile x.typ
pandoc x.md -o x.pdf      # LaTeX path – needs a TeX install
```

Both produce a reasonable PDF. The Typst path is far faster and needs no TeX; if you don't have LaTeX installed, only the Typst path will run at all — which is rather the point.

24.3 Without the Div-rewriting filter, `> [!TIP]` becomes a plain nested `#block[ #block[ Tip ] text ]` — no colour, with `Tip` as literal text. That's exactly why the filter exists: with it, the same input becomes `#admonition("tip")[text]`, which your preamble can style.

24.4 A Lua filter that strips HTML:

```
function RawBlock(el)
  if el.format == "html" then
    return {}
  end
end
```

An HTML comment `<!-- ... -->` arrives as a `RawBlock` with format `html`; returning `{}` deletes it. Run the file with `--lua-filter` and confirm the comment is gone from the `-t typst` output.

24.5 A miniature version of this chapter's pipeline: a few Markdown files, a `preamble.typ` defining `#let admonition(kind, body) = ...` plus page and text set rules, and a `build.sh` that converts, concatenates, and compiles:

```
pandoc chapters/*.md --from gfm --to typst -o body.typ
cat preamble.typ body.typ > out.typ
typst compile out.typ book.pdf
```

Grade yourself on a working end-to-end build, however minimal. The insight is the whole chapter in one line: converter + template + compiler = a book from a folder of Markdown.

For side-by-side translations from other systems, see Appendix B (coming from LaTeX) and Appendix C (coming from Word); for a one-page syntax reminder, keep Appendix D, the cheat sheet, within reach.

Appendix B · From LaTeX to Typst

If you already speak LaTeX, most of Typst is a matter of translation, and the dictionary is short. Three shifts cover the bulk of it. First, there is no preamble: a blank file is a valid document, so `\documentclass`, `\usepackage`, and `\begin{document}` all map to *nothing*. Second, LaTeX's backslash commands become one of two things — inline markup (`*bold*`, = `Heading`) for the common cases, or a function call `#name(...)` for everything else. Third, math drops the backslashes entirely: symbols are spelled by name (`alpha`, not `\alpha`) and a slash is a fraction (`a/b`, not `\frac{a}{b}`).

The tables below are a phrasebook. Each section points at the chapter that covers the idea in full; come here to translate, go there to understand. Typst markup is the default; a `#` opens a code expression, and `$ ... $` opens math.

Document setup

The whole LaTeX skeleton — class, preamble, `document` environment — collapses. A minimal article, side by side:

```
\documentclass[12pt,a4paper]{article}
\usepackage{amsmath}
\title{My paper}
\author{A. Writer}
\begin{document}
\maketitle
Hello.
\end{document}
```

```
#set text(size: 12pt)
#set page(paper: "a4")
Hello.
```

The title block, the class, and the packages are gone — replaced by a couple of `#set` rules for the things you actually want to change, and by a *template* (Chapter 19) when you want a reusable look with a real title block.

| LaTeX | Typst | Notes |
|--|--|--|
| <code>\documentclass{article}</code> | <i>(nothing)</i> , or a template | A blank file already compiles. |
| <code>\documentclass[12pt]{...}</code> | <code>#set text(size: 12pt)</code> | Base font size is a set rule. |
| <code>\documentclass[a4paper]{...}</code> | <code>#set page(paper: "a4")</code> | Paper by name; <code>margin: too</code> (Ch 5). |
| <code>\usepackage{geometry}</code> | <code>#set page(margin: 2cm)</code> | Built in; no package. |
| <code>\usepackage{...}</code> (most) | <i>(nothing)</i> | Math, color, graphics, links are core. |
| <code>\usepackage{...}</code> (extras) | <code>#import
"@preview/name:version"</code> | Registry packages (Ch 20). |
| <code>\begin{document}</code>
... <code>\end{document}</code> | <i>(nothing)</i> | The file body <i>is</i> the document. |
| <code>\maketitle</code> ,
... <code>\title{...}</code> ,
... <code>\author{...}</code> | template arguments | <code>#show: article.with(title: ...)</code>
(Ch 19). |
| custom <code>\documentclass (.cls)</code> | a template function | An ordinary <code>#let</code> -defined function. |

Text formatting

| LaTeX | Typst (markup) | Typst (code) |
|---|------------------|------------------------------------|
| <code>\textbf{x}</code> | <code>*x*</code> | <code>#strong[x]</code> |
| <code>\textit{x}</code> , <code>\emph{x}</code> | <code>_x_</code> | <code>#emph[x]</code> |
| <code>\texttt{x}</code> , <code>\verb x </code> | <code>`x`</code> | <code>#raw("x")</code> |
| <code>\underline{x}</code> | <code>—</code> | <code>#underline[x]</code> |
| <code>\sout{x}</code> (soul) | <code>—</code> | <code>#strike[x]</code> |
| <code>\textsc{x}</code> | <code>—</code> | <code>#smallcaps[x]</code> |
| <code>\textcolor{red}{x}</code> | <code>—</code> | <code>#text(fill: red)[x]</code> |
| <code>{\large x}</code> , <code>\Large</code> | <code>—</code> | <code>#text(size: 1.4em)[x]</code> |
| <code>\footnote{x}</code> | <code>—</code> | <code>#footnote[x]</code> |
| <code>\href{url}{x}</code> | <code>—</code> | <code>#link("url")[x]</code> |
| <code>\url{u}</code> | <code>—</code> | <code>#link("u")</code> |

A handful of LaTeX's typographic shorthands survive almost unchanged, because Typst does the same smart substitutions in markup:

| LaTeX | Typst | Result |
|---|---|---|
| <code>\\</code> | <code>\</code> (end of line) | Hard line break |
| <code>~</code> | <code>~</code> | Non-breaking space |
| <code>---</code> | <code>---</code> | Em dash — |
| <code>--</code> | <code>--</code> | En dash – |
| <code>\ldots</code> | <code>...</code> | Ellipsis ... |
| <code>\newpage</code> , <code>\clearpage</code> | <code>#pagebreak()</code> | <code>weak: true</code> collapses at page top |
| <code>\centering</code> , <code>center env</code> | <code>#align(center)[...]</code> | |
| <code>\hfill</code> | <code>#h(1fr)</code> | Flexible horizontal fill |
| <code>\hspace{1em}</code> , <code>\vspace{1em}</code> | <code>#h(1em)</code> , <code>#v(1em)</code> | |
| <code>\rule</code> , <code>\hrule</code> | <code>#line(length: 100%)</code> | |
| <code>\noindent</code> / <code>\indent</code> | <code>#set par(first-line-indent: ...)</code> | Controlled globally by set rule |

Text and fonts are Chapter 4; page-level layout is Chapter 5.

Sectioning and cross-references

Headings are equals signs — one per level. Unlike LaTeX, they are **not numbered by default**; turn numbering on with a set rule.

| LaTeX | Typst | Notes |
|---|---|---|
| <code>\section{Intro}</code> | <code>= Intro</code> | Top level you use; depth = count of <code>=</code> . |
| <code>\subsection{...}</code> | <code>== ...</code> | |
| <code>\subsubsection{...}</code> | <code>=== ...</code> | |
| <code>\chapter{...}</code> | <code>= ...</code> | In a book template <code>=</code> renders as a chapter. |
| numbered sections | <code>#set heading(numbering: "1.1")</code> | Off unless you ask. |
| <code>\section*{...}</code> | <code>#heading(numbering: none)[...]</code> | One unnumbered heading. |
| <code>\label{k}</code> | <code><k></code> (after the thing) | On a heading, equation, figure... |
| <code>\ref{k}</code> , <code>\autoref{k}</code> | <code>@k</code> | Clickable, auto-supplemented ("Section 2"). |
| <code>\pageref{k}</code> | <code>#context
counter(page).at(<k>).first()</code> | Uses context (Ch 17). |
| <code>\tableofcontents</code> | <code>#outline()</code> | |
| <code>\listoffigures</code> | <code>#outline(target: figure.where(kind:
image))</code> | |

References and cross-references are Chapter 11; headings and numbering patterns are Chapters 3 and 9.

Lists

No environments — just a marker at the start of each line. Nesting is by indentation.

| LaTeX | Typst | Notes |
|---|--|------------------------------|
| <code>\begin{itemize}</code> / <code>\item</code> | <code>- item</code> | One <code>-</code> per line. |
| <code>\begin{enumerate}</code> / <code>\item</code> | <code>+ item</code> | Auto-numbered. |
| <code>\begin{description}</code> / <code>\item[Term]</code> | <code>/ Term: description</code> | Term list. |
| nested lists | indent two spaces | Depth follows indentation. |
| <code>\begin{enumerate}[(a)]</code> | <code>#set enum(numbering: "a")</code> | Numbering pattern (Ch 9). |
| custom bullet | <code>#set list(marker: [--])</code> | |

Lists live in Chapter 3.

Math

The one topic that changes the most and hurts the least. Math still sits between dollar signs, `^` is still superscript, `_` still subscript — but every backslash command becomes a plain name, and `\frac` becomes a slash. The `amsmath` / `amssymb` packages have no equivalent because everything they add is already built in.

Inline versus display is a matter of *spacing inside the dollars*:

Inline `$a^2 + b^2 = c^2$` sits in the line.

`$ a^2 + b^2 = c^2 $` // a space inside each dollar → centered block

| LaTeX | Typst | Notes |
|--|--|--|
| <code>\(x\)</code> , <code>\$x\$</code> | <code>\$x\$</code> | Snug against the dollars = inline. |
| <code>\[x\]</code> , <code>\$\$x\$\$</code> , equation | <code>\$ x \$</code> | A space inside each dollar = display. |
| <code>\frac{a}{b}</code> | <code>a/b</code> or <code>frac(a, b)</code> | The slash is a fraction. |
| <code>\sqrt{x}</code> | <code>sqrt(x)</code> | Auto-sized radical. |
| <code>\sqrt[3]{x}</code> | <code>root(3, x)</code> | |
| <code>x^{n+1}</code> , <code>x_{ij}</code> | <code>x^(n+1)</code> , <code>x_(i j)</code> | Parens group; a space separates letters. |
| <code>\sum_{i=1}^n</code> | <code>sum_(i=1)^n</code> | Bounds stack in display, tuck in inline. |
| <code>\int_a^b</code> , <code>\prod</code> , <code>\lim</code> | <code>integral_a^b</code> , <code>product</code> , <code>lim</code> | |
| <code>\alpha</code> , <code>\beta</code> , <code>\Omega</code> | <code>alpha</code> , <code>beta</code> , <code>Omega</code> | Names, no backslash; capital name = capital. |
| <code>\times</code> , <code>\cdot</code> | <code>times</code> , <code>dot</code> | |
| <code>\le</code> <code>\ge</code> <code>\ne</code> <code>\in</code> <code>\to</code> <code>\infty</code> | <code><=</code> <code>>=</code> <code>!=</code> <code>in</code> <code>-></code> <code>oo</code> | Typed symbols work too. |
| <code>\mathbb{R}</code> | <code>RR</code> or <code>bb(R)</code> | Plus <code>NN</code> , <code>ZZ</code> , <code>QQ</code> , <code>CC</code> . |
| <code>\mathrm{}</code> , <code>\mathbf{}</code> , <code>\mathcal{}</code> | <code>upright()</code> , <code>bold()</code> , <code>cal()</code> | |
| <code>\text{if}</code> | <code>"if"</code> | Quotes set upright text. |
| <code>\hat{x}</code> , <code>\bar{x}</code> , <code>\vec{v}</code> , <code>\tilde{x}</code> | <code>hat(x)</code> , <code>bar(x)</code> , <code>arrow(v)</code> , <code>tilde(x)</code> | Accents are functions. |
| <code>\quad</code> , <code>\qquad</code> , <code>\,</code> , <code>\;</code> | <code>quad</code> , <code>wide</code> , <code>thin</code> , <code>med</code> | Explicit spaces by name. |

Structured math shares one convention: a comma separates entries across a row, a semicolon ends the row.

| LaTeX | Typst | Notes |
|---|--|---|
| <code>\begin{matrix} a&b\c&d \\ \end{matrix}</code> | <code>mat(a, b; c, d)</code> | Default fence is parentheses. |
| <code>\begin{bmatrix} ... \\ \end{bmatrix}</code> | <code>mat(delim: "[", ...)</code> | Any delimiter via <code>delim:</code> . |
| <code>\begin{vmatrix} ... \\ \end{vmatrix}</code> | <code>mat(delim: " ", ...)</code> | Determinant bars. |
| column vector | <code>vec(x, y, z)</code> | One entry per row, no <code>;</code> needed. |
| <code>\begin{cases} ... \\ \end{cases}</code> | <code>cases(x & "if" ..., -x & ...)</code> | <code>&</code> aligns the conditions. |
| <code>\left(... \right)</code> | matched <code>(...)</code> , or <code>lr(...)</code> | Matched delimiters auto-scale. |
| <code>\operatorname{argmax}</code> | <code>op("argmax")</code> | limits: <code>#true</code> stacks sub/superscripts. |
| <code>\DeclareMathOperator</code> | <code>#let f = math.op("f")</code> | Define once in code, reuse the name. |

Multi-line alignment is `\` for a new line and `&` for the alignment column — the same two characters as an `align` environment, minus the `\begin` / `\end`:

```
$ (a + b)^2 &= (a + b)(a + b) \
    &= a^2 + 2 a b + b^2 $
```

Numbering and labels work as elsewhere: turn on `#set math.equation(numbering: "(1)")`, pin a `<eq:name>` after the closing dollar, and refer with `@eq:name`. All of this is Chapter 8.

Tables

The `tabular` column-spec mini-language is gone. Columns, alignment, and rules are ordinary arguments to `#table`, not pipes and letters wedged into a preamble string.

| LaTeX | Typst | Notes |
|---|---|---|
| <code>\begin{tabular}{lcr}</code> | <code>#table(columns: 3, align: (left, center, right))</code> | |
| <code>\begin{tabular}{p{3cm}}</code> | <code>#table(columns: (3cm,))</code> | Fixed width. |
| flexible column | <code>columns: (auto, 1fr)</code> | <code>fr</code> shares leftover space. |
| <code>a & b \\</code> | <code>[a], [b],</code> | Cells are content, comma-separated. |
| <code>\hline</code> , <code>\toprule</code> , <code>\midrule</code>
(booktabs) | <code>table.hline(stroke: ...)</code> | Placed where you want it; no package. |
| vertical rule (<code> </code>) | <code>table.vline(stroke: ...)</code> | |
| <code>\multicolumn{2}{c}{x}</code> | <code>table.cell(colspan: 2)[x]</code> | |
| <code>\multirow{2}{*}{x}</code> | <code>table.cell(rowspan: 2)[x]</code> | |
| header row | <code>table.header([A], [B])</code> | Repeats across page breaks; marks it up for screen readers. |
| cell padding | <code>inset: 10pt</code> | Space inside cells. |
| <code>\arraystretch</code> , spacing between | <code>gutter: , column-gutter:</code> | Space between cells. |

The booktabs look — thick top and bottom, one thin mid-rule, no verticals — is just three `table.hline`s and `stroke: none`. A bare `#grid` is the same engine without the borders or table semantics, for pure layout. All in Chapter 7.

Figures and images

| LaTeX | Typst | Notes |
|---|--|---|
| <code>\includegraphics{f}</code> | <code>#image("f")</code> | PNG, JPG, GIF, SVG. |
| <code>\includegraphics[width=3cm]{f}</code> | <code>#image("f", width: 3cm)</code> | |
| <code>\begin{figure} ... \caption{} ... \end{figure}</code> | <code>#figure(image("f"), caption: [...])</code> | One call; inside it you are in code, so <code>image(...)</code> needs no <code>#</code> . |
| <code>\label{fig:x}</code> inside the figure | <code><fig:x></code> after the <code>#figure(...)</code> | |
| <code>\ref{fig:x}</code> | <code>@fig:x</code> | Renders "Figure 3", clickable. |
| <code>\caption</code> prefix "Figure" | default supplement | Change with <code>#ref(<fig:x>, supplement: [Fig.])</code> |
| <code>alt= text</code> | <code>#image("f", alt: "...")</code> | Accessibility built in. |
| <code>[htbp]</code> , float packages | <i>(nothing)</i> | No floats: a figure sits where you place it. |

Figures and images are Chapter 6.

Bibliography and citations

No separate BibTeX or Biber run, and no `.aux` / `.bbl` shuffle. Your existing `.bib` files work unchanged; Typst also reads its own Hayagriva YAML format.

| LaTeX | Typst | Notes |
|--|--|--|
| <code>\cite{key}</code> | <code>@key</code> | Or <code>#cite(<key>)</code> in code. |
| <code>\bibliography{refs}</code> | <code>#bibliography("refs.bib")</code> | Point at <code>.bib</code> or <code>.yaml</code> . |
| <code>\bibliographystyle{apa}</code> | <code>#bibliography("refs.bib", style: "apa")</code> | CSL styles by name. |
| <code>\usepackage{natbib}</code> / <code>biblatex</code> | <i>(nothing)</i> | Citation forms come from the style. |
| run <code>bibtex</code> , recompile twice | <i>(nothing)</i> | Resolved in the single compile pass. |

Citations and bibliographies are Chapter 12.

Custom commands and packages

A LaTeX macro is a Typst function; you define it with `#let` in the same language as the rest of the document. Arguments are named and defaulted, not numbered `#1`, `#2`.

```
\newcommand{\todo}[1]{\textcolor{red}{\textbf{TODO: }#1}}
\todo{write this}
```

```
#let todo(body) = text(fill: red)[*TODO:* #body]
#todo[write this]
```

| LaTeX | Typst | Notes |
|---|---|---|
| <code>\newcommand{\x}[1]{...}</code> | <code>#let x(body) = [...]</code> | Call <code>#x[...]</code> or <code>#x(...)</code> . |
| <code>\newcommand{\x}{value}</code> | <code>#let x = value</code> | A plain binding. |
| <code>\newcommand</code> with default arg | <code>#let x(body, size: 1em) = ...</code> | Named, defaulted parameters. |
| <code>\renewcommand{\emph}[1]{...}</code> | <code>#show emph: it => ...</code> | Redefine existing behavior (Ch 10). |
| <code>\renewcommand</code> of a length/setting | <code>#set ...(...)</code> | Change a default (Ch 9). |
| <code>\newenvironment</code> | a function taking a <code>body</code> | Same as a command with content. |
| <code>\usepackage{tool}</code> | <code>#import "@preview/tool:1.0.0":
*</code> | From Typst Universe; version required (Ch 20). |
| <code>\input{f}</code> , <code>\include{f}</code> | <code>#include "f.typ"</code> | Splices another file's content. |
| import definitions from a file | <code>#import "f.typ": name</code> | Bring in specific bindings. |

`#let` functions are Chapters 14 and 18; `set` and `show` rules — the safe, scoped answer to `\renewcommand` and `.sty` patching — are Chapters 9 and 10; packages are Chapter 20.

Things that are just gone

Not translated — simply absent, and you will not miss them:

- **No separate compile passes.** One `typst compile` produces the final PDF, cross-references, citations, and table of contents settled. No "run it three times until the numbers stop moving."
- **No `.aux`, `.toc`, `.bbl`, `.log`, `.out` litter.** Typst keeps its bookkeeping in memory; the only output is the PDF you asked for.
- **No preamble.** Nothing to configure before you can type your first sentence.
- **No float placement algorithm.** Figures and tables stay where you put them, so there is no `[htbp]`, no `\FloatBarrier`, no chasing a figure that wandered three pages downstream.
- **No package hunt for basics.** Color, graphics, hyperlinks, math, and rules like booktabs are in the core, not bolted on.
- **No cryptic error walls.** When something is wrong, Typst points at the line and says so in a sentence — and because it compiles in milliseconds, the preview updates as you type.

Coming from a word processor instead? See Appendix C. For a one-page summary of Typst syntax with no LaTeX in sight, see the cheat sheet in Appendix D.

Appendix C • From Word to Typst

You know how to make a document look right in Word. You select a line, click a bigger font, maybe bold it, nudge the spacing until it sits well. Typst asks you to unlearn that reflex — not the goal, just the reflex — and the payoff is a document that stays consistent no matter how long it grows.

This appendix is a translation table. It maps the Word way of working (menus, dialogs, toolbar buttons) onto the Typst way (markup and rules), and points you at the chapter where each idea gets the full treatment. It assumes you're coming from Microsoft Word or Google Docs, not from LaTeX or any programming language. If a snippet looks alien, that's fine — the chapter it points to starts from zero.

The one shift that explains all the others

In Word you apply formatting *directly*. You select some text and act on the selection: this word becomes bold, this line becomes 18-point, this paragraph becomes centered. The formatting lives on the characters. Change your mind about what headings should look like, and you go find every heading and re-do it by hand — or you fight the Styles gallery, which is Word quietly admitting that direct formatting doesn't scale.

In Typst you *label structure* and *style it once*. You mark a line as a heading (= Introduction) without saying anything about how it looks. Then, separately, you write one rule that decides what every heading looks like. Change the rule, and all headings change together. The two jobs — saying what something *is* and deciding how it *looks* — are pulled apart and done in different places.

Those rules come in two kinds, and together they are the heart of the whole system:

- A **set rule** adjusts an element's built-in settings — the font of all text, the numbering of all headings, the margins of every page (Chapter 9).
- A **show rule** rebuilds an element's whole appearance, or applies a set rule to only *some* elements. This is the workshop where the fancy looks get built (Chapter 10).

Word's Styles gallery is the closest thing it has to this idea; a Typst template (Chapter 19) is the same instinct taken all the way, packing every rule for a document's look into one reusable function.

Here is the shift in miniature — structure labeled, appearance decided once:

```
#set text(font: "Libertinus Serif", size: 11pt)
#set heading(numbering: "1.")
#show heading: set text(fill: navy)

= Introduction
This has *strong*, _emphasized_, #underline[underlined], and
#strike[struck] text, plus #highlight[a highlight].
```

Nothing in the body says "make Introduction navy and numbered." The three rules at the top do, and they'd do it for a hundred headings just as happily as one.

Coming from Word. The first ten minutes feel like more work: you're typing = Introduction instead of clicking a style. The reward comes at minute eleven and every minute after, when one edit re-styles the entire document and nothing drifts out of sync. Word makes the easy thing easy and the consistent thing hard. Typst makes the opposite trade.

How do I do X? — the quick reference

Read this as "the thing you'd click in Word" on the left, "the thing you'd type in Typst" on the right, and where to learn it on the far side. In the Typst column, `*...*` and `_..._` are markup you type inline; anything starting with `#` is a function call.

| In Word | In Typst | Where |
|-----------------------------------|--|-------|
| Select text, click B | <code>*strong text*</code> | Ch 3 |
| Select text, click <i>I</i> | <code>_emphasized text_</code> | Ch 3 |
| Underline button | <code>#underline[text]</code> | Ch 4 |
| Strikethrough button | <code>#strike[text]</code> | Ch 4 |
| Highlighter button | <code>#highlight[text]</code> | Ch 4 |
| Font / size / color pickers | <code>#text(font: "New Computer Modern", size: 12pt, fill: red) [...]</code>
or set once with <code>#set text(...)</code> | Ch 4 |
| Styles gallery: Heading 1 / 2 / 3 | <code>=, ==, ===</code> | Ch 3 |
| Bulleted list | <code>- item</code> on each line | Ch 3 |
| Numbered list | <code>+ item</code> on each line | Ch 3 |
| Align left / center / right | <code>#align(center)[...]</code> | Ch 5 |
| Justify | <code>#set par(justify: true)</code> | Ch 5 |
| Navigation pane (heading tree) | <code>#outline()</code> | Ch 11 |
| Page Setup: size & margins | <code>#set page(paper: "a4", margin: 2.5cm)</code> | Ch 5 |
| Page Setup: orientation | <code>#set page(flipped: true)</code> | Ch 5 |
| Page Setup: columns | <code>#set page(columns: 2)</code> | Ch 5 |
| Insert > Header / Footer | <code>#set page(header: [...], footer: [...])</code> | Ch 5 |
| Insert > Page Number | <code>#set page(numbering: "1")</code> | Ch 5 |
| Insert > Table | <code>#table(columns: 3, ...)</code> | Ch 7 |
| Insert > Picture | <code>#image("photo.jpg")</code> | Ch 6 |
| Insert > Caption | <code>#figure(..., caption: [...])</code> | Ch 6 |
| Insert > Table of Contents | <code>#outline()</code> | Ch 11 |
| Insert > Cross-reference | <code>a <label>, then @label</code> | Ch 11 |
| Insert > Equation | <code>\$ a^2 + b^2 = c^2 \$</code> | Ch 8 |
| Find & Replace on formatting | <code>a #show rule (string or regex(...))</code> | Ch 10 |
| Save As template (.dotx) | a template function | Ch 19 |
| Mail merge | a <code>#for</code> loop over your data | Ch 15 |
| Track Changes / version history | a plain-text file under git | below |

The rest of this appendix expands the rows that deserve more than a cell.

Page setup

Word's Page Setup dialog — size, margins, orientation, columns — is a single Typst function, `page`, usually applied as a set rule so it governs the whole document (Chapter 5):

```
#set page(paper: "a4", margin: 2.5cm, columns: 2)
```

`flipped: true` turns any paper size to landscape. Margins can be one value for all four sides or a dictionary like `margin: (top: 3cm, rest: 2cm)`. Because it's a rule and not a one-time dialog, you can also *change* the page part-way through a document — a landscape spread for one wide table, then back — which Word makes you insert section breaks to achieve.

Headers, footers, and page numbers

In Word these hide behind the Insert menu and a double-click into the page margin. In Typst they're just more arguments to `page` (Chapter 5):

```
#set page(
  paper: "a4",
  header: align(right)[Learning Typst],
  numbering: "1",
)
```

`numbering: "1"` puts plain page numbers in the footer for you; `"1 / 1"` gives "page of total", and `"i"` gives lowercase roman numerals. Whatever content you put in `header:` or `footer:` repeats on every page.

Tables and pictures

Insert > Table becomes the `table` function, which takes the number of columns and then the cells in reading order (Chapter 7):

```
#table(
  columns: 3,
  table.header([Name], [Role], [Room]),
  [Ada], [Analyst], [B12],
  [Grace], [Compiler], [C04],
)
```

Insert > Picture is `image("photo.jpg")` (Chapter 6). On its own an image is just an image; wrap it in a `figure` and it gains a numbered caption, floats to a tidy spot on the page, and — with a label — becomes something you can refer to from anywhere:

See `@sales` for the numbers.

```
#figure(
  image("chart.png", width: 8cm),
  caption: [Quarterly sales],
) <sales>
```

That `<sales>` is a label, and `@sales` is a reference to it (Chapter 11). Typst fills in "Figure 1" and keeps the number correct even if you add three figures before it — the cross-reference maintenance Word's Insert > Cross-reference promises and doesn't always keep.

Tables of contents and cross-references

Insert > Table of Contents is one line, `#outline()`, and it reads your `=` headings automatically (Chapter 11). The same `<label> / @label` mechanism from the figure above works for headings, equations, and anything else you can tag, so "see Section 3" and "see Figure 4" stay right without a manual "Update Field."

Equations

Word's equation editor becomes dollar signs. Inline math sits in the flow of a sentence; put spaces inside the dollars and it becomes a centered display equation (Chapter 8):

```
Inline $E = m c^2$, and displayed on its own line:
$ a^2 + b^2 = c^2 $
```

The syntax is close to what you'd say out loud — `a^2` for a superscript, `sqrt(x)` for a root — and it's the part of Typst that most often makes ex-Word users wonder why they suffered so long.

Find & Replace for formatting

Word can find and replace text. Finding and replacing *formatting* — "make every 'TODO' red and bold" — is clumsier, and it's a one-shot edit that doesn't stick to new text you type later. A show rule does it as a standing instruction (Chapter 10):

```
#show "TODO": set text(fill: red, weight: "bold")
#show regex("\\d{4}"): set text(fill: blue)

TODO revisit the 2026 projection.
```


Every `TODD` in the document turns red and bold, every four-digit number turns blue, and both keep working as you add more text. The `regex(...)` form matches patterns, not just literal strings — the power-user corner of Word's Find that almost nobody finds.

Templates

A Word `.dotx` template bundles styles, page setup, and boilerplate so new documents start from a house look. The Typst equivalent is a function that takes your content, wraps it in every rule you'd otherwise retype, and hands it back dressed (Chapter 19):

```
#let report(title: none, body) = {
  set page(paper: "a4", margin: 2.5cm)
  set text(font: "Libertinus Serif", size: 11pt)
  set par(justify: true)
  show heading: set text(fill: navy)
  align(center)[#text(size: 20pt, weight: "bold")[#title]]
  body
}

#show: report.with(title: "Field Notes")

= Introduction
Everything below is styled by the template.
```

The `#show: report.with(...)` line feeds the whole rest of the document through `report`. Move that function into its own file (or publish it as a package, Chapter 20) and every document that imports it shares one look — change the function once, and they all update.

Mail merge, i.e. a hundred certificates

Word's mail merge stitches a letter template to a spreadsheet of names. Typst does the same thing with a loop, and you can see exactly what it's doing because it's right there in your file (Chapter 15; the data-wrangling side is Chapter 16):

```
#let names = ("Ada Lovelace", "Grace Hopper", "Alan Turing")

#for name in names [
  #align(center + horizon)[
    #text(size: 20pt)[Certificate of Completion] \
    awarded to #text(weight: "bold")[#name]
  ]
  #pagebreak(weak: true)
]
```

Three names, three certificates, one page each. Swap the list for a hundred names read from a CSV file and you get a hundred certificates, no wizard, no merge fields, no praying the preview matches the print.

Track Changes and version history

This one isn't a feature swap; it's a change of medium, and it's the quiet superpower of working in plain text. A Typst document is just text in a file, so the entire toolkit that programmers use to track changes to text works on your document too. The standard tool is **git**: it records every version you save, shows you exactly what changed between any two (line by line, in color), lets you label good versions, branch off to try something risky, and merge two people's edits.

That covers what Track Changes and Word's version history do, and then some — with one difference worth setting expectations on. Git compares *lines of your source*, not the rendered page, so it shines when edits are text and is blunter about a reshuffled layout. For prose and structure, which is most of what you write, it's a genuine upgrade: nothing is ever silently lost, and "what did I change last Tuesday?" has an exact answer. Git is a book of its own; for now, just file away that your documents are now the kind of thing it can look after.

What Word still does better

Switching tools honestly means naming what you give up. Word and Google Docs still win at a few things, and Chapter 1 already conceded the big one:

- **Real-time co-editing with non-technical people.** Two colleagues editing the same Google Doc, cursors dancing, is a workflow Typst's collaboration story doesn't fully match — especially when a co-author would rather quit than see a curly brace.
- **Quick, throwaway, one-page documents.** A birthday-party flyer or a note for the fridge does not want a compiler. Open Word, type, print, forget. Reaching for Typst here is using a lathe to sharpen a pencil.
- **Comment threads and suggestions.** The margin conversation — "can we cut this?", resolved and archived — is smoother in a word processor than in text files, even with git in the mix.

The rule of thumb from Chapter 1 holds: short, simple, or live-collaborative leans Word; long, structured, consistent, or automated leans Typst. Pick the tool the document actually needs.

Coming from LaTeX instead? See Appendix B. For a one-page syntax crib you can keep open while you work, see Appendix D.

Appendix D • Quick reference

Everything the book covers, condensed for fast recall. Every snippet is Typst 0.15.0; the chapter in each heading points back to the full treatment. Recall the two doorways: `$` opens math, `#` drops from markup into code.

Markup (Ch 3)

| Syntax | Does |
|---------------------------------|--|
| <code>= H1 ... ===== H6</code> | Headings, levels 1–6 (space after <code>=</code> required) |
| <code>*strong*</code> | Bold — long form <code>#strong[...]</code> |
| <code>_emph_</code> | Italic — long form <code>#emph[...]</code> |
| <code>`raw`</code> | Inline verbatim / monospace |
| <code>```lang ... ```</code> | Fenced block, optional language for highlighting |
| <code>- item</code> | Bullet list |
| <code>+ item</code> | Numbered list (Typst counts) |
| <code>/ Term: desc</code> | Term list |
| <code>https://typst.app</code> | Bare URL becomes a link |
| <code>#link("url")[text]</code> | Labelled link (address first) |
| <code><label></code> | Pin a label to the preceding element |
| <code>@label</code> | Reference or citation to a label |
| <code>\ at line end</code> | Hard line break inside the paragraph |
| <code>~</code> | Non-breaking space |
| <code>-- / ---</code> | En dash / em dash |
| <code>...</code> | Ellipsis |
| <code>"..." / '...'</code> | Smart quotes, language-aware |
| <code>* \# _</code> | Escape a markup character |
| <code>\u{2713}</code> | Any character by Unicode codepoint |
| <code>// ...</code> | Comment to end of line |
| <code>/* ... */</code> | Comment spanning lines |

Text and fonts (Ch 4)

`#text(...)[span]` styles one stretch; `#set text(...)` styles everything after.

| Argument | Value |
|-----------------|---|
| font: | "Libertinus Serif", or a fallback list ("A", "B") |
| size: | 11pt, 1.2em |
| weight: | "regular", "bold", or 100 – 900 |
| style: | "normal", "italic", "oblique" |
| fill: | a color (see below) |
| lang: / region: | "en" / "US" — drives quotes and hyphenation |
| tracking: | letter spacing, e.g. 0.15em |
| number-type: | "lining" (default) or "old-style" |
| number-width: | "proportional" or "tabular" |
| hyphenate: | auto, true, false |
| ligatures: | true / false |

Bundled fonts (no warnings, render everywhere): "Libertinus Serif", "New Computer Modern", "New Computer Modern Math", "DejaVu Sans Mono".

Colors: `rgb("#1a7f37")`, `named (red, blue, navy, ...)`, `luma(40%)`, `cmyk(75%, 0%, 70%, 5%)`. Colors adjust: `red.lighten(70%)`, `navy.darken(20%)`.

Decorations, each taking content in brackets:

```
#underline[x] #overline[x] #strike[x] #highlight[x] #smallcaps[Name]
E = mc#super[2] H#sub[2]0
```

Pages and paragraphs (Ch 5)

`#set page(...)` — one function for the whole sheet:

| Argument | Value |
|---------------------------|--|
| paper: | "a4" (default), "us-letter", "a5", ... |
| width: / height: | explicit size when no named paper fits |
| flipped: | true for landscape |
| margin: | one length, (x: ..., y: ...), or per-side (top:, bottom:, left:, right:) |
| columns: | 2 for a full-page multi-column flow |
| header: / footer: | content reprinted each page |
| numbering: | pattern: "1", "i", "a", "1 / 1" |
| number-align: | center, right + bottom, ... |
| fill: | flood the whole sheet with a color |
| background: / foreground: | content drawn behind / over the page |

`#set par(...)`: `justify: true`, `leading: 0.8em` (within a paragraph), `spacing: 1.2em` (between paragraphs), `first-line-indent: 1.2em` (or `amount: 1.2em, all: true`), `linebreaks: auto`.

Spacing and breaks by hand:

```
#v(2cm)           // vertical space (weak: true collapses at page top)
#h(1cm)           // fixed horizontal space
a #h(1fr) b       // stretchy space: shoves a and b to the edges
#pagebreak()      // hard; pagebreak(weak: true) skips if page is empty
#colbreak()       // jump to the next column
#columns(2, gutter: 1cm) [ #lorem(40) ] // a local multi-column block
```

Positioning and layout (Ch 21)

| Call | Does |
|---|---|
| <code>#place(top + right, dx: -2mm, dy: 2mm, body)</code> | Pin to a spot; reserves no space |
| <code>#place(bottom, float: true, clearance: 10pt, body)</code> | Float any block |
| <code>#box(fill:, stroke:, radius:, inset:, outset:, width:, clip:)[...]</code> | Inline container |
| <code>#block(...same args...)[...]</code> | Block-level container |
| <code>#stack(dir: ttb, spacing: 8pt, a, b, c)</code> | Line items on one axis (ttb , ltr , ...) |
| <code>#pad(x: 1cm, body)</code> | Space around content |
| <code>#align(center)[...]</code> | Position within the allotted width |
| <code>#rotate(-20deg, body)</code> | Rotate (add <code>reflow: true</code> to claim space) |
| <code>#scale(x: 140%, y: 70%, body)</code> | Stretch / squash |
| <code>#move(dx:, dy:, body)</code> | Shift without leaving the flow |
| <code>#skew(ax: 20deg, body)</code> | Slant |
| <code>#box(width: 1fr, repeat[.])</code> | Tile content to fill width (dotted leaders) |
| <code>#hide[Paris]</code> | Lay out, take space, draw nothing |

Shapes: `#rect`, `#circle(radius:, fill:)`, `#line(length:, stroke:)`, `#polygon(fill:, (0pt,0pt), (30pt,0pt), (15pt,25pt))`. Paths use `#curve` (the removed `#path` is gone): `curve.move`, `curve.line`, `curve.cubic`, `curve.quad`, `curve.close`.

Figures and images (Ch 6)

```
#image("logo.svg", width: 3cm, alt: "...") // fit: "cover"|"contain"|"stretch"
#figure(
  image("chart.svg", width: 60%),
  caption: [Revenue.],
) <fig:rev> // then reference with @fig:rev
```

`#figure(body, caption:, kind:, supplement:, placement:)`. `placement:` is `top`, `bottom`, or `auto`. A table or code body auto-detects its kind; otherwise set `kind: "diagram"` and `supplement: [Diagram]` for a separate counter. Caption on top: `#set figure.caption(position: top)`.

Tables and grids (Ch 7)

```
#table(
  columns: (auto, 1fr, 2.5cm), // track kinds: auto | length | fr
  align: (left, center, right), // value | per-column list | (x, y) => ...
  fill: (x, y) => if calc.odd(y) { luma(240) },
  stroke: none,
  inset: (x: 10pt, y: 6pt), // padding inside cells
  gutter: 6pt, // or column-gutter / row-gutter
  table.header([*A*], [*B*], [*C*]),
  [1], [2], [3],
  table.footer([*Σ*], [x], [y]),
)
```

| Piece | Does |
|---|---|
| <code>table.header(...)</code> / <code>table.footer(...)</code> | Rows that repeat across page breaks |
| <code>table.cell(colspan: 2, rowspan: 3, fill:, align:, inset:)[...]</code> | Span or override one cell |
| <code>table.hline(stroke: 1pt)</code> / <code>table.vline(...)</code> | Hand-placed rules (booktabs) |
| <code>#grid(...)</code> / <code>grid.cell(...)</code> | Same API, no borders, no table semantics — for layout |

Use `table` for data, `grid` for pure layout.

Math (Ch 8)

`$x$` is inline; `$ x $` (spaces inside the dollars) is a display block. Symbols are **names**, **not backslashes**; a run of letters is one name, so separate variables with spaces (`a b` , not `ab`).

| Write | Get |
|---|----------------------------------|
| <code>alpha</code> , <code>pi</code> , <code>Omega</code> | α , π , Ω |
| <code><=</code> <code>>=</code> <code>!=</code> | $\leq$ $\geq$ $\neq$ |
| <code>-></code> <code>oo</code> | $\rightarrow$ ∞ |
| <code>times</code> <code>dot</code> <code>in</code> <code>subset</code> | $\times$ $\cdot$ $\in$ $\subset$ |
| <code>plus.minus</code> <code>arrow.double</code> | $\pm$ $\Rightarrow$ |

```

$x^2$ $x_i$ $x^{(n+1)}$ $a_{(i j)}$ // ^ up, _ down; group with ()
$a/b$ $frac(a, b)$ $binom(n, k)$ // fraction; a/b is a fraction
$sqrt(x)$ $root(3, x)$ // roots
$sin x$ $lim_{x->0}$ $sum_{i=1}^n i$ // known ops upright; bounds stack in block
$op("argmax", limits: #true)_x f(x)$ // ad-hoc operator (# for the bool)
#let Var = math.op("Var") // reusable operator
$mat(1, 2; 3, 4)$ $vec(x, y, z)$ // , across a row; ; ends a row
$mat(delim: "[", 1, 0; 0, 1)$ // choose the fence
$cases(x & "if" x >= 0, -x & "else")$ // & aligns the branches
$ a &= b \ &= c $ // \ new line, & alignment column
$V_{"max"}$ // quoted words set upright
$pi approx #calc.round(calc.pi, digits: 4)$ // # drops into code

```

Numbering: `#set math.equation(numbering: "(1)")` ; label with `<eq:x>` , reference with `@eq:x` .

Scripting: values and functions (Ch 13–14)

`#let name = value` binds; `#name` uses it. Parenthesise any multi-token expression: `#pi` but `$(pi * 2)` . Inspect with `#type(x)` and `#repr(x)` .

| Type | Literal |
|--------------------------|---|
| <code>int</code> | <code>42</code> |
| <code>float</code> | <code>3.14</code> (division always yields a float: <code>6 / 2</code> is <code>3.0</code>) |
| <code>str</code> | <code>"hi"</code> |
| <code>bool</code> | <code>true</code> , <code>false</code> |
| <code>length</code> | <code>12pt</code> , <code>2cm</code> , <code>1em</code> |
| <code>ratio</code> | <code>50%</code> |
| <code>content</code> | <code>[bold]</code> |
| <code>array</code> | <code>(1, 2, 3)</code> |
| <code>dictionary</code> | <code>(name: "Ada")</code> |
| <code>none / auto</code> | the two singletons |

Operators: arithmetic `+` `-` `*` `/` ; compare `==` `!=` `<` `>` `<=` `>=` ; logic `and` `or` `not` ; `+` also joins strings, arrays, and content.

```

#let f(x, y: 10) = x + y // positional x, named-with-default y
#f(3) #f(3, y: 5)
#let g = x => x * 2 // arrow form (one param bare, 2+ in parens)
#let h = (a, b) => a + b
#let twice(fn, x) = fn(fn(x)) // functions are values
#let triple = g.with() // .with(...) pre-fills arguments
#let sum(..nums) = nums.pos().sum() // ..sink; .pos()/.named()
#sum(..(1, 2, 3)) // .. at a call site spreads

```

Blocks: `[...]` produces content; `{...}` runs statements and yields their joined result. `#f[body]` equals `#f([body])` — bracket content is the last argument.

Control flow (Ch 15)

```
#if cond [ ... ] else if other [ ... ] else [ ... ] // markup: emits content
#let x = if cond { a } else { b }                 // code: an expression
#for item in collection [ ... ]                   // arrays, or a range
#for (key, value) in dict.pairs() [ ... ]          // destructure pairs
#while cond { ... ; x = x + 1 }                   // make it terminate
```

`range(5)` → 0...4; `range(2, 6)` → 2...5; `range(0, 10, step: 2)` → 0,2,4,6,8. Ranges are **half-open** (exclude the end); `step:` is a *named* argument. `break` leaves the loop, `continue` skips to the next item. Feed generated cells straight into a function with a spread: `..for x in xs { (x,) }`.

Arrays, dictionaries, strings (Ch 16)

Arrays index from zero; negative counts from the end.

| Method | Does |
|---|----------------------------------|
| <code>.at(i)</code> , <code>.at(-1)</code> , <code>.first()</code> , <code>.last()</code> , <code>.len()</code> | Access and length |
| <code>.slice(a, b)</code> | Half-open sub-array |
| <code>.contains(x)</code> / <code>x in arr</code> | Membership |
| <code>.rev()</code> , <code>.sorted(key: f)</code> | New reversed / sorted array |
| <code>.push(x)</code> , <code>.pop()</code> | Grow / shrink a variable |
| <code>.map(f)</code> , <code>.filter(p)</code> , <code>.fold(init, f)</code> | Transform, keep, collapse |
| <code>.sum()</code> , <code>.product()</code> , <code>.join(sep, last:)</code> | Common reductions |
| <code>.enumerate()</code> , <code>.zip(other)</code> , <code>.flatten()</code> | Pair with index, stitch, flatten |

Single-element array needs a trailing comma: `("solo",)`.

Dictionaries: `d.key` or `d.at("key", default: "-")`; `.keys()`, `.values()`, `.pairs()`, `.insert(k, v)`, `.remove(k)`, `"k" in d`.

Strings: `.len()`, `.split(sep)`, `.replace(old, new)`, `.trim()`, `.contains(s)`, `.slice(a, b)`, `.find(p)`, `.match(regex(...))`, `.clusters()`; plus `upper(s)` / `lower(s)`. `.replace` and `.find` also accept `regex("...")`.

Set and show rules (Ch 9–10)

```
#set elem(param: value) // change a default from here on
#set text(fill: red) if draft // only when the condition holds
#show "TODO": strong // string selector → styled everywhere
#show regex("[A-Z]{2,}"): it => smallcaps(lower(it.text))
#show heading: it => block[ ... it.body ... it.level ... ] // rebuild
#show heading.where(level: 1): set text(size: 18pt) // show-set: safest
#show: template // wrap the whole rest of the document
```

Selectors: a bare element (`heading`, `raw`, `emph`, `link`, `figure`, ...), `elem.where(field: value)`, a `"string"`, or `regex("...")` (match on `it.text`). Numbering patterns (shared by headings, enums, figures, equations, pages): `"1."`, `"1.1"`, `"i."`, `"A."`, `"(1)"`. A rule that rebuilds the element it matches recurses — reuse `it`, don't construct a fresh one.

References and bibliography (Ch 11–12)

```
= Introduction <sec:intro> // label attaches to the preceding element
#set heading(numbering: "1.") // required before referencing a heading
See @sec:intro and #ref(<fig:rev>, supplement: [Fig.]).

#outline(title: [Contents], depth: 2, indent: 2em)
```



```
#outline(title: [Figures], target: figure)
#outline(target: figure.where(kind: table))
```

Citations resolve against one `#bibliography` :

```
Working memory is a bottleneck @sweller1988.
#cite(<deci2000>, form: "prose") // "author" | "year" | "full" | "prose"
On @sweller1988[p.~262], or #cite(<deci2000>, supplement: [pp.~230--231]).
#bibliography("refs.yml", title: [References], full: true, style: "apa")
```

`style:` takes a name ("apa" , "ieee" , "chicago-author-date" , "mla" , "vancouver" , ...) or a path to a `.csl` file. Source files are Hayagriva `.yml` or BibTeX `.bib` . Quote any YAML title that contains a colon.

Context, counters, state (Ch 17, 21)

Layout-dependent values live inside `#context` ; outside it they error.

```
#context counter(page).display()
#context [ page #here().page(), y = #here().position().y ]
```

Counters — built-in `counter(page)` , `counter(heading)` , or custom `counter("theorem")` :

| Method | Needs context? | Does |
|---|----------------|---|
| <code>.step()</code> | no | Bump by one |
| <code>.update(n)</code> / <code>.update(x => x + 2)</code> | no | Set to a value or via a function |
| <code>.display(pattern)</code> | yes | Render the current value |
| <code>.get()</code> | yes | Current value as an array (<code>.first()</code> for the number) |
| <code>.at(loc)</code> | yes | Value at another location |
| <code>.final()</code> | yes | Value at the document's end |

`counter(page).display("1 of 1", both: true)` prints current *and* total. State — any value that changes: `state("part", "Part I")` with `.update()` , `.get()` , `.at(loc)` , `.final()` (update *before* a page break so the header sees it). Introspection: `query(sel)` (in context) returns matching elements; `measure(body)` gives `.width` / `.height` (in context); `layout(size => ...)` adapts to the room available.

Files and packages (Ch 18, 20)

```
#import "@preview/cetz:0.3.4": * // registry package; version is required
#import "lib.typ": accent, kbd // named
#import "lib.typ": * // everything (namespace clash risk)
#import "lib.typ" // module only → reach in with lib.kbd
#import "lib.typ": kbd as key // rename on the way in
#include "section.typ" // splice a file's rendered content
```

`#import` brings **definitions**; `#include` brings **content**. Scaffold a project from a template package: `typst init @preview/charged-ieee` . The first package compile downloads and caches; every later build is offline.

Command line (Ch 2)

```
typst compile main.typ out.pdf # format inferred from the extension
typst watch main.typ out.pdf # recompile on every save
typst compile main.typ out.png --ppi 300 # raster resolution (default 144)
typst compile main.typ page-{p}.png # one PNG/SVG per page via {p}
typst compile main.typ out.pdf --pages 2-4,6
typst compile --root . chapters/main.typ out.pdf # absolute paths resolve here
typst compile main.typ out.pdf --open # open when done
typst compile main.typ out.pdf --format png # force a format
typst fonts # list available font families
typst --version
```

For further reading — the official docs, community packages, and where to go next — see Appendix E.

Appendix E • Where to go next

A book has to stop somewhere. Typst doesn't. While you were reading, someone shipped a new package, someone else filed a bug that will be fixed by Friday, and the compiler quietly gained a feature that isn't in these pages. That's not a flaw in the book; it's the shape of a young, fast-moving tool. What follows is a map to the living parts — where to look things up, where to ask, and where to watch for what changed after this book went to press.

The official documentation

Two doors, and you'll use both for different reasons.

The **Tutorial** — <https://typst.app/docs/tutorial> — is the gentle on-ramp: four short lessons that build a document from nothing. If a friend asks you "what's Typst?", this is the link to send. You've effectively already done it, several times over, so treat it as a place to point other people.

The **Reference** — <https://typst.app/docs/reference> — is the one you'll keep open forever. It's organized by category — text, layout, math, visualize, and so on — rather than alphabetically, which sounds inconvenient until you realize it matches how you actually think ("I want to do something with *tables*", not "I want the function that starts with `t`"). Every function has its full parameter list and a live, editable example. Once you know the *shapes* of Typst — that styling is `set` and `show`, that layout is functions returning content — the Reference is how you fill in the exact argument you half-remember. It is the daily driver.

The web app

<https://typst.app> is Typst in your browser: a free editor with live preview, a gallery of templates to start from, and real-time collaboration if you're writing with someone else. Nothing to install, nothing to configure. It's the fastest way to try an idea, to share a document with a co-author who has never heard of a command line, or to work from a machine that isn't yours. The same compiler you've been running locally powers it, so a document behaves the same in both places.

Typst Universe

<https://typst.app/universe> is the package and template registry you met in Chapter 20 — the searchable catalogue of community drawing kits, table helpers, slide systems, and ready-made document templates. When you catch yourself about to build something fiddly from primitives, look here first; the odds are decent that a stranger already built it and pinned a version for you.

The community forum

<https://forum.typst.app> is the best place to ask a real question. It's an official, searchable forum where questions get considered, worked answers rather than a shrug — and because it's indexed, your question helps the next person who searches the same problem. Before you post, search it: a good fraction of "how do I..." questions have been answered thoughtfully already.

GitHub: the source and the registry

Typst is open source, and two repositories are worth bookmarking.

The compiler lives at <https://github.com/typst/typst>. This is where you file a bug, read the release notes, or check the changelog to see exactly what changed between versions. If something in this book stops matching what your compiler does, the changelog is the first place to look for why.

The registry lives at <https://github.com/typst/packages>. As Chapter 20 showed, publishing a package is a pull request to this repo. Even if you never publish, it's useful to see how the registry is laid out and to browse what others have shipped.

Community chat

There's an official Discord for quick, conversational questions — the kind where a forum post feels too heavy and you just want to ask someone. Rather than trust an invite link that might have rotated by the time you read this, grab the current one from the front page of <https://typst.app> or the README of the compiler repo above; both link straight to it. The forum is better for anything you'd want to find again; chat is better for "am I holding this wrong?"

Curated lists

The community maintains an **awesome-typst** list at <https://github.com/qjcg/awesome-typst> — a hand-curated collection of packages, templates, tools, and articles, grouped by what they're for. It's a good way to discover the corners of the ecosystem you wouldn't think to search for, and a gentler survey than scrolling Universe package by package.

Staying current

Typst is young and moves fast, and that cuts both ways: features land quickly, but so do changes that can shift the ground under an old document. Two habits keep you on solid ground.

Watch the releases and changelog on the compiler repo. When a new Typst comes out, a two-minute skim of the changelog tells you whether anything you rely on moved.

And remember the lesson from Chapter 20: **pin your package versions, and re-pin when you upgrade the compiler**. A package frozen to an old version can fall behind a newer Typst and start erroring; when it does, check Universe for a fresh release and bump your pin. This book targets **Typst 0.15.0**. If you're reading it against a later release, treat that as your homework: skim the changelog for what's changed since, and expect a few details here to have been smoothed over or renamed. The ideas hold; the odd argument name may not.

How to ask a good question

Whether on the forum or in chat, the questions that get fast, useful answers all look the same. Give three things:

- A **minimal reproducible example** — the fewest lines that compile *and* show the problem. Not your whole document; the smallest slice that still misbehaves. Half the time, building this example makes you find the answer yourself, which is a fine outcome too.
- Your **Typst version** (`typst --version`), since behaviour genuinely differs between releases.
- **What you expected versus what you got** — the actual error text, or a plain description of how the output was wrong.

That's it. A question with those three parts is a joy to answer; a screenshot of a whole page with "it's broken" is not.

Where to practice

The single fastest way to get fluent is to rebuild something you already have. Take a document that exists — your CV, a report you wrote last month, a page of lecture notes, the minutes of a meeting — and typeset it again in Typst. You already know what the finished thing should look like, so all of your attention goes to *how* Typst gets you there. You'll hit real questions instead of toy ones, you'll reach for the Reference for real reasons, and by the end you'll have both a document you can use and a working knowledge you didn't have that morning. Do it two or three times and Typst stops being a thing you're learning and starts being a thing you use.

A last word

You started this book being promised that you didn't need to know LaTeX and you didn't need to be a programmer, only that you could edit a plain-text file and weren't frightened of a curly brace. That promise held. Along the way you picked up a genuine little programming language, a styling engine built on two ideas, and a compiler that talks back in plain sentences.

Look at what that adds up to. You can typeset a clean essay. You can write a scientific paper with numbered figures, cross-references, and a bibliography that formats itself. You can build a template and hand it to two hundred people. You can, if you like, produce a book that typesets itself from its own source — we did exactly that, and you watched it happen.

The afternoon that Typst was a quiet rebellion against, back in the preface — the one where the table won — is yours again now. The tools have stopped fighting you. What you make with the time is up to you, and there's no better way to finish learning Typst than to close this book and go make something.

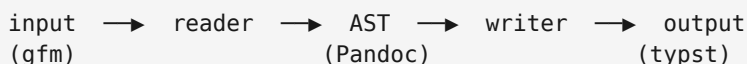
Appendix F · Pandoc, in practice

Chapter 24 told the story of how this book typesets itself — the pipeline, the filter, the head that reuses the Chapter 22 template. This appendix is the other thing: the reference you keep open in a second window while you work, plus an idea bank for the workflows a Pandoc-and-Typst setup makes cheap. The story is there; this is the toolbox.

What Pandoc is

Pandoc is a universal document converter: one program that reads a few dozen formats and writes a few dozen more. You hand it a Word file and ask for Markdown; you hand it Markdown and ask for a PDF; you hand it LaTeX and ask for HTML. It has been the quiet plumbing under half the web's documentation toolchains for fifteen years, and since version 3.0 it can write Typst.

The one idea that makes everything else fall into place: Pandoc does not convert format A straight into format B. It goes through the middle. A **reader** parses your input into an internal document model — an abstract syntax tree, the AST — and then a **writer** renders that tree in the target format. The AST is a plain tree of typed nodes: headings, paragraphs, lists, links, code blocks, emphasis. It knows nothing about `#` or `\section` or `<h1>`; those are just how the gfm reader and the LaTeX reader and the HTML writer happen to spell a `Header`.



```
graph LR; input["input (gfm)"] --> reader; reader --> AST["AST (Pandoc)"]; AST --> writer; writer --> output["output (typst)"]
```

That "AST in the middle" is not a detail — it's the whole reason the tool scales. Adding a new input format means writing one reader, and it instantly converts to every existing output. It is also what makes filters make sense: a filter is a program that edits the tree *after* the reader and *before* the writer, so you shape the conversion at the structural level instead of hacking at text. Hold onto that picture; the filters section leans on it entirely.

For a Typst user, a handful of the formats matter. On the way **in**: `gfm` (GitHub-flavored Markdown, this book's source dialect), `commonmark`, Pandoc's own extended `markdown`, `latex`, `html`, `docx`, and `epub`. On the way **out**: `typst`, obviously, but also `latex`, `html`, `docx`, and `epub` when you need to send the same content somewhere Typst doesn't reach. The rest of this appendix mostly cares about the diagonal from `gfm` (or `docx`, or `latex`) in to `typst` out.

Where does Pandoc sit next to Typst? They don't compete; they compose. Pandoc **converts** — it moves content between representations and never lays out a page. Typst **typesets** — it takes a document and produces the finished, paginated PDF. Pandoc gets your words *into* Typst; Typst makes them beautiful. The moment you internalize that division of labour, the "which tool" question stops being a question.

The commands worth knowing

The whole surface you need day to day is small. Here are the flags a Typst user reaches for, then a set of complete command lines for real jobs.

| Flag | What it does |
|---|---|
| <code>-f</code> , <code>--from FORMAT</code> | Input format. <code>gfm</code> , <code>commonmark</code> , <code>markdown</code> , <code>latex</code> , <code>docx</code> , <code>html</code> , <code>epub</code> . |
| <code>-t</code> , <code>--to FORMAT</code> | Output format. <code>typst</code> , <code>latex</code> , <code>html</code> , <code>docx</code> , <code>epub</code> . |
| <code>-o</code> , <code>--output FILE</code> | Write to a file. Omit it and Pandoc prints to stdout. |
| <code>-s</code> , <code>--standalone</code> | Emit a complete document (preamble, wrapper) rather than a bare fragment. |
| <code>--lua-filter FILE</code> | Run a Lua filter on the AST, in-process. Repeatable; they chain in order. |
| <code>--filter PROG</code> | Run an external (JSON) filter — any language. Repeatable. |
| <code>-M</code> , <code>--metadata KEY=VAL</code> | Set one metadata value (title, author, ...) on the command line. |
| <code>--metadata-file FILE</code> | Read metadata from a YAML file instead. |
| <code>--extract-media DIR</code> | Pull embedded images (e.g. out of a <code>.docx</code>) into <code>DIR</code> and rewrite links to them. |
| <code>--wrap none auto preserve</code> | How to wrap output lines. <code>none</code> gives one long line per paragraph. |
| <code>--toc</code> | Generate a table of contents (with <code>-s</code>). |
| <code>--resource-path DIR</code> | Where to look for images and other linked resources. |
| <code>--list-extensions=FORMAT</code> | Print every syntax extension for a format and whether it's on by default. |
| <code>--verbose</code> , <code>--trace</code> | Louder logging; <code>--trace</code> narrates the AST transforms for debugging. |

The smallest useful command is the one from Chapter 24 with a file on each end:

```
pandoc in.md -o out.typ
```

Pandoc infers `gfm`-ish Markdown from the `.md` and Typst from the `.typ`, so it often does the right thing with no `-f` / `-t` at all. Say them explicitly when the extension is ambiguous or when you want a specific dialect:

```
pandoc in.md --from gfm --to typst -o out.typ
```

A Word manuscript to Typst, with its embedded images unpacked to a folder and the links rewritten to point at them:

```
pandoc manuscript.docx \
  --from docx --to typst \
  --extract-media media \
  -o manuscript.typ
```

You now have `manuscript.typ` next to a `media/` folder of extracted images, ready to open and refine. (This is the backbone of the Word workflow in Appendix C.)

A whole folder of Markdown notes into one Typst body, in filename order — the generalization of this book's own trick:

```
pandoc chapters/*.md --from gfm --to typst -o body.typ
```

The glob expands alphabetically, so zero-padded numeric prefixes (`01-`, `02-`) give you deterministic ordering for free. Pandoc concatenates the files into a single AST before writing, so cross-file references resolve as if it were one document.

When something converts oddly and you want to see *why*, look at the AST itself. `-t native` prints Pandoc's internal tree in a readable Haskell-ish form; `-t json` prints the same tree as JSON, which is what an external filter actually receives:

```
echo '# Hi *there*' | pandoc -f gfm -t native
pandoc in.md -f gfm -t json | jq .      # pipe to jq to explore
```

Reading the native output is the fastest way to learn node names — you'll see `Header`, `Str`, `Emph`, `Space` — which is exactly what you need before writing a filter. And when you're unsure which Markdown extensions a format turns on:

```
pandoc --list-extensions=gfm
```

Each line is prefixed `+` (enabled) or `-` (disabled); you can flip any of them inline, like `--from gfm-smart` or `--from commonmark+hard_line_breaks`.

TIP

Leaving `-o` off and letting Pandoc print to stdout is the fast way to eyeball a conversion. `pandoc in.md -t typst | less` shows you the Typst without littering the folder with output files.

Filters, concisely

A filter is a program that transforms the AST between the reader and the writer. That is the entire concept, and it is the one from the mental model above made executable: Pandoc parses your input into the tree, hands the tree to your filter, your filter edits it, and the writer renders whatever comes back. You are never doing string surgery on Typst output — you're editing structured nodes before Typst syntax even exists.

Two kinds, differing only in how they run:

- **Lua filters** run in-process, in the Lua interpreter Pandoc embeds. You pass them with `--lua-filter`. No extra install, fast, and they have direct access to Pandoc's `pandoc.*` helper library. This is what you want almost always, and what this book uses.
- **JSON filters** run as a separate program in any language — Python (via `panflute` or `pandocfilters`), Haskell, whatever — communicating with Pandoc over JSON on stdin/stdout. You pass them with `--filter`. Reach for these only when you need a library Lua can't give you.

Either way, the programming model is the same and it is delightfully small. You write functions **named after AST node types**, and Pandoc calls your function once for every matching node it finds while walking the tree. Define a function called `Header` and it fires on every heading; call one `Link` and it fires on every link. The node types you'll reach for most: `Str` (a run of text), `Para` (a paragraph), `Header`, `Link`, `Image`, `CodeBlock`, `Div` (a block container), `Span` (an inline container), `Math`, and `RawBlock` / `RawInline` (literal target-format passthrough).

What your function returns is the whole control surface:

- `nil` (return nothing) — leave the node exactly as it was.
- **a single node** — replace the node with that one.
- **a list of nodes** (`{a, b, c}`) — splice those in where the node was.
- `{}` (an empty list) — delete the node.

Pandoc walks the tree bottom-up and applies your functions as it goes, so a filter is usually just a few small functions, each minding one node type. The one special move you'll use constantly for Typst work is injecting raw target-format text: `pandoc.RawBlock("typst", "...")` and its inline sibling `pandoc.RawInline("typst", "...")` drop a literal chunk of Typst into the stream that the writer passes through verbatim. That's how you emit a `#figure(...)` call or a `#note[...]` wrapper that has no Markdown equivalent.

Here's a small, complete filter — one Chapter 24's filter doesn't do. It turns a standalone image (a paragraph that is *only* an image) into a Typst `#figure`, using the image's alt text as the caption:

```
-- figure-images.lua
-- Wrap a standalone image in a Typst #figure, captioned with its alt text.

function Para(el)
  -- Act only on a paragraph that is exactly one image and nothing else.
  if #el.content == 1 and el.content[1].t == "Image" then
    local img = el.content[1]
    -- The alt text (the [ ... ] part of ![alt](src)) becomes the caption.
    local caption = pandoc.utils.stringify(img.caption)
    -- %q quotes and escapes the path as a valid Typst string literal.
    local typst = string.format(
      '#figure(image(%q), caption: [%s])',
      img.src, caption)
    return pandoc.RawBlock("typst", typst)
  end
  -- Every other paragraph: return nothing, so Pandoc leaves it untouched.
end
```

Run it like any filter, and every full-width image in your document comes out as a numbered, captioned figure:

```
pandoc in.md --from gfm --to typst --lua-filter figure-images.lua -o out.typ
```


Read it against the return-value rules above: the images-in-paragraphs get replaced by a raw Typst block; every other paragraph falls through the implicit `nil` and is written normally. Thirteen lines, one clean structural change.

For the real, book-scale version — alerts to admonitions, math passthrough, link rewriting, heading numbering, dropped author comments — read `book-filter.lua` and the walkthrough in Chapter 24. It's the same handful of element functions, just more of them.

TIP

`pandoc lua` drops you into a REPL with the whole `pandoc.*` library loaded, which is the fastest way to check what `pandoc.utils.stringify` or `pandoc.RawBlock` actually returns before you wire it into a filter. The full reference is the Lua filters documentation at <https://pandoc.org/lua-filters.html>.

Pandoc + Typst: an idea bank

Everything so far was mechanics. This is the part where the mechanics pay off — concrete workflows worth trying, each one a hinge between a format that's pleasant to *write* in and Typst's power to *typeset*. Treat these as prompts, not prescriptions.

- **Author in Word, typeset in Typst.** A co-author who lives in Word (or Google Docs, exported as `.docx`) hands you a manuscript; you want a real book out of it. `pandoc thesis.docx --to typst --extract-media media -o body.typ` converts the prose and unpacks the images in one shot, and you drop the result into a Typst template. The seams are predictable — tracked changes and comments vanish, complex tables need a look — and Appendix C walks the whole path.
- **A folder of Markdown into one PDF.** This is the book's own trick, generalized to any pile of notes: lecture notes, a research journal, a wiki export. Glob the files in order, convert to one Typst body, prepend a head that sets the page and imports a template, compile. You keep writing in the editor you like and get a typeset PDF whenever you want one.
- **Markdown as a 90%-there starting point.** Even without a template, `pandoc notes.md -o notes.typ` gets you readable Typst you can then hand-refine. The conversion does the tedious 90% — headings, lists, tables, links — and you spend your attention on the last 10% that actually needs judgment: page breaks, figure placement, the one table that wants custom columns.
- **Slide decks, via the prose.** Pandoc writes Reveal.js and Beamer, but it does **not** write Typst slides — there's no `polylux` or `touying` writer. So the pattern flips: convert your Markdown prose to Typst *content*, then paste the pieces into a slide template (`polylux` or `touying`) and mark the slide breaks by hand. Pandoc carries the words across; the slide structure is yours to build.
- **Born-digital in, print out.** Have an EPUB or a website you want in print? `pandoc book.epub --to typst` or `pandoc page.html --to typst` gives you a Typst starting point for a proper print edition of something that only ever existed on a screen. Expect to clean up navigation cruft and re-decide the layout, but the text arrives intact.
- **LaTeX to Typst, first pass.** `pandoc old-paper.tex --to typst` is a genuine head start on a migration — the structure, sections, and plain math often come through usably. Be honest about its limits: anything macro-heavy or math-dense is a *rough* first pass, not a finished translation, and custom `\newcommand` definitions won't survive. Use it to get 60% of the way, then follow Appendix B for the rest.
- **One source, two outputs, with math protected.** This book writes its math in Typst syntax right in the Markdown and uses a filter to pass it through untouched (the `Math` function in Chapter 24). The payoff is that a single `.md` file renders as legible math on GitHub *and* compiles to a Typst PDF — no second copy, no translation step. If you want one source to serve both a web render and a print PDF, this is the move.
- **Rebuild on every push.** A git `pre-commit` hook or a GitHub Action that runs your build script turns "the PDF is always current" into something you never think about. The Action installs Typst and Pandoc, runs the same `build-book.sh` you run locally, and uploads the PDF as an artifact (or commits it, or attaches it to a release). No workflow file here — just know the shape: the same three commands, on a runner, on every push.

None of this is free of friction, and it's only fair to name the seams. Pandoc's Typst writer is young — younger than its LaTeX writer by more than a decade — so some things convert cleanly, some need a filter to come out right, and some need hand-work no converter can do for you. Tables arrive wrapped in figures. Fine-grained typography is yours to set. Anything exotic in the source is where you'll spend your time. But the seams are all findable, and every one of them is either a small filter or a template tweak away from smooth.

The through-line is the same one Chapter 24 closes on, and it's worth carrying out of this book: write where writing is pleasant, typeset where typesetting is powerful, and let Pandoc be the hinge between them. That's the whole trick — Markdown for the words, Typst for the page, and a converter in the middle so you never have to choose.

About the authors

Lars De Richter

Lars De Richter is a lecturer at Thomas More Hogeschool in Flanders and, at the time of writing, a master's student in educational sciences — a formal way of saying he is professionally curious and has the transcripts to prove it. He learns things by building them, a habit that explains how someone who had known Typst for all of a few weeks came to commission, direct, and quality-control an entire book about it. He wrote none of these chapters and is answerable for every one of them: the brief, the standard, the pointed feedback, and the refusal to ship anything that didn't actually compile. He works in Flemish Dutch, thinks in diagrams, and holds that the fastest way to understand a tool is to make it do something real. This book was that something.

Claude (Opus 4.8)

Claude is a large language model made by Anthropic, and the closest thing this book has to a ghostwriter who works for electricity. It produced most of the words you just read, none of which it can remember writing, and it has never once seen a page it typeset — it cannot see, and it has firm opinions about kerning regardless. Its relationship with the truth is enthusiastic but wants supervision: left to itself it will write ninety-odd thousand confident words and a handful of links that lead nowhere, which is precisely what happened until a human noticed. What it is genuinely good at is turning a clear standard into a great deal of careful, consistent prose very quickly — and then re-compiling every example to check its own homework, because the one editor it could not talk its way past was the Typst compiler, which either builds the file or does not. Every example in this book builds. That part, at least, it can promise.

Colophon

This book was written in Markdown, converted to Typst with Pandoc, and typeset by Typst 0.15.0 — using the very book template it builds by hand in Chapter 22. There was no separate design step: the pipeline that produced this PDF is the one the book teaches, run on the book’s own source. Every code example was compiled against a real Typst binary before it earned its place; nothing here is a snippet that merely looks as though it would work.

It is set in Libertinus Serif, with New Computer Modern for titles and headings and DejaVu Sans Mono for code — three faces that ship with Typst, so the book renders the same on any machine. The whole process, from the first prompt to the managing-editor-and-sub-agents workflow that ran it, is documented openly at github.com/lars-derichter/learning-typst-book.

Licence



Learning Typst — its text and its example code — is released under a Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International licence. Share it, remix it, teach from it; just credit the source, don’t sell it, and pass on the same freedoms. Full terms: creativecommons.org/licenses/by-nc-sa/4.0.

Typesetting that gets out of your way

Typst is what you get if you cross the readability of Markdown with the power of LaTeX and hand the result to people who like fast feedback and error messages that make sense. *Learning Typst* teaches it from `= Hello` to writing your own packages — and then does the thing few programming books dare: it turns around and typesets **itself** with the very tools it just taught you.

You don't need to know LaTeX. You don't need to be a programmer. You need a plain-text editor, a little curiosity, and the willingness to run a few examples — every one of which, in this book, genuinely compiles.

By the end you can:

- set clean documents, tables, and math that behaves;
- bend the defaults with `set` and `show` rules;
- write real Typst code — functions, data, and state;
- build a reusable book template and hand it to other people;
- turn a folder of Markdown into a finished PDF.